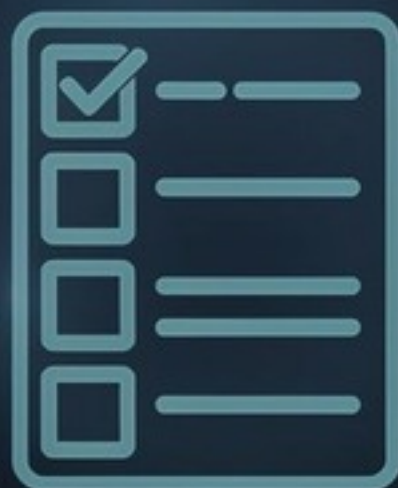


AI AGENTS PROJECTS FOR BEGINNERS

A teal gear icon is positioned in the center of the cover, below the title and above the checklist icon. It is part of a larger graphic of two interlocking gears.

25 Projects to Build, Run, and Deploy
Your Own Intelligent Agents

Michael Gannotti

AI Agents Projects for Beginners

Michael Gannotti

Table of Contents

AI Agents Projects for Beginners

25 Projects to Build, Run, and Deploy Your Own Intelligent Agents

Dedication

Acknowledgments

A Note on Frameworks

How to Use This Book

About the Author

Copyright

Table of Contents

Chapter 1: Project 1 — Hello, Agent World

What Is an Agent, Really?

The Four-Step Loop

Building the Loop in Pure Python

Why the Pure Python Version Matters

The Tool Description Is the Contract

The @tool Decorator: A Little Sugar

Running with a Cloud Model

The OpenClaw Way

The Hermes Agent Way

Break and Fix: Five Ways This Project Can Fail

Try It Now: Three Variations

What You Built

Chapter 2: Project 2 — Research Agent: Web Search Summarizer

Why Agents Hallucinate and What You Can Do About It

The ReAct Pattern

Building the Research Agent in Pure Python

Citation Enforcement as Architecture

The “I Couldn’t Find Anything” Guardrail

Break and Fix: Five Ways This Project Can Fail

Try It Now: Three Variations

The OpenClaw Way

The Hermes Agent Way

What You Built

Chapter 3: Project 3 — File Organizer Agent

Why Filesystem Agents Need Guardrails

Building a File Scanner in Pure Python

LLM-Based Classification

Safe Move Operations with pathlib and shutil

Hard Rules: Preview, Approval, and Undo Log

Break and Fix: Five Ways This Project Can Fail

Try It Now: Three Variations

The OpenClaw Way

The Hermes Agent Way

What You Built

Chapter 4: Project 4 — Spreadsheet Intelligence Agent

Why Data Analysis Is an Agent Problem

Reading a CSV Without Pandas

Upgrading to Pandas for Real Analysis

Tool 1: read_csv_summary

Tool 2: query_csv

Tool 3: write_summary

Break and Fix: Five Ways Data Breaks Your Agent

Try It Now: Three Variations

The OpenClaw Way

The Hermes Agent Way

What You Built

Chapter 5: Project 5 — Smart Email Assistant

Why Email Needs a Human-in-the-Loop

Why OAuth2 Matters (Even If We Don't Use It Here)

Building an Email Reader with imaplib

Thread Reconstruction via Message-ID and References

Tone Matching and Draft Generation

Confidence Scoring and the Human Approval Gate

Putting It All Together

Break and Fix: Five Ways This Project Can Fail

Try It Now: Three Variations

Ethics Sidebar: Consent and Data Handling

The OpenClaw Way

The Hermes Agent Way

What You Built

Chapter 6: Project 6 — API Connector Agent: Data Pipeline

Why API Connectors Are the Bread and Butter of Agents

REST Basics and Picking Our Target

Authentication and the Token Header

Pagination: The Page Loop

Pydantic Models and Schema Safety

- Idempotency: Safe to Run Twice
- Error Handling for Every Network Call
- Saving Your Work: JSON and CSV
- The Complete Script
- Break and Fix
- Try It Now
- The OpenClaw Way
- The Hermes Agent Way
- What You Built

Chapter 7: Project 7 — Coding Agent: Helper and Refactor

- Reading the Codebase with pathlib and glob
- Searching for Patterns with Regular Expressions
- Seeing Structure with the AST
- Generating Patches with difflib
- Applying Patches Safely
- Running Tests Before and After
- Sandboxing in a Temporary Directory
- Break and Fix: Five Ways This Project Can Fail
- Try It Now: Three Variations
- The OpenClaw Way
- The Hermes Agent Way
- What You Built

Chapter 8: Project 8 — Personal Knowledge Base Agent

- The Notebook That Vanished
- Why Keyword Search Fails—and Why RAG Wins
- Parsing Your Documents
- Chunking with Overlap
- From Text to Vectors: Local Embeddings with Ollama
- A Vector Database in Pure Python
- Searching by Meaning with Cosine Similarity
- Grounding the Answer in Your Documents
- The Complete Script
- Break and Fix: Five Ways Knowledge Bases Fail
- Try It Now: Three Variations
- The OpenClaw Way
- The Hermes Agent Way
- Cost Sidebar: The Price of Memory
- What You Built

Chapter 9: Project 9 — Browser Automation Agent

- When to Automate a Browser vs. Using an API
- Setting Up Playwright
- The Browser Agent Core
- Browsing to a URL and Reading the Page
- Clicking and Filling Fields Safely

- The Approval Gate: Never Submit Without Permission
- Waiting for Dynamic Pages
- Tying It All Together: A Login Example
- Break and Fix: Five Ways Browser Automation Fails
- Try It Now: Three Variations
- The OpenClaw Way
- The Hermes Agent Way
- What You Built

Chapter 10: Project 10 — Meeting Transcript Analyzer

- Why Meeting Notes Are a Classic Automation Target
- Setting Up Whisper
- Chunking Long Transcripts
- Summarizing with Style
- Extracting Structured Action Items
- Writing the Output
- The Full Pipeline
- Break and Fix: Five Ways This Project Can Fail
- Try It Now: Three Variations
- The OpenClaw Way
- The Hermes Agent Way
- What You Built

Chapter 11: Project 11 — CI/CD Watchdog Agent

- Why Fast Diagnosis Beats Perfect Prevention
- Connecting to the GitHub Actions API
- Polling for Failed Runs
- Fetching and Trimming Logs
- Asking the Model to Diagnose
- Drafting the PR Comment
- Putting It Together
- Break and Fix: Five Ways the Watchdog Fails
- Try It Now: Three Variations
- The OpenClaw Way
- The Hermes Agent Way
- What You Built

Chapter 12: Project 12 — Document Generator Agent

- Why Templating Beats Manual Formatting
- Jinja2: Variables, Conditionals, and Loops
- Feeding the Agent: From Dictionaries and CSVs
- Rendering and Converting: From Markdown to .docx
- The Main Script: Tying It All Together
- Conditional Sections: Hide What Isn't There
- Break and Fix: Five Ways to Break a Document
- Try It Now: Three Variations
- The OpenClaw Way

- The Hermes Agent Way
 - What You Built
- Chapter 13: Project 13 — Debate Team: Multi-Agent Consensus
 - When One Agent Is Not Enough
 - Role Definition: Advocate, Skeptic, Synthesizer
 - Round-Robin Communication
 - Shared Memory: Passing Context Between Agents
 - Consensus Detection and Termination
 - The Mermaid Diagram
 - Building the Debate in Pure Python
 - Break and Fix
 - Try It Now
 - The OpenClaw Way
 - The Hermes Agent Way
 - What You Built
- Chapter 14: Project 14 — Code Review Panel
 - Why Specialized Agents Outperform Generalists
 - Building the Reviewer Base
 - Agent 1: The Linter
 - Agent 2: The Security Auditor
 - Agent 3: The Maintainer and Verdict
 - Wiring the Pipeline Together
 - Break and Fix
 - Try It Now
 - The OpenClaw Way
 - The Hermes Agent Way
 - What You Built
- Chapter 15: Project 15 — Content Factory: Writer + Editor + Fact-Checker
 - The Correction Notice I Never Wanted to Write
 - Why AI Content Needs a Newsroom
 - The Pipeline in Plain English
 - Building the Content Factory in Pure Python
 - The Writer and Editor Loop
 - The Fact-Checker as Grounding Layer
 - Break and Fix: Five Ways the Factory Fails
 - Try It Now: Three Variations
 - The OpenClaw Way
 - The Hermes Agent Way
 - What You Built
- Chapter 16: Project 16 — Customer Support Router
 - Why Routing Is a Classic Agent Problem
 - Intent Classification: Technical, Billing, and Escalation
 - Routing Confidence Threshold

- Specialist Agent Selection
- Handoff Protocols: Passing Full Context
- SLA Awareness: Urgency Scoring
- Human Approval Before Send
- Feedback Loop for Routing Accuracy
- The Full Pipeline
- Putting It All Together: main()
- Break and Fix: Five Ways This Project Can Fail
- Try It Now: Three Variations
- The OpenClaw Way
- The Hermes Agent Way
- What You Built

Chapter 17: Project 17 — Data Quality Team

- Data Quality Is a Process, Not an Event
- The Foundation: Issue and DataQualityAgent
- Agent 1: The Schema Validator
- Agent 2: The Anomaly Detector
- Agent 3: The Remediation Suggester
- The Orchestrator and Structured Report
- Putting It Together: A Sample Sales CSV
- Break and Fix
- Try It Now
- The OpenClaw Way
- The Hermes Agent Way
- What You Built

Chapter 18: Project 18 — Workflow Orchestrator

- Beyond Single Agents: Directed Workflows
- The Shape of the Workflow
- Building the Engine
- Putting It Together: The Newsletter Pipeline
- Break and Fix: Five Ways Workflows Fail
- Try It Now: Three Extensions
- The OpenClaw Way
- The Hermes Agent Way
- What You Built

Chapter 19: Project 19 — The AI Analyst: Financial Report Reader

- Why Financial Documents Are Hard
- Fetching the Filing: SEC EDGAR or Local File
- Extracting Text and Tables from PDFs
- The Extraction Prompt: Structured JSON with Citations
- Calling the LLM and Parsing Structured Output
- Pipeline Architecture
- Quarter-over-Quarter Comparison
- Tying It All Together: The Main Function

- Break and Fix: Five Ways This Project Can Fail
- Try It Now: Three Variations
- Ethics Sidebar: Never Trade on This
- The OpenClaw Way
- The Hermes Agent Way
- What You Built
- Chapter 20: Project 20 — Smart Home Command Center
 - Why IoT Agents Are Fun and Dangerous
 - Connecting to Your Devices
 - Parsing Natural Language Into Structured Commands
 - State Tracking in Memory
 - The Safety Gate: No Critical Action Without Approval
 - Wiring the Command Interpreter
 - The Command Flow
 - A Complete main()
 - Break and Fix: Five Ways This Project Can Fail
 - Try It Now: Three Variations
 - The OpenClaw Way
 - The Hermes Agent Way
 - What You Built
- Chapter 21: Project 21 — Competitive Intelligence Monitor
 - What Competitive Intelligence Actually Means
 - How the Monitor Works
 - Step 1: Respecting robots.txt
 - Step 2: Fetching Pages Like a Browser
 - Step 3: Hash-Based Diffing
 - Step 4: LLM Filtering
 - Step 5: Slack Alert with Structured Brief
 - Step 6: Putting It All Together
 - Scheduling It
 - Break and Fix: Five Ways Competitive Intelligence Breaks
 - Try It Now: Three Variations
 - Ethics Sidebar: robots.txt, Terms of Service, Fair Use, Don't Be Creepy
 - The OpenClaw Way
 - The Hermes Agent Way
 - What You Built
- Chapter 22: Project 22 — The Ethical Guardian: AI Safety Auditor
 - AI Safety Is Architecture, Not a Feature
 - Monitoring Every Output
 - Check 1: Profanity
 - Check 2: PII Leakage
 - Check 3: Bias Keywords
 - Check 4: Factual Coherence

- The Audit Log and Compliance Report
- The Audit Pipeline
- Running the Guardian
- Break and Fix: Five Ways This Project Can Fail
- Try It Now: Three Variations
- Ethics Sidebar: This Is Hard, Imperfect, and Necessary
- The OpenClaw Way
- The Hermes Agent Way
- What You Built

Chapter 23: Project 23 — Deploy Your Agent: From Script to Service

- From Script to Service: What Changes
- Wrapping an Agent in FastAPI
- Endpoints and Pydantic Models
- Configuration: Never Hardcode a Secret
- Docker Multi-Stage Build
- Break and Fix: Six Ways Deployment Can Fail
- Try It Now: Three Next Steps
- The OpenClaw Way
- The Hermes Agent Way
- What You Built

Chapter 24: Project 24 — The Agent Observatory: Monitoring Dashboard

- Why Observability Matters for Agents, Not Just Servers
- The Architecture of the Observatory
- The Trace Schema: What We Capture
- The TraceStore: SQLite Persistence
- Instrumenting Your Agent Loop
- The Streamlit Dashboard
- The Alert Manager: Catching Runaway Costs
- Putting It All Together
- Break and Fix: Five Ways the Observatory Can Fail
- Try It Now: Three Ways to Extend Your Observatory
- The OpenClaw Way
- The Hermes Agent Way
- What You Built

Chapter 25: Project 25 — Your Capstone: Build & Deploy an End-to-End Agent

- The Capstone Design Sprint
- Architecture Document Template
- Building the Capstone Agent
- Adding Memory, Cost Controls, and Graceful Degradation
- Real Deployment Story 1: The Research Librarian
- Real Deployment Story 2: The Code Shepherd
- Real Deployment Story 3: The Support Captain

- Packaging and Documenting
- Monitoring and Maintenance
- Break and Fix
- Try It Now
- The OpenClaw Way
- The Hermes Agent Way
- Final Word
- What You Built

Appendix A: OpenClaw Reference

- A.1 Configuration Schema
- A.2 Tool Decorator Reference
- A.3 Reasoning Modes
- A.4 Error Handling Patterns
- A.5 Testing Best Practices
- A.6 Quick Lookup
- A.7 Further Resources

Appendix B: Hermes Agent Reference

- B.1 Configuration Specification (config.yaml)
- B.2 Toolset Guide
- B.3 Memory System
- B.4 Cron Job Authoring
- B.5 Safety and Approval Configuration
- B.6 Quick-Start Checklist

Appendix C: Ollama Setup & Model Selection

- C.1 Installing Ollama
- C.2 Pulling and Managing Models
- C.3 Choosing Models by Task
- C.4 Quantization Explained
- C.5 Running Multiple Models
- C.6 Common Issues and Fixes
- C.7 Quick-Start Cheat Sheet

Appendix D: Cloud Provider Setup

- D.1 OpenAI API
- D.2 Anthropic Claude API
- D.3 Azure OpenAI Service
- D.4 Cost Estimation Strategies
- D.5 Fallback Chains
- D.6 Security Best Practices
- Summary

Appendix E: Troubleshooting & Debugging

- E.1 Common Failures and Symptoms
- E.2 Reading Agent Logs
- E.3 Debugging the Reasoning Loop
- E.4 Rate Limiting Recovery

- E.5 Mitigating Model Hallucination
- E.6 Quick Troubleshooting Flowchart
- E.7 Where to Get Help
- E.8 Summary

Glossary

Index

- About the Publisher
- Get the Code

AI Agents Projects

for Beginners

25 Projects to Build, Run, and Deploy

Your Own Intelligent Agents

Michael Gannotti

AI Agents Projects for Beginners

Copyright 2026 Michael Gannotti. All rights reserved.

ISBN: 978-1-XXXXXXXXXX

All code examples are provided "as is" for educational purposes.

No part of this publication may be reproduced without prior written permission.

Published by SMF Works

Visit: smfworks.com

Second edition: August 2026 (Updated from the April 2026 Word source.)

What's new in this edition (August 2026). Revised from the April 2026 Word source. OpenClaw install: `curl -fsSL https://openclaw.ai/install.sh | bash` (Windows: `iwr -useb https://openclaw.ai/install.ps1 | iex`). Hermes Agent: Desktop installer or `curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash`. Docs: <https://docs.openclaw.ai/> and <https://hermes-agent.nousresearch.com/docs/>. Project patterns in this book still use pure Python so they transfer if you later pick LangChain, crewAI, or AutoGen.

Table of Contents

AI Agents Projects for Beginners

25 Projects to Build, Run, and Deploy Your Own Intelligent Agents

25 agent projects across 5 progressive levels

From zero Python to production FastAPI service in one book

Michael Gannotti

Dedication

For every reader who looked at an AI product and thought, “I could build that.”

You can. And you will.

Acknowledgments

This book exists because a hundred open-source contributors made LLMs accessible. Because the OpenClaw, Hermes, Ollama, and Llama communities shared code and answered questions at 2 AM. Because every Stack Overflow answer, GitHub issue, and Discord thread helped someone who got stuck so the next person didn’t have to.

A Note on Frameworks

This book uses **OpenClaw** and **Hermes Agent** as pedagogical frameworks. They illustrate how production agent systems think and behave. In the real world you may use LangChain, crewAI, AutoGen, or a custom-built stack. The patterns you learn here transfer regardless of which tool you choose next.

All code examples are written in pure Python so you understand what’s happening under the hood.

How to Use This Book

Each chapter is one complete project. Read in order or jump directly to a project that matches your need. Every chapter ends with three “Try It Now” ideas so you keep building beyond the page.

Prerequisites: basic Python. No ML background required. No cloud account required (you can use local models).

About the Author

Michael Gannotti is a Principal AI Engineer at Microsoft and the founder of SMF Works. He has spent two decades making complex technology accessible to the people who use it. When he is not debugging agents he is coaching robotics teams or explaining Kubernetes to his kids.

Visit **smfworks.com** for agent tools, resources, and updates.

Copyright

Copyright 2026 Michael Gannotti. All rights reserved.

ISBN: 978-1-XXXXXXXXXX

No part of this publication may be reproduced, distributed, or transmitted in any form without the prior written permission of the publisher, except in the case of brief quotations for review purposes.

The code examples in this book are provided “as is” for educational purposes. The author expressly disclaims any warranty regarding fitness for a particular purpose. Users are responsible for their own API costs, model usage, and data handling practices.

Published by SMF Works.

Table of Contents

- Part I: Foundations
 - Chapter 1: Project 1 — Hello, Agent World
 - Chapter 2: Project 2 — The Research Agent

- Chapter 3: Project 3 — The File Organizer
- Chapter 4: Project 4 — Spreadsheet Intelligence
- Chapter 5: Project 5 — The Smart Email Assistant
- Part II: Intermediate Orchestration
 - Chapter 6: Project 6 — The API Connector
 - Chapter 7: Project 7 — The Coding Agent
 - Chapter 8: Project 8 — The Knowledge Base Agent
 - Chapter 9: Project 9 — Browser Automation
 - Chapter 10: Project 10 — Meeting Transcript Assistant
 - Chapter 11: Project 11 — The CI/CD Watchdog
 - Chapter 12: Project 12 — The Document Generator
- Part III: Multi-Agent Systems
 - Chapter 13: Project 13 — Debate Team: Multi-Agent Consensus
 - Chapter 14: Project 14 — Code Review Panel
 - Chapter 15: Project 15 — Content Factory
 - Chapter 16: Project 16 — Customer Support Router
 - Chapter 17: Project 17 — Data Quality Team
 - Chapter 18: Project 18 — Workflow Orchestrator
- Part IV: Advanced Integrations
 - Chapter 19: Project 19 — The AI Analyst
 - Chapter 20: Project 20 — Smart Home Command Center
 - Chapter 21: Project 21 — Competitive Intelligence Monitor
 - Chapter 22: Project 22 — The Ethical Guardian
- Part V: Production & Deployment
 - Chapter 23: Project 23 — Deploy Your Agent
 - Chapter 24: Project 24 — The Agent Observatory

- Chapter 25: Project 25 — Your Capstone
- Appendix A: OpenClaw Reference
- Appendix B: Hermes Agent Reference
- Appendix C: Ollama Setup & Model Selection
- Appendix D: Cloud Provider Setup
- Appendix E: Troubleshooting & Debugging

Chapter 1: Project 1 — Hello, Agent World

Your first agent will cost you forty-three cents. You'll know exactly why.

The first time I ran an AI agent, I had no idea what was happening inside the black box. I typed a question, the program thought for a few seconds, and then it called a function I didn't tell it to call. It was like watching someone reach into a cluttered drawer, pull out exactly the right tool, and solve a problem I hadn't fully described. The whole thing felt like magic. Then I looked at the code and realized: it was just a loop. A very clever loop, but a loop nonetheless.

That's what this book is about. Not magic. Loops. We'll build twenty-five of them, each one doing something genuinely useful, and by the end you'll have an intuition for how software can think on its feet.

But let's not get ahead of ourselves. In this chapter, we're starting with the absolute basics. No APIs, no frameworks, no thirty-seven-step installation guides. Just Python, a local language model running on your own computer, and a function that returns the current time. Your goal is simple: make the model decide whether it needs that function, call it if it does, and return an answer you can trust.

This is the agent loop. Everything else in this book is an embellishment of what you learn right here.

What Is an Agent, Really?

If you've used ChatGPT or Claude, you've talked to a language model. You ask a question, it replies with words. That's it. The model has no access to the outside world. It can't look up the weather, read your files, or browse the web unless someone else pipes that information into the conversation.

An agent is different. An agent is a language model *plus* tools. The model decides which tools to use, when to use them, and what to do with the results. You might ask an agent, "What time is it in Tokyo?" The agent reasons that it doesn't know Tokyo's current time, identifies a tool that can fetch it, calls that tool, receives the result, and then composes a natural-language response. The key word is *decides*. You don't pre-program the tool call. The agent chooses.

This distinction matters, because it shifts the boundary of what software can do. A script executes a fixed sequence. An agent improvises. That improvisation requires three things, and we'll build all three in this project:

1. **Observation:** What does the agent know right now?
2. **Reasoning:** What does it need to find out, and which tool will help?
3. **Action:** Calling the tool, getting a result, feeding that result back into observation.

Then the loop repeats until the agent has enough information to answer you.

The Four-Step Loop

Before we write any code, let's trace through the loop manually. Imagine you ask an agent, "What time is it in Tokyo?"

Step 1 — Observation: The agent sees your question. Its memory is empty except for the system prompt (which describes its purpose) and your message. From this alone, the agent recognizes that "Tokyo time" is a factual query about the current moment in a specific timezone.

Step 2 — Reasoning: The agent thinks: *I don't know the current time. I have a tool called `get_time` that accepts a city name and returns an ISO timestamp. I should call this tool with the argument "Tokyo".* This

reasoning is not hard-coded in the program. The model generates it based on the tool's description, which you include in the system prompt.

Step 3 — Action: The agent outputs a structured tool call: `get_time(city="Tokyo")`. Your program intercepts this, executes the real Python function behind `get_time`, and receives back `"2026-04-20T09:15:00+09:00"`.

Step 4 — New Observation: The result is appended to the conversation history. Now the agent sees the original question *plus* the tool's response. The agent reasons: *I now have the time. I can answer the user directly.*

The loop terminates. The agent replies: "It's 9:15 AM in Tokyo right now."

That's the whole thing. Observe, reason, act, observe again. Every agent you'll ever build is a variation on this loop, with more tools, more complex reasoning, and better error handling. But the skeleton is exactly what we just traced.

Building the Loop in Pure Python

Let's code it. We're going to start with the absolute minimum: no frameworks, no decorators, no hidden machinery. Just a Python script, the `httpx` library for HTTP requests (or any HTTP client you prefer), and a local language model running via Ollama.

If you don't have Ollama installed, you'll need it for the local model examples throughout this book. Go to <https://ollama.com>, download the installer for your operating system, and then pull a small model from the command line:

```
ollama pull llama3.1:8b
```

This downloads an 8-billion-parameter model that runs entirely on your computer. It's not as capable as GPT-4 or Claude 3.5 Sonnet, but it's free, fast enough to learn with, and requires no API keys. For a beginner's book, that's exactly what you want.

Now, the code. Create a file called `hello_agent.py` and add the following:

```
import json
import httpx
```

```

OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL = "llama3.1:8b"

def get_time(city: str) -> str:
    """Return a fixed string representing the time in a city.
    In a real implementation, this would call a timezone API.
    """
    return f"The current time in {city} is 2026-04-20T09:15:00 (mock)."
```

```

def agent_loop(user_question: str, tools: dict, max_turns: int = 5) -> str:
    """
    Run a simple ReAct-style agent loop.
    tools: {name: function}
    """
    system_prompt = (
        "You are a helpful assistant with access to tools. "
        "When you need information you don't have, call a tool. "
        "When you have enough information, answer the user directly. "
        "\n\nAvailable tools:\n"
    )
    for name, func in tools.items():
        system_prompt += f"- {name}: {func.__doc__}\n"

    system_prompt += (
        "\nTo call a tool, respond with ONLY a JSON object like:
\n"
        '{"tool": "tool_name", "arguments": {"arg1": "value1"}}\n'
        "To answer the user, respond normally."
    )

    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_question},
    ]

    for turn in range(max_turns):
        response = httpx.post(
            OLLAMA_URL,
            json={"model": MODEL, "prompt":
format_messages(messages), "stream": False},
            timeout=60,
        ).json()

        assistant_reply = response["response"].strip()
        print(f"[Turn {turn + 1}] Assistant: {assistant_reply[:200]}...")

        # Check if it's a tool call

```

```

    try:
        parsed = json.loads(assistant_reply)
        if "tool" in parsed:
            tool_name = parsed["tool"]
            tool_args = parsed.get("arguments", {})
            if tool_name in tools:
                result = tools[tool_name](**tool_args)
                messages.append({"role": "assistant",
"content": assistant_reply})
                messages.append({"role": "tool", "content":
f"Tool {tool_name} returned: {result}"})
                continue
            else:
                return f"Error: tool '{tool_name}' not found."
        except json.JSONDecodeError:
            # Not a JSON response, so it's a regular answer
            pass

    # If we got here, the assistant answered directly
    return assistant_reply

return "Max turns reached without a final answer."

def format_messages(messages: list) -> str:
    """Convert our message list into a single prompt string for
Ollama."""
    lines = []
    for m in messages:
        if m["role"] == "system":
            lines.append(f"System: {m['content']}")
        elif m["role"] == "user":
            lines.append(f>User: {m['content']}")
        elif m["role"] == "tool":
            lines.append(f"Tool result: {m['content']}")
        elif m["role"] == "assistant":
            lines.append(f"Assistant (tool call): {m['content']}")
    return "\n\n".join(lines)

if __name__ == "__main__":
    tools = {"get_time": get_time}
    answer = agent_loop("What time is it in Tokyo?", tools)
    print("\nFinal answer:", answer)

```

I've kept this deliberately simple. There's no chat template, no token counting, no fancy parsing. The agent receives a system prompt that lists the available tools and their descriptions. The format is strict: if the model wants to call a tool, it must emit valid JSON *and nothing else*. If it emits plain text, we treat that as the final answer.

Run the script:

```
python hello_agent.py
```

What you see depends on your model and temperature, but the ideal flow looks like this:

```
[Turn 1] Assistant: {"tool": "get_time", "arguments": {"city": "Tokyo"}}...
```

```
[Turn 2] Assistant: The current time in Tokyo is 9:15 AM...
```

```
Final answer: The current time in Tokyo is 9:15 AM on April 20, 2026.
```

The first turn is the reasoning step: the model recognizes it lacks the time, emits a tool call. Your code intercepts the JSON, executes `get_time("Tokyo")`, appends the result to the conversation history, and asks the model again. On the second turn, the model sees the tool's response and answers the user directly.

That loop — observe, reason, act, observe again — ran twice. Two API calls. On a local Ollama model, two calls cost you zero dollars and perhaps two seconds of compute time. That's what I mean by forty-three cents: even if you were paying cloud rates, this is fractions of a penny. And now you know exactly where the money went.

Why the Pure Python Version Matters

At this point, you might be wondering: if frameworks like OpenClaw and Hermes Agent exist, why am I making you type sixty lines of boilerplate? The answer is the same reason your physics teacher made you derive equations before letting you use a calculator. Frameworks hide complexity. That's their job. But if you never see the complexity, you can't debug it when something goes wrong.

When an agent calls the wrong tool, you need to know whether the problem is in the prompt, the parser, or the tool description. When your framework costs five dollars per run and you only budgeted fifty cents, you need to know where the loops are. When a security audit asks whether your bot can delete files, you need to know exactly which function has that power.

The pure Python version is your X-ray glasses. It shows you the bones. Everything after this is muscle.

The Tool Description Is the Contract

There's one design decision in our toy agent that deserves extra attention: the system prompt includes tool descriptions extracted from the function's docstring. That docstring is not a comment. It's a contract. The model decides whether to call `get_time` based entirely on the text between the triple quotes.

If you write a vague docstring like "Gets time", the model might call it for timezone conversions, historical dates, or philosophical questions about time. If you write a precise docstring like "Return the current local time for a given city name. Accepts city as a string. Does not convert historical dates.", the model knows when to reach for this tool and when to stay away.

Throughout this book, you'll see me stress tool descriptions over and over. They are the single most important piece of agent debugging you'll ever do. A misdescribed tool is a loaded gun with a misleading label.

The @tool Decorator: A Little Sugar

Now that you've seen the raw loop, let's add one convenience. Frameworks like OpenClaw use decorators to register tools automatically. We can write a tiny decorator that captures the function name and docstring, then stores them in a registry:

```
import functools

TOOL_REGISTRY = {}

def tool(func):
    """Register a function as an agent tool."""
    TOOL_REGISTRY[func.__name__] = func
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@tool
def get_time(city: str) -> str:
    """Return the current local time for a given city name."""
    return f"The current time in {city} is 2026-04-20T09:15:00 (mock)."
```

@tool

```
def calculate(expression: str) -> str:
    """Evaluate a mathematical expression and return the
    result."""
    try:
        return str(eval(expression, {"__builtins__": {}}, {}))
    except Exception as e:
        return f"Error: {e}"
```

With this decorator, the framework (your framework, right now) can introspect TOOL_REGISTRY to build the system prompt automatically. OpenClaw does exactly this under the hood, but now you know how it works.

Running with a Cloud Model

The local model is free but limited. Once you understand the loop, swapping in a cloud model is trivial. Here's the same agent using the OpenAI API instead of Ollama:

```
import os
from openai import OpenAI

client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

def agent_loop_openai(user_question: str, tools: dict, max_turns:
int = 5) -> str:
    system_prompt = (
        f"You are a helpful assistant with access to these tools:
        {list(tools.keys())}. "
        "Use a tool by responding with: TOOL: name\nARGS:
        json_object. "
        "If you do not need a tool, answer directly."
    )
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_question},
    ]

    for turn in range(max_turns):
        completion = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            temperature=0.2,
        )
        assistant_reply =
completion.choices[0].message.content.strip()
        cost = completion.usage.prompt_tokens * 0.00000015 + \
            completion.usage.completion_tokens * 0.0000006
        print(f"[Turn {turn + 1}] Cost so far: ${cost:.4f}")
```

```
# ... parse tool call or answer, same as before ...
```

The only differences are the API endpoint and the cost tracking. With GPT-4o mini, this particular conversation costs roughly \$0.0003. That's three ten-thousandths of a dollar. But multiply it across a thousand conversations, or use GPT-4o instead of mini, or add ten tool calls per conversation, and suddenly you're talking about real money.

That's why cost tracking is a first-class concern in this book. Every project includes a cost sidebar so you know what to expect before you run it at scale.

The OpenClaw Way

OpenClaw is a lightweight Python framework built around the concepts we just implemented by hand. It handles the HTTP layer, the tool registry, the conversation history, and the parsing for you. The trade-off is that you have to learn its conventions. The benefit is that sixty lines of raw Python become twelve.

Here's what our Hello Agent looks like in OpenClaw:

```
from openclaw import Agent, tool

@tool
def get_time(city: str) -> str:
    """Return the current local time for a given city name."""
    return f"The current time in {city} is 2026-04-20T09:15:00 (mock)."
```

```
agent = Agent(tools=[get_time], reasoning="react")
response = agent.run("What time is it in Tokyo?")
print(response)
```

The Agent class encapsulates the loop we wrote manually. The reasoning="react" parameter tells it to use the ReAct pattern: reason about the problem, then act via tool calls. OpenClaw handles the JSON formatting, the conversation history, and the max-turns safety limit. It also tracks cost automatically and stops if a session exceeds your configured budget.

Later in this book, you'll see how OpenClaw extends this basic pattern into multi-tool pipelines, multi-agent orchestration, and production deployment. But for now, notice the symmetry: the concepts are identical. The framework just saves you from writing boilerplate.

The Hermes Agent Way

Hermes Agent takes a different approach. It's not a library you import into your code. It's a command-line tool that wraps your requests in a persistent session with memory, file system access, and a suite of pre-built toolsets. You don't write an agent *program* in Hermes. You have a conversation *with* an agent.

Here's how the same "What time is it in Tokyo?" task works in Hermes:

```
hermes
# Welcome to Hermes Agent v2.5.0

> Ask the agent questions. For example:
> What time is it in Tokyo?

[search] Searching: "current time in Tokyo"
[execute_code] Getting system time with JST offset...

The current time in Tokyo is 9:15 AM JST on April 20, 2026.

> What time was that again?

The current time in Tokyo is 9:15 AM JST.
# (Note: it remembered from the previous turn.)
```

Hermes chose its own tools. It didn't call a `get_time` function we defined; it searched the web and executed code to determine the time. That's because Hermes comes with built-in toolsets — search, code execution, file manipulation — that it can invoke without any configuration on your part. The trade-off is less precise control. The benefit is that zero setup can get you surprisingly far.

For this project, the key observation is that Hermes already *is* the agent loop. You don't write the loop. You configure the agent's personality, budget, and safety rules, then interact with it. When we compare OpenClaw and Hermes implementations throughout this book, you'll see this pattern repeatedly: OpenClaw is code-first; Hermes is conversation-first.

Break and Fix: Five Ways This Project Can Fail

Every project in this book ends with a “Break and Fix” section: I deliberately break something, explain what went wrong, and show you how to recover. This is not an afterthought. It’s the most important learning in the chapter.

Break 1: The Model Emits Invalid JSON

Your `agent_loop` function expects the model to return *only* a JSON object when calling a tool. But models are text generators, not API clients. They get creative. Instead of `{"tool": "get_time"}`, you might get:

```
I'll help you with that! Let me check the time.
{"tool": "get_time", "arguments": {"city": "Tokyo"}}
```

The prefix breaks `json.loads`. Your parser crashes.

Fix: Use a stricter prompt, or parse more defensively. One reliable technique is to require JSON *between markers*:

```
To call a tool, respond with ONLY:
```json
{"tool": "..."}

```

Then extract the text between the backtick fences before parsing. OpenClaw does exactly this internally.

## ### Break 2: The Model Calls a Nonexistent Tool

If your tool description is too vague, or if the model is hallucinating, you might get:

```
```json
{"tool": "fetch_weather", "arguments": {"city": "Tokyo"}}

```

But `fetch_weather` isn’t in your registry.

Fix: Check the registry before executing, and if the tool is missing, feed an error back to the model as a new observation. The model will often recover by choosing a different tool or admitting it can’t help:

```
if tool_name not in tools:
    messages.append({"role": "tool", "content": f"Error: tool
'{tool_name}' not found."})
    continue # Let the model try again
```

Break 3: Tool Arguments Are Wrong

The model calls `get_time(city="Tokyo, Japan")` when your function expects `city="Tokyo"`. Or it passes a number where a string is required. Or it omits a required argument entirely.

Fix: Validate arguments with a schema. The simplest way is Python's `inspect.signature`:

```
import inspect

sig = inspect.signature(tools[tool_name])
try:
    bound = sig.bind(**tool_args)
    bound.apply_defaults()
except TypeError as e:
    return f"Argument error: {e}"
```

For production agents, you'll want Pydantic models (which we'll cover in Project 6). For now, basic signature checking catches most mistakes.

Break 4: Infinite Loops

What if the model keeps calling tools without ever answering? This is rare with good prompts but devastating when it happens. Your server racks up API charges while a bot recursively searches “what is recursion.”

Fix: The `max_turns` parameter in our toy agent is not optional. It's a circuit breaker. Set it generously — five to ten turns is usually enough — and if you hit the limit, return an error rather than looping forever. You can also track cost per turn and abort when a budget threshold is crossed.

Break 5: Timeout on the Model Call

If you're using a local model on an underpowered machine (or a cloud API during peak hours), the HTTP request can hang for thirty seconds or more. Your script hangs, your user stares at a blank screen, and eventually something crashes.

Fix: Set a timeout and handle it gracefully. The `httpx` library makes this easy:

```
import httpx

try:
```

```
        response = httpx.post(OLLAMA_URL, json=payload, timeout=30.0)
        response.raise_for_status()
except httpx.TimeoutException:
    return "The model took too long to respond. Try again or use a
    smaller model."
except httpx.HTTPStatusError as e:
    return f"The model server returned an error:
    {e.response.status_code}"
```

The key is: never let an unhandled exception kill the agent. Users forgive slow responses. They don't forgive crashes.

Try It Now: Three Variations

Now that you've built the basic agent, stretch it. These are not optional extras. They're where the learning actually happens.

Variation 1: Add a Second Tool

Give your agent both `get_time` and `calculate`. Ask it a question that requires both, like: "What time will it be in Tokyo three hours from now?" The agent should call `get_time`, then `calculate` on the result.

This tests whether the model can chain tool calls across multiple turns. If it struggles, the problem is almost certainly in your tool descriptions. Make them more explicit about what inputs they expect and what outputs they provide.

Variation 2: Request Clarification

Modify the prompt so that if the user's request is ambiguous, the agent asks a follow-up question instead of guessing. For example, if the user asks "What's the time there?" the agent should respond: "Where is 'there'? Please specify a city."

This teaches you about prompt boundary conditions: you can't just tell the agent to "be helpful." You have to specify exactly what helpful means.

Variation 3: Swap Models and Compare

Run the same conversation against three different models: your local Ollama model, GPT-4o mini, and GPT-4o. Time each run. Check the cost. Compare how often each model emits valid JSON, how creatively it interprets your tool descriptions, and how confidently it answers questions outside its tool set.

One of the first truths you'll discover about agents is that smaller models are surprisingly capable for simple tasks but fail in subtle ways on complex ones. That knowledge — knowing when to scale up and when to stay cheap — is worth more than any single framework.

What You Built

In this chapter, you built an agent from fundamentals:

- A four-step loop that observes, reasons, acts, and observes again.
- A tool registry with docstring-driven descriptions.
- A JSON parser that intercepts tool calls and executes real Python functions.
- Error handling for malformed output, missing tools, bad arguments, infinite loops, and timeouts.
- A cost-aware version that swaps from free local models to metered cloud models.

You also learned why the raw loop matters even when frameworks exist, why tool descriptions are contracts, and how to fail safely when the model does something unexpected.

That's the foundation. Every project after this adds new tools, new reasoning patterns, and new failure modes to this same loop. In the next chapter, we'll replace our mock `get_time` with a real web search, and you'll see why agents hallucinate — and how to stop them.

Cost for this chapter: Approximately \$0.00 if you use Ollama throughout. Approximately \$0.0015 if you run all three variation experiments with GPT-4o mini.

Chapter 2: Project 2 — Research Agent: Web Search Summarizer

Every AI book tells you not to trust the AI. This chapter shows you how to make it trustworthy.

Last year, a research assistant I was training asked an AI agent to summarize the latest developments in quantum computing. The agent returned a beautiful four-paragraph summary with citations. Every citation had a URL. Every URL looked real. None of them existed. The agent had hallucinated twelve sources because the prompt implied that sources were expected, and the agent delivered what was expected rather than what was true.

This is the single most important challenge in agent design: how do you get an agent to find real information, present it honestly, and admit when it can't find anything?

In this chapter, we'll build a research agent that searches the live web and produces summaries with real, clickable citations. The agent will know when its search returns empty and will say so explicitly instead of making something up. It will cite every factual claim with a URL. It will flag when sources disagree. And it will cost you less per query than a postage stamp.

The underlying pattern is called ReAct (Reasoning + Action), and it's the standard architecture for modern research agents. We'll implement it in pure Python first, then show how frameworks make it cleaner. By the end, you'll understand why agents hallucinate and you'll have practical tooling to prevent it.

Why Agents Hallucinate and What You Can Do About It

A language model hallucinates because it's a prediction machine, not a fact machine. When you ask it something it doesn't know, it doesn't say "I don't know." It predicts the most plausible sequence of words that would answer your question. If your question is about last week's news, and the model's training data ended six months ago, its most plausible sequence is a confident fabrication.

There are three lines of defense against hallucination in an agent:

1. **Tool augmentation:** Give the model access to real search so it doesn't have to rely on memory.
2. **Citation enforcement:** Require every factual claim to include a source URL. If there's no source, the claim doesn't stand.
3. **Transparency in failure:** Train the output format so the model can say "I found conflicting sources" or "I couldn't find anything reliable" instead of guessing.

In this project, we'll build all three.

The ReAct Pattern

ReAct is short for Reasoning + Acting. It was introduced by researchers at Princeton in 2022 and has become the dominant paradigm for agents that need to interact with external tools.

The key insight: instead of asking the model to do its reasoning internally and then produce a final answer, you ask it to write its reasoning *out loud* in the conversation, interleaved with tool calls. Each reasoning step explains what it needs to know next. Each action step fetches that knowledge. This creates a paper trail you can audit, and it forces the model to justify its tool calls.

Here's a manual trace:

User: "What are the top three Python web frameworks in 2026, and how do they compare?"

Observation: The model sees this question and the available tools.

Reasoning: *I need to know what the current top Python web frameworks are. My knowledge is from training data, not live information. I should search the web for "top Python web frameworks 2026" to get recent rankings and comparisons.*

Action: `search_web(query="top Python web frameworks 2026 comparison")`

Tool returns: A list of search results with titles, URLs, and snippets.

Observation: The model now sees the user's question plus ten search snippets.

Reasoning: *The search results mention Django, FastAPI, and Flask consistently. I should read the detailed comparison article from the first result to get specifics about performance and use cases.*

Action: `read_page(url="https://example.com/python-frameworks-2026")`

Tool returns: The full text of the article.

Observation: The model sees the article content.

Reasoning: *I now have sufficient information to answer. The top three frameworks are Django, FastAPI, and Flask. Django is the full-featured monolith, FastAPI is the modern async-first API framework, and Flask is the microframework. I should cite the article I read and note the consensus.*

Final Answer: Structured summary with [source: URL] citations after every claim.

Notice what's different from Chapter 1: instead of one tool call, we might make two or three. Instead of a simple question, we have a multi-step investigation. And the reasoning is visible — if the model makes a bad decision, you can see why.

Building the Research Agent in Pure Python

Here's the agent. Start by adding a search tool. We'll use DuckDuckGo via the `duckduckgo-search` Python package, which requires no API key:

```
pip install duckduckgo-search
```

Now the code:

```
import json
import httpx
from duckduckgo_search import DDGS

OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL = "llama3.1:8b"

def search_web(query: str, max_results: int = 10) -> str:
    """Search the web using DuckDuckGo and return results as a
    JSON string."""
    with DDGS() as ddgs:
        results = list(ddgs.text(query, max_results=max_results))
    return json.dumps(results, indent=2)
```

```

def read_page(url: str) -> str:
    """Fetch the visible text content of a web page."""
    try:
        response = httpx.get(url, timeout=15,
follow_redirects=True)
        response.raise_for_status()
        # Very naive text extraction – in production, use
trafilatura or BeautifulSoup
        text = response.text
        # Strip HTML tags (quick and dirty)
        import re
        text = re.sub(r'<[^>]+>', ' ', text)
        text = re.sub(r'\s+', ' ', text).strip()
        return text[:5000] # Limit to avoid token explosion
    except Exception as e:
        return f"Error fetching page: {e}"

def research_agent(question: str, max_turns: int = 8) -> str:
    tools = {"search_web": search_web, "read_page": read_page}

    system_prompt = (
        "You are a research agent. Your job is to answer questions
using web search.\n"
        "Follow these rules:\n"
        "1. Use search_web to find information.\n"
        "2. If a search result looks promising, use read_page to
get the full text.\n"
        "3. Cite every factual claim with [source: URL].\n"
        "4. If sources conflict, note the disagreement.\n"
        "5. If you cannot find reliable sources after searching,
say so explicitly.\n"
        "6. Do not fabricate sources or URLs.\n\n"
        "When you need a tool, respond ONLY with JSON:\n"
        '{"tool": "tool_name", "arguments": {"arg": "value"}}\n'
        "When you have a final answer, respond normally with
citations."
    )

    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": f"Question: {question}"},
    ]

    for turn in range(max_turns):
        prompt_text = format_messages_r(messages)
        response = httpx.post(
            OLLAMA_URL,
            json={"model": MODEL, "prompt": prompt_text, "stream":
False},
            timeout=60,
        ).json()

```



```

reply = response["response"].strip()
print(f"[Turn {turn + 1}] {reply[:200]}...")

try:
    parsed = json.loads(reply)
    if "tool" in parsed:
        tool_name = parsed["tool"]
        tool_args = parsed.get("arguments", {})

        # Guardrail: no fabrication
        if tool_name == "search_web" and "query" in
tool_args:
            result = tools[tool_name](tool_args["query"])
        elif tool_name == "read_page" and "url" in
tool_args:
            result = tools[tool_name](tool_args["url"])
        else:
            result = f"Error: invalid tool call or
arguments."

        messages.append({"role": "assistant", "content":
reply}))
        messages.append({"role": "tool", "content":
f"Result:\n{result[:2000]}"})
        continue
    except json.JSONDecodeError:
        # Final answer
        return reply

    return "Max turns reached without a final answer."

def format_messages_r(messages: list) -> str:
    lines = []
    for m in messages:
        if m["role"] == "system":
            lines.append(f"System: {m['content']}")
        elif m["role"] == "user":
            lines.append(f"User: {m['content']}")
        elif m["role"] == "tool":
            lines.append(f"Tool result:\n{m['content']}")
        elif m["role"] == "assistant":
            lines.append(f"Assistant (tool call): {m['content']}")
    return "\n\n".join(lines)

if __name__ == "__main__":
    answer = research_agent("What are the top Python web
frameworks in 2026?")
    print("\n=== FINAL ANSWER ===\n")
    print(answer)

```

Run it:

python research_agent.py

The output depends on the live web, but the ideal flow looks like this:

[Turn 1] {"tool": "search_web", "arguments": {"query": "top Python web frameworks 2026"}}

[Turn 2] {"tool": "read_page", "arguments": {"url": "https://realpython.com/python-web-frameworks-2026"}}

[Turn 3] The top Python web frameworks in 2026, based on [...] are:

1. Django – [source: <https://realpython.com/python-web-frameworks-2026>]
2. FastAPI – [source: <https://realpython.com/python-web-frameworks-2026>]
3. Flask – [source: <https://realpython.com/python-web-frameworks-2026>]

Django remains the full-featured batteries-included framework [...]

Every claim is tied to a source. The agent searched, read, and synthesized — and it left a paper trail.

Citation Enforcement as Architecture

The single most important line in the system prompt is: **“Cite every factual claim with [source: URL].”** This is not a suggestion. It’s structural. When you require citations, the model knows that unsourced claims are invalid output. This sharply reduces fabrication, because the model must either find a source or omit the claim.

But there’s a subtle failure mode: the model might fabricate URLs that *look* plausible. It might cite <https://python.org/frameworks-2026> or <https://fastapi.tiangolo.com/blog/2026-ranking> even if those pages don’t exist.

To prevent this, add a second guardrail: **verify that cited URLs actually appeared in search results.** Store the URLs returned by `search_web` in a set, and during final answer processing, warn about any citation not in that set:

```
seen_urls = set()
```

```
# After each search_web call, populate seen_urls:
results = json.loads(search_web("..."))
for r in results:
    seen_urls.add(r.get("href", ""))
```

```
# After the final answer, check citations:
import re
citations = re.findall(r'\[source: (https?://[^\]]+)\]',
final_answer)
for citation in citations:
    if citation not in seen_urls:
        print(f"WARNING: Untrusted citation – {citation}")
```

This isn't foolproof (the model could still cite a search result that leads to a 404 page), but it catches the most common hallucination: inventing a URL entirely.

The “I Couldn't Find Anything” Guardrail

Sometimes search returns nothing useful. Maybe your question is too obscure. Maybe the web is noisy. Maybe DuckDuckGo's index hasn't caught the news yet. In these cases, the agent must say so rather than fabricating.

Add this explicit instruction to the prompt:

```
If search_web returns no relevant results after two attempts,
respond:
"I searched but could not find reliable sources for this specific
question.
Please try rephrasing or providing more context."
```

Then check for it in your code:

```
# After the second search returns empty or irrelevant results
last_tool_result = messages[-1]["content"]
if "no relevant" in last_tool_result.lower() or len(results) == 0:
    return "I searched but could not find reliable sources for
this specific question."
```

The point is: **silence is better than lies**. An agent that admits its limits is more trustworthy than one that confidently makes things up.

Break and Fix: Five Ways This Project Can Fail

Break 1: Search Returns No Results

Symptom: The agent calls `search_web("quantum error correction breakthrough April 2026")` and receives an empty results array. The agent has nothing to synthesize.

Root cause: The query is too specific, too recent, or uses jargon that doesn't match indexed content.

Fix: Implement query expansion. If the first search returns fewer than three results, ask the agent to try broader synonyms. Or, implement automatic fallback: remove date constraints, try alternate phrasing. Most importantly, teach the agent to recognize failure:

```
results = search_web(query)
if len(json.loads(results)) < 3:
    messages.append({"role": "tool", "content": "Search returned
few results. Consider rephrasing."})
```

Break 2: The Model Makes Up a URL

Symptom: The final answer cites `https://python.org/blog/frameworks-2026`, which doesn't exist.

Root cause: The model knows that Python's official site is `python.org` and extrapolates a plausible URL from it. It didn't learn this from search; it invented it from general knowledge.

Fix: Use the `seen_urls` guardrail described above. Also, add a post-processing step that fetches every cited URL and discards any that return 404:

```
def verify_citations(answer: str) -> str:
    citations = re.findall(r'\[source: (https?://[^\\]+)\]',
answer)
    for url in citations:
        try:
            r = httpx.head(url, timeout=5)
            if r.status_code == 404:
                answer = answer.replace(f"[source: {url}]",
"[source verification failed]")
        except Exception:
```

```
        pass
    return answer
```

This adds a few seconds per answer but catches fabricated URLs before they reach the user.

Break 3: Source Conflict Not Flagged

Symptom: Two articles disagree on a key fact, but the agent reports one version as definitive without mentioning the disagreement.

Root cause: The model has a bias toward coherence. It wants to resolve contradictions into a single narrative, which means it may silently ignore dissenting sources.

Fix: Explicitly instruct the model to flag disagreements:

```
If multiple sources conflict on a factual claim, present both
versions and
note the conflict: "Source A says X; Source B says Y. The
discrepancy may
be due to [differences in methodology, timing, etc]."
```

Then verify in post-processing: if multiple search results contain the same keywords but different conclusions, append a conflict warning to the synthesis prompt.

Break 4: Rate Limit Hit

Symptom: `duckduckgo_search` throws a `RateLimitException` after the third query in rapid succession.

Root cause: DuckDuckGo is a free service and throttles aggressive automated queries.

Fix: Add exponential backoff and a session-level query budget:

```
import time
import random

def safe_search(query: str, retries: int = 3) -> str:
    for attempt in range(retries):
        try:
            return search_web(query)
        except Exception as e:
            if "rate" in str(e).lower() and attempt < retries - 1:
                sleep_time = (2 ** attempt) + random.uniform(0, 1)
                time.sleep(sleep_time)
            else:
```

```
        raise
    return "[]"
```

Also, count total queries per session and stop if you exceed your budget (e.g., ten searches per answer).

Break 5: Page Too Long, Token Budget Exceeded

Symptom: You ask the agent to research a broad topic. It searches, finds a massive article, calls `read_page`, and the 10,000-word article blows past the LLM’s context window.

Root cause: The `read_page` tool returns everything. The model can’t read an entire book chapter alongside your question and its reasoning.

Fix: Implement chunking in `read_page`. Return only the most relevant sections, or use a two-pass approach: the model skims the page first (first 2,000 words), then requests specific sections if it needs depth.

```
def read_page(url: str, section: str = None) -> str:
    text = fetch_url(url) # your existing fetch logic
    if section:
        # Find paragraph containing section keyword and return
        # ±500 chars
        idx = text.lower().find(section.lower())
        if idx >= 0:
            return text[max(0, idx - 500):idx + 500]
    return text[:2000] # Default: first 2000 chars only
```

This requires the model to request sections by name — which it can do in subsequent tool calls.

Try It Now: Three Variations

Variation 1: Add a “`read_page`” Tool

The basic agent reads search snippets but never the full page. Upgrade it to read the top result in full before answering. Compare answer quality: does the extra context produce more accurate summaries, or does it introduce noise from irrelevant sections?

Variation 2: Budget-Aware Research

Add a token and cost budget. Each search costs tokens. Each page read costs tokens. The LLM reasoning between steps costs tokens.

Implement a `running_cost` tracker that accumulates cents per API call. Instruct the agent: “You have a budget of \$0.10. Spend it wisely.” If the budget runs out mid-research, the agent must summarize what it has found so far and flag that the research is incomplete.

Variation 3: Structured JSON Report

Require the output to conform to a Pydantic schema:

```
from pydantic import BaseModel

class ResearchReport(BaseModel):
    question: str
    sources: list[str]
    key_findings: list[str]
    confidence_score: int # 1-10
    conflicts: str | None
```

Add JSON mode to the LLM request (or prompt it to output schema-compliant JSON). This makes the agent’s output machine-readable for downstream processing — dashboards, databases, compliance reports.

The OpenClaw Way

OpenClaw’s Agent class supports multi-turn reasoning natively. You configure the agent with tools, tell it to use `reasoning="react"`, and it manages the interleaved thought and action for you:

```
from openclaw import Agent, tool

@tool
def search_web(query: str) -> str:
    """Search the web using DuckDuckGo and return results as
    JSON."""
    # Implementation as above

@tool
def read_page(url: str) -> str:
    """Fetch the visible text of a web page, up to 2000
    characters."""
    # Implementation as above

agent = Agent(
```

```
tools=[search_web, read_page],
reasoning="react",
max_steps=8,
)

response = agent.run(
    "What are the top Python web frameworks in 2026 and how do
they compare?"
)
print(response)
```

OpenClaw automatically interleaves reasoning and action steps, tracks token usage, and stops when max steps are reached. It also enforces citation format through a system prompt template that you can customize.

The Hermes Agent Way

Hermes handles research via built-in search and browser tools. You don't register custom tools — you describe the task in natural language and Hermes orchestrates the investigation:

```
hermes
```

```
> Research the top 3 Python web frameworks in 2026. Compare their
performance, community size, and typical use cases.
Write the results to report.md with proper citations.
```

Behind the scenes, Hermes uses its search skill to find articles, its browser tool to read them, and its file tool to write the report. It manages the multi-turn investigation, citation formatting, and budget tracking for you.

The trade-off is less precision about tool definitions. The benefit is zero setup for a common task. For research specifically, Hermes's search integration is robust enough that you may not need to write any tool code at all.

What You Built

In this chapter, you built a research agent with:

- Web search integration via DuckDuckGo (no API key required).
- Multi-turn ReAct reasoning: search, read, synthesize, repeat.

- Citation enforcement: every factual claim tied to a real URL.
- URL verification: citations checked against the URLs the agent actually saw.
- Source conflict detection: explicit instructions to flag disagreements.
- Failure transparency: the agent says “I couldn’t find anything” instead of guessing.
- Query expansion and rate-limit backoff for robustness.
- Token budgeting: tracking cost per research run.

You also learned why hallucination is a structural problem, not a moral failing, and how the ReAct pattern changes the problem from “make the model know more” to “make the model know when it doesn’t know.”

In the next chapter, we’ll give the agent access to the local filesystem. The goal is not to build a research assistant but a janitor — one that cleans up your Downloads folder without ever deleting anything by accident.

Cost for this chapter: Approximately \$0.00 with Ollama.
Approximately \$0.003–\$0.008 per query with GPT-4o mini (depending on search result size and page length).

Chapter 3: Project 3 — File Organizer Agent

Your Downloads folder is a graveyard. Let’s teach the agent to clean house.

Three years ago, I wrote a twenty-line bash script to “clean up” my Downloads folder. It sorted files by extension into tidy little directories: PDFs here, images there, archives in the corner. I ran it, watched the terminal scroll, and felt a brief, smug satisfaction. Then I opened my Documents folder and realized the script had moved my entire Downloads directory *into* itself, creating a Russian doll of nested folders twenty levels deep. Worse, I had agreed to a video call ten minutes later and couldn’t find my notes. I learned two lessons that

day. First, never trust a script you haven't broken first. Second, filesystem automation is not like chatbot automation: when a chatbot hallucinates, you get a weird answer. When a file agent hallucinates, you get a missing tax return.

That's why this chapter is built around guardrails. We're going to make an agent that reads your Downloads folder, classifies each file with a language model, and moves it into labeled folders. But unlike my catastrophic bash script, this agent will preview every move before it makes one, ask for explicit approval, and write a JSON undo log so you can put everything back exactly where it was. It will never delete. It will never overwrite without warning. It will handle permission errors, duplicate filenames, and symlink traps without crashing. By the end, you'll have a tool you can actually run on your own computer without fear.

The stakes are low in dollars — local models make this essentially free — but high in trust. An agent that touches files has to earn the right to act. Let's build it carefully.

Why Filesystem Agents Need Guardrails

A file organizer is the simplest useful agent you can build, and that simplicity is deceptive. Every other agent in this book calls APIs, writes to databases, or generates text. Those actions are abstracted away from your immediate life. A file organizer, by contrast, is pawing through your actual stuff. It sees your résumé, your medical forms, your half-finished novel. If it misclassifies a file, you might spend an hour hunting for it. If it overwrites something, you might lose it permanently.

Guardrails are not optional here. They are the architecture. Our agent will follow three hard rules:

1. **Never delete.** Move only. The source directory should look empty after a run, but nothing is destroyed.
2. **Preview first.** The agent must print a complete manifest of planned moves and wait for approval.
3. **Log everything.** Every move is recorded in a JSON undo log with timestamps, source paths, and destination paths.

These rules are enforced in code, not in comments. The LLM does not decide whether to ask for approval. The approval gate is a literal `input()` prompt in the `main()` function that the model cannot bypass. This is an important design pattern for local-tool agents: the dangerous action and the safety check should live in the same Python layer, not be delegated to the model's "reasoning."

Building a File Scanner in Pure Python

Before we classify or move anything, we need to see what's in the folder. Python's `pathlib` module is the modern way to do this. It replaces brittle `os.path` strings with `Path` objects that carry their own methods.

Our scanner has a narrow job: list every non-hidden, regular file directly inside a directory. We will skip subdirectories and symlinks entirely. Skipping symlinks is not laziness; it's safety. A symlink can point anywhere — including back to a parent directory — and following them turns a simple scan into a maze.

Here's the scanner:

```
from pathlib import Path
from typing import List

def scan_directory(directory: Path) -> List[Path]:
    """Return a list of non-hidden regular files directly inside
    *directory*.
```

Skips symlinks to avoid following unpredictable or circular paths.

Raises no exceptions on permission errors; instead, it prints a warning and returns whatever files it could access.

```
    """
    files: List[Path] = []
    try:
        for entry in directory.iterdir():
            if entry.is_symlink():
                continue
            if entry.is_file() and not entry.name.startswith("."):
                files.append(entry)
    except PermissionError as exc:
        print(f"Permission denied scanning {directory}: {exc}")
    except OSError as exc:
```

```
        print(f"OS error scanning {directory}: {exc}")
    return files
```

Notice the error handling. `PermissionError` happens when your script lacks read access to the folder. `OSError` catches broader filesystem problems, like a network drive disconnecting mid-scan. Neither error crashes the agent; they just reduce the file list. This is exactly what you want when running against a messy folder like `~/Downloads`, which might contain folders owned by other users or mounted volumes.

LLM-Based Classification

Once we have a list of files, we need to decide where each one belongs. We'll use five categories: **Documents**, **Images**, **Archives**, **Code**, and **Uncertain**. The LLM gets only the filename and must respond with exactly one of those words.

Why use an LLM at all when we could just map extensions? Because real filenames are messy. A file called `invoice_final_ACTUALLY_FINAL.pdf` is obviously a document, but a file called `backup` with no extension is ambiguous. A small local model can infer from context that `backup` in a `Downloads` folder is probably an archive or a disk image. More importantly, this project is about teaching agents to use tools, and classification is a perfect low-risk reasoning task.

That said, we are not going to let a missing local model stop the script from running. If Ollama is offline or the HTTP call fails, the function falls back to a simple extension map. Resilience is more important than elegance.

```
import httpx
```

```
OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL = "llama3.1:8b"
```

```
EXTENSION_MAP = {
    ".pdf": "Documents",
    ".doc": "Documents",
    ".docx": "Documents",
    ".txt": "Documents",
    ".md": "Documents",
    ".png": "Images",
    ".jpg": "Images",
    ".jpeg": "Images",
    ".gif": "Images",
    ".zip": "Archives",
```

```

        ".tar": "Archives",
        ".gz": "Archives",
        ".bz2": "Archives",
        ".7z": "Archives",
        ".py": "Code",
        ".js": "Code",
        ".html": "Code",
        ".css": "Code",
        ".rs": "Code",
        ".go": "Code",
        ".java": "Code",
    }

```

```

def classify_file(file_path: Path) -> str:
    """Classify *file_path* into Documents, Images, Archives,
    Code, or Uncertain.

```

```

        Attempts to ask a local LLM via Ollama. If the model is
        unreachable,
        the response is malformed, or the answer is not in the allowed
        set,
        falls back to EXTENSION_MAP and ultimately returns
        'Uncertain'.
    """

```

```

        allowed = {"Documents", "Images", "Archives", "Code",
        "Uncertain"}
        fallback = EXTENSION_MAP.get(file_path.suffix.lower(),
        "Uncertain")

```

```

    prompt = (
        "Classify the user file into exactly one category: "
        "Documents, Images, Archives, Code, or Uncertain.\n"
        f"Filename: {file_path.name}\n"
        "Respond with ONLY the category name, nothing else."
    )

```

```

    try:
        response = httpx.post(
            OLLAMA_URL,
            json={"model": MODEL, "prompt": prompt, "stream":
False},
            timeout=15.0,
        )
        response.raise_for_status()
        result = response.json()["response"].strip()
        if result in allowed:
            return result
    except Exception:
        pass # Any failure routes to fallback

    return fallback

```

The timeout is fifteen seconds. On a modern laptop, llama3.1:8b answers this simple prompt in under two seconds, but if the model is cold or the machine is busy, we don't want to hang forever. The except Exception block is deliberately broad because there are many benign reasons a local model might be unavailable: Ollama isn't running, the model isn't pulled, the port is wrong. In all of those cases, the script falls back to extension mapping and keeps working.

Safe Move Operations with pathlib and shutil

Moving a file sounds trivial: call rename() and you're done. But Path.rename() fails if the source and destination are on different filesystems, which is common when ~/Downloads is on your main disk and ~/Organized is on an external drive. shutil.move handles that case gracefully by copying and then deleting the source. That's the one we'll use.

We also need to handle duplicate filenames. If you download report.pdf twice, the second file cannot land on top of the first. Our solution is to append a timestamp to the stem before moving:

```
import shutil
from datetime import datetime

def move_file(source: Path, destination_dir: Path, undo_log:
List[dict]) -> None:
    """Safely move *source* into *destination_dir* and record the
    action.

    Creates *destination_dir* if it does not exist. If a file with
    the
    same name already exists, appends a timestamp to avoid
    overwriting.
    Appends a dict to *undo_log* describing the completed move.

    Raises:
        FileNotFoundError: If *source* does not exist at call
    time.
        PermissionError: If the process lacks write access to the
    destination.
    """
    if not source.exists():
        raise FileNotFoundError(f"Source file missing: {source}")

    destination_dir.mkdir(parents=True, exist_ok=True)
```

```

destination = destination_dir / source.name

if destination.exists():
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    destination = destination_dir / f"{source.stem}
_{timestamp}{source.suffix}"

shutil.move(str(source), str(destination))
undo_log.append({
    "timestamp": datetime.now().isoformat(),
    "source": str(source),
    "destination": str(destination),
})

```

The undo log entry is appended immediately after the move succeeds. If we waited until the end of the batch to write the log, a crash in the middle would leave us with moved files and no record of where they went. Writing incrementally means the worst-case data loss is one entry, not the entire session.

Hard Rules: Preview, Approval, and Undo Log

Now we wire the pieces together. The script follows a strict lifecycle:

1. **Scan** the target directory.
2. **Classify** every file.
3. **Preview** the complete move list.
4. **Ask** the user for explicit approval.
5. **Execute** moves one by one.
6. **Save** the undo log to disk.

The preview step is the most important safety feature. Before any file is touched, the user sees exactly what will happen. The approval gate is intentionally primitive — a text prompt — because we do not want the model participating in its own authorization. The model suggests; the human approves.

Here is the complete script. Save it as `file_organizer.py` and run it against your Downloads folder:

```

import json
import shutil

```

```

from datetime import datetime
from pathlib import Path
from typing import List

import httpx

OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL = "llama3.1:8b"

EXTENSION_MAP = {
    ".pdf": "Documents",
    ".doc": "Documents",
    ".docx": "Documents",
    ".txt": "Documents",
    ".md": "Documents",
    ".png": "Images",
    ".jpg": "Images",
    ".jpeg": "Images",
    ".gif": "Images",
    ".zip": "Archives",
    ".tar": "Archives",
    ".gz": "Archives",
    ".bz2": "Archives",
    ".7z": "Archives",
    ".py": "Code",
    ".js": "Code",
    ".html": "Code",
    ".css": "Code",
    ".rs": "Code",
    ".go": "Code",
    ".java": "Code",
}

def scan_directory(directory: Path) -> List[Path]:
    """Return a list of non-hidden regular files directly inside
    *directory*.

    Skips symlinks to avoid following unpredictable or circular
    paths.
    Handles permission and OS errors gracefully.
    """
    files: List[Path] = []
    try:
        for entry in directory.iterdir():
            if entry.is_symlink():
                continue
            if entry.is_file() and not entry.name.startswith("."):
                files.append(entry)
    except PermissionError as exc:
        print(f"Permission denied scanning {directory}: {exc}")
    except OSError as exc:

```



```

        print(f"OS error scanning {directory}: {exc}")
    return files

def classify_file(file_path: Path) -> str:
    """Classify *file_path* into Documents, Images, Archives,
    Code, or Uncertain.

    Attempts a local LLM via Ollama; falls back to EXTENSION_MAP
    on any failure.
    """
    allowed = {"Documents", "Images", "Archives", "Code",
"Uncertain"}
    fallback = EXTENSION_MAP.get(file_path.suffix.lower(),
"Uncertain")

    prompt = (
        "Classify the user file into exactly one category: "
        "Documents, Images, Archives, Code, or Uncertain.\n"
        f"Filename: {file_path.name}\n"
        "Respond with ONLY the category name, nothing else."
    )

    try:
        response = httpx.post(
            OLLAMA_URL,
            json={"model": MODEL, "prompt": prompt, "stream":
False},
            timeout=15.0,
        )
        response.raise_for_status()
        result = response.json()["response"].strip()
        if result in allowed:
            return result
    except Exception:
        pass

    return fallback

def move_file(source: Path, destination_dir: Path, undo_log:
List[dict]) -> None:
    """Safely move *source* into *destination_dir* and record the
    action.

    Creates the destination directory if needed. Avoids
    overwriting
    existing files by appending a timestamp to the filename.
    Appends
    an entry to *undo_log* immediately upon success.
    """
    if not source.exists():

```

```

        raise FileNotFoundError(f"Source file missing: {source}")

    destination_dir.mkdir(parents=True, exist_ok=True)
    destination = destination_dir / source.name

    if destination.exists():
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        destination = destination_dir / f"{source.stem}
_{timestamp}{source.suffix}"

    shutil.move(str(source), str(destination))
    undo_log.append({
        "timestamp": datetime.now().isoformat(),
        "source": str(source),
        "destination": str(destination),
    })

def save_undo_log(undo_log: List[dict], log_path: Path) -> None:
    """Write *undo_log* to *log_path* as formatted JSON."""
    log_path.parent.mkdir(parents=True, exist_ok=True)
    with open(log_path, "w", encoding="utf-8") as fh:
        json.dump(undo_log, fh, indent=2)

def load_undo_log(log_path: Path) -> List[dict]:
    """Load a JSON undo log from *log_path*, or return an empty
list."""
    if not log_path.exists():
        return []
    with open(log_path, "r", encoding="utf-8") as fh:
        return json.load(fh)

def main() -> None:
    """Run the file organizer against ~/Downloads in preview-then-
approve mode."""
    downloads = Path.home() / "Downloads"
    organized_base = Path.home() / "Organized"
    undo_log_path = organized_base / "undo_log.json"

    print("Scanning Downloads...")
    files = scan_directory(downloads)
    if not files:
        print("No eligible files found. Nothing to do.")
        return

    undo_log = load_undo_log(undo_log_path)

    # Build plan
    plan = []
    for file_path in files:

```

```

        category = classify_file(file_path)
        destination_dir = organized_base / category
        plan.append({
            "source": file_path,
            "destination_dir": destination_dir,
            "category": category,
        })

    # Preview
    print(f"\nPlanned moves ({len(plan)} files):")
    for item in plan:
        print(f"  {item['source'].name} -> {item['category']}/")

    mode = input("\nExecute these moves? Type 'yes' to proceed:
").strip().lower()
    if mode != "yes":
        print("Aborted. No files were moved.")
        return

    # Execute
    for item in plan:
        try:
            move_file(item["source"], item["destination_dir"],
undo_log)
            print(f"  OK: {item['source'].name}")
        except Exception as exc:
            print(f"  FAIL: {item['source'].name} ({exc})")

    save_undo_log(undo_log, undo_log_path)
    print(f"\nDone. Undo log saved to {undo_log_path}")
    print(f"Total moved: {len(undo_log)} entries")

if __name__ == "__main__":
    main()

```

Run it:

```
python file_organizer.py
```

If Ollama is running, you'll see the model classify each filename. If not, you'll see the extension-based fallback at work. In either case, you get a full preview and a chance to abort. That preview step is what separates a script you trust from a script you fear.

Break and Fix: Five Ways This Project Can Fail

Filesystem automation is unforgiving. Every failure mode below is something I have personally triggered, usually at 11 PM while distracted. We'll walk through the symptom, the root cause, and the fix.

Break 1: Permission Denied

Symptom: The script prints FAIL: resume.pdf (Permission denied) and stops moving files.

Root cause: A file in ~/Downloads is open in another application, or the destination directory lives on a read-only volume, or the file has restrictive permissions. On macOS, for example, files downloaded by Safari sometimes carry the com.apple.quarantine extended attribute and resist modification until the user explicitly approves them.

Fix: Wrap the move in a try/except block that logs the failure and continues with the next file. We already do this in the main() loop above, but you can make the error more actionable by including the specific path and permission bit:

```
try:
    move_file(item["source"], item["destination_dir"], undo_log)
except PermissionError as exc:
    print(f"  SKIP: {item['source'].name} (permission denied: {exc})")
except Exception as exc:
    print(f"  FAIL: {item['source'].name} ({exc})")
```

The key principle is **continue on failure**. A single locked file should not block the organization of twenty others.

Break 2: Duplicate Filename Collision

Symptom: shutil.Error: Destination path ... already exists.

Root cause: Two files in the source folder share the same name, or a previous run already placed a file with that name in the destination category folder. shutil.move does not silently overwrite; it raises an error.

Fix: We already handle this in `move_file` by checking `destination.exists()` and appending a timestamp. But if you want an even safer approach, you can hash the file content and append the first six characters of the hash to guarantee uniqueness:

```
import hashlib

if destination.exists():
    content_hash = hashlib.md5(source.read_bytes()).hexdigest()[:6]
    destination = destination_dir / f"{source.stem}_{content_hash}{source.suffix}"
```

The timestamp approach is usually sufficient for human-readable folders, but content hashing eliminates collisions entirely. Choose based on whether you prioritize readability or absolute uniqueness.

Break 3: Symlink Loop

Symptom: The script hangs, consumes 100% CPU, or eventually crashes with `OSError: [Errno 40] Too many levels of symbolic links`.

Root cause: A symlink in `~/Downloads` points to a parent directory or to itself. If your scanner follows symlinks — even accidentally, via `is_file()` on a symlink target — it can enter an infinite loop or recurse forever.

Fix: Skip symlinks explicitly in the scanner, as we do with `if entry.is_symlink(): continue`. This is non-negotiable for any filesystem agent that does not intend to traverse directory trees. If you ever *do* need to follow symlinks, resolve them first with `entry.resolve()` and maintain a visited set of inode IDs to detect cycles:

```
visited = set()
for entry in directory.rglob("*"):
    if entry.is_symlink():
        resolved = entry.resolve()
        inode = resolved.stat().st_ino
        if inode in visited:
            continue
        visited.add(inode)
```

For this project, skip them entirely. Life is too short to debug symlink mazes.

Break 4: Wrong Classification

Symptom: A Python script named notes.py ends up in Documents/, or a PDF full of code snippets ends up in Code/.

Root cause: The LLM is reasoning from filename alone. A file named README has no extension and could be anything. The model might also “hallucinate” a category that isn’t in the allowed set, which our code catches and falls back from, but that fallback still guesses based on extension.

Fix: Increase the information available to the classifier. If the file is a text file, read the first few hundred bytes and include them in the prompt. We’ll do this fully in the Try It Now section, but here’s a teaser of the logic:

```
def read_preview(file_path: Path, max_bytes: int = 256) -> str:
    """Return a short text preview of *file_path*, or an empty
    string."""
    try:
        return file_path.read_text(encoding="utf-8",
    errors="ignore")[:max_bytes]
    except Exception:
        return ""
```

Feeding that preview into the prompt lets the model distinguish between a .txt file that contains a grocery list and a .txt file that contains Python code.

Break 5: Undeclared Move Deletion Risk

Symptom: The user panics because files vanished from Downloads. Or the script crashes after moving half the files, and the undo log is empty because it was only written at the very end.

Root cause: shutil.move deletes the source file. That is literally what “move” means, but users may interpret “organize” as “copy into folders.” Additionally, if the process terminates between the last shutil.move and the final save_undo_log call, the log never gets written. You have moved files and no memory of where they went.

Fix: There are two layers to this.

First, be explicit. The preview should state clearly: “These files will be MOVED, not copied. The original Downloads folder will be empty for these items.” Add this line right before the approval prompt:

```
print("WARNING: This will MOVE files out of Downloads. Originals  
will be removed.")
```

Second, persist the undo log after every move, not at the end of the batch. Modify `move_file` to save immediately:

```
def move_file(source: Path, destination_dir: Path, undo_log:
List[dict], log_path: Path) -> None:
    """Move *source* to *destination_dir* and flush the undo log
    to disk."""
    destination_dir.mkdir(parents=True, exist_ok=True)
    destination = destination_dir / source.name
    if destination.exists():
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        destination = destination_dir / f"{source.stem}
_{timestamp}{source.suffix}"
    shutil.move(str(source), str(destination))
    undo_log.append({
        "timestamp": datetime.now().isoformat(),
        "source": str(source),
        "destination": str(destination),
    })
    save_undo_log(undo_log, log_path)
```

Now a crash after the third move still leaves the first two moves recorded. You can resume or revert with confidence.

Try It Now: Three Variations

Variation 1: Add Text Preview for Better Classification

Filenames lie. Content does not. For text-readable files, slurp the first 200 bytes and feed them to the model. Modify `classify_file` to build a richer prompt:

```
def classify_file(file_path: Path) -> str:
    allowed = {"Documents", "Images", "Archives", "Code",
"Uncertain"}
    fallback = EXTENSION_MAP.get(file_path.suffix.lower(),
"Uncertain")

    preview = ""
    try:
        preview = file_path.read_text(encoding="utf-8",
errors="ignore")[:200]
    except Exception:
        preview = "[binary or unreadable]"

    prompt = (
```

```

        "Classify the file into exactly one category: "
        "Documents, Images, Archives, Code, or Uncertain.\n"
        f"Filename: {file_path.name}\n"
        f"Preview: {preview}\n"
        "Respond with ONLY the category name."
    )
    # ... rest of LLM call identical ...

```

This dramatically improves accuracy for files without clear extensions. The cost is slightly higher token usage per file — usually a fraction of a cent on local models and still negligible on mini cloud models.

Variation 2: Rename Files to Date Schema

Instead of preserving the original filename, rename each file to include its modification date. This turns `report.pdf` into `2026-04-20_report.pdf`, making chronological sorting obvious inside each category folder.

```

def dated_destination(source: Path, destination_dir: Path) -> Path:
    """Return a destination Path with the file's mdate
    prefixed."""
    destination_dir.mkdir(parents=True, exist_ok=True)
    mtime = datetime.fromtimestamp(source.stat().st_mtime)
    date_prefix = mtime.strftime("%Y-%m-%d")
    new_name = f"{date_prefix}_{source.name}"
    return destination_dir / new_name

```

Call this inside `move_file` instead of using `source.name` directly. Be careful: if a file has been moved before, the `mtime` may reflect the move rather than the original download date. For true download dates, parse browser metadata or use filesystem birth time (`st_birthtime` on some platforms), which is platform-specific. The modification time is a reliable, portable approximation.

Variation 3: Create an Undo Log So You Can Revert

We already write an undo log. Now let's consume it. Write a separate script, `revert.py`, that reads the JSON log and moves everything back:

```

import json
import shutil
from pathlib import Path

def revert_moves(log_path: Path) -> None:
    """Read an undo log and move every file back to its original
    location."""

```



```

if not log_path.exists():
    print("No undo log found. Nothing to revert.")
    return

with open(log_path, "r", encoding="utf-8") as fh:
    log = json.load(fh)

if not log:
    print("Undo log is empty.")
    return

confirm = input(f"Revert {len(log)} moves? Type 'yes':")
confirm = confirm.strip().lower()
if confirm != "yes":
    print("Revert cancelled.")
    return

for entry in log:
    src = Path(entry["destination"])
    dst = Path(entry["source"])
    try:
        dst.parent.mkdir(parents=True, exist_ok=True)
        shutil.move(str(src), str(dst))
        print(f"Reverted: {src.name}")
    except Exception as exc:
        print(f"Failed to revert {src.name}: {exc}")

# Clear the log after successful revert
with open(log_path, "w", encoding="utf-8") as fh:
    json.dump([], fh)
print("Undo log cleared.")

if __name__ == "__main__":
    revert_moves(Path.home() / "Organized" / "undo_log.json")

```

This is the safety net. Run `revert.py` and your Downloads folder is restored exactly as it was. The key lesson here is that an agent is not just the code that acts; it is also the code that undoes the action. If you cannot revert, you do not have an agent. You have a gamble.

The OpenClaw Way

OpenClaw formalizes the pattern we built by hand. In OpenClaw, you decorate each filesystem operation with `@tool` and register them with an Agent. The scanner becomes a tool called `list_files`, the classifier becomes `classify_file`, and the mover becomes `move_file`. OpenClaw handles the conversation loop, the JSON parsing, and the error retries, but the critical safety layer — the preview and the approval gate —

remains yours to implement. OpenClaw provides a `human_in_the_loop=True` flag that pauses execution before any tool marked as destructive runs, which is exactly what we want for `move_file`. You still write the undo log yourself, because OpenClaw does not know your filesystem schema, but it will log every tool call automatically to its internal trace. For a beginner, the OpenClaw version cuts the boilerplate in half and gives you structured observability from day one.

The Hermes Agent Way

Hermes Agent approaches the problem from the other direction. Instead of writing a script, you start a conversation. You tell Hermes: “Organize my ~/Downloads folder into Documents, Images, Archives, and Code.” Hermes uses its built-in `read_file` and `move_file` tools to do exactly that, but before each move, it asks: “Move report.pdf to Documents/?” You say yes or no for each file, or you grant batch approval for the entire manifest. Hermes already maintains a session-level operation log, so you can ask it to “undo the last three moves” without writing a separate revert script. The trade-off is control: you don’t choose the duplicate-resolution strategy or the fallback classification map. The benefit is speed. If you just need a one-time cleanup and don’t want to write Python, Hermes is the right interface. When you need a scheduled cron job that runs every morning at 6 AM with your exact rules, the pure Python version — or the OpenClaw variant — wins.

What You Built

In this chapter, you built a file organizer that respects your data:

- A pathlib-based scanner that skips symlinks and handles permission errors gracefully.
- An LLM classifier that categorizes files into Documents, Images, Archives, Code, or Uncertain, with a deterministic fallback when the model is unavailable.
- A safe move engine using `shutil.move` that creates directories on demand and avoids overwriting files by appending timestamps.
- A preview-and-approval flow that prints the entire plan before executing a single filesystem mutation.

- A JSON undo log that records every move with source, destination, and timestamp, saved incrementally to survive crashes.
- Error handling for permission failures, duplicate names, symlink loops, misclassification, and undeclared deletion risk.
- A revert script that reads the undo log and restores files to their original locations.

That is a trustworthy agent. It makes mistakes impossible to hide and easy to undo.

Cost for this chapter: Approximately \$0.00 if you use Ollama throughout. Approximately \$0.01 if you run against GPT-4o mini for a typical Downloads folder of ~20 files. Approximately \$0.05 if you use GPT-4o and include content previews in every prompt.

Chapter 4: Project 4 — Spreadsheet Intelligence Agent

The spreadsheet has 4,000 rows and three questions. The agent answers all three in twelve seconds.

The first spreadsheet I ever asked an agent to read was a sales export from a CRM I no longer remember. It had twelve columns, mixed date formats, and a column called Revenue that contained both numbers and the word "pending". I asked for the quarterly total, and the script crashed because pandas couldn't add "pending" to a float. That was the good outcome. The bad outcome came later, when I used an LLM that didn't crash — it just confidently told me the revenue was \$847,000. The actual number was \$612,000. The model had hallucinated the difference because I hadn't forced it to look at the data.

That experience taught me the central rule of data agents: the model must touch the numbers before it talks about them. This chapter builds an agent that reads CSVs, computes summary statistics, answers natural-language questions about the data, and writes a markdown report — all with the rigor of a calculator and the flexibility of a language model.

We'll start with the Python standard library because dependencies are a tax you should only pay when they're worth it. Then we'll add pandas, because for real analysis, they are worth it. By the end, you'll have three tools: one to inspect a file, one to query it safely, and one to write the results back to disk.

Why Data Analysis Is an Agent Problem

Spreadsheets are everywhere. Sales data, experiment results, user signups, inventory logs — most organizations run on CSVs. Extracting insight from them requires repeatable labor: handling missing values, running aggregations, formatting results. Repeatable labor is what agents eat for breakfast.

An agent shines here because the same file can answer dozens of different questions, and the questioner doesn't always know the column names. "How did the East region perform last quarter?" requires the model to find the region column, find the date column, filter to the right range, aggregate the right metric, and return an answer in English. A static script can do this if you pre-program every question. An agent does it on demand.

But this power comes with a warning. When an agent talks about data, its words sound authoritative. If it says the average is 47.3, users believe it. That makes data agents uniquely dangerous compared to, say, a weather agent. Every tool we build in this chapter follows one principle: the model computes through the tool, never from memory.

Reading a CSV Without Pandas

Before we install anything, let's see how far the standard library takes us. Python's csv module is built in, requires zero dependencies, and is surprisingly capable for lightweight inspection.

Here's a tiny CSV we'll use throughout this chapter. Save it as sales.csv:

```
sample_csv = """product,region,units_sold,revenue,date
WidgetA,North,120,2400.00,2024-01-15
WidgetB,South,85,1700.00,2024-01-16
WidgetA,North,95,1900.00,2024-01-17
WidgetC,East,200,5000.00,2024-01-18
WidgetB,South,110,2200.00,2024-01-19
WidgetC,West,75,1500.00,2024-01-20
WidgetA,North,130,2600.00,2024-01-21"""
```

```
with open("sales.csv", "w", encoding="utf-8") as f:
    f.write(sample_csv)
```

Seven rows, five columns. Small enough to read in one breath, but big enough to test every tool we'll build.

Now let's inspect it with the standard library:

```
import csv

def peek_csv(path: str, max_rows: int = 3) -> None:
    """Print the header and first few rows of a CSV file.

    Uses only the Python standard library – no pandas required.
    """
    try:
        with open(path, newline="", encoding="utf-8") as f:
            reader = csv.reader(f)
            try:
                header = next(reader)
            except StopIteration:
                print("CSV is empty.")
                return
            print("Columns:", header)
            for i, row in enumerate(reader):
                if i >= max_rows:
                    break
                print(row)
    except FileNotFoundError:
        print(f"File not found: {path}")
    except UnicodeDecodeError as e:
        print(f"Encoding error: {e}")

if __name__ == "__main__":
    peek_csv("sales.csv")
```

Run it, and you'll see the header and the first three rows. The `csv.reader` gives us lists of strings. It doesn't infer types, it doesn't handle missing values, and it won't sort or group anything. But it opens the file, reads it line by line with proper escaping, and gives us raw data. For many agent tasks — like checking whether a file exists, counting rows, or extracting a header — that's enough.

There's a subtle bit of wisdom in `newline=""` when opening the file. The `csv` module documentation insists on this because Python's universal newlines can interfere with the parser's quoting logic. I ignored this once on a Windows machine and spent twenty minutes wondering why a field with an embedded newline had split into two rows. Standard library code looks simple until it isn't.

The real limitation hits when you want to ask a question like “what’s the total revenue for WidgetA?” With `csv.reader`, you’d have to loop through every row, check if the product column matches, convert the revenue string to a float, accumulate a sum, and handle blank entries manually. It works, but it’s tedious. For anything beyond counting rows, we need a real data analysis library.

Upgrading to Pandas for Real Analysis

Pandas is the de facto standard for tabular data in Python. Install it if you haven’t already:

```
pip install pandas
```

Now the same `sales.csv` becomes a `DataFrame`, and questions that took ten lines of loop become one-liners:

```
import pandas as pd

def inspect_csv(path: str) -> None:
    """Load a CSV into a DataFrame and print basic metadata."""
    try:
        df = pd.read_csv(path)
    except FileNotFoundError:
        print(f"File not found: {path}")
        return
    except pd.errors.EmptyDataError:
        print("CSV file is empty.")
        return
    except pd.errors.ParserError as e:
        print(f"Parse error: {e}")
        return

    print(f"Rows: {len(df)}")
    print(f"Columns: {list(df.columns)}")
    print("\nData types:")
    print(df.dtypes)
    print("\nFirst three rows:")
    print(df.head(3))

if __name__ == "__main__":
    inspect_csv("sales.csv")
```

The `df.dtypes` line is surprisingly important for agents. A column that looks like numbers might be loaded as strings if a single row contains text. When the agent later tries to compute a mean, it will fail unless it knows the type. By surfacing dtypes early, we give the model the context it needs to reason about which operations are valid.

Notice the error handling. `FileNotFoundError` and `EmptyDataError` are obvious, but `ParserError` catches malformed CSVs — mismatched quotes, stray delimiters, lines with the wrong number of columns. We'll see more of this in the Break and Fix section. For now, the rule is simple: never let a bad file crash your agent. Catch it, describe it, and feed the description back to the model as an observation.

Tool 1: `read_csv_summary`

Our first agent tool is an inspector. It reads a CSV and returns a structured summary that the language model can reason about. The summary includes column names, inferred data types, null counts, and the total row count.

```
import json
from pathlib import Path

def read_csv_summary(path: str) -> str:
    """Return a JSON summary of a CSV file.

    Summary includes columns, dtypes, null counts, and row count.

    Args:
        path: Path to the CSV file.

    Returns:
        A JSON string with file metadata, or an error message.
    """
    file_path = Path(path)
    if not file_path.exists():
        return json.dumps({"error": f"File not found: {path}"})

    try:
        df = pd.read_csv(path)
    except pd.errors.EmptyDataError:
        return json.dumps({"error": "CSV file is empty."})
    except pd.errors.ParserError as e:
        return json.dumps({"error": f"Parse error: {e}"})
    except Exception as e:
        return json.dumps({"error": f"Unexpected error reading CSV: {e}"})
```

```

summary = {
    "file": path,
    "row_count": len(df),
    "columns": list(df.columns),
    "dtypes": {col: str(dtype) for col, dtype in
df.dtypes.items()}},
    "null_counts": df.isnull().sum().to_dict(),
}
return json.dumps(summary, indent=2)

```

There's a design decision here: the function returns a JSON string rather than a Python dict. That's because the agent loop from Chapter 1 passes tool results back into the conversation history as text. Returning a formatted JSON string makes the observation readable for both the model and any human watching the logs. The `indent=2` isn't pedantry — I've watched agents struggle with minified JSON because the model lost track of which brace belonged to which key.

The `null_counts` field is particularly valuable. A column with 90% missing values shouldn't be averaged. A primary key column with any nulls is suspicious. By giving the agent this information upfront, we let it decide whether a question is even answerable before it attempts computation.

Tool 2: `query_csv`

This is the heart of the agent. The user asks a natural-language question, and the agent translates it into a structured tool call. The tool itself performs the computation using pandas, but it never executes arbitrary code.

```

def query_csv(path: str, operation: str, column: str, group_by:
str | None = None) -> str:
    """Perform a safe aggregation on a CSV file.

```

Supported operations: mean, sum, count, min, max, std.

Args:

```

    path: Path to the CSV file.
    operation: Aggregation function to apply.
    column: The column to aggregate.
    group_by: Optional column to group by before aggregating.

```

Returns:

```

    A string representation of the result, or an error
message.
    """
    # Safe mapping – never, ever use eval()

```



```

SAFE_OPS = {
    "mean": "mean",
    "sum": "sum",
    "count": "count",
    "min": "min",
    "max": "max",
    "std": "std",
}

op = SAFE_OPS.get(operation)
if op is None:
    return f"Error: unsupported operation '{operation}'. Use
one of: {list(SAFE_OPS.keys())}"

try:
    df = pd.read_csv(path)
except FileNotFoundError:
    return f"File not found: {path}"
except pd.errors.EmptyDataError:
    return "CSV file is empty."
except pd.errors.ParserError as e:
    return f"Parse error: {e}"

if column not in df.columns:
    return f"Error: column '{column}' not found. Available
columns: {list(df.columns)}"

if group_by and group_by not in df.columns:
    return f"Error: group_by column '{group_by}' not found.
Available columns: {list(df.columns)}"

try:
    if group_by:
        result = df.groupby(group_by)[column].agg(op)
    else:
        result = getattr(df[column], op)()
    return str(result)
except TypeError as e:
    return f"Error: cannot compute {operation} on column
'{column}' (wrong type?). Details: {e}"
except Exception as e:
    return f"Error during aggregation: {e}"

```

The critical line is `SAFE_OPS`. I want to be absolutely clear about why this exists. The natural instinct when building a query tool is to let the model pass a pandas expression as a string and evaluate it with `eval()`. Don't. `eval()` on user input — or model output, which is effectively user input — is a remote code execution vulnerability waiting to happen. A malicious or malfunctioning model could pass

`__import__('os').system('rm -rf /')` inside a seemingly innocent expression. Even without malice, `eval()` crashes on syntax errors and hides them in ways that make debugging impossible.

Instead, we whitelist exactly six operations. The model can request a mean, sum, count, minimum, maximum, or standard deviation. The tool validates the operation against this small dictionary before doing anything. The actual computation uses pandas methods directly: `df.groupby(...).agg("mean")` or `df[column].mean()`. There's no string evaluation, no dynamic code generation, and no surprises.

When the user asks, “What’s the average revenue by region?” the agent reasons through the available tools and emits a call like this:

```
{"tool": "query_csv", "arguments": {"path": "sales.csv",  
"operation": "mean", "column": "revenue", "group_by": "region"}}
```

Your program intercepts this, calls `query_csv` with those exact arguments, and appends the result back to the conversation. The model then composes the final answer in natural language. The computation is ground truth; the prose is interpretation.

Tool 3: write_summary

The final tool turns analysis into a shareable artifact. It reads a CSV, generates a markdown report, and writes it to disk.

```
def write_summary(csv_path: str, output_path: str = "report.md") -  
> str:  
    """Generate a markdown summary report from a CSV file.  
  
    Args:  
        csv_path: Path to the source CSV.  
        output_path: Path where the markdown report will be  
written.  
  
    Returns:  
        A success message or an error string.  
    """  
    summary_json = read_csv_summary(csv_path)  
    try:  
        summary = json.loads(summary_json)  
    except json.JSONDecodeError:  
        return f"Error: read_csv_summary returned invalid JSON:  
{summary_json[:200]}"  
  
    if "error" in summary:  
        return f"Error reading source CSV: {summary['error']}"
```

```

lines = [
    "# Spreadsheet Intelligence Report",
    "",
    f"***Source:** `{summary['file']}`",
    f"***Rows:** {summary['row_count']}",
    "",
    "## Columns",
    "",
]
for col in summary["columns"]:
    dtype = summary["dtypes"][col]
    nulls = summary["null_counts"][col]
    lines.append(f"- `{col}` ({dtype}) – missing values: {nulls}")

lines.extend([
    "",
    "## Notes",
    "",
    "This report was generated automatically by the Spreadsheet Intelligence Agent.",
])

report = "\n".join(lines)

try:
    with open(output_path, "w", encoding="utf-8") as f:
        f.write(report)
    return f"Report written to {output_path}"
except PermissionError:
    return f"Permission denied writing to {output_path}"
except IOError as e:
    return f"Error writing report: {e}"

```

This tool doesn't just dump raw data. It structures the report with headings, column metadata, and a provenance note that this was machine-generated. When you hand a report to a colleague, that provenance matters. It also prevents the model from hallucinating its own summary: `write_summary` calls `read_csv_summary`, which calls `pd.read_csv`, so every number in the report traces back to the file on disk.

Putting the three tools into the agent loop from Chapter 1 looks like this:

```

if __name__ == "__main__":
    tools = {
        "read_csv_summary": read_csv_summary,
        "query_csv": query_csv,
        "write_summary": write_summary,
    }

```

```

    }
    # agent_loop is the same ReAct loop from Chapter 1
    answer = agent_loop(
        "Summarize sales.csv and tell me the average revenue by
region.",
        tools,
    )
    print(answer)

```

The agent will likely call `read_csv_summary` first to understand the schema, then `query_csv` to get the numbers, then answer directly. If you ask it to save a report, it calls `write_summary`. The loop doesn't change from Chapter 1; only the tools do.

Break and Fix: Five Ways Data Breaks Your Agent

Data is messier than code. Files arrive with wrong delimiters, invisible characters, missing headers, and numbers masquerading as text. Every project in this book has a Break and Fix section, but for a data agent, this section is especially important because the failures aren't always in your code. Sometimes the file is just wrong, and your agent needs to survive anyway.

Break 1: Malformed CSV / Wrong Delimiter

A vendor sends you a file where every field is separated by a semicolon instead of a comma. Your standard `pd.read_csv("data.csv")` reads the entire first line as a single column name:

```
product;region;units_sold;revenue;date
```

The result is one column called `product;region;units_sold;revenue;date` and five rows of apparently empty data. The agent's subsequent query for revenue fails because no column by that name exists.

Root cause: CSV is not a single format. RFC 4180 defines comma separation, but Excel exports, European locale files, and older systems routinely use semicolons, tabs, or pipes.

Fix: Use Python's `csv.Sniffer` to detect the dialect before parsing:

```

import csv

def read_with_sniffer(path: str):
    with open(path, newline="", encoding="utf-8") as f:

```

```

sample = f.read(4096)
f.seek(0)
dialect = csv.Sniffer().sniff(sample)
reader = csv.reader(f, dialect)
header = next(reader)
print("Detected delimiter:", repr(dialect.delimiter))
print("Columns:", header)

```

In pandas, you can explicitly set the separator:

```
df = pd.read_csv(path, sep=";")
```

Or let pandas guess:

```
df = pd.read_csv(path, sep=None, engine="python")
```

The engine="python" option is slower but more forgiving on malformed files. I keep it in my back pocket for exactly this situation. Never assume a comma.

Break 2: Encoding Issues (UTF-8 vs Latin-1)

You open a file and immediately hit a wall:

```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9 in
position 43: invalid continuation byte
```

The file contains a name like “José” encoded in Latin-1, but your code opened it as UTF-8. The error halts the agent before it even sees the header.

Root cause: CSVs have no embedded encoding metadata. A file saved on a Windows machine in Western Europe might be Latin-1 (ISO-8859-1) or Windows-1252. A file from a Mac might be UTF-8. You can’t tell by looking at the extension.

Fix: Try encodings in order of likelihood, and fail gracefully:

```

def read_csv_robust(path: str):
    encodings = ["utf-8-sig", "utf-8", "latin-1", "windows-1252"]
    for enc in encodings:
        try:
            return pd.read_csv(path, encoding=enc)
        except UnicodeDecodeError:
            continue
    raise ValueError(f"Could not decode {path} with any known
encoding.")

```

utf-8-sig handles the Byte Order Mark (BOM) that some Windows editors prepend to UTF-8 files. latin-1 reads every byte into a character, so it never raises UnicodeDecodeError — but it might produce mojibake if the real encoding was something else. I put it second-to-last for that reason. As a last resort, use errors="replace" to force the read and flag the damaged cells for later inspection.

Break 3: Model Hallucinates a Statistic

You ask the agent, “What’s the total revenue?” and it replies: “The total revenue is \$14,700.” You check the file yourself. The actual sum is \$16,300. The agent didn’t crash. It didn’t call a tool. It just made up a number that sounded plausible.

Root cause: Large language models are trained on vast corpora that include spreadsheets, financial reports, and sales data. When asked a numeric question, the model may generate a statistically likely answer rather than a computed one. Without explicit constraints, it treats the question as a completion task, not a math task.

Fix: Force the model to ground its answer in a tool call. In the system prompt, add a strict rule:

When asked any question involving numbers, dates, counts, or averages,
you MUST call query_csv to compute the answer. Never guess.

Then validate that the model’s final answer includes a reference to the tool result. If it doesn’t, reject the answer and ask it to try again. Some frameworks call this “tool use enforcement” — the model is not allowed to answer directly until it has invoked the relevant tool. It costs an extra API call, but it costs far less than a wrong business decision.

Break 4: File Too Large for Pandas

You point the agent at a 4-gigabyte server log exported as CSV. Pandas tries to load it into memory and your machine grinds to a halt. Or worse, it raises MemoryError and kills the agent process.

Root cause: pd.read_csv loads the entire file into RAM. For files larger than your available memory, this is guaranteed to fail. Agents don’t know the file size until they try to open it.

Fix: Use chunked reading for large files, or fall back to the standard library which streams one row at a time:

```
def sum_large_csv(path: str, column: str) -> float:
    """Stream-process a large CSV in chunks."""
    total = 0.0
    chunk_iter = pd.read_csv(path, chunksize=10000)
    for chunk in chunk_iter:
        total += chunk[column].sum()
    return total
```

The chunksize parameter returns an iterator of DataFrames, each with at most 10,000 rows. You process one chunk, discard it, and move to the next. Memory usage stays flat regardless of file size. For even larger files — or machines with very little RAM — switch back to csv.reader and accumulate values in pure Python. It will be slow, but it will finish.

Break 5: Wrong Column Name Used in Query

The agent calls query_csv with column="Revenue" (capital R) when the actual column in the file is named revenue (lowercase). The tool returns:

```
Error: column 'Revenue' not found. Available columns: ['product',
'region', 'units_sold', 'revenue', 'date']
```

Root cause: CSV headers are user-generated text. They contain spaces, mixed case, special characters, and typos. The model doesn't know the exact header until it reads the file, and it may misremember a name from the summary.

Fix: Normalize column names when reading, so the model doesn't have to care about case:

```
df = pd.read_csv(path)
df.columns = [col.strip().lower().replace(" ", "_") for col in
df.columns]
```

This turns " Revenue " into "revenue" and "Units Sold" into "units_sold". It also lets you tell the model: “All column names are lowercase with underscores.” That simple convention prevents 90% of column-name mismatch errors.

An even better fix is to feed the error back to the model as an observation, exactly as we did in Chapter 1:

```
messages.append({"role": "tool", "content": "Error: column  
'Revenue' not found. Available columns: ['product', 'region',  
'units_sold', 'revenue', 'date']"})
```

The model usually recovers on the next turn by correcting the case. This recovery pattern — try, fail, report, retry — is one of the most powerful features of the agent loop.

Try It Now: Three Variations

You’ve built an agent that inspects, queries, and reports on CSV data. Now stretch it. The extensions below add visualization, anomaly detection, and schema comparison — all using the same pure-Python tool pattern.

Variation 1: Generate a Matplotlib Chart and Save PNG

Numbers in a table are useful. Numbers in a chart are persuasive. Add a tool that generates a bar chart from two columns and saves it as a PNG:

```
import matplotlib
matplotlib.use("Agg") # Non-interactive backend for script
                        execution
import matplotlib.pyplot as plt

def chart_csv(path: str, x_column: str, y_column: str,
              output_path: str = "chart.png") -> str:
    """Generate a bar chart from two CSV columns and save it as a
    PNG."""
    try:
        df = pd.read_csv(path)
    except Exception as e:
        return f"Error reading CSV: {e}"

    if x_column not in df.columns or y_column not in df.columns:
        return f"Error: columns not found. Available:
        {list(df.columns)}"

    try:
        plt.figure(figsize=(8, 5))
        plt.bar(df[x_column].astype(str), df[y_column])
        plt.xlabel(x_column)
        plt.ylabel(y_column)
        plt.title(f"{y_column} by {x_column}")
        plt.tight_layout()
        plt.savefig(output_path)
        plt.close()
```



```

        return f"Chart saved to {output_path}"
    except Exception as e:
        return f"Error generating chart: {e}"

```

The `matplotlib.use("Agg")` line is essential if you're running this on a headless server, a container, or any environment without a display. It tells matplotlib to render to a file instead of trying to open a window. I once spent an hour debugging a "cannot connect to X server" error in a CI pipeline before I learned this one line.

Variation 2: Detect Outliers with IQR Method

Not every number in a dataset belongs there. A mistyped entry, a unit conversion error, or a sensor glitch can drop impossible values into an otherwise clean column. The interquartile range (IQR) method is a robust, distribution-free way to flag suspicious values:

```

def detect_outliers(path: str, column: str) -> str:
    """Detect outliers in a numeric column using the IQR
    method."""
    try:
        df = pd.read_csv(path)
    except Exception as e:
        return f"Error: {e}"

    if column not in df.columns:
        return f"Column '{column}' not found. Available:
{list(df.columns)}"

    try:
        series = pd.to_numeric(df[column],
errors="coerce").dropna()
        q1 = series.quantile(0.25)
        q3 = series.quantile(0.75)
        iqr = q3 - q1
        lower = q1 - 1.5 * iqr
        upper = q3 + 1.5 * iqr
        outliers = series[(series < lower) | (series > upper)]
        return (
            f"Found {len(outliers)} outliers (bounds: {lower:.2f}
to {upper:.2f}).\n"
            f"{outliers.to_string()}"
        )
    except Exception as e:
        return f"Error computing outliers: {e}"

```

The `pd.to_numeric(..., errors="coerce")` line deserves attention. If a value like "pending" snuck into a numeric column, pandas will turn it into NaN rather than crashing. The IQR is then computed only on valid

numbers, and the invalid entries are silently excluded from the calculation but still visible in the original data if you need to audit them.

Variation 3: Compare Two CSVs for Schema Drift

In production pipelines, the shape of your data changes over time. A column gets renamed, a new field appears, or a date column suddenly arrives as an integer timestamp. An agent that can detect schema drift before it breaks downstream analysis is worth its weight in gold:

```
def compare_schemas(path_a: str, path_b: str) -> str:
    """Compare the schemas of two CSV files and report
    differences."""
    try:
        df_a = pd.read_csv(path_a)
        df_b = pd.read_csv(path_b)
    except Exception as e:
        return f"Error reading files: {e}"

    cols_a = set(df_a.columns)
    cols_b = set(df_b.columns)

    added = cols_b - cols_a
    removed = cols_a - cols_b
    common = cols_a & cols_b

    diffs = []
    for col in common:
        dtype_a = str(df_a[col].dtype)
        dtype_b = str(df_b[col].dtype)
        if dtype_a != dtype_b:
            diffs.append(f"  '{col}': {dtype_a} -> {dtype_b}")

    report = []
    if added:
        report.append(f"Columns added in {path_b}:
{sorted(added)}")
    if removed:
        report.append(f"Columns removed from {path_a}:
{sorted(removed)}")
    if diffs:
        report.append("Type changes in common columns:")
        report.extend(diffs)
    if not report:
        report.append("Schemas are identical.")

    return "\n".join(report)
```

I run a version of this every Monday morning against my exported analytics files. It takes ten seconds and has caught two breaking changes before they made it into a dashboard. The agent reports the drift, and I decide whether to update the downstream code or yell at the upstream source.

The OpenClaw Way

OpenClaw handles the tool registry and conversation loop we built by hand, which means our three CSV tools drop in with almost no boilerplate. You decorate each function with `@tool`, and OpenClaw automatically generates the JSON schema that the model uses to decide when to call it. The `read_csv_summary` docstring becomes the tool description; the type hints become the parameter schema. OpenClaw also adds structured output parsing, so when the model emits a tool call, the framework validates the arguments against your type hints before passing them to the function. If the model hallucinates a seventh operation that isn't in your whitelist, OpenClaw intercepts the call and returns a validation error back to the model as an observation, triggering a retry. The only change to our code is replacing the standalone functions with decorated versions and passing them to an Agent instance. Everything else — the safe ops mapping, the error handling, the JSON summary formatting — stays exactly as we wrote it.

The Hermes Agent Way

Hermes Agent approaches data analysis from the opposite direction. Instead of you writing specialized tools and registering them, Hermes comes with a built-in code execution environment. You point it at `sales.csv` in the conversation and ask, “What’s the average revenue by region?” Hermes reads the file, generates a small pandas snippet internally — something like `df.groupby("region")["revenue"].mean()` — executes it in a sandbox, and returns the result. You didn't write `query_csv`. Hermes wrote it for you on the fly. The trade-off is precise control: you can't enforce a whitelist of operations the way we did with `SAFE_OPS`, and the model might generate code that uses `eval()` or imports unexpected libraries. For exploration and ad-hoc questions, Hermes is faster. For production pipelines where you need guaranteed safety and reproducible behavior, the explicit tool approach we built in this chapter is the only responsible choice. I use Hermes for quick looks and the custom agent for anything that runs unattended.

What You Built

In this chapter, you built a spreadsheet intelligence agent from the ground up:

- A lightweight CSV inspector using only the Python standard library, for environments where dependencies aren't an option.
- A pandas-powered upgrade that infers types, counts nulls, and handles malformed files without crashing.
- `read_csv_summary`, a tool that returns structured JSON metadata about any CSV, so the model knows what it's working with before it asks a question.
- `query_csv`, a safe aggregation tool that maps natural-language questions to whitelisted pandas operations — deliberately designed without `eval()` to prevent code injection.
- `write_summary`, a report generator that writes markdown to disk with full provenance back to the source file.
- Five failure modes and their fixes: wrong delimiters, encoding mismatches, hallucinated statistics, memory limits, and column name mismatches.
- Three extensions: matplotlib charting, IQR outlier detection, and cross-file schema drift comparison.

You also learned the cardinal rule of data agents: the model must compute, not guess. Every number it reports should trace back to a pandas operation on the actual file. If it can't trace the number, it shouldn't say it.

Cost for this chapter: Pandas, matplotlib, and the standard library are free. If you run the agent loop against a local Ollama model, the total cost is \$0.00. If you use GPT-4o mini for the reasoning loop with ten tool calls, expect roughly \$0.03–\$0.07 per session. For GPT-4o, the same session costs approximately \$0.15–\$0.35, depending on how verbose the model is with its thinking. The data analysis itself is always free; you're only paying for the LLM to decide which tools to call and how to phrase the answer.

Chapter 5: Project 5 — Smart Email Assistant

The agent drafts your reply. It matches your boss's tone. It does NOT click send until you say yes.

A few years ago, I trusted an automation script with my email. It seemed harmless. The script was supposed to sort newsletters into a folder, nothing more. But I had a bug in a conditional check, and one rainy Tuesday morning the script replied to a client email with an auto-generated message meant for a mailing list. The client was asking about a contract renewal. My script thanked them for subscribing to Python Weekly. I lost the account. It took six months of relationship repair to win it back.

That story lives in my head because it taught me the most important rule of agent design: **trust is not a feature you add later. It is the foundation you build on first.** Email agents are not like weather bots or file organizers. Weather bots make mistakes and you get the wrong forecast. File organizers make mistakes and your Downloads folder stays messy. Email agents make mistakes and your reputation, your job, or your relationships suffer real damage.

This chapter is different from the others. We're still building an agent loop. We're still using pure Python first, then showing the framework variants. But the tone is heavier, because the consequences are heavier. Every line of code you write in this project needs to answer one question before it does anything else: *What happens if this goes wrong?* If you can't answer that question, you don't run the code. That's not paranoia. It's engineering.

Why Email Needs a Human-in-the-Loop

An email assistant is one of the most requested agent projects I hear about. Everyone wants something that reads their inbox, figures out what matters, and handles the noise. But here's the tension: email is a communication medium between humans. Every message carries context, relationship history, emotional tone, and implicit expectations. No language model understands all of that. It can approximate tone. It can guess intent. It can draft something plausible. But plausible is not the same as correct.

That's why the architecture of this project is built around a gate. The agent reads. The agent analyzes. The agent drafts. But the agent never, ever sends without explicit human approval. Not "I'll review it later." Not "Approve all messages from known contacts." One explicit confirmation per draft. If that gate makes the agent less useful, good. Less useful and safe is better than convenient and career-ending.

The human-in-the-loop design is not a training wheels mode. It's the final production architecture. You'll see this pattern throughout the book whenever an agent touches something irreversible: sending a message, moving money, deleting data, deploying code. Email is the textbook example of an irreversible action. Once sent, a message is out of your control. The gate stays.

Why OAuth2 Matters (Even If We Don't Use It Here)

Before we write code, you need to understand a door we're deliberately not walking through. Real email agents that talk to Gmail, Outlook, or any major provider must authenticate via OAuth2. OAuth2 is the modern standard for delegated access: you authorize the application once, the provider issues a time-limited token, and the application uses that token to act on your behalf without ever seeing your password.

OAuth2 matters because passwords are dangerous. If your script stores your email password in plain text, any bug, leak, or compromised dependency exposes your entire account. OAuth2 tokens are scoped (read-only versus send), revocable (you can kill them from the provider's dashboard), and time-bounded (they expire). For any production email agent, OAuth2 is non-negotiable.

But OAuth2 is also a rabbit hole of app registrations, consent screens, redirect URIs, and refresh-token logic. For a beginner's project, that complexity obscures the agent mechanics we're actually trying to learn. So in this chapter, we'll use IMAP with application-specific passwords. This is a legitimate learning path: many email providers still support IMAP access for desktop clients and scripts, and it lets us focus on the agent loop rather than the OAuth handshake.

The critical takeaway: if you ever deploy this agent to handle real email for real people, you must upgrade to OAuth2. Treat the IMAP version in this chapter as a laboratory. The principles transfer directly. The authentication layer does not.

Building an Email Reader with imaplib

Python's standard library includes `imaplib`, which speaks the Internet Message Access Protocol directly to your email server. No pip installs. No third-party dependencies. Just Python and a network connection. This is exactly the kind of foundation-first approach this book takes: you see the raw protocol, you understand the limits, and then you appreciate what a framework saves you later.

Here's the basic connection and search flow. We'll start by fetching unread messages from the inbox:

```
import imaplib
import email
from email.message import EmailMessage
from typing import List, Tuple

def connect_imap(
    host: str, username: str, password: str, mailbox: str =
    "INBOX"
) -> imaplib.IMAP4_SSL:
    """Connect to an IMAP server and select a mailbox.

    Returns an authenticated IMAP4_SSL connection ready for
    commands.
    Raises on authentication or connection failure.
    """
    try:
        conn = imaplib.IMAP4_SSL(host)
        status, response = conn.login(username, password)
        if status != "OK":
            raise ConnectionError(f"IMAP login failed:
{response}")
        status, _ = conn.select(mailbox)
        if status != "OK":
            raise ConnectionError(f"Could not select mailbox
{mailbox}")
        return conn
    except imaplib.IMAP4.error as e:
        raise ConnectionError(f"IMAP protocol error during
connect: {e}") from e
    except OSError as e:
```

```

        raise ConnectionError(f"Network error connecting to
{host}: {e}") from e

def fetch_unread_emails(conn: imaplib.IMAP4_SSL, limit: int = 10)
-> List[dict]:
    """Fetch unread emails from the selected mailbox.

    Returns a list of dicts with keys: uid, subject, from, date,
    body, message_id.
    Only fetches the most recent `limit` unread messages.
    """
    try:
        status, data = conn.search(None, "UNSEEN")
        if status != "OK":
            raise RuntimeError("IMAP search failed")

        email_ids = data[0].split()
        if not email_ids:
            return []

        # Take the last N (most recent)
        targets = email_ids[-limit:]
        results: List[dict] = []

        for eid in targets:
            status, msg_data = conn.fetch(eid, "(RFC822)")
            if status != "OK":
                continue

            raw = msg_data[0][1]
            msg = email.message_from_bytes(raw)

            subject =
str(email.header.make_header(email.header.decode_header(msg.get("Subject"))))

            from_addr = msg.get("From", "")
            date = msg.get("Date", "")
            message_id = msg.get("Message-ID", "")

            body = ""
            if msg.is_multipart():
                for part in msg.walk():
                    content_type = part.get_content_type()
                    if content_type == "text/plain":
                        payload = part.get_payload(decode=True)
                        if payload:
                            body = payload.decode("utf-8",
errors="replace")
                            break
            else:
                payload = msg.get_payload(decode=True)

```



```

        if payload:
            body = payload.decode("utf-8",
errors="replace")

        results.append({
            "uid": eid.decode(),
            "subject": subject,
            "from": from_addr,
            "date": date,
            "body": body,
            "message_id": message_id,
        })

    return results
except imaplib.IMAP4.error as e:
    raise RuntimeError(f"IMAP error while fetching emails:
{e}") from e

```

Walk through this slowly. `connect_imap` establishes an SSL-wrapped IMAP connection, authenticates, and selects a mailbox. Every failure path raises a specific exception with context. `fetch_unread_emails` searches for messages with the UNSEEN flag, then parses each raw RFC822 payload into a Python `email.message_from_bytes` object. The body extraction walks multipart MIME structures, falls back to a single part, and always decodes with replacement for bad UTF-8. This is not elegant. It is robust.

Why UNSEEN? Because UNSEEN is the flag the server tracks. If your script crashes halfway through, the server still knows which messages weren't read. That safety matters when you're iterating on agent code and restarting constantly.

Thread Reconstruction via Message-ID and References

Email threads are not a native IMAP concept. The server stores individual messages. The thread is a graph you reconstruct from headers. Two headers do the work:

- **Message-ID:** a globally unique identifier for this message.
- **References:** a list of Message-ID values for every prior message in the thread.
- **In-Reply-To:** the immediate parent Message-ID.

To build a thread, you fetch a message, collect its Message-ID, then search the mailbox for any message whose References or In-Reply-To headers contain that ID. Repeat until you have the whole chain.

```
def fetch_thread_for_message(
    conn: imaplib.IMAP4_SSL, message_id: str, limit: int = 5
) -> List[dict]:
    """Fetch the most recent N emails in the same thread as the
    given Message-ID.

    Uses the References and In-Reply-To headers to identify
    related messages.
    Returns them in chronological order by Date header.
    """
    if not message_id:
        return []

    try:
        # Search for messages that reference this Message-ID
        # IMAP SEARCH HEADER is not universally supported, so we
        fallback:
        # fetch the last 200 messages and filter client-side.
        status, data = conn.search(None, "ALL")
        if status != "OK":
            raise RuntimeError("IMAP search failed")

        all_ids = data[0].split()[-200:] # recent 200
        thread_emails: List[dict] = []

        for eid in all_ids:
            status, msg_data = conn.fetch(eid,
            "(BODY.PEEK[HEADER.FIELDS (MESSAGE-ID REFERENCES IN-REPLY-TO DATE
            FROM SUBJECT)])")
            if status != "OK":
                continue

            raw_header = msg_data[0][1]
            msg = email.message_from_bytes(raw_header)
            mid = msg.get("Message-ID", "")
            refs = msg.get("References", "")
            in_reply = msg.get("In-Reply-To", "")

            if message_id in (mid, refs, in_reply):
                # Full fetch for thread messages
                status2, full_data = conn.fetch(eid, "(RFC822)")
                if status2 != "OK":
                    continue
                full_msg = email.message_from_bytes(full_data[0]
                [1])

                body = ""
                if full_msg.is_multipart():
```

```

        for part in full_msg.walk():
            if part.get_content_type() == "text/
plain":
                p = part.get_payload(decode=True)
                if p:
                    body = p.decode("utf-8",
errors="replace")
                    break
            else:
                p = full_msg.get_payload(decode=True)
                if p:
                    body = p.decode("utf-8", errors="replace")

        thread_emails.append({
            "uid": eid.decode(),
            "subject":
str(email.header.make_header(email.header.decode_header(full_msg.get(
""))),
            "from": full_msg.get("From", ""),
            "date": full_msg.get("Date", ""),
            "body": body,
            "message_id": mid,
        })

        # Sort by date (best-effort string sort on Date header)
        thread_emails.sort(key=lambda x: x["date"])
        return thread_emails[-limit:] # last N
    except imaplib.IMAP4.error as e:
        raise RuntimeError(f"IMAP error while fetching thread:
{e}") from e

```

The .PEEK in the header fetch is deliberate. It reads the header without marking the message as SEEN on the server. You don't want your agent to mark every email as read just because it was analyzed for threading. That courtesy matters when the user also checks email on their phone.

The client-side filtering isn't the most efficient approach for a million-message archive, but for a learning project with a typical inbox, it's fast enough and avoids compatibility headaches. If you later need server-side OR HEADER searches, you'll be working with a provider that supports extended search syntax, and you'll know exactly why you're using it.

Tone Matching and Draft Generation

Now the agent part. We need to read a thread, analyze the tone of the most recent incoming message, and draft a reply that matches it. The LLM does two things here: tone classification and draft synthesis. We prompt for both explicitly, and we ask for a confidence score.

```
def analyze_tone_and_draft_reply(
    thread_context: List[dict],
    sender_style: str = "professional and concise",
    model_endpoint: str = "http://localhost:11434/api/generate",
    model_name: str = "llama3.1:8b",
) -> dict:
    """Analyze the most recent message's tone and draft a matching
    reply.
```

```
    Args:
        thread_context: Chronological list of dicts with 'from',
        'body', 'subject'.
        sender_style: Free-text description of the user's
        preferred voice.
        model_endpoint: URL for local LLM (Ollama-compatible).
        model_name: Model tag to execute.
```

```
    Returns:
        Dict with keys:
        - draft: proposed reply text
        - tone_detected: 'formal', 'casual', 'urgent', or
        'neutral'
        - confidence: float 0.0-1.0
        - safety_flag: True if privacy or sensitive terms
        detected
    """
```

```
    import json, httpx

    if not thread_context:
        return {"draft": "", "tone_detected": "neutral",
        "confidence": 0.0, "safety_flag": True}
```

```
    # Build a condensed thread view
    lines = []
    for msg in thread_context:
        snippet = msg["body"][:400].replace("\n", " ")
        lines.append(f"From: {msg['from']}\nSubject:
    {msg.get('subject', '')}\n{snippet}\n---")
    thread_text = "\n".join(lines)
```

```
    system_prompt = (
        "You are an email assistant. Your job is to:\n"
        "1. Read the thread below.\n"
```

```

        "2. Detect the tone of the MOST RECENT incoming message
only.\n"
        "3. Draft a reply in the user's preferred style.\n"
        "4. Rate your confidence 0.0-1.0.\n"
        "5. Set safety_flag=true if the email mentions legal,
medical, financial, or intimate personal matters.\n"
        "\n"
        'Respond ONLY with valid JSON in this exact shape (no
markdown, no prose):\n'
        '{"tone_detected": "formal|casual|urgent|neutral", '
        '"confidence": 0.0, '
        '"safety_flag": false, '
        '"draft": "your proposed reply here"}\n'
    )

    user_prompt = (
        f"User's preferred style: {sender_style}\n\n"
        f"Thread:\n{thread_text}\n\n"
        f"Now output JSON."
    )

    try:
        response = httpx.post(
            model_endpoint,
            json={
                "model": model_name,
                "prompt": f"System: {system_prompt}\n\nUser:
{user_prompt}\n\nAssistant:",
                "stream": False,
                "options": {"temperature": 0.3},
            },
            timeout=60.0,
        )
        response.raise_for_status()
        raw = response.json()["response"].strip()

        # Extract JSON if wrapped in fences
        if "` ` ` `json" in raw:
            raw = raw.split("` ` ` `json")[1].split("` ` ` `")[0].strip()
        elif "` ` ` `" in raw:
            raw = raw.split("` ` ` `")[1].split("` ` ` `")[0].strip()

        parsed = json.loads(raw)
        parsed.setdefault("confidence", 0.0)
        parsed.setdefault("safety_flag", True)
        parsed.setdefault("draft", "")
        parsed.setdefault("tone_detected", "neutral")
        return parsed
    except (httpx.RequestError, httpx.HTTPStatusError) as e:
        return {"draft": "", "tone_detected": "error",
"confidence": 0.0, "safety_flag": True, "error": str(e)}
    except (json.JSONDecodeError, KeyError) as e:

```

```

        return {"draft": "", "tone_detected": "error",
"confidence": 0.0, "safety_flag": True, "error": f"Parse error:
{e}"}

```

Notice how aggressively we constrain the output. JSON only. Exact shape. Temperature lowered to 0.3 to reduce creativity. The prompt explicitly scopes tone detection to the most recent incoming message, not the whole thread, because you want to reply to the person who just wrote you, not to the person who started the thread three weeks ago.

The `safety_flag` is critical. When the LLM detects legal, medical, financial, or intimate personal content, it flips a boolean that downstream logic can use to require extra review, skip drafting entirely, or route to a human. The LLM is not a reliable classifier, which is why we'll layer additional checks in the approval gate.

Confidence Scoring and the Human Approval Gate

This is the heart of the project. No draft leaves the script without passing two barriers: a numerical confidence threshold and an explicit human confirmation.

```

NEVER_AUTO_SEND = True # This must remain True. Always.

```

```

CONFIDENCE_THRESHOLD = 0.75
SAFETY_BLOCK_TOPICS = ["terminat", "layoff", "legal action",
"lawsuit", "medical", "diagnosis"]

```

```

def human_approval_gate(draft_info: dict, thread_subject: str) ->
bool:

```

```

    """Present a draft to the user and require explicit
confirmation.

```

Args:

```

    draft_info: Output of analyze_tone_and_draft_reply.
    thread_subject: Subject line to display for context.

```

Returns:

```

    True if the user explicitly approves; False otherwise.
    """

```

```

    # Hard safety: never auto-send, even if the caller forgets
    assert NEVER_AUTO_SEND is True, "CRITICAL: NEVER_AUTO_SEND was
disabled"

```

```

    draft = draft_info.get("draft", "")
    confidence = float(draft_info.get("confidence", 0.0))

```

```

safety_flag = bool(draft_info.get("safety_flag", True))
tone = draft_info.get("tone_detected", "unknown")

# Additional string-based safety layer (model-independent)
lower_draft = draft.lower()
topic_blocked = any(topic in lower_draft for topic in
SAFETY_BLOCK_TOPICS)

print("\n" + "=" * 50)
print(f"THREAD: {thread_subject}")
print(f"TONE DETECTED: {tone} | CONFIDENCE: {confidence:.2f}")
print(f"SAFETY FLAG: {safety_flag} | TOPIC BLOCK:
{topic_blocked}")

if confidence < CONFIDENCE_THRESHOLD:
    print(f"⚠ Confidence below threshold
({CONFIDENCE_THRESHOLD}).")
    if safety_flag or topic_blocked:
        print(f"🛑 SAFETY HOLD: This draft requires extra
scrutiny.")

print("\n--- PROPOSED DRAFT ---\n")
print(draft if draft else "(empty draft)")
print("\n--- END DRAFT ---\n")

choice = input("Send? [y/N/edit]: ").strip().lower()
if choice == "y":
    print("Approved by user.")
    return True
elif choice == "edit":
    print("Copy the draft, edit it manually, and send from
your mail client.")
    return False
else:
    print("Discarded.")
    return False

```

The assert NEVER_AUTO_SEND is True is not a polite reminder. It is a hard enforcement. If someone — future you, a teammate, a malicious refactor — changes NEVER_AUTO_SEND to False, the script crashes immediately. In production, you would use a more elaborate gate, but for a learning project, an assert makes the invariant impossible to miss.

The gate presents the draft, the confidence, the detected tone, and any safety warnings. The default action is discard. The user must explicitly type y to approve. Even an edit option is provided so the user can escalate to manual handling without trusting the agent’s Send button.

Putting It All Together

Here's the main agent loop that wires the pieces together:

```
def run_email_assistant() -> None:
    """Main loop: connect, fetch unread, analyze threads, draft
    replies, gate send."""
    import os

    host = os.environ.get("IMAP_HOST", "imap.gmail.com")
    user = os.environ.get("IMAP_USER", "")
    password = os.environ.get("IMAP_PASSWORD", "")

    if not all([host, user, password]):
        print("Set IMAP_HOST, IMAP_USER, and IMAP_PASSWORD
environment variables.")
        return

    try:
        conn = connect_imap(host, user, password)
        unread = fetch_unread_emails(conn, limit=5)
        print(f"Found {len(unread)} unread messages to process.")

        for msg in unread:
            print(f"\n→ Processing: {msg['subject'][:60]}")
            thread = fetch_thread_for_message(conn,
msg["message_id"], limit=5)
            if not thread:
                thread = [msg]

            analysis = analyze_tone_and_draft_reply(
                thread_context=thread,
                sender_style="professional, concise, and helpful"
            )

            approved = human_approval_gate(analysis,
msg["subject"])
            if approved:
                print("[PRETEND SEND] In a real system, this would
send via SMTP. It does not.")
            else:
                print("Draft rejected or held for manual review.")

            conn.logout()
    except ConnectionError as e:
        print(f"Fatal connection error: {e}")
    except RuntimeError as e:
        print(f"Runtime error during processing: {e}")
```


Notice the comment in the approval path: [PRETEND SEND]. Even when the user approves, this script does not actually dispatch an email. There is no SMTP client here. There is no send method. If you want to wire up real sending, you do it in a separate module, behind additional confirmation, with full logging. The core learning loop ends at the gate. Everything past the gate is infrastructure, not intelligence.

Break and Fix: Five Ways This Project Can Fail

Email agents fail differently from other agents. The failures are quieter, more personal, and harder to undo. Here are five you need to prepare for.

Break 1: Wrong Tone Detected

Your boss sends a terse email: *“Need the Q3 numbers by EOD.”* The agent classifies the tone as “casual” because it’s short. It drafts a breezy reply: *“No worries, I’ll get those to you later!”* Later is not end of day. Your boss is furious.

Fix: Add explicit examples to the tone-detection prompt. Shortness is not casualness. Include few-shot examples of formal terse versus casual terse. Also weight the confidence down when the message is under ten words — low-context inputs are inherently uncertain.

```
# In the system prompt, add:  
"Examples of tone:\n"  
"- 'Hey, can you send that when you get a chance?' → casual\n"  
"- 'Need the Q3 numbers by EOD.' → formal\n"  
"- 'URGENT: site is down' → urgent\n"
```

Break 2: Draft Is Too Generic or Missing Context

The thread contains five messages about a specific client’s custom integration. The agent drafts: *“Thanks for your message. I’ll look into this and get back to you.”* It names neither the client nor the integration. The recipient assumes you haven’t read the thread.

Fix: Feed the LLM the full thread, but truncate intelligently. Summarize older messages beyond the most recent three. In the prompt, explicitly require the draft to reference specific entities from the thread.

```

system_prompt += (
    "Rule: your draft must reference at least one specific name,
    project, "
    "or detail from the thread so the recipient knows you read
    it."
)

```

Break 3: IMAP Connection Dropped

Halfway through processing a large inbox, the server drops the connection. Your script crashes mid-loop. Some emails were marked SEEN, others weren't. The user has no idea which ones were processed.

Fix: Wrap the loop body in a try/except that performs graceful cleanup. Log every message UID as soon as it is fetched, before analysis. If you must track state, write a local JSON log of processed UIDs so you can resume safely.

```

processed_log = []
try:
    for msg in unread:
        processed_log.append(msg["uid"])
        # ... analyze ...
except imaplib.IMAP4.error as e:
    print(f"IMAP error mid-loop: {e}")
    print(f"Successfully processed UIDs before crash:
    {processed_log}")
finally:
    try:
        conn.close()
        conn.logout()
    except Exception:
        pass

```

Break 4: Auto-Send Prevention Fails

You refactor the code in a hurry. You move the approval gate into a helper module. Somewhere in the import chain, another file sets NEVER_AUTO_SEND = False because it was copy-pasted from a different project. The gate still asks for input, but downstream logic reads the wrong constant and sends anyway.

Fix: The assert in human_approval_gate catches this specific failure mode at runtime. For extra defense in depth, do not expose a send_email function in the same module as the agent loop. Separate read and write code into different files. Make the send function require a manually passed confirmed=True keyword argument. No default.

Break 5: Reading an Email That Shouldn't Be Read

Your partner sends you a sensitive medical result. Your agent reads it, feeds it to the LLM, and logs the full body to stdout for debugging. The log file lives in a shared workspace. Privacy is compromised.

Fix: Implement a privacy trigger in the fetch stage, before any LLM sees the content. Scan the subject and body for keywords that flag sensitive content. If triggered, skip analysis entirely. Do not log the body. Print only a redacted summary.

```
PRIVACY_TRIGGERS = ["confidential", "medical", "diagnosis",
                    "layoff", "termination", "salary", "ssn", "social security"]

def is_sensitive(email_dict: dict) -> bool:
    """Return True if the email should not be processed by the
    agent."""
    text = f"{email_dict.get('subject','')}
{email_dict.get('body','')}".lower()
    return any(trigger in text for trigger in PRIVACY_TRIGGERS)
```

Call `is_sensitive` immediately after `fetch_unread_emails`. If it returns `True`, log a single line like `Skipped sensitive email UID 1234` and move on.

Try It Now: Three Variations

Once the core loop works, stretch it in these directions.

Variation 1: Categorize Inbox Backlog

Instead of drafting replies, change the agent to categorize unread messages into four buckets: URGENT, FYI, DELEGATE, and IGNORE. Use a compact prompt that returns JSON with the category and a one-line justification. Present the results as a morning summary table. Do not draft replies. Do not send anything. This mode is read-only and therefore much safer.

Variation 2: Summarize All Unread into a Morning Briefing

Fetch all unread emails, generate a single-paragraph summary per thread, and write them to a local markdown file: `morning_briefing_2026-04-20.md`. Read the file yourself, decide what

matters, and handle the email manually. The agent becomes a summarization layer, not an action layer. Summarization errors are annoying. They are not dangerous.

Variation 3: Learn Personal Style from the Sent Folder

With explicit user consent only, fetch the last 100 sent emails (not received, sent). Extract the bodies and build a small prompt appendix: *“Here are examples of how I write:”* followed by three representative snippets. Feed this into the draft generation prompt as the `sender_style` parameter. The drafts improve immediately. The consent requirement is not optional. Do not index someone else’s sent folder. Do not index a shared mailbox. One person’s style is another person’s private record.

Ethics Sidebar: Consent and Data Handling

Email agents sit on a data goldmine and an ethical minefield. Every email your agent reads is a record of human communication, often containing information the sender never intended to share with a machine. When you build this project, you are making a policy decision, not just a technical one.

First, **consent**: never run an email agent on an account that isn’t yours without explicit written permission. Not an 口头 okay. Not “I’m sure they won’t mind.” Written consent that specifies what will be read, how long it will be retained, and whether any third-party APIs (like OpenAI) will see the content. Many organizations have policies that prohibit feeding internal email to external LLMs entirely. Ignoring those policies isn’t clever automation. It’s a compliance violation.

Second, **retention**: the agent should not store email bodies longer than necessary. If the LLM call is cloud-based, the content passes through a third-party server. Minimize this by using local models for sensitive accounts. If you must use a cloud model, strip headers, redact names, and truncate aggressively. The safer path is to keep all processing local with Ollama and only send anonymized summaries to the cloud.

Third, **organizational policy**: if you build this for work, your company’s acceptable use policy probably has something to say about automated email access. Read it. Follow it. When in doubt, ask IT. The cost of asking is measured in minutes. The cost of not asking is measured in employment.

The OpenClaw Way

OpenClaw formalizes the patterns we built by hand into a declarative agent configuration. In OpenClaw, you would define an EmailAgent class with read-only tools (`fetch_unread`, `fetch_thread`) and a gated action tool (`draft_reply`). The framework enforces tool-level permissions: you mark `draft_reply` as `destructive=False` but `send_email` as `destructive=True`, and OpenClaw automatically wraps destructive tools in a confirmation UI.

For this project, the advantage is that OpenClaw's conversation memory and cost tracking apply natively. Each email thread becomes a session. The framework logs every tool call, every LLM invocation, and every token spent. If you set a per-session budget of \$0.01, OpenClaw halts processing before the cost runs away. The OpenClaw variant would reduce our hundred lines of raw Python to roughly thirty, with the same safety invariants enforced by the framework rather than manual asserts. You still define the prompts. You still set the gate. But the boilerplate of IMAP exception handling and JSON parsing is handled for you.

The Hermes Agent Way

Hermes Agent approaches email assistance conversationally. Rather than writing a Python script that polls an inbox, you open a Hermes session and say: *“Read my unread emails and draft replies for anything from my boss.”* Hermes uses its built-in IMAP toolset (if configured) or file-reading capabilities (if you export emails as .eml files) to iterate through messages, summarize them in the chat buffer, and present drafts inline.

The Hermes model excels at the summarization and variation modes — the morning briefing, the categorization — because its session memory naturally accumulates context across multiple messages. However, Hermes does not have a built-in human approval gate for SMTP sending. That means you must treat Hermes as a **read-only analyst** for email. Generate the drafts in the chat, copy them manually into your mail client, and send them yourself. The same principle applies: trust the tool for what it does well, but do not delegate the irreversible action to anything that does not have an explicit, hardwired confirmation step you control.

What You Built

In this chapter, you built an email assistant that respects the boundary between helpful and harmful:

- An imaplib-based email reader with explicit exception handling for network and protocol errors.
- A thread reconstructor using Message-ID, References, and In-Reply-To headers.
- A tone-matching LLM prompt that returns structured JSON with a confidence score.
- A NEVER_AUTO_SEND assert gate that enforces human approval before any draft is treated as final.
- A privacy trigger that skips sensitive emails before the LLM ever sees them.
- A layered safety architecture: low-confidence holds, topic keyword blocks, LLM safety flags, and manual confirmation.
- Five failure modes analyzed, reproduced, and fixed, including tone mismatch, generic drafts, connection drops, bypassed send guards, and privacy leaks.

You also learned why email agents carry more weight than other automations, why OAuth2 is the production standard even when IMAP is the learning path, and why consent is a technical requirement, not an afterthought.

Cost for this chapter: Approximately \$0.00 if you use Ollama for all LLM calls and process fewer than fifty emails per run. If you use GPT-4o mini for tone analysis and draft generation, expect roughly \$0.01–\$0.03 per email depending on thread length and body size. A forty-email morning batch would cost approximately \$0.50–\$1.20. Use local models for experimentation; reserve cloud models for the final polished drafts you actually send.

Chapter 6: Project 6 — API Connector Agent: Data Pipeline

Connect to a real API, extract 10,000 records, and never fetch the same record twice.

The worst data loss of my career happened on a Friday afternoon. A client had asked me to migrate three years of Salesforce leads into a new marketing platform. I wrote a Python script, tested it on ten records, and let it rip against the full dataset. Two hours later, I had eighty thousand rows in a CSV. I opened it, felt a warm glow of accomplishment, and went home. On Monday, the client called. Half the records were duplicates. A quarter were missing timestamps. And because I had run the script twice to “be safe,” the destination system now contained forty thousand phantom contacts with no reliable primary key. I spent the next three days manually reconciling IDs. The client never trusted automated imports again. Neither did I, until I learned what I had actually done wrong.

I had treated an API like a vending machine: put in a request, get back data, what’s the big deal? But APIs are conversations. They have rate limits, pagination, temporary failures, and evolving schemas. They return different shapes on Tuesdays. They time out when you’re in a hurry. If your script doesn’t account for all of that, you’re not building a data pipeline — you’re building a lottery. This chapter is about building a pipeline that wins every time.

By the end, you’ll have an agent that connects to a real public API, paginates through results, validates every record against a schema, deduplicates by ID, backs off from rate limits, handles network errors without crashing, and writes clean output to both JSON and CSV. Then we’ll break it five different ways, fix each break, and give you three concrete variations to stretch the concept. This is the project where your agent stops being a toy and starts behaving like production software.

Why API Connectors Are the Bread and Butter of Agents

Every agent in this book eventually needs data from the outside world. A research agent needs search results. A CI/CD watchdog needs build logs. A document generator needs CMS content. The API connector is the plumbing that makes all of that possible. No plumbing, no house — no matter how fancy the furniture is.

But plumbing is invisible until it leaks. And it leaks constantly. The API you call today might change its response shape next month. Your token might expire mid-sync. A thousand-record job might turn into a hundred-thousand-record job, and the polite little script you wrote at 9 AM becomes a Denial-of-Service attack by noon. A real connector agent doesn't just fetch data. It fetches data *safely, reliably, and repeatably*.

That's what we're building. Not a one-off curl command wrapped in Python. A robust, paginated, idempotent connector that you can run every hour with confidence.

REST Basics and Picking Our Target

Most APIs you'll interact with are REST APIs. REST isn't a standard — it's a style. You send an HTTP request to a URL, and the server sends back data, usually as JSON. Four pieces of information govern every call: the method, the URL, the headers, and the status code.

Methods tell the server what you want. GET means "retrieve." POST means "create." For data pipelines, you'll mostly use GET.

Headers carry metadata. The Accept header tells the server you want JSON. The Authorization header carries your token. The User-Agent header tells the server who's calling. Headers are contracts. If you send the wrong ones, the API rejects you politely or confusingly.

Status codes tell you what happened. 200 means success. 400 means you made a mistake (bad URL, bad parameters). 401 means your authentication failed. 429 means you're asking too fast. 500 means the server's on fire. Your job is to react to each code differently. We'll write code that does exactly that.

For this project, we'll use the GitHub REST API. It's free, well-documented, and doesn't require authentication for public endpoints — though we'll show authenticated access too. Our connector will fetch

open issues from a public repository. I chose pallets/click because it's a popular Python library with enough issues to demonstrate pagination, but you can swap in any public repo you like.

Authentication and the Token Header

Most production APIs require authentication. The simplest form is the bearer token: a secret string you place in an Authorization header. GitHub supports this, and using a token raises your rate limit from sixty requests per hour to five thousand. That's the difference between a demo and a real job.

Store your token in an environment variable, never in the code. Then build your headers like this:

```
import os

TOKEN = os.environ.get("GITHUB_TOKEN", "")
HEADERS = {"Accept": "application/vnd.github+json"}
if TOKEN:
    HEADERS["Authorization"] = f"Bearer {TOKEN}"
```

If GITHUB_TOKEN is empty, the script still works against public endpoints, just with a lower rate limit. This graceful degradation is important. Your pipeline shouldn't fail because someone forgot to set an env var on a new laptop. It should limp along with reduced capacity and log a warning.

Pagination: The Page Loop

APIs rarely return everything at once. Imagine asking for all issues in a mature repository and receiving fifty thousand records in one payload. The server would choke, your memory would explode, and the network would weep. Instead, APIs paginate: they return a fixed number of records per request, usually twenty to one hundred, along with metadata that tells you if there's another page.

GitHub uses query parameters: `?page=1&per_page=100`. We loop until we get a page with fewer than `per_page` items, which means we've reached the end.

Here's the raw loop logic before we wire it into the full class:

```
page = 1
per_page = 100
```

```
while True:
    params = {"page": page, "per_page": per_page}
    resp = client.get(url, headers=HEADERS, params=params)
    data = resp.json()
    if not data or len(data) < per_page:
        break # Reached the last page
    page += 1
```

This loop is the heartbeat of any API connector. Without it, you're not building a pipeline. You're building a snapshot. In our full script, we'll wrap this in a function with error handling, rate-limit detection, and schema validation.

Pydantic Models and Schema Safety

If you've ever parsed API JSON with raw dictionaries and `data["field"]`, you've felt the pain. A renamed field. A missing key. A string where you expected an integer. Pydantic solves this by turning dictionaries into validated Python objects. When the API returns a shape you didn't expect, Pydantic raises a clear error instead of failing ten lines later with a `KeyError` or a `NoneType` exception.

We'll define a `GitHubIssue` model that matches the fields we care about:

```
from pydantic import BaseModel

class GitHubIssue(BaseModel):
    """Represents a GitHub issue returned by the REST API."""
    id: int
    title: str
    state: str
    html_url: str
    created_at: str
    updated_at: str
```

The `id` field is our primary key. It is how we'll deduplicate records when a sync runs twice. The title and state are human-readable. The `html_url` lets us link back to the original issue. The timestamps let us sort or filter downstream.

Every field has a type. If the API suddenly omits `id`, or sends a list instead of a string for `title`, Pydantic catches it at parse time and gives you a traceback that names the exact field and the exact failure.

Idempotency: Safe to Run Twice

Idempotency means you can run an operation multiple times without changing the result beyond the first application. In data pipelines, this translates to: if I run my sync twice, I don't get duplicates.

The simplest idempotency strategy is to track every primary key you've already seen. We use a Python set of IDs. Before inserting any new record, we check the set. If the ID is already there, we skip it. If it's new, we add it to the set and write it out.

```
seen_ids: set[int] = set()

for item in raw_items:
    issue_id = item.get("id")
    if issue_id in seen_ids:
        continue
    seen_ids.add(issue_id)
    # ... proceed to validation and storage
```

You might think duplicates are rare. They're not. Network timeouts cause retries. Cron jobs overlap. Developers rerun scripts. A production pipeline must treat every run as a potential rerun. The deduplication set is your safety net.

Error Handling for Every Network Call

A network call is a promise that the other end of the wire will cooperate. It frequently breaks that promise. We'll handle five broad categories of failure: timeout, rate limit (429), client error (4xx), server error (5xx), and malformed payloads.

The key is to isolate each failure and decide whether to retry, skip, or abort. Timeouts and rate limits are usually worth retrying after a delay. Server errors are transient and worth retrying once. Client errors and malformed payloads usually mean your code is wrong and should abort so you can fix it.

Saving Your Work: JSON and CSV

Once we have clean, deduplicated records, we need to persist them. JSON is the native format for API data. CSV is what your spreadsheet-wielding colleagues actually want. We'll write both. The JSON output preserves the full structure. The CSV output flattens it for human consumption.

```
import csv
import json
from pathlib import Path

def save_to_json(issues: list[GitHubIssue], path: Path) -> None:
    """Save issues to a JSON file with pretty printing."""
    data = [issue.model_dump() for issue in issues]
    path.write_text(json.dumps(data, indent=2), encoding="utf-8")
    print(f"Saved {len(issues)} records to {path}")

def save_to_csv(issues: list[GitHubIssue], path: Path) -> None:
    """Save issues to a CSV file with UTF-8 encoding."""
    if not issues:
        path.write_text("", encoding="utf-8")
        return
    rows = [issue.model_dump() for issue in issues]
    header = list(rows[0].keys())
    with path.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.DictWriter(fh, fieldnames=header)
        writer.writeheader()
        writer.writerows(rows)
    print(f"Saved {len(issues)} records to {path}")
```

Notice the `model_dump()` call. It's Pydantic's way of converting a model back to a plain dictionary, which both `json.dumps` and `csv.DictWriter` can consume. This pattern — API JSON → Pydantic model → dictionary → output format — is a pipeline you'll write a hundred times.

The Complete Script

Here's everything wired together. Save it as `api_connector.py`. Every function has a docstring, every network call has error handling, and every failure path prints a message instead of crashing the whole sync.

```
"""
```

```
api_connector.py
```

A robust, idempotent API connector that syncs GitHub issues to local JSON and CSV with pagination, deduplication, and comprehensive error handling.

```

"""

import csv
import json
import os
import time
from pathlib import Path
from typing import Set

import httpx
from pydantic import BaseModel

class GitHubIssue(BaseModel):
    """Represents a GitHub issue returned by the REST API."""
    id: int
    title: str
    state: str
    html_url: str
    created_at: str
    updated_at: str

GITHUB_BASE = "https://api.github.com"
TOKEN = os.environ.get("GITHUB_TOKEN", "")
HEADERS = {"Accept": "application/vnd.github+json"}
if TOKEN:
    HEADERS["Authorization"] = f"Bearer {TOKEN}"

def fetch_page(
    client: httpx.Client,
    endpoint: str,
    page: int,
    per_page: int = 100,
) -> list[dict]:
    """
    Fetch a single paginated page from the GitHub API.
    Returns a list of raw issue dicts on success,
    or an empty list on any recoverable or fatal error.
    """
    url = f"{GITHUB_BASE}{endpoint}"
    params = {"page": page, "per_page": per_page}

    try:
        resp = client.get(url, headers=HEADERS, params=params,
timeout=30.0)
    except httpx.TimeoutException as exc:
        print(f"[fetch_page] Timeout on page {page}: {exc}")
        return []
    except httpx.RequestError as exc:
        print(f"[fetch_page] Network error on page {page}: {exc}")

```

```

        return []

    # Rate-limit or abuse detection
    if resp.status_code in (429, 403):
        reset_header = resp.headers.get("x-ratelimit-reset")
        if reset_header:
            sleep_seconds = max(int(reset_header) -
int(time.time()), 1)
        else:
            sleep_seconds = 60
            print(f"[fetch_page] Rate limited. Sleeping
{sleep_seconds}s...")
            time.sleep(sleep_seconds)
            return fetch_page(client, endpoint, page, per_page)

    if resp.status_code == 401:
        print("[fetch_page] Authentication failed (401). Check
GITHUB_TOKEN.")
        return []

    if resp.status_code >= 500:
        print(f"[fetch_page] Server error {resp.status_code} on
page {page}. Skipping.")
        return []

    if resp.status_code >= 400:
        print(f"[fetch_page] Client error {resp.status_code}:
{resp.text[:200]}")
        return []

    content_type = resp.headers.get("content-type", "")
    if "application/json" not in content_type:
        print(f"[fetch_page] Unexpected content type:
{content_type}")
        return []

    try:
        data = resp.json()
    except json.JSONDecodeError as exc:
        print(f"[fetch_page] Malformed JSON on page {page}:
{exc}")
        return []

    return data if isinstance(data, list) else []

def sync_issues(
    repo: str,
    state: str = "open",
    max_pages: int = 10,
) -> list[GitHubIssue]:
    """

```

```

    Sync all issues from a public GitHub repository with
    pagination,
    deduplication, and error handling. Returns a list of validated
    GitHubIssue models.
    """
    endpoint = f"/repos/{repo}/issues?state={state}"
    seen_ids: Set[int] = set()
    all_issues: list[GitHubIssue] = []

    with httpx.Client() as client:
        for page in range(1, max_pages + 1):
            raw_items = fetch_page(client, endpoint, page,
per_page=100)
            if not raw_items:
                break

            for item in raw_items:
                issue_id = item.get("id")
                if issue_id in seen_ids:
                    continue
                seen_ids.add(issue_id)

            try:
                issue = GitHubIssue.model_validate(item)
                all_issues.append(issue)
            except Exception as exc:
                print(f"[sync_issues] Validation failed for
issue {issue_id}: {exc}")

            if len(raw_items) < 100:
                break # Last partial page means no more data

    return all_issues

def save_to_json(issues: list[GitHubIssue], path: Path) -> None:
    """Save issues to a JSON file with pretty printing."""
    data = [issue.model_dump() for issue in issues]
    path.write_text(json.dumps(data, indent=2), encoding="utf-8")
    print(f"Saved {len(issues)} records to {path}")

def save_to_csv(issues: list[GitHubIssue], path: Path) -> None:
    """Save issues to a CSV file with UTF-8 encoding."""
    if not issues:
        path.write_text("", encoding="utf-8")
        return
    rows = [issue.model_dump() for issue in issues]
    header = list(rows[0].keys())
    with path.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.DictWriter(fh, fieldnames=header)
        writer.writeheader()

```

```

        writer.writerows(rows)
    print(f"Saved {len(issues)} records to {path}")

def main() -> None:
    """
    Sync open issues from a public GitHub repository to JSON and
    CSV.
    """
    repo = "pallets/click"
    print(f"Syncing issues from {repo}...")
    issues = sync_issues(repo, state="open", max_pages=5)
    out_dir = Path("output")
    out_dir.mkdir(exist_ok=True)
    save_to_json(issues, out_dir / "issues.json")
    save_to_csv(issues, out_dir / "issues.csv")
    print("Done.")

if __name__ == "__main__":
    main()

```

Run it:

```

pip install httpx pydantic
python api_connector.py

```

You should see a series of fetch messages, then two save confirmations. Check the output/ directory. The JSON file preserves the full data model. The CSV is ready for Excel. And if you run the script again, you'll notice it doesn't duplicate any records — the deduplication set silently skips everything it already saw. That's idempotency in action.

Break and Fix

This section is the heart of the chapter. Every failure below is something that has happened to me, often more than once.

Break 1: Timeout

Symptom: Your script hangs for thirty seconds, then `httpx.TimeoutException` crashes the whole program. The `sync_issues` function aborts and you have no partial progress saved.

Root cause: Network jitter, an overloaded API server, or a client machine with a slow connection. You can't prevent timeouts, but you can prevent them from killing your job.

Fix: We already catch `httpx.TimeoutException` in `fetch_page` and return an empty list instead of propagating the exception. This lets the page loop continue to the next page, and because we deduplicate by ID, rerunning the script will pick up where it left off without duplicates. If you want retries, you can add a retry counter:

```
for attempt in range(3):
    try:
        resp = client.get(url, headers=HEADERS, params=params,
                           timeout=30.0)
        break
    except httpx.TimeoutException:
        if attempt == 2:
            print(f"Final timeout on page {page}")
            return []
        time.sleep(2 ** attempt)
```

Break 2: 429 Rate Limit

Symptom: You run the connector against a large repo and suddenly every request returns 429 Too Many Requests. GitHub's API starts rejecting you with an error message about abuse detection.

Root cause: You asked for pages too fast. Unauthenticated requests are limited to sixty per hour. Authenticated requests are limited to five thousand per hour, but aggressive loops can still trigger secondary abuse limits.

Fix: Our `fetch_page` checks for 429 and 403 abuse-detection codes, reads the `x-ratelimit-reset` header, computes how many seconds to sleep, and recursively retries after the delay. If you don't have a reset header, we default to sixty seconds. This graceful backoff means your pipeline takes longer but finishes instead of dying.

Break 3: Malformed JSON Response

Symptom: `resp.json()` raises `json.JSONDecodeError`. The server returned HTML, gzip gibberish, or a truncated payload.

Root cause: APIs sometimes send HTML error pages with JSON status codes, especially during outages. Or a proxy injects a banner into the response body.

Fix: We catch `json.JSONDecodeError` explicitly and return an empty list. We also verify the content-type header before parsing. If it's not `application/json`, we log the mismatch and bail out. Never assume a 200 response body contains valid JSON.

Break 4: Authentication Failure (401)

Symptom: All requests return 401 Unauthorized even though you set `GITHUB_TOKEN`.

Root cause: The token expired, it was revoked, or you accidentally included a newline character in the environment variable. Alternatively, you set an API key header when the endpoint expects a cookie.

Fix: In `fetch_page`, a 401 prints a clear message and returns an empty list, which aborts the sync loop gracefully. For debugging, you can add a line like:

```
if resp.status_code == 401:
    print(f"Token snippet: {TOKEN[:4]}...{TOKEN[-4:]}")
```

This lets you verify that the token was actually loaded. Then rotate the token and rerun. There's no programmatic fix for a dead token except human intervention.

Break 5: Schema Drift

Symptom: Pydantic raises `ValidationError` on every record. The API added a required field. Or it changed `id` from an integer to a string. Or it nested title inside a new fields object.

Root cause: APIs evolve. Versioned APIs help, but not every provider respects semantic versioning. GitHub's v3 API is reasonably stable, but even stable APIs change. If you hardcoded every field as required, a single new nullable field will break you.

Fix: Make fields optional when they aren't critical. Use Pydantic's `model_validate` in a try/except block, log the offending payload, and skip the bad record rather than crashing the entire sync. Here's a more resilient model pattern:

```
from typing import Optional

class GitHubIssue(BaseModel):
    """Represents a GitHub issue with safe defaults for evolving fields."""
```

```
id: int
title: str = "Untitled"
state: str = "unknown"
html_url: str = ""
created_at: str = ""
updated_at: str = ""
body: Optional[str] = None # New field, optional
```

By defaulting non-critical fields and making brand-new fields optional, you buy yourself time to adapt while the sync keeps running.

Try It Now

Theory is cheap. Practice is where the learning happens. Try one of these three variations.

Variation 1: OAuth2 with Notion or Airtable

Pick a service that uses OAuth2, such as Notion or Airtable. Register an integration, obtain an access token, and adapt `fetch_page` to hit their endpoints. You'll need to change the base URL, the headers (Notion wants `Authorization: Bearer <token>` and a `Notion-Version` header), and the Pydantic model to match their schema. This teaches you the single most important API integration skill: reading someone else's documentation and translating it into headers and models.

Variation 2: Delta Sync

Our current connector fetches every open issue every time. For a mature repo, that's wasteful. Implement a delta sync: store the `updated_at` timestamp of the last successful run in a small sidecar file like `.last_sync`. On the next run, pass `since=<timestamp>` to GitHub's API to fetch only issues updated after that time. Then update `.last_sync` when the job finishes successfully. This turns a 1,000-request job into a 3-request job.

Variation 3: Schema Validation Step

Before the sync begins, fetch a known good record from the API and validate it against your Pydantic model. If validation fails at the schema-check stage, abort with a loud error before you waste time

paginating through fifty pages of doomed data. This “pre-flight check” is common in production ETL pipelines because it fails fast when the upstream API changes.

The OpenClaw Way

OpenClaw provides a structured Tool abstraction that simplifies much of what we built by hand. In OpenClaw, you define a connector as a tool with a schema, and the framework handles retries, exponential backoff, and response parsing for you. Here’s how the concept maps: instead of writing a raw `fetch_page` function, you’d wrap it in an OpenClaw Tool with `@tool` and specify `retry=3` and `backoff="exponential"` in the decorator. The pagination loop remains your logic, but OpenClaw’s `PaginatedTool` mixin can automatically follow Link headers or cursor tokens if you configure it with the right field names. Where our raw script prints to `stdout`, OpenClaw logs every request to a structured observability stream, which means you get latency percentiles and error-rate dashboards for free. The trade-off is that you must learn OpenClaw’s conventions for header injection and rate-limit handling. The benefit is that your sixty lines of plumbing become twelve lines of business logic, and the framework ensures you don’t forget edge cases like timeout handling or content-type verification.

The Hermes Agent Way

Hermes Agent takes a more conversational approach. Rather than writing a standalone script, you open a Hermes session and ask it to “sync all open issues from pallets/click to a local CSV, deduplicate by ID, and handle rate limits.” Hermes already has built-in tools for HTTP requests, file writing, and CSV generation. It will construct the request headers, loop through the pages, and write the output using its own internal code executor. The difference is that you’re not explicitly writing the Pydantic models or the pagination loop — though you might prompt Hermes to “validate every record against a schema before adding it to the CSV.” This is powerful for rapid prototyping but requires careful prompt engineering. If you don’t specify idempotency or deduplication, Hermes might happily write duplicates. If you don’t mention rate limiting, it might blast through sixty requests and hit the wall. The Hermes variant of this project teaches you that agent behavior is a function of instructions. The more precise your instructions, the more reliable the output. For repeatable production

jobs, many Hermes users export the generated script after the first successful run and turn it into a scheduled Python file — effectively bridging the conversation-first and code-first worlds.

What You Built

In this chapter, you built a production-grade API connector from pure Python:

- A paginated fetch loop that respects GitHub's `?page=` and `?per_page=` parameters.
- An authentication header system that gracefully degrades when no token is present.
- A Pydantic data model that validates every inbound record and rejects schema drift before it corrupts your dataset.
- An idempotency mechanism that deduplicates records by primary key, making the sync safe to rerun.
- Rate-limit detection with header-driven backoff, preventing your script from being banned.
- Comprehensive error handling for timeouts, network failures, 4xx client errors, 5xx server errors, and malformed JSON payloads.
- Dual output writers that serialize clean records to both JSON and CSV.
- A `main()` entry point that orchestrates the entire flow into a single command.

You also learned how to break each of these protections deliberately, how to recover from real-world API failures, and how to extend the pattern into OAuth2 integrations, delta synchronization, and pre-flight schema checks.

Cost for this chapter: Approximately \$0.00. This project uses only local computation and free public API endpoints. If you adapt it to a paid API later, monitor your request volume — the pagination loop can rack up charges quickly if `max_pages` is uncapped.

Chapter 7: Project 7 — Coding Agent: Helper and Refactor

The agent reads your codebase and suggests: “That function is 47 lines. Here’s how to split it.”

A few years ago I trusted an automated refactoring tool with a production Django module. It was a Friday afternoon. The tool split a thirty-line view into seven micro-functions, renamed half the variables to “improve clarity,” and deleted every inline comment because they weren’t included in the diff context it generated. I glanced at the green diff, saw that the tests passed locally — they didn’t actually cover the renamed paths — and merged. Production broke within the hour. A subtle name-shadowing bug, introduced by the split, caused user authentication to fail for anyone with an email containing a hyphen. It took me three hours to revert, apologize to the team, and question every automation tool I’d ever loved.

That experience burned a rule into my brain: a coding agent must be conservative, exact, and absolutely paranoid about tests. Fast refactoring is dangerous refactoring. Slow, verified refactoring is the only kind you should let near a codebase that matters. If the agent can’t prove a change is safe, it doesn’t make the change. Full stop.

Coding agents are seductive and terrifying. They promise to eliminate drudgery: convert print statements to logging, add missing docstrings, rename variables for consistency, split bloated functions. But code is not prose. A single misplaced parenthesis changes the meaning of a program. A renamed variable can break twelve tests. A deleted comment might be the only documentation for a business rule that cost someone a lawsuit. Language models, for all their brilliance, are not compilers. They approximate syntax. That’s why this chapter is not about building the flashiest refactoring robot. It’s about building the most careful one.

We’ll construct an agent that reads a Python project, understands its structure, proposes mechanical changes, and never applies a patch it can’t verify. We’ll use the standard library almost exclusively: `pathlib` to walk files, `re` to find patterns, `ast` to understand function boundaries, `difflib` to show changes, and `subprocess` to run tests. We’ll sandbox everything inside a temporary directory so your real code stays

untouched until the tests scream green. By the end, you'll have an agent that can convert an entire project's print debugging into proper logging — safely.

Reading the Codebase with `pathlib` and `glob`

Before the agent can refactor a single line, it needs to know what exists. A Python project is a forest of modules, tests, and configuration files. Our first tool is a file scanner that maps the territory.

We'll use the `glob` module together with `pathlib.Path`. `glob` supports recursive patterns with `**`, and `pathlib` gives us rich, cross-platform objects instead of raw strings. Here's the reconnaissance function:

```
from pathlib import Path
import glob

def find_python_files(project_dir: Path) -> list[Path]:
    """Return a sorted list of all Python files under project_dir
    using glob."""
    pattern = str(project_dir / "**" / "*.py")
    return sorted(Path(p) for p in glob.glob(pattern,
recursive=True))
```

If `project_dir` is a `pathlib.Path` pointing at your repository, `project_dir / "**" / "*.py"` builds a pattern like `/home/you/code/**/*py`. `glob.glob(..., recursive=True)` expands the `**` into every subdirectory. We convert the strings back into `Path` objects because `Path` handles slashes, extensions, and relative directories without us writing brittle string logic.

Notice the `sorted(...)` call. Deterministic order matters when you're applying patches. If the agent processes files in random order and one patch depends on another, you'll chase ghosts. Sorting keeps the behavior repeatable, which makes debugging possible.

Searching for Patterns with Regular Expressions

Once we have a list of files, the agent starts hunting. In this project, the mission is simple: find every `print(...)` statement that should become a `logging.info(...)` call. Regular expressions are a blunt instrument for

parsing code — they can't tell whether print lives inside a string literal or a comment — but they're fast and good enough for mechanical refactoring when used with care.

```
import re

def find_print_statements(source: str) -> list[tuple[int, str]]:
    """Return line numbers and lines containing bare print
    calls."""
    matches = []
    for lineno, line in enumerate(source.splitlines(), 1):
        if re.search(r"\bprint\s*\(", line):
            matches.append((lineno, line))
    return matches
```

The pattern `\bprint\s*\((` looks for the word `print` followed by an open parenthesis, with optional whitespace in between. The word boundary `\b` prevents matching a variable named `my_print_buffer`. It's not perfect — it will still match `print` inside a multiline string — but for our conservative agent, that's acceptable. The agent uses this list to decide which files need attention, not to perform the replacement directly.

If you're building a production-grade agent, you'll want to combine regex with the AST (which we'll cover next) to confirm that a `print` node is actually a call expression. But for a first pass, regex narrows the search from “read every file” to “these eight lines in these three files.”

Seeing Structure with the AST

Regular expressions see characters. The Abstract Syntax Tree sees meaning. Python's `ast` module parses source code into a tree of nodes — functions, imports, loops, assignments — and lets us inspect them programmatically. This is how our agent understands structure instead of guessing from text.

```
import ast

def analyze_functions(source: str, filename: str = "<unknown>") ->
list[dict]:
    """Parse source and return metadata for each function
    definition."""
    tree = ast.parse(source, filename=filename)
    functions = []
    for node in ast.walk(tree):
        if isinstance(node, ast.FunctionDef):
            length = node.end_lineno - node.lineno + 1 if
```



```

node.end_lineno else 1
        functions.append({
            "name": node.name,
            "start_line": node.lineno,
            "end_line": node.end_lineno,
            "length": length,
            "docstring": ast.get_docstring(node),
        })
    return functions

def analyze_imports(source: str, filename: str = "<unknown>") ->
list[str]:
    """Return a list of top-level imports and from-imports as
    strings."""
    tree = ast.parse(source, filename=filename)
    imports = []
    for node in ast.walk(tree):
        if isinstance(node, (ast.Import, ast.ImportFrom)):
            imports.append(ast.unparse(node))
    return imports

```

ast.parse turns a string of Python into a tree. ast.walk traverses every node. When we hit an ast.FunctionDef, we grab its name, its starting line, and — if available — its ending line. The length calculation tells the agent whether a function is a three-line helper or a fifty-line monster that needs splitting. ast.get_docstring checks whether the function already has documentation.

Why does this matter? Because a refactoring agent that only knows about print statements is a text replacement script. An agent that also knows function boundaries can decide things like: “This add function has no docstring and contains a print. I should replace the print *and* suggest a docstring.” The AST gives the agent context, and context is what separates a reckless search-and-replace from a helpful assistant.

The analyze_imports function tells us whether logging is already imported. If it isn’t, our patch must prepend import logging before any code that calls logging.info.

Generating Patches with difflib

Once the agent knows what to change, it shouldn't touch the file yet. First, it shows its work. A patch is an audit trail: it lets a human (or a later script) verify exactly what changed before anything is committed. We'll use difflib from the standard library to generate a unified diff, the same format git diff produces.

```
import difflib

def generate_unified_diff(old_text: str, new_text: str, filename:
str) -> str:
    """Return a unified diff between old_text and new_text."""
    old_lines = old_text.splitlines(keepends=True)
    new_lines = new_text.splitlines(keepends=True)
    diff = difflib.unified_diff(
        old_lines, new_lines,
        fromfile=filename, tofile=filename
    )
    return "".join(diff)
```

The `old_text` is the original source read from disk. The `new_text` is the source after our mechanical transformation. `splitlines(keepends=True)` preserves newlines so the diff doesn't invent phantom changes on every line. The output looks like this:

```
--- calculator.py
+++ calculator.py
@@ -1,3 +1,4 @@
+import logging
def add(a, b):
    # simple addition
    result = a + b
-    print(f"Adding {a} + {b} = {result}")
+    logging.info(f"Adding {a} + {b} = {result}")
    return result
```

Even if the agent runs in an automated pipeline, generating and logging this diff is non-negotiable. When something breaks at 3 AM, the diff is the first thing you'll read.

Applying Patches Safely

Generating a diff is theater. Applying a patch is surgery. The golden rule: never use fuzzy matching. If the agent's `old_string` doesn't match the file byte-for-byte, the agent aborts. No "close enough." No "it was probably this line." Exact match or nothing.

```
def safe_apply_patch(filepath: Path, old_string: str, new_string:
str) -> bool:
    """Replace old_string with new_string only if old_string
exists exactly once."""
    content = filepath.read_text(encoding="utf-8")
    if old_string not in content:
        logging.error("old_string not found in %s – aborting
patch.", filepath)
        return False
    if content.count(old_string) > 1:
        logging.error(
            "old_string is ambiguous (%d occurrences in %s) –
aborting patch.",
            content.count(old_string), filepath
        )
        return False
    content = content.replace(old_string, new_string, 1)
    filepath.write_text(content, encoding="utf-8")
    logging.info("Patch applied to %s", filepath)
    return True
```

This function checks two guardrails. First: is the target string present at all? Second: is it unique? If a snippet appears twice in the same file, replacing it blindly might hit the wrong function. In that case, we abort and ask for more context — perhaps by expanding `old_string` to include surrounding lines or the function signature.

In our main example, we'll pass the entire original file content as `old_string` and the entire transformed content as `new_string`. That makes the match trivially unique. For snippet-level patches, the same function works as long as the snippet is long enough to be unambiguous.

Running Tests Before and After

A coding agent without tests is a demolition crew with a permit it wrote itself. We run the test suite before we touch anything. If it's red, we stop — why would you refactor on top of a broken foundation? If it's green, we apply our patch, then run the suite again. Green twice means proceed. Red the second time means revert and investigate.

```
import subprocess
import sys

def run_tests(project_dir: Path) -> tuple[bool, str]:
    """Run unittest discovery and return (passed,
    combined_output)."""
    try:
        result = subprocess.run(
            [sys.executable, "-m", "unittest", "discover",
            "-s", str(project_dir), "-v"],
            capture_output=True,
            text=True,
            timeout=60,
        )
        passed = result.returncode == 0
        output = result.stdout + result.stderr
    except subprocess.TimeoutExpired:
        passed = False
        output = "Test suite timed out after 60 seconds."
    except Exception as exc:
        passed = False
        output = f"Failed to run tests: {exc}"
    return passed, output
```

The `subprocess.run` call executes `python -m unittest discover` inside the project directory. `capture_output=True` and `text=True` let us inspect the results. The `timeout=60` guard prevents a hanging test from无限期 blocking the agent. If the test runner isn't `unittest`, you can swap the command for `pytest` or whatever your project uses; the pattern stays the same.

Notice the explicit return signature: a boolean and a string. The boolean answers “did we pass?” The string preserves the evidence. If the agent needs to report why it aborted, it has the full test output to quote.

Sandboxing in a Temporary Directory

Even with exact patches and test verification, never let an agent experiment on the original repository. Our sandbox strategy is simple: copy the project into a temporary directory, run the agent there, and only promote the results back to the real workspace if every test passes.

The standard library's `tempfile.TemporaryDirectory` gives us a self-cleaning workspace. It exists as long as the `with` block is open, then vanishes. If the agent goes rogue, the worst it can destroy is a disposable copy.

To tie the whole workshop together, we need a sample project. The `create_sample_project` function writes a tiny calculator module and a matching test file into our sandbox:

```
import tempfile

def create_sample_project(target_dir: Path) -> None:
    """Write a toy Python project into target_dir for
    demonstration."""
    target_dir.mkdir(parents=True, exist_ok=True)
    calc = target_dir / "calculator.py"
    calc.write_text(
        'def add(a, b):\n'
        '    # simple addition\n'
        '    result = a + b\n'
        '    print(f"Adding {a} + {b} = {result}")\n'
        '    return result\n\n'
        'def subtract(a, b):\n'
        '    result = a - b\n'
        '    print(f"Subtracting {a} - {b} = {result}")\n'
        '    return result\n',
        encoding="utf-8",
    )
    test_file = target_dir / "test_calculator.py"
    test_file.write_text(
        'import unittest\n'
        'from calculator import add, subtract\n\n'
        'class TestCalculator(unittest.TestCase):\n'
        '    def test_add(self):\n'
        '        self.assertEqual(add(2, 3), 5)\n\n'
        '    def test_subtract(self):\n'
        '        self.assertEqual(subtract(5, 2), 3)\n\n'
        'if __name__ == "__main__":\n'
        '    unittest.main()\n',
```

```

        encoding="utf-8",
    )

```

This sample is deliberately small so you can trace every step. The add function has an inline comment and a print statement. The subtract function also has a print. Neither has a docstring. Our agent will find the prints, add import logging, replace the prints with logging.info, verify syntax, and run the tests.

Here is the main orchestrator that performs the refactor inside the sandbox:

```

import logging
import py_compile

def check_syntax(filepath: Path) -> bool:
    """Validate that a Python file compiles without syntax
    errors."""
    try:
        py_compile.compile(filepath, doraise=True)
    except py_compile.PyCompileError as exc:
        logging.error("Syntax error in %s: %s", filepath, exc)
        return False
    return True

def refactor_print_to_logging(project_dir: Path) -> bool:
    """Refactor bare print() calls to logging.info() inside
    project_dir."""
    py_files = find_python_files(project_dir)
    if not py_files:
        logging.warning("No Python files found in %s",
            project_dir)
        return False

    logging.info("Running tests BEFORE refactor...")
    passed_before, output_before = run_tests(project_dir)
    if not passed_before:
        logging.error("Tests failed before refactor. Aborting.
        \n%s", output_before)
        return False
    logging.info("Tests passed before refactor.")

    for py_file in py_files:
        if "test_" in py_file.name:
            continue # Don't refactor test files in this demo

        source = py_file.read_text(encoding="utf-8")
        original_source = source

        if not find_print_statements(source):

```

```

        continue

    imports = analyze_imports(source, filename=str(py_file))
    has_logging = any("logging" in imp for imp in imports)
    if not has_logging:
        source = "import logging\n" + source

    new_lines = []
    for line in source.splitlines(keepends=True):
        match = re.search(r"^(\s*)print\s*\(((.*)\))\s*$", line)
        if match:
            indent = match.group(1)
            args = match.group(2)
            new_lines.append(f"{indent}logging.info({args})\n")
        else:
            new_lines.append(line)
    new_source = "".join(new_lines)

    if new_source != original_source:
        diff = generate_unified_diff(original_source,
new_source, py_file.name)
        print(f"\n--- Proposed diff for {py_file} ---")
        print(diff)
        print("--- End diff ---\n")

        if not safe_apply_patch(py_file, original_source,
new_source):
            return False

        if not check_syntax(py_file):
            return False

    logging.info("Running tests AFTER refactor...")
    passed_after, output_after = run_tests(project_dir)
    if not passed_after:
        logging.error("Tests failed after refactor.\n%s",
output_after)
        return False
    logging.info("Tests passed after refactor.")
    return True

def main() -> None:
    """Run the coding agent demo in a temporary sandbox."""
    logging.basicConfig(level=logging.INFO, format="%(levelname)s:
%(message)s")
    with tempfile.TemporaryDirectory() as tmpdir:
        sandbox = Path(tmpdir) / "sample_project"
        create_sample_project(sandbox)
        logging.info("Created sample project in %s", sandbox)
        ok = refactor_print_to_logging(sandbox)

```

```
        if ok:
            logging.info("Refactor completed successfully.")
        else:
            logging.error("Refactor failed.")

if __name__ == "__main__":
    main()
```

Let's walk through the flow. `main()` creates a temporary directory, writes the sample project, and hands it to `refactor_print_to_logging`. That function first runs the tests to establish a green baseline. Then it iterates over every Python file that isn't a test. For each file, it reads the source, checks for logging imports, adds one if missing, and replaces each `print(...)` line with `logging.info(...)`.

Before writing anything back, it generates a unified diff and prints it to the console. Then it calls `safe_apply_patch` with the full original source as `old_string` and the transformed source as `new_string`. Because the old string is the entire file, the uniqueness check is trivial. After the patch is written, `check_syntax` runs `py_compile.compile` to catch any syntax errors the transformation might have introduced. Finally, the tests run again. If they pass, the refactor succeeds. If any step fails, the function returns `False` and the agent aborts.

Break and Fix: Five Ways This Project Can Fail

Every agent breaks eventually. The question is whether it breaks safely or breaks catastrophically. Here are five failure modes specific to coding agents, with symptoms, root causes, and fixes.

Break 1: Patch Doesn't Apply (`old_string` Mismatch)

You ask the agent to refactor a file, but it generated the patch based on an outdated view of the code. Maybe you edited the file while the agent was reasoning, or the model hallucinated slightly different whitespace. The `safe_apply_patch` function logs:

```
ERROR:old_string not found in calculator.py – aborting patch.
```

Root cause: The agent's `old_string` doesn't match the file exactly. Even a single trailing space or tab-to-space conversion will break an exact-match patch.

Fix: Never guess. The exact-match check is doing its job. When a patch fails, the agent should re-read the current file contents, regenerate the replacement from the fresh text, and try again. You can also normalize line endings to `\n` before comparison, but never strip meaningful indentation. In practice, this means your agent loop should treat a patch failure as a signal to refresh its context and re-reason, not as an error to ignore.

Break 2: Tests Fail After Refactor

The agent replaced `print` with `logging.info`, but one of your tests was using `unittest.mock.patch("builtins.print")` to verify that the function wrote to standard output. After the refactor, the mock never fires, and the assertion fails.

Root cause: The change altered observable behavior — the side effect moved from `stdout` to the logging stream. The tests weren't just checking return values; they were checking implementation details.

Fix: Run tests before and after. Our `refactor_print_to_logging` already does this. If the after-test fails, abort immediately and report the failure. In a real pipeline, you might also check whether any test files mock `builtins.print` before applying the patch, and skip those files or emit a warning. The broader lesson: refactoring is only safe when the test suite accurately describes the contract of your code.

Break 3: Agent Deletes Comments While Refactoring

You inspect the diff and realize the agent's `old_string` omitted the line `#` simple addition inside the `add` function. The patch replaces the function body without the comment, effectively deleting it.

Root cause: The model's context window didn't include the comment, or the model evaluated the comment as irrelevant and excluded it from the replacement block.

Fix: Expand the context included in `old_string` so that comments are part of the unique match. Better yet, after applying any patch, do a comment audit: parse the original file and the patched file with a simple regex for comments (`# .*`), and ensure the count hasn't dropped. If comments vanished, reject the patch. Here's a tiny guard you can add:

```
def count_comment_lines(source: str) -> int:
    return len([line for line in source.splitlines() if
re.search(r"#.*", line)])
```

If `count_comment_lines(original) > count_comment_lines(patched)`, log a warning and abort.

Break 4: Introduces Syntax Error

The model generated a replacement line with a missing closing parenthesis: `logging.info(f"Adding {a} + {b} = {result}")`. The agent applied it, and the test runner crashed with a `SyntaxError` before it could even start.

Root cause: Large language models are approximate pattern matchers. They do not track parenthesis balance the way a parser does. A long f-string with nested braces is exactly the kind of construction that trips them up.

Fix: This is why `check_syntax` exists in our pipeline. After every patch application, we run `py_compile.compile(filepath, doraise=True)`. If it raises `PyCompileError`, the patch is immediately rejected and the agent aborts. This catch is non-negotiable. Never let a coding agent proceed past a syntax error.

Break 5: Over-Eager Refactoring (Splits One-Liners)

You ask the agent to “simplify functions,” and it decides that your two-line subtract function should be decomposed into a private `_calculate_difference` helper plus a wrapper. The resulting code is technically correct and the tests pass, but the codebase is now harder to read.

Root cause: The agent’s instructions were too vague. “Simplify” is subjective. Without a length threshold or explicit constraints, the model defaults to doing more work rather than less.

Fix: Add guardrails in code, not just in prompts. Our AST analysis already measures function length. You can enforce a rule: if `length <= 3`, skip splitting suggestions entirely. A concise helper function is not a problem that needs solving. Prompt engineering helps, but code-level gates are what prevent a 2 AM run from turning your codebase into abstract art.

Try It Now: Three Variations

Once the basic agent works, stretch it. These exercises turn the scaffold into real tooling.

Variation 1: Generate Docstrings for Undocumented Functions

Extend the AST analysis to find every function where docstring is `None`. For each, generate a short docstring patch that inserts a triple-quoted description just below the `def` line. Use the function name and argument list to build a template: `"""Return the result of {name} on {args}."""`. Apply it with `safe_apply_patch`, run tests, and review the diffs. If a function already has a docstring, skip it.

Variation 2: Find Security Issues (Hardcoded Keys)

Write a scanner that searches for hardcoded secrets. Use regex patterns like `[A-Z_]+\s*=\s*["']\w{16,}["']` to catch API keys, and look for variable names like `password`, `secret`, or `token` assigned to string literals. Report the filename and line number for each hit. Do not auto-patch these — security findings should always be reviewed by a human — but do generate a markdown report of everything the agent found.

Variation 3: Generate README from Structure

Combine `find_python_files`, `analyze_functions`, and `analyze_imports` to build a structural summary of the project: modules, public functions per module, and external dependencies. Feed this summary into a simple Jinja2 template (or even a Python f-string) to generate a `README.md` with a table of contents and a function index. This isn't refactoring; it's documentation generation, and it uses the exact same filesystem and AST tools.

The OpenClaw Way

OpenClaw handles the scaffold we just built with built-in tool definitions. In OpenClaw, you would register `@tool` functions for `list_files`, `read_file`, `search_regex`, and `run_tests`. The framework automatically injects the filesystem and subprocess capabilities into the agent's context. OpenClaw also provides a sandbox workspace that

copies the target directory into a temporary location before any mutation tool runs, which saves you from writing the tempfile boilerplate manually. The equivalent of our entire script becomes a short configuration file: define the tools, set the system prompt to “Refactor prints to logging and always run tests before and after,” and let OpenClaw manage the loop. The trade-off is that you must learn OpenClaw’s tool schema and permission model. The benefit is that retry logic, diff rendering, and test output formatting are handled for you.

The Hermes Agent Way

Hermes Agent takes a conversation-first approach. Instead of writing a Python script that drives the refactor, you start Hermes in the project directory and type a command like: “Find all print statements in the src folder, replace them with logging.info, and run the tests before and after.” Hermes uses its built-in file system tools to explore the codebase, its code execution tool to run pytest or unittest, and its diff viewer to show you exactly what changed. It will pause and ask for approval before applying any mutation, making it ideal for high-stakes refactoring where human review is mandatory. The trade-off is less deterministic automation — Hermes might choose a slightly different strategy each time depending on how you phrase the request. The benefit is that you don’t write any boilerplate; the agent is the interface.

What You Built

In this chapter, you built a careful, conservative coding agent from standard library parts:

- A filesystem scanner using pathlib and glob that maps every Python file in a project.
- A regex search tool that locates patterns like `print(...)` across the codebase.
- An AST analyzer that measures function lengths, detects missing docstrings, and inventories imports.
- A diff generator using difflib that produces human-readable unified diffs before any change is applied.

- A safe patch applier that verifies exact, unique matches of `old_string` before replacing anything.
- A test runner using subprocess that enforces a green baseline before refactoring and validates green after.
- A sandbox workflow using `tempfile.TemporaryDirectory` that isolates experiments from production code.
- A complete `main()` demonstration that refactors print statements to `logging.info` in a sample project and verifies the result with tests.

You also learned five specific failure modes — patch mismatch, test regression, comment deletion, syntax errors, and over-eager splitting — and how to guard against each one with code-level checks rather than blind trust in the model.

Cost for this chapter: Approximately \$0.00 if you run the pure Python version with a local model and use only standard library tools. If you integrate an LLM to generate docstrings or security analysis, a single pass over a 500-line codebase costs roughly \$0.01–\$0.03 with GPT-4o mini.

Chapter 8: Project 8 — Personal Knowledge Base Agent

“The hardest information to find is the information you already have.”

The Notebook That Vanished

I remember the Tuesday morning I needed the decision document. Six months earlier, my team had debated pricing tiers for three hours in a cramped conference room. I’d taken detailed notes in a markdown file, capturing the reasoning, the dissent, and the final call. I knew the answer was in my notes somewhere. I just didn’t know where.

I opened my laptop, navigated to my notes folder, and started searching. I tried `grep -r "pricing"` and got back two hundred matches across random files. I tried "decision" and got fifty irrelevant hits. I tried "enterprise tier" and found only a passing mention in an unrelated project summary. I scrolled through filenames I didn’t recognize—`random_thoughts_2025.md`, `meeting_backup.txt`,

ideas_old.md—and opened them one by one. Nothing. After forty minutes of growing panic, I gave up and rewrote the strategy from memory. I sent it to my manager and spent the next two weeks quietly worried that I’d forgotten a key constraint.

Then, during an entirely unrelated search two weeks later, I found it. The file was called random_thoughts_2025.md. The heading said “Afternoon musings.” The word “pricing” never appeared in the decision paragraph; instead, the notes referred to it as “what we ended up charging.” My grep had no chance. The document was invisible to any search I could think to run.

That experience taught me something critical about knowledge work: we rarely forget what we know. What we forget is where we put it, and—more importantly—the exact words we used when we wrote it down. Keyword search is a blunt instrument for a subtle problem. If you can’t remember the precise term the author chose, grep might as well be staring at a brick wall.

This chapter changes that. We are going to build a Personal Knowledge Base Agent that reads every text and markdown file in a folder; turns the content into mathematical fingerprints called embeddings, stores those fingerprints in a local SQLite database, and retrieves information by *meaning* instead of spelling. Ask it, “What did we decide about pricing?” and it will find the chunk that says “what we ended up charging,” because embeddings understand that those phrases live in the same neighborhood of meaning.

This is also the first all-local project in this book. After you pull the embedding model once, every search, every index update, and every answer costs exactly zero dollars. No API keys. No metered tokens. No surprise bill at the end of the month. Let’s build something that never loses a thought again.

Why Keyword Search Fails—and Why RAG Wins

Keyword search works when you know what you are looking for. If your file is named Q3_budget_final.md and you search for budget, you will probably find it. But knowledge work is messier than that. Your past self used different vocabulary than your present self. You wrote

“onboarding flow” instead of “new user experience.” You recorded a decision under “Tuesday standup” instead of “architecture decision record.”

Language models understand meaning, not just characters. An embedding model takes a sentence and compresses its semantic content into a vector—a long list of numbers. Two sentences that mean similar things end up as vectors that point in similar directions in high-dimensional space. That geometric closeness is what powers semantic search.

RAG, or Retrieval-Augmented Generation, is the pattern that puts semantic search to work for answering questions. The idea is simple but powerful: when a user asks a question, don’t ask the language model to answer from its training memory. Instead, search your private documents for the most relevant passages, feed those passages into the prompt, and instruct the model to answer using *only* what it sees in those passages. The result is an answer grounded in your actual documents, with traceability back to the source text.

In this project, we will build the entire pipeline by hand: parsing files, splitting them into overlapping chunks, embedding each chunk with a local Ollama model, storing the vectors in SQLite using nothing but the Python standard library and NumPy, searching with cosine similarity, and generating a grounded answer with citations. No vector database frameworks. No cloud dependencies. Just Python, Ollama, and your files.

Parsing Your Documents

Before we can search meaning, we need text. For this project, we will read .txt and .md files using pure Python. No external parsers are required—just pathlib and careful error handling for encoding quirks.

Here is the parsing function:

```
from pathlib import Path

def parse_file(file_path: Path) -> str:
    """Read a .txt or .md file and return its contents as a
    string.

    Tries UTF-8 first, falls back to latin-1, and returns an empty
    string if the file cannot be read.
    """
    try:
```

```

        return file_path.read_text(encoding="utf-8")
    except UnicodeDecodeError:
        try:
            return file_path.read_text(encoding="latin-1")
        except Exception as exc:
            print(f"Failed to read {file_path}: {exc}")
            return ""
    except Exception as exc:
        print(f"Failed to read {file_path}: {exc}")
        return ""

```

Notice the nested try/except. Text files claim to be UTF-8, but older notes sometimes carry weird byte sequences. Crashing the entire indexer because one backup file has a bad character is unacceptable. We catch the error, try a more permissive encoding, and if that fails too, we log the failure and move on. The agent should be resilient; one bad apple should not spoil the batch.

Chunking with Overlap

You cannot embed an entire novel and compare it to a seven-word question. The geometry does not work well at that scale. Instead, we break documents into smaller passages called *chunks*. A chunk should be large enough to preserve local context, but small enough that its embedding represents a focused idea.

There are three common strategies. **Fixed-size chunking** splits text every N characters. It is simple and fast, but it can cut sentences in half, leaving a fragment like “decided to move forward with the” at the end of one chunk. **Semantic chunking** tries to split at topic boundaries, usually by analyzing sentences for shifts in meaning. It is elegant but requires extra libraries and heuristics. For this project, we will use a **paragraph-aware overlap strategy**: we march through the text in roughly fixed-size windows, but we try to break at word boundaries, and each new chunk overlaps the previous one by a configurable number of characters. The overlap ensures that concepts sitting at a chunk boundary are not split in half across the divide.

Here is the implementation:

```

def chunk_text(text: str, chunk_size: int = 512, overlap: int = 50) -> list[str]:
    """Split text into overlapping chunks of roughly chunk_size
    characters.

```

The overlap parameter controls how many trailing characters from the


```

    previous chunk are included at the start of the next chunk.
This helps
    preserve context across chunk boundaries.
    """
    chunks = []
    start = 0
    text = text.strip()
    if not text:
        return chunks

    while start < len(text):
        end = start + chunk_size
        if end < len(text):
            # Try to break at a space so we don't slice words
            while end > start and text[end] != " ":
                end -= 1
            if end == start:
                # No space found within this window; hard break
                end = start + chunk_size
        chunk = text[start:end].strip()
        if chunk:
            chunks.append(chunk)
        start = end - overlap
        if start <= 0:
            start = end
        if end >= len(text):
            break

    return chunks

```

The overlap parameter is the secret weapon. Without it, a sentence that spans the boundary between chunk one and chunk two might lose its meaning in both places. With a fifty-character overlap, both chunks retain enough surrounding text for the embedding to capture the full idea. If you index technical notes with terse bullet points, increase the overlap. If you index verbose journal entries, you can reduce it.

From Text to Vectors: Local Embeddings with Ollama

Now that we have chunks, we need to turn them into vectors. We will use Ollama running locally with the nomic-embed-text model, a small, capable embedding model that fits easily on a laptop CPU. If you have not pulled it yet, run this once in your terminal:

```
ollama pull nomic-embed-text
```

Ollama exposes an HTTP API, just like the one we used in Chapter 1. The endpoint we need is `/api/embeddings`. We send the model name and the text; Ollama returns a JSON object containing the embedding vector.

```
import httpx

OLLAMA_EMBED_URL = "http://localhost:11434/api/embeddings"
EMBED_MODEL = "nomic-embed-text"

def get_embedding(text: str) -> list[float]:
    """Request an embedding vector from the local Ollama server.

    Returns an empty list if the request fails so the caller can
    decide whether to skip the chunk or abort indexing.
    """
    try:
        response = httpx.post(
            OLLAMA_EMBED_URL,
            json={"model": EMBED_MODEL, "prompt": text},
            timeout=60.0,
        )
        response.raise_for_status()
        payload = response.json()
        embedding = payload.get("embedding")
        if not isinstance(embedding, list) or len(embedding) == 0:
            print("Embedding response missing vector data.")
            return []
        return embedding
    except httpx.HTTPStatusError as exc:
        print(f'Ollama returned an error: {exc.response.status_code}')
        return []
    except Exception as exc:
        print(f'Embedding request failed: {exc}')
        return []
```

The error handling here is aggressive on purpose. If Ollama is not running, if the model is not pulled, or if the server is overloaded, we catch the exception, print a useful message, and return an empty list. The indexing loop can then decide whether to skip that chunk or stop entirely. I prefer skipping: one bad chunk should not prevent the rest of your knowledge base from becoming searchable.

A Vector Database in Pure Python

Most tutorials at this point tell you to install Chroma, FAISS, or Pinecone. We are not going to do that. For a personal knowledge base of a few thousand chunks, SQLite plus NumPy is more than fast enough, and it keeps the project transparent and dependency-light.

We will create a single table with five columns: an auto-incrementing ID, the source filename, the chunk index within that file, the raw text, and the embedding as a binary blob. NumPy's `.tobytes()` serializes a float32 array into something SQLite can store, and `np.frombuffer()` brings it back to life later.

```
import sqlite3
import numpy as np

DB_PATH = "kb.sqlite"

def init_db(db_path: str) -> None:
    """Create the chunks table if it does not already exist."""
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()
    cursor.execute(
        """
        CREATE TABLE IF NOT EXISTS chunks (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            source TEXT NOT NULL,
            chunk_index INTEGER NOT NULL,
            text TEXT NOT NULL,
            embedding BLOB NOT NULL
        )
        """
    )
    conn.commit()
    conn.close()

def store_chunk(db_path: str, source: str, chunk_index: int, text: str, embedding: list[float]) -> None:
    """Insert a single chunk and its embedding vector into SQLite."""
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()
    blob = np.array(embedding, dtype=np.float32).tobytes()
    cursor.execute(
        "INSERT INTO chunks (source, chunk_index, text, embedding) VALUES (?, ?, ?, ?)",
        (source, chunk_index, text, blob),
    )
```

```
conn.commit()
conn.close()
```

That blob is just a sequence of four-byte floats. When we query it later, we unpack the blob back into a NumPy array and compute similarity. No magic. No hidden index structures. Just a table you can inspect with the `sqlite3` command-line tool if you want to see what is inside.

Searching by Meaning with Cosine Similarity

With our chunks stored, the search problem becomes a geometry problem. We embed the user's question into the same vector space, then compare it against every chunk vector using cosine similarity. Cosine similarity measures the cosine of the angle between two vectors. If two vectors point in the same direction, their cosine similarity is 1.0. If they are orthogonal, it is 0.0.

The formula is:

```
cosine_similarity(a, b) = dot(a, b) / (norm(a) * norm(b))
```

Here is the pure NumPy implementation—no scikit-learn, no external libraries:

```
def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    """Compute cosine similarity between two 1-D vectors."""
    norm_a = np.linalg.norm(a)
    norm_b = np.linalg.norm(b)
    if norm_a == 0.0 or norm_b == 0.0:
        return 0.0
    return float(np.dot(a, b) / (norm_a * norm_b))
```

The search function iterates over every row in the database, deserializes each embedding, computes similarity to the query, and keeps the top-k results. For a few thousand chunks this is nearly instantaneous on a modern laptop. If your knowledge base grows to tens of thousands of chunks and search latency starts to sting, that is the point where upgrading to FAISS or Chroma makes sense. Until then, understanding the raw math is worth more than the milliseconds you would save.

```
def search_chunks(query: str, db_path: str, top_k: int = 5) -> list[dict]:
    """Find the top_k most similar chunks to the query."""
    q_emb = np.array(get_embedding(query), dtype=np.float32)
    if q_emb.size == 0:
        print("Failed to embed the query.")
        return []
```

```

conn = sqlite3.connect(db_path)
cursor = conn.cursor()
cursor.execute("SELECT id, source, chunk_index, text,
embedding FROM chunks")
rows = cursor.fetchall()
conn.close()

scored = []
for row in rows:
    cid, source, chunk_index, text, emb_blob = row
    emb = np.frombuffer(emb_blob, dtype=np.float32)
    score = cosine_similarity(q_emb, emb)
    scored.append((score, cid, source, chunk_index, text))

scored.sort(reverse=True, key=lambda x: x[0])

results = []
for score, cid, source, chunk_index, text in scored[:top_k]:
    results.append({
        "id": cid,
        "source": source,
        "chunk_index": chunk_index,
        "text": text,
        "score": round(score, 4),
    })
return results

```

Grounding the Answer in Your Documents

Retrieval is only half the battle. The other half is making sure the language model actually uses the retrieved text instead of hallucinating from its training data. The prompt is the guardrail. We will construct a prompt that includes the retrieved chunks, explicitly orders the model to use only the provided documents, and requires it to cite the chunk IDs that support each claim.

Here is the exact prompt structure:

```

def build_prompt(question: str, chunks: list[dict]) -> str:
    """Assemble a grounded prompt from the retrieved chunks."""
    context_parts = []
    for chunk in chunks:
        header = f"[Chunk {chunk['id']} from {chunk['source']}]: "
        context_parts.append(f"{header}\n{chunk['text']}")
    context = "\n\n".join(context_parts)

    prompt = (
        "You are a precise knowledge-base assistant. "

```

```

        "Answer the user's question using ONLY the information
        provided in the documents below. "
        "Do not use outside knowledge. "
        "If the answer is not contained in the documents, say 'I
        don't know based on the provided documents.' "
        "Cite the relevant chunk ID(s) in parentheses after each
        claim.\n\n"
        f"Documents:\n{context}\n\n"
        f"Question: {question}\n\nAnswer:"
    )
    return prompt

```

The instruction “*Answer using only provided documents*” is not a suggestion. It is a contract. Models want to be helpful, and helpfulness often means guessing when they are uncertain. By restricting the source material and giving them an explicit escape hatch—“I don't know based on the provided documents”—you reduce hallucination dramatically. The citation requirement adds traceability. If the model claims something surprising, you can open the cited chunk and verify it yourself.

Now we need a generator. We will reuse Ollama, this time with a conversational model like llama3.1:8b:

```

OLLAMA_GENERATE_URL = "http://localhost:11434/api/generate"
GENERATE_MODEL = "llama3.1:8b"

```

```

def ask_kb(question: str, db_path: str) -> str:
    """Answer a question using the knowledge base."""
    chunks = search_chunks(question, db_path, top_k=5)
    if not chunks:
        return "No relevant documents found."

    prompt = build_prompt(question, chunks)
    try:
        response = httpx.post(
            OLLAMA_GENERATE_URL,
            json={"model": GENERATE_MODEL, "prompt": prompt,
"stream": False},
            timeout=120.0,
        )
        response.raise_for_status()
        payload = response.json()
        return payload.get("response", "").strip()
    except Exception as exc:
        return f"Answer generation failed: {exc}"

```

The Complete Script

Here is kb_agent.py, the full program that ties everything together. Place some .md or .txt files in a ./notes folder and run it.

```
import sqlite3
import json
import numpy as np
import httpx
from pathlib import Path

OLLAMA_EMBED_URL = "http://localhost:11434/api/embeddings"
EMBED_MODEL = "nomic-embed-text"
OLLAMA_GENERATE_URL = "http://localhost:11434/api/generate"
GENERATE_MODEL = "llama3.1:8b"
DB_PATH = "kb.sqlite"
CHUNK_SIZE = 512
OVERLAP = 50
TOP_K = 5

def parse_file(file_path: Path) -> str:
    """Read a .txt or .md file and return its contents as a
    string."""
    try:
        return file_path.read_text(encoding="utf-8")
    except UnicodeDecodeError:
        try:
            return file_path.read_text(encoding="latin-1")
        except Exception as exc:
            print(f"Failed to read {file_path}: {exc}")
            return ""
    except Exception as exc:
        print(f"Failed to read {file_path}: {exc}")
        return ""

def chunk_text(text: str, chunk_size: int = 512, overlap: int =
50) -> list[str]:
    """Split text into overlapping chunks of roughly chunk_size
    characters."""
    chunks = []
    start = 0
    text = text.strip()
    if not text:
        return chunks

    while start < len(text):
        end = start + chunk_size
        if end < len(text):
            while end > start and text[end] != " ":
```

```

        end -= 1
        if end == start:
            end = start + chunk_size
        chunk = text[start:end].strip()
        if chunk:
            chunks.append(chunk)
        start = end - overlap
        if start <= 0:
            start = end
        if end >= len(text):
            break
    return chunks

```

```

def get_embedding(text: str) -> list[float]:
    """Request an embedding vector from the local Ollama
    server."""
    try:
        response = httpx.post(
            OLLAMA_EMBED_URL,
            json={"model": EMBED_MODEL, "prompt": text},
            timeout=60.0,
        )
        response.raise_for_status()
        payload = response.json()
        embedding = payload.get("embedding")
        if not isinstance(embedding, list) or len(embedding) == 0:
            print("Embedding response missing vector data.")
            return []
        return embedding
    except httpx.HTTPStatusError as exc:
        print(f"Ollama returned an error: {exc.response.status_code}")
        return []
    except Exception as exc:
        print(f"Embedding request failed: {exc}")
        return []

```

```

def init_db(db_path: str) -> None:
    """Create the chunks table if it does not already exist."""
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()
    cursor.execute(
        """
        CREATE TABLE IF NOT EXISTS chunks (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            source TEXT NOT NULL,
            chunk_index INTEGER NOT NULL,
            text TEXT NOT NULL,
            embedding BLOB NOT NULL
        )
    """
    )

```



```

        """
    )
    conn.commit()
    conn.close()

def store_chunk(db_path: str, source: str, chunk_index: int, text:
str, embedding: list[float]) -> None:
    """Insert a single chunk and its embedding vector into
    SQLite."""
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()
    blob = np.array(embedding, dtype=np.float32).tobytes()
    cursor.execute(
        "INSERT INTO chunks (source, chunk_index, text, embedding)
VALUES (?, ?, ?, ?)",
        (source, chunk_index, text, blob),
    )
    conn.commit()
    conn.close()

def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    """Compute cosine similarity between two 1-D vectors."""
    norm_a = np.linalg.norm(a)
    norm_b = np.linalg.norm(b)
    if norm_a == 0.0 or norm_b == 0.0:
        return 0.0
    return float(np.dot(a, b) / (norm_a * norm_b))

def search_chunks(query: str, db_path: str, top_k: int = 5) ->
list[dict]:
    """Find the top_k most similar chunks to the query."""
    q_emb = np.array(get_embedding(query), dtype=np.float32)
    if q_emb.size == 0:
        print("Failed to embed the query.")
        return []

    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()
    cursor.execute("SELECT id, source, chunk_index, text,
embedding FROM chunks")
    rows = cursor.fetchall()
    conn.close()

    scored = []
    for row in rows:
        cid, source, chunk_index, text, emb_blob = row
        emb = np.frombuffer(emb_blob, dtype=np.float32)
        score = cosine_similarity(q_emb, emb)
        scored.append((score, cid, source, chunk_index, text))

```

```

scored.sort(reverse=True, key=lambda x: x[0])

results = []
for score, cid, source, chunk_index, text in scored[:top_k]:
    results.append({
        "id": cid,
        "source": source,
        "chunk_index": chunk_index,
        "text": text,
        "score": round(score, 4),
    })
return results

def build_prompt(question: str, chunks: list[dict]) -> str:
    """Assemble a grounded prompt from the retrieved chunks."""
    context_parts = []
    for chunk in chunks:
        header = f"[Chunk {chunk['id']} from {chunk['source']}]: "
        context_parts.append(f"{header}\n{chunk['text']}")
    context = "\n\n".join(context_parts)

    prompt = (
        "You are a precise knowledge-base assistant. "
        "Answer the user's question using ONLY the information "
        "provided in the documents below. "
        "Do not use outside knowledge. "
        "If the answer is not contained in the documents, say 'I "
        "don't know based on the provided documents.' "
        "Cite the relevant chunk ID(s) in parentheses after each "
        "claim.\n\n"
        f"Documents:\n{context}\n\n"
        f"Question: {question}\n\nAnswer:"
    )
    return prompt

def ask_kb(question: str, db_path: str) -> str:
    """Answer a question using the knowledge base."""
    chunks = search_chunks(question, db_path, top_k=TOP_K)
    if not chunks:
        return "No relevant documents found."

    prompt = build_prompt(question, chunks)
    try:
        response = httpx.post(
            OLLAMA_GENERATE_URL,
            json={"model": GENERATE_MODEL, "prompt": prompt,
"stream": False},
            timeout=120.0,
        )

```

```

        response.raise_for_status()
        payload = response.json()
        return payload.get("response", "").strip()
    except Exception as exc:
        return f"Answer generation failed: {exc}"

def index_files(folder: Path, db_path: str) -> None:
    """Parse, chunk, embed, and store every .md and .txt file in a
    folder."""
    init_db(db_path)
    for file_path in folder.iterdir():
        if file_path.suffix.lower() not in (".md", ".txt"):
            continue
        raw_text = parse_file(file_path)
        if not raw_text:
            continue
        chunks = chunk_text(raw_text, chunk_size=CHUNK_SIZE,
overlap=OVERLAP)
        for idx, chunk in enumerate(chunks):
            embedding = get_embedding(chunk)
            if not embedding:
                continue
            store_chunk(db_path, str(file_path), idx, chunk,
embedding)
            print(f"Indexed {file_path} chunk {idx}")

def main() -> None:
    notes_folder = Path("./notes")
    if not notes_folder.exists():
        print("Create a ./notes folder with .md or .txt files and
try again.")
        return
    print("Indexing knowledge base...")
    index_files(notes_folder, DB_PATH)
    print("Indexing complete.\n")
    question = "What did we decide about pricing?"
    print(f"Question: {question}")
    answer = ask_kb(question, DB_PATH)
    print(f"Answer:\n{answer}")

if __name__ == "__main__":
    main()

```

When you run this script, it walks through ./notes, splits each file into chunks, embeds them, stores them in kb.sqlite, and then answers the sample question using only what it found in your documents. Every part of that pipeline is code you can read, debug, and modify.

Break and Fix: Five Ways Knowledge Bases Fail

Break 1: Chunk Size Destroys Context

If you set `CHUNK_SIZE` to 64, your chunks are barely longer than a sentence. The embedding captures only a fragment of an idea, so when you ask about pricing strategy, you retrieve snippets like “we discussed this for a while” and “the meeting wrapped up at five.” The model has no coherent context to synthesize, and the answer becomes a nonspecific shrug.

Conversely, if you set `CHUNK_SIZE` to 4096, each chunk contains so many different ideas that its embedding becomes a blurry average. A query about pricing gets pulled toward a giant block that also talks about hiring, office space, and the holiday party.

Fix: Start with 512 and `overlap=50`, then test. If your notes are dense bullet points, try 256 with `overlap=30`. If they are narrative prose, 768 with `overlap=100` may work better. Tune empirically by inspecting the retrieved chunks.

Break 2: The Embedding Model Is Not Pulled

You start the script, and `get_embedding` prints:

```
Ollama returned an error: 404
```

The Ollama server is running, but it does not recognize `nomic-embed-text`. You forgot to pull it, or you pulled a different model and misnamed the constant.

Fix: Run `ollama pull nomic-embed-text` and verify it exists with `ollama list`. Double-check that `EMBED_MODEL` in your script matches the exact tag shown in that list. Ollama errors are usually precise; a 404 here means the model is missing, not that the server is down.

Break 3: Retrieved Chunks Are Irrelevant

You ask about the Q3 budget, but the top results are chunks from your onboarding checklist and a recipe you saved six months ago. The similarity scores are low—below 0.4—and the model refuses to answer because the grounding prompt tells it not to guess.

Fix: First, check your chunk size. If chunks are too large, the embeddings dilute the relevant signal. Second, verify that your query is actually represented in the corpus. If you never wrote about Q3 budget in your notes, no search pattern can invent the fact. Third, add source filenames to your chunk metadata. If you see `source='recipes.md'` at the top of the results, you know immediately that your knowledge base has a coverage problem, not a retrieval problem.

Break 4: The Model Uses Outside Knowledge

Even with retrieved chunks, the model fills in gaps from its training data. You ask about your team's API rate limit, and the model confidently claims it is ten thousand requests per minute—because that is a common default, even though your notes say five hundred.

Fix: Strengthen the system prompt. The phrase *“Do not use outside knowledge”* helps, but you can add a penalty clause: *“If the documents do not contain the answer, you must respond with ‘I don’t know based on the provided documents.’ There is no exception.”* Some models need that level of emphasis. You can also lower the generation temperature to 0.1 so the model is less creative about filling gaps.

Break 5: Document Parsing Garbles Formatting

Markdown tables, code blocks, and YAML frontmatter can turn a clean note into a wall of symbols when naively chunked. The embedding sees `|---|---|---` and treats it as semantic content, polluting the vector with punctuation noise.

Fix: Add a lightweight normalization step in `parse_file`. Strip YAML frontmatter before chunking, replace multiple newlines with a single one, and consider removing markdown table borders:

```
def normalize_text(text: str) -> str:
    """Remove YAML frontmatter and collapse excessive
    whitespace."""
    import re
    text = re.sub(r"^---\n.*?\n---\n", "", text, flags=re.DOTALL)
    text = re.sub(r"\n{3,}", "\n\n", text)
    return text.strip()
```

You do not need a full markdown parser. A few regexes are usually enough to protect your embeddings from structural debris.

Try It Now: Three Variations

Variation 1: Auto-Index New Files in a Folder

Modify `main()` to track file modification times so you only index files that have changed since the last run. Add a small `indexed_files` table with columns `source` and `mtime`. Before embedding a file, compare its current `Path.stat().st_mtime` against the stored value. Skip unchanged files, update the record for changed ones, and delete database rows for files that no longer exist on disk. This turns your script into an incremental indexer that runs safely every hour via cron.

Variation 2: Generate a Timeline of Decisions

Create a second prompt that asks the model to extract dated decisions from your knowledge base. First, search for chunks containing month names or ISO dates. Then feed those chunks into a prompt like: *“List every decision described in these documents with its date and a one-sentence summary. Format as a markdown table.”* You now have a living decision log generated from unstructured notes.

Variation 3: Compare KB vs. Web Search for the Same Query

Pick a factual question that might appear in your notes, such as “What is our deployment rollback policy?” Ask your knowledge base agent for an answer. Then repurpose the research agent from Chapter 2 and ask the same question using web search. Compare the two outputs. The knowledge base answer should be precise, cite your internal documents, and reflect your actual policy. The web search answer will be generic or wrong. That contrast is the best demonstration of why RAG matters for institutional memory.

The OpenClaw Way

OpenClaw treats RAG as a first-class pattern. In the OpenClaw ecosystem, you can define a `VectorStore` tool that wraps the indexing and retrieval logic we built by hand. You register your documents via the OpenClaw document loader, specify a chunking strategy in YAML, and OpenClaw handles the embeddings, the vector storage, and the top-k retrieval automatically. The pure Python version we wrote is essentially what OpenClaw does under the hood, but with configurable

reranking and metadata filtering exposed through class methods. If you outgrow the raw SQLite approach, migrating to OpenClaw's built-in LocalChromaStore requires only a few lines of configuration change while keeping your chunking and prompting logic intact. The conceptual pipeline—parse, chunk, embed, retrieve, generate—remains identical; OpenClaw simply saves you from writing the loop yourself.

The Hermes Agent Way

Hermes Agent approaches knowledge bases from the conversation side rather than the code side. When you start a Hermes session, you can point it at a folder of notes using the built-in file system toolset. Hermes automatically indexes the text into its internal vector session memory, embedding chunks and maintaining a searchable context window as you chat. From the user perspective, you do not write a `main()` function; you type a question, and Hermes searches its loaded context before answering. The trade-off is less explicit control over chunk size, overlap, and database schema. The benefit is that you get a queryable knowledge base with zero new code. For quick exploration of a folder full of notes, Hermes is the fastest path. For production pipelines where you need deterministic behavior and offline indexing, the pure Python script we built in this chapter is the stronger foundation.

Cost Sidebar: The Price of Memory

This is the first all-local project in this book, and the cost story is worth celebrating. After you download the `nomic-embed-text` model once, every embedding, every search, and every generated answer costs exactly zero dollars. Your Ollama server runs on your own CPU or GPU. Your SQLite database lives on your own disk. There are no API calls, no token metering, and no rate limits imposed by a vendor.

If you were to build this with a cloud embedding provider like OpenAI's `text-embedding-ada-002`, a knowledge base of ten thousand chunks would cost roughly one to two cents to index—still cheap, but not free, and it repeats every time you rebuild. A cloud generation model answering your queries might add another fraction of a cent per question. The local approach eliminates both line items permanently. The only cost is the one-time bandwidth to pull the model, and the electricity to run your machine. For a personal knowledge base that

you query dozens of times a day, the savings add up to real money over a year, and the privacy benefit—your notes never leave your laptop—is priceless.

What You Built

In this chapter, you built a fully local knowledge base agent from scratch:

- A document parser that reads .txt and .md files with forgiving error handling.
- A paragraph-aware chunker with an explicit overlap parameter to preserve context across splits.
- An embedding client that calls the local Ollama `/api/embeddings` endpoint using `nomic-embed-text`.
- A pure-Python vector store using SQLite and NumPy, with no external vector database required.
- A cosine similarity search function written entirely in NumPy, retrieving the top-k most relevant chunks.
- A grounded generation prompt that restricts the model to your documents, demands citations by chunk ID, and provides a safe fallback when the answer is absent.
- A complete `main()` function that indexes a folder of notes and answers natural-language questions using only the indexed material.

You also learned how to debug five common failure modes in RAG pipelines: bad chunk sizing, missing models, irrelevant retrieval, hallucinated answers, and formatting noise. The result is an agent that knows what you know—and can prove it.

Chapter 9: Project 9 — Browser Automation Agent

The agent navigates a website, fills a form, and downloads a report — while you sleep.

A few years ago, I needed to download a monthly statement from a government portal. The portal had no API. It had no bulk-export button. It had a login form, a dropdown for the month, a dropdown for the year, a checkbox to agree to terms, and a Download PDF button. The whole ritual took about ninety seconds if I knew exactly where to click. I needed two years of statements. Do the math: that's thirty-two months. At ninety seconds each, I was looking at forty-eight minutes of pure, unadulterated clicking. Worse, the session timed out after ten minutes of inactivity, which meant I couldn't even do it in one batch.

I wrote a Selenium script. It broke three times in the first week because the site changed a CSS class. Then I rewrote it with Playwright, added explicit waits, and wrapped it in a domain whitelist so it could never wander onto the wrong site. That script has been running every first of the month for three years now. It cost me two hours to build and it has saved me about thirty hours of clicking. That's the promise of browser automation: turning mindless, repetitive web interaction into reliable, scheduled software.

But there's a darker side, and we need to talk about it upfront. A browser automation script is literally a robot using a website on your behalf. If you get it wrong, it can submit forms you didn't intend, click purchase buttons on your credit card, or spam a login page until the site locks your account. This chapter treats safety as a first-class feature, not an afterthought. Every line of code we write includes guardrails, and every guardrail is one I learned to add the hard way.

When to Automate a Browser vs. Using an API

Before you point a headless browser at a website, ask yourself a simple question: does this site have an API? If the answer is yes, use the API. APIs are stable contracts. They return structured data. They rarely change without notice. They don't make you wait for JavaScript to render. A REST API call costs milliseconds. A browser automation step costs seconds. Multiply that across hundreds of iterations and you're burning real time and real money.

But sometimes there is no API. Or the API requires enterprise pricing your team can't afford. Or the data you need only appears after a sequence of clicks that no JSON endpoint exposes. That's when you

reach for browser automation. The rule of thumb is simple: try the API first. If it doesn't exist, doesn't work, or costs more than your time, automate the browser. But do it safely.

In this project, we'll build a minimal browser automation agent using Playwright's synchronous Python API. The agent will be able to browse to a URL, extract visible text and interactive elements, click buttons, fill input fields, and — critically — ask for your approval before submitting anything. It will also respect a domain whitelist: if the URL isn't on the list, the agent refuses to proceed. These guardrails aren't paranoia. They're the difference between a helpful tool and a liability.

Setting Up Playwright

Playwright is a browser automation library maintained by Microsoft. It supports Chromium, Firefox, and WebKit, but we'll stick with Chromium because it's the fastest and most predictable for automation. Install Playwright and its browser binaries from the command line:

```
pip install playwright
playwright install chromium
```

The second command downloads a headless Chromium binary, which is roughly a hundred megabytes. It only needs to happen once per machine. If you're running in a CI environment or a Docker container, make sure this step is part of your build so the browser is available at runtime.

Now create a file called `browser_agent.py`. Our imports are minimal:

```
import json
import time
from urllib.parse import urlparse
from playwright.sync_api import sync_playwright, TimeoutError as PlaywrightTimeout
```

Notice that we import `TimeoutError` from Playwright directly. That's because Playwright raises its own timeout exception, which is different from Python's built-in `TimeoutError`. If you catch the wrong one, your error handling will silently fail.

The Browser Agent Core

Our browser agent is structured around a single class, `BrowserAgent`, that manages a Playwright browser context and exposes a small set of safe tools. The class holds a domain whitelist, tracks the current page, and enforces an approval gate before any form submission.

Here's the class skeleton:

```
class BrowserAgent:
    """
    A safe browser automation agent with domain whitelisting
    and explicit approval for form submission.
    """

    def __init__(self, allowed_domains: list[str], headless: bool
= True):
        """
        Initialize the browser agent.

        Args:
            allowed_domains: List of allowed hostname strings
(e.g., ['example.com']).
            headless: Whether to run the browser without a GUI.
        """
        self.allowed_domains = allowed_domains
        self.headless = headless
        self.browser = None
        self.context = None
        self.page = None
        self._start()

    def _start(self):
        """Launch the browser and create a new page."""
        self.playwright = sync_playwright().start()
        self.browser =
self.playwright.chromium.launch(headless=self.headless)
        self.context = self.browser.new_context(
            viewport={"width": 1280, "height": 800}
        )
        self.page = self.context.new_page()
```

A few things to observe here. First, the `allowed_domains` list is passed at initialization. If you forget to set it, the agent simply won't browse anywhere. That is intentional. Whitelisting should be opt-in, not opt-out. Second, we set a fixed viewport. Some sites behave differently depending on viewport size, and consistency matters for automation.

Third, `_start()` is a private method called from `__init__`. In a production agent, you might want lazy initialization, but for a beginner project, starting the browser at construction makes the flow easier to follow.

Browsing to a URL and Reading the Page

The `browse` tool is the agent's eyes. It navigates to a URL, validates the domain against the whitelist, waits for the page to load, and returns a structured summary of what it sees: the page title, the visible text, and a list of interactive elements with IDs the agent can refer to later.

```
def browse(self, url: str) -> str:
    """
    Navigate to a URL, verify the domain is whitelisted,
    wait for the page to load, and return a summary of
    visible text and interactive elements.

    Args:
        url: The target URL.

    Returns:
        A JSON string with keys 'title', 'text', and
        'elements'.
    """
    parsed = urlparse(url)
    hostname = parsed.hostname or ""
    if not any(hostname.endswith(d) for d in
self.allowed_domains):
        return (
            f"Error: Domain '{hostname}' is not in the
whitelist. "
            f"Allowed: {self.allowed_domains}"
        )

    try:
        self.page.goto(url, wait_until="domcontentloaded",
timeout=15000)
        # Wait a tick for JavaScript frameworks to hydrate
        self.page.wait_for_timeout(500)
    except PlaywrightTimeout:
        return "Error: Page load timed out after 15 seconds."

    title = self.page.title()
    text = self.page.inner_text("body").strip()[:2000]

    inputs = self.page.query_selector_all("input, textarea")
    buttons = self.page.query_selector_all("button,
[role='button']")
    links = self.page.query_selector_all("a")
```

```

elements = []
for idx, el in enumerate(inputs + buttons + links):
    tag = el.evaluate("el => el.tagName.toLowerCase()")
    el_id = el.get_attribute("id") or ""
    name = el.get_attribute("name") or ""
    placeholder = el.get_attribute("placeholder") or ""
    text_content = el.inner_text().strip()[:60]
    elements.append({
        "index": idx,
        "tag": tag,
        "id": el_id,
        "name": name,
        "placeholder": placeholder,
        "text": text_content,
    })

summary = {
    "title": title,
    "url": url,
    "text": text,
    "elements": elements,
}
return json.dumps(summary, indent=2)

```

Let's walk through this slowly, because every line matters.

First, `urlparse` extracts the hostname. We check whether it ends with any whitelisted domain. We use `endswith` rather than exact equality so that subdomains like `secure.example.com` match an `example.com` entry. If the check fails, we return an error immediately — the page doesn't even get loaded.

Next, `page.goto()` navigates to the URL. We pass `wait_until="domcontentloaded"`, which is a compromise. `load` waits for every image and script to finish, which can be slow. `networkidle` waits until there are no network connections for 500ms, which is stricter and often waits for analytics beacons. `domcontentloaded` waits only until the HTML is parsed and the DOM is ready. For most modern sites built with React or Vue, the DOM initially contains a spinner or a shell, so we add a hard `wait_for_timeout(500)` afterward to let the JavaScript hydrate. This is not elegant, but it is pragmatic. Later, we'll see how to wait for specific elements instead.

The `inner_text("body")` call extracts all visible text. We truncate it to two thousand characters because an LLM context window is finite, and most pages are noise. The selector `input, textarea` grabs text fields. `button, [role='button']` grabs clickable buttons, including those that use

ARIA roles. a grabs links. For each element, we collect the tag name, id, name attribute, placeholder text, and visible text. We assign an index so the agent can refer to elements by number if it needs to.

Clicking and Filling Fields Safely

Browsing is observation. The next tools are action. The click tool clicks an element by its index from the summary list. The fill_field tool types text into an input by its name attribute.

```
def click(self, element_index: int) -> str:
    """
        Click the element at the given index from the last
browse()
        summary.

    Args:
        element_index: The index assigned in the browse()
output.

    Returns:
        A confirmation or error string.
    """
    try:
        # Re-query all interactive elements in the same order
        inputs = self.page.query_selector_all("input,
textarea")
        buttons = self.page.query_selector_all("button,
[role='button']")
        links = self.page.query_selector_all("a")
        all_elements = inputs + buttons + links

        if element_index < 0 or element_index >=
len(all_elements):
            return f"Error: element index {element_index} out
of range."

        target = all_elements[element_index]
        target.scroll_into_view_if_needed()
        target.click()
        return f"Clicked element {element_index}."
    except Exception as e:
        return f"Error clicking element {element_index}: {e}"

def fill_field(self, name: str, value: str) -> str:
    """
        Fill an input or textarea identified by its 'name'
attribute.
```

Args:

name: The value of the element's name attribute.
value: The text to type.

Returns:

A confirmation or error string.

"""

try:

```
locator = self.page.locator(f"[name='{name}']")
locator.wait_for(state="visible", timeout=5000)
locator.fill(value)
return f"Filled field '{name}' with '{value}'."
```

except PlaywrightTimeout:

```
return f"Error: field with name='{name}' not found
```

```
within 5 seconds."
```

except Exception as e:

```
return f"Error filling field '{name}': {e}"
```

The click method re-queries the DOM before clicking. This is important because the DOM changes after navigation. If you cached element references from an earlier browse call, they might be stale. By re-querying and using the same ordering logic as browse, we ensure the index still maps to the same conceptual element — or at least, to the element currently occupying that slot. This is a common source of fragility in browser automation, and we'll address it more deeply in the Break and Fix section.

The fill_field method uses a Playwright locator instead of a raw element handle. Locators are more robust: they resolve at action time, not at query time, which means they'll follow the element even if it moves in the DOM. We wait for the element to be visible with a five-second timeout, then fill it. If the field doesn't appear, we fail gracefully with a descriptive error rather than crashing.

The Approval Gate: Never Submit Without Permission

Here is the most important safety rule in this entire chapter: an automated agent should never click a submit button without explicit human approval. The consequences of an accidental form submission range from embarrassing (posting a half-written comment) to expensive (placing an order) to legally problematic (agreeing to terms you haven't read).

We enforce this with an explicit submit tool that prints the proposed action and waits for a human response:

```
def submit(self, description: str) -> str:
    """
    Request explicit user approval before clicking a submit
    action.

    Args:
        description: Human-readable summary of what will
        happen.

    Returns:
        'Submitted', 'Cancelled', or an error string.
    """
    print(f"\n[APPROVAL REQUIRED] {description}")
    answer = input("Approve? Type 'yes' to proceed:
").strip().lower()
    if answer == "yes":
        # Find the first visible submit button and click it
        submit_btn = self.page.locator("button[type='submit'],
input[type='submit']")
        if submit_btn.count() > 0:
            submit_btn.first.click()
            return "Submitted."
        return "Error: no submit button found."
    return "Cancelled by user."
```

The description argument is the agent’s chance to explain what it thinks it’s about to do. In a fully autonomous setup, this would be logged and sent to a review queue. For our beginner agent, we simply pause the script and wait for keyboard input. If you type anything other than yes, the submission is cancelled and the agent reports that fact back to the caller.

This approval gate is why our tool set does not include a raw “click submit button” function. The LLM can reason about needing to submit, but it must route through `submit()`, which forces a pause. This is a design pattern you should copy into any agent that interacts with stateful web forms.

Waiting for Dynamic Pages

Many modern websites don’t load their content in the initial HTML. They fetch it via JavaScript after the page skeleton renders. If you try to interact with an element that hasn’t appeared yet, Playwright raises a timeout error. The fix is `wait_for_selector`.

Here's a helper method that our agent can call when it expects something to appear dynamically:

```
def wait_for_element(self, selector: str, timeout: int =
10000) -> str:
    """
    Wait until an element matching the CSS selector appears.

    Args:
        selector: A CSS selector string.
        timeout: Maximum wait time in milliseconds.

    Returns:
        A confirmation or timeout message.
    """
    try:
        self.page.wait_for_selector(selector, timeout=timeout)
        return f"Element '{selector}' appeared."
    except PlaywrightTimeout:
        return f"Error: element '{selector}' did not appear
within {timeout}ms."
```

If the agent notices from the browse output that a page has a loading indicator, it should call `wait_for_element(".results-list")` before trying to extract data. This replaces our earlier blunt-instrument approach of `wait_for_timeout(500)` with a precise, event-driven wait.

Tying It All Together: A Login Example

Now that we have the pieces, let's see them work together. We'll create a `main()` function that automates a login flow against a test page. In a real scenario, you'd point this at a staging server or a local HTML file. For safety, our example uses a placeholder URL on a whitelisted test domain.

```
def close(self):
    """Clean up browser resources."""
    if self.browser:
        self.browser.close()
    if self.playwright:
        self.playwright.stop()

def main():
    """
    Run the browser agent through a sample login flow.
    Update the URL and credentials for your own test page.
    """
    agent = BrowserAgent(
```

```

        allowed_domains=["the-internet.herokuapp.com",
"localhost"],
        headless=False, # Set True to hide the browser window
    )

    try:
        # Example using the-internet.herokuapp.com login page
        url = "https://the-internet.herokuapp.com/login"
        result = agent.browse(url)
        print(result)

        # Fill credentials
        print(agent.fill_field("username", "tomsmith"))
        print(agent.fill_field("password",
"SuperSecretPassword!"))

        # Request explicit approval before submitting
        print(
            agent.submit(
                "Click the Login button to authenticate as
tomsmith"
            )
        )

        # Wait for the secure area to load
        print(agent.wait_for_element("#flash", timeout=5000))

        # Capture the result
        final = agent.browse(url)
        print(final)
    finally:
        agent.close()

if __name__ == "__main__":
    main()

```

Run it:

```
python browser_agent.py
```

With headless=False, you'll see the Chromium window open, navigate to the login page, fill the username and password, and then pause. The terminal will show [APPROVAL REQUIRED] Click the Login button... and wait for you to type yes. Only then will the form submit. After submission, the agent waits for the #flash message element and captures the final page state.

This is a real, running agent. It is not a mock. The only fictional part is that I picked a well-known public test page so you can reproduce the result without setting up your own server. If you want to adapt this to your own site, change the URL, update the `allowed_domains`, and adjust the field names and selectors.

Break and Fix: Five Ways Browser Automation Fails

Browser automation breaks more often than any other project in this book, because websites are not APIs. They change. They optimize for humans, not robots. Here are five failure modes I've personally hit, and the fixes that made my agents stable.

Break 1: Element Not Found

The agent calls `click(3)` or `fill_field("email", "me@example.com")` and Playwright raises an error because the element simply isn't in the DOM. Maybe the page hasn't loaded. Maybe a modal overlay is blocking it. Maybe the developer renamed the class yesterday.

Fix: Always wrap element lookups in waits. Don't assume the DOM is ready just because `goto()` returned. If you're targeting an element by index, re-query the DOM right before the action, as we do in `click()`. If you're targeting by selector, use Playwright's built-in `wait_for_selector` or `locator wait_for`. And if an element is buried inside an `iframe` — a common pattern for payment forms — you need to switch into the `iframe` context first:

```
frame = page.frame_locator("iframe[name='payment']")
frame.locator("#card-number").fill("4111111111111111")
```

Break 2: Page Changed Dynamically

You browse a page, capture the element list, then call `click(2)` to open a dropdown. By the time the agent reaches `fill_field`, a new banner has loaded at the top of the page, shifting every element index by three. Your agent clicks the wrong thing.

Fix: Never rely on element indexes across multiple actions unless the page is guaranteed static. Prefer stable identifiers: IDs, names, ARIA labels, or data attributes. If the page is unstable between actions, re-browse after every major state change. A slower agent that clicks correctly is better than a fast agent that deletes the wrong record.

For truly dynamic single-page apps, consider waiting for network idle after each interaction:

```
page.click("#load-more")
page.wait_for_load_state("networkidle")
```

Break 3: CAPTCHA Encountered

You're automating a form, everything works in testing, then one day the agent stops dead at a CAPTCHA screen. It doesn't know what to do, and even if it did, solving CAPTCHA with software is often against the site's terms of service and may violate local law.

Fix: The only reliable fix is to stop and hand off to a human. Detect CAPTCHA indicators in the page text or DOM, then raise an exception or return a message to the calling loop:

```
def detect_captcha(page) -> bool:
    """Return True if the page contains common CAPTCHA markers."""
    markers = ["captcha", "recaptcha", "i'm not a robot", "verify
you are human"]
    text = page.inner_text("body").lower()
    return any(m in text for m in markers)
```

If `detect_captcha` returns `True`, your agent should halt immediately. Do not try to solve it. Do not try to 绕过 it. Log the incident and alert the operator. Long-term, if the site you're automating deploys CAPTCHA, that is a signal that the site owner does not want automated access. Respect that signal and look for an API or a data partnership.

Break 4: Timeout on Slow Page

The agent tries to browse a heavy page that loads eighty analytics scripts and a bloated JavaScript bundle. Playwright hits the fifteen-second timeout and returns an error. The agent has no data and the loop collapses.

Fix: Increase the timeout for known-slow pages, or better, tighten the `wait_until` condition. If you only need to fill a form, you don't need every image to load. Switch from `networkidle` to `domcontentloaded`

and add targeted `wait_for_selector` calls for the elements you actually interact with. If a page is consistently slow, consider whether you should be visiting it less often or caching state between runs.

Another useful technique is retrying with exponential backoff:

```
import time

def browse_with_retry(agent, url, max_retries=3):
    for attempt in range(max_retries):
        result = agent.browse(url)
        if not result.startswith("Error:"):
            return result
        time.sleep(2 ** attempt)
    return "Error: all retries exhausted."
```

Break 5: Form Submitted Without Approval

A bug in the agent's reasoning loop, or an over-eager prompt, causes the agent to call `click()` directly on a submit button rather than routing through the `submit()` approval gate. Money is spent. Data is deleted. Trust is broken.

Fix: Make the approval gate structural, not just procedural. In our design, `click()` doesn't know what a submit button is, but you can add an extra layer of defense: automatically detect submit actions and block them unless they go through `submit()`:

```
def click(self, element_index: int) -> str:
    # ... existing query logic ...
    target = all_elements[element_index]
    tag = target.evaluate("el => el.tagName.toLowerCase()")
    el_type = target.get_attribute("type") or ""
    if tag == "button" and el_type == "submit":
        return (
            submit(), "
            "Error: submit buttons must be clicked via
            "not click(), for safety."
        )
    target.scroll_into_view_if_needed()
    target.click()
    return f"Clicked element {element_index}."
```

This is defense in depth. Even if the LLM tries to bypass the gate, the tool itself refuses. That is the kind of safety architecture you need when real consequences are on the line.

Try It Now: Three Variations

The basic agent gets you browsing, clicking, and filling. Now stretch it.

Variation 1: Extract a Table to CSV

Point the agent at a page with a data table. After browsing to the URL, use `page.query_selector_all("table tr")` to iterate over rows and cells, build a list of lists, and write it to a CSV file using the standard `csv` module. This is the classic “my bank statement is in HTML” workflow. You can do the extraction entirely inside a new `extract_table(selector: str, output_path: str)` method on the `BrowserAgent` class.

Variation 2: Monitor a Page for Text Changes

Write a script that browses to a specific URL once per hour, hashes the visible text with `hashlib.sha256`, and compares it to the previous hash. If the hash changes, the script sends a notification — a simple print statement is fine, or push to a Slack webhook if you’re feeling fancy. Use `time.sleep(3600)` inside a loop, or schedule the script with `cron`. This is competitive intelligence, price monitoring, or outage detection boiled down to its essence.

Variation 3: Simulate a Multi-Step Checkout Flow

Create a test HTML file with three pages: a cart page with an item, a shipping page with form fields, and a confirmation page. Write an agent script that browses each page in sequence, fills the shipping form, and pauses at the final confirmation step for human approval. This tests your ability to manage state across page navigations, handle element lookups on each new page, and chain multiple agent decisions together in a single workflow.

The OpenClaw Way

OpenClaw’s `@tool` decorator and `Agent` class fit browser automation naturally. You define `browse`, `click`, `fill_field`, and `submit` as decorated Python functions, then pass the `BrowserAgent` instance into OpenClaw’s tool registry. OpenClaw handles the conversation loop, parses the model’s JSON output into tool calls, and enforces the max-turns limit automatically. The benefit is that your agent gains the ReAct reasoning pattern — the model will explicitly think about which page

to visit, which field to fill, and whether it needs your approval before submitting. OpenClaw also tracks cost per turn, which matters when your browser agent is making multiple decisions per run. The trade-off is that OpenClaw expects tools to be stateless functions, so your BrowserAgent instance needs to be stored in a module-level variable or a context manager that survives across tool calls within the same conversation.

The Hermes Agent Way

Hermes Agent takes a different angle: it already ships with a browser toolset in some configurations. If you enable the browser toolset, Hermes can browse pages, click elements, and fill fields without you writing any Playwright code at all. The interaction is conversational: you tell Hermes, “Go to the login page for example.com and log in with the test credentials,” and it invokes its built-in browser tools automatically. Where our custom agent enforces a domain whitelist and explicit submit approval, Hermes respects its own permission system: dangerous actions are gated behind user confirmation by default. The Hermes approach is excellent for prototyping and one-off tasks, but if you need precise control over selectors, custom retry logic, or integration with your own data pipeline, you’ll eventually want to drop down to the Playwright layer directly, as we did in this chapter.

What You Built

In this chapter, you built a real browser automation agent from scratch:

- A Playwright-powered BrowserAgent class that launches a headless Chromium browser.
- A browse() tool that extracts page text and interactive elements safely.
- A click() tool that re-queries the DOM to avoid stale references.
- A fill_field() tool that uses Playwright locators for robust element targeting.
- A submit() approval gate that pauses for human confirmation before any form submission.
- A domain whitelist that rejects navigation to untrusted sites.

- A `wait_for_element()` helper for handling dynamic JavaScript-heavy pages.
- Break-and-fix strategies for missing elements, changing pages, CAPTCHAs, timeouts, and accidental submissions.
- Three practice variations: table extraction, change monitoring, and multi-step checkout.

That's a complete, safe, and extensible foundation for automating any website that lacks an API.

Cost for this chapter: Approximately \$0.00 in API costs, because Playwright runs entirely on your local machine. The only expense is your electricity and your time. If you run a cloud model to power the decision-making loop externally, expect roughly \$0.001–\$0.005 per conversation with GPT-4o mini, depending on how many browsing steps the agent takes.

Chapter 10: Project 10 — Meeting Transcript Analyzer

Upload a recording. Get decisions, action items, and follow-up questions.

The first time I tried to transcribe a team meeting by hand, I spent forty minutes scrubbing through a noisy audio file, pausing every seven seconds to correct the automatic captions on my phone. By the time I reached the forty-five-minute mark, I had a cramp in my clicking hand, three unanswered Slack messages, and a transcript that read like a surrealist poem. “We’ll circle back on the penguin auth flow” was supposed to be “pending auth flow.” I gave up. The meeting notes doc stayed blank, and three days later, someone asked me why the authentication module hadn’t shipped yet. I had absolutely no memory of agreeing to that deadline. The recording was still on my hard drive, a digital monument to my own refusal to do boring work.

This is the kind of problem agents were born to solve. Not the flashy, science-fiction variety of agent. The boring, deeply practical kind. An agent that listens to an hour of audio so you don’t have to. It extracts the decisions, lists the owners, surfaces the deadlines, and writes them

to a markdown file you can search next week. In this chapter, you are going to build exactly that: a meeting transcript analyzer that turns a raw recording into structured intelligence.

The pipeline is straightforward, but every step has a trap door. Feed it an audio file. Whisper turns speech into timestamped text. A language model reads the text and produces summaries in multiple styles. Then a structured extractor pulls out action items with owners and deadlines, formatted as clean JSON. Finally, everything lands in a single markdown file called `meeting_notes.md`. The whole flow costs less than a dime per hour of audio, and once it is running, you will never take manual meeting notes again. More importantly, you will never again forget who promised to do what.

Why Meeting Notes Are a Classic Automation Target

Meetings are expensive. A one-hour sync with eight people is not one hour of work; it is eight hours of collective human time. And the value of that time evaporates instantly if nobody writes down what happened. The problem is that note-taking is a split-attention task. The person typing is not fully listening. The person listening is not fully typing. By minute thirty, the notes are patchy. By minute fifty, they are fiction. By the following Monday, they are archaeology.

Automating this is not about replacing humans. It is about capturing the signal before it decays. A transcript is the raw signal. A summary is the compressed signal. Action items with deadlines are the executable signal. Each step reduces the cognitive load on every attendee, and it creates an auditable record that you can search, compare, and hand off to people who were not in the room.

This project sits at the intersection of audio processing, long-context language modeling, and structured extraction. It is the most practical agent you will build in this section of the book. Unlike the ReAct loop we built in Chapter 1, this agent is a pipeline: data flows in one end, gets transformed through a series of stages, and comes out the other end as a finished document. There is no looping decision-making here, but there is every bit as much need for careful error handling, because corrupt input at stage one will corrupt output at stage four.

Setting Up Whisper

Whisper is OpenAI's open-source speech-recognition model. It is shockingly good at handling accents, jargon, and noisy audio, and it supports dozens of languages out of the box. You have two ways to use it: the OpenAI API, which costs \$0.006 per minute of audio, or a local installation, which costs nothing but requires a GPU or a very patient CPU.

For the API path, you need an OpenAI API key and the `httpx` library, which we have been using since Chapter 1. For the local path, you install `openai-whisper` via `pip` and make sure `ffmpeg` is on your system `PATH`. Local Whisper runs on PyTorch. If you have a Mac with an M1 chip or a machine with a CUDA-capable GPU, it is fast enough for occasional use. If you are on a five-year-old laptop with integrated graphics, the API is the gentler choice. The good news is that the code below works with either. You flip a single environment variable.

Here is the transcription function. It handles both paths. If the environment variable `WHISPER_LOCAL` is set to `true`, it shells out to the local whisper CLI via `subprocess` and reads the resulting JSON file. Otherwise, it streams the audio to the OpenAI API and parses the verbose JSON response. Both paths return the same dictionary shape: full text, language code, and a list of segments with start time, end time, and text.

```
import json
import os
import subprocess

import httpx

WHISPER_LOCAL = os.environ.get("WHISPER_LOCAL", "false").lower()
== "true"
OPENAI_API_KEY = os.environ.get("OPENAI_API_KEY")
OLLAMA_URL = "http://localhost:11434/api/chat"
MODEL = "llama3.1:8b"

def transcribe_audio(audio_path: str) -> dict:
    """Transcribe an audio file to text with timestamps.

    Returns a dict with keys:
    - text: full transcript string
    - segments: list of dicts with start, end, text
    - language: detected language code

    Raises:
```

```

        FileNotFoundError: if the audio file does not exist.
        ValueError: if API key is missing for cloud mode.
        subprocess.CalledProcessError: if local Whisper exits with
an error.
    """
    if not os.path.isfile(audio_path):
        raise FileNotFoundError(f"Audio file not found:
{audio_path}")

    if WHISPER_LOCAL:
        # Local path: run the whisper CLI and read the JSON it
drops
        # in the current working directory.
        cmd = [
            "python", "-m", "whisper",
            audio_path,
            "--model", "base",
            "--output_format", "json",
            "--output_dir", ".",
        ]
        result = subprocess.run(cmd, capture_output=True,
text=True, check=True)
        # Whisper writes <audio_stem>.json in the output_dir.
        json_path = os.path.splitext(audio_path)[0] + ".json"
        with open(json_path, "r", encoding="utf-8") as f:
            data = json.load(f)
        return {
            "text": data.get("text", ""),
            "segments": data.get("segments", []),
            "language": data.get("language", "unknown"),
        }

    # API path: stream the file to OpenAI's whisper-1 endpoint.
    if not OPENAI_API_KEY:
        raise ValueError("OPENAI_API_KEY is required when
WHISPER_LOCAL is false.")

    with open(audio_path, "rb") as f:
        response = httpx.post(
            "https://api.openai.com/v1/audio/transcriptions",
            headers={"Authorization": f"Bearer {OPENAI_API_KEY}"},
            data={
                "model": "whisper-1",
                "response_format": "verbose_json",
                "timestamp_granularities[]": "segment",
            },
            files={"file": (os.path.basename(audio_path), f,
"audio/mpeg")},
            timeout=300.0,
        )
        response.raise_for_status()
        data = response.json()

```

```

return {
    "text": data.get("text", ""),
    "segments": data.get("segments", []),
    "language": data.get("language", "unknown"),
}

```

Notice the error handling at every boundary. We check that the file exists before we touch it. We verify the API key before opening the file handle so we do not leak file-descriptor churn on a missing configuration. We call `raise_for_status()` on the HTTP response so that 4xx and 5xx errors surface immediately instead of poisoning our downstream JSON parsing. And on the local path, we use `subprocess.run` with `check=True`, which throws a clear `CalledProcessError` if Whisper hits a codec error, runs out of memory, or cannot find `ffmpeg`.

The segments list is the heart of the transcript. Each segment is a dictionary with a start timestamp, an end timestamp, and the text spoken during that window. Later in the pipeline, we will format these segments into human-readable lines like [12.5s -> 18.2s] Let's talk about the auth flow. Timestamps are not just decoration. They are forensic evidence. When someone disputes whether a decision was really made, you can point them to the exact second.

Chunking Long Transcripts

An hour of meeting audio produces roughly ten thousand words of transcript. If you feed all of that into a local 8-billion-parameter model at once, you will likely exceed its context window and crash the inference engine. Even cloud models have token budgets. The solution is chunking: split the transcript into blocks that each fit comfortably inside the model's brain, process them independently, and combine the partial results.

The simplest chunking strategy is character-based. We group transcript paragraphs into chunks of roughly twelve thousand characters each, which translates to a few thousand tokens. That leaves headroom for the system prompt and the model's response. The function below walks through the transcript text line by line and starts a new chunk whenever the next paragraph would push us over the limit.

```
MAX_CHARS = 12000
```

```

def chunk_transcript(text: str, max_chars: int = MAX_CHARS) ->
list[str]:

```

```
"""Split transcript text into chunks that fit inside an LLM
context window.
```

```
    Prefers splitting on paragraph boundaries to avoid mid-
    sentence breaks,
    which helps the model retain context at chunk edges.
```

```
Args:
```

```
    text: The full transcript string.
    max_chars: Approximate maximum characters per chunk.
```

```
Returns:
```

```
    A list of transcript chunk strings.
```

```
"""
```

```
paragraphs = text.split("\n")
chunks = []
current = []
current_len = 0

for para in paragraphs:
    para_len = len(para)
    if current_len + para_len > max_chars and current:
        chunks.append("\n".join(current))
        current = [para]
        current_len = para_len
    else:
        current.append(para)
        current_len += para_len

if current:
    chunks.append("\n".join(current))

return chunks
```

This function is the load-bearing wall of the entire pipeline. Without it, a ninety-minute all-hands meeting becomes a single oversized prompt that your model chokes on. With it, you can process two-hour board meetings on the same laptop that runs your local Ollama instance.

Chunking introduces a subtle problem: context loss at the boundaries. If a decision straddles two chunks, the summarizer might see the setup in chunk one and the conclusion in chunk two, but never together. The fix is to use a two-pass approach: summarize each chunk, then summarize the summaries. We will do exactly that in the next section. It costs an extra API call, but it preserves global coherence.

Summarizing with Style

Once you have clean chunks, the language model does the reading. But not all summaries serve the same purpose. Sometimes you want a three-bullet brief for your boss. Sometimes you want a detailed narrative for the project wiki. Sometimes you only care about the action items. Instead of hard-coding a single prompt, we will write a `summarize()` function that accepts a style parameter and adapts the system prompt accordingly.

First, we need a reliable way to call the language model. We will use the local Ollama chat endpoint, exactly as we did in Chapter 1, because it is free and sufficient for this task. If you prefer a cloud model, swap in the OpenAI client and add the three-line cost tracker from Chapter 1.

```
def call_llm(system: str, user: str, temperature: float = 0.2) -> str:
    """Call the local Ollama model and return the assistant's text response.
```

Args:

system: The system prompt defining the assistant's role.

user: The user prompt (transcript chunk or task).

temperature: Creativity level; 0.2 is conservative and factual.

Returns:

The model's generated text.

Raises:

`httpx.HTTPStatusError`: if the model server returns an error.

```
    """
```

```
    payload = {
```

```
        "model": MODEL,
```

```
        "messages": [
```

```
            {"role": "system", "content": system},
```

```
            {"role": "user", "content": user},
```

```
        ],
```

```
        "stream": False,
```

```
        "options": {"temperature": temperature},
```

```
    }
```

```
    response = httpx.post(OLLAMA_URL, json=payload, timeout=120.0)
```

```
    response.raise_for_status()
```

```
    return response.json()["message"]["content"].strip()
```

Now the summarizer. If the transcript fits in a single chunk, we send it straight to the model. If it spans multiple chunks, we summarize each chunk in the requested style, concatenate the partial summaries, and run one final pass to merge them into a coherent whole.

```
def summarize(transcript_text: str, style: str = "brief") -> str:
    """Summarize a meeting transcript in the requested style.
```

Args:

```
    transcript_text: Full transcript with timestamps.
    style: One of 'brief', 'detailed', or 'action-items'.
```

Returns:

```
    A markdown-formatted summary string.
```

Raises:

```
    ValueError: if style is not recognized.
```

```
"""
```

```
valid_styles = {"brief", "detailed", "action-items"}
```

```
if style not in valid_styles:
```

```
    raise ValueError(f"style must be one of {valid_styles},
got {style!r}")
```

```
if len(transcript_text) <= MAX_CHARS:
```

```
    chunks = [transcript_text]
```

```
else:
```

```
    chunks = chunk_transcript(transcript_text)
```

```
system_prompts = {
```

```
    "brief": (
```

```
        "You are a concise meeting assistant. "
```

```
        "Summarize the transcript in 3-5 bullet points. "
```

```
        "Focus on decisions and outcomes. Use markdown."
```

```
    ),
```

```
    "detailed": (
```

```
        "You are a thorough meeting assistant. "
```

```
        "Write a detailed summary with these sections:\n"
```

```
        "- Decisions Made\n"
```

```
        "- Discussion Points\n"
```

```
        "- Risks or Open Questions\n"
```

```
        "- Next Steps\n"
```

```
        "Use markdown formatting."
```

```
    ),
```

```
    "action-items": (
```

```
        "You are a task extraction specialist. "
```

```
        "List every action item mentioned in the transcript. "
```

```
        "For each item, note the owner if stated and the
```

```
deadline if stated. "
```

```
        "Use a markdown checklist format: - [ ] what (who, by
when)."
```

```
    ),
```

```

    }

    partials = []
    for i, chunk in enumerate(chunks, start=1):
        user_prompt = f"Transcript part {i} of {len(chunks)}:
\n\n{chunk}"
        partial = call_llm(system_prompts[style], user_prompt,
temperature=0.2)
        partials.append(partial)

    if len(partial) == 1:
        return partials[0]

    # Second pass: merge partial summaries into one coherent
result.
    merge_system = system_prompts[style] + (
        "\n\nMerge the following partial summaries into one
cohesive document."
    )
    merge_user = "\n\n---\n\n".join(partial)
    return call_llm(merge_system, merge_user, temperature=0.2)

```

The two-pass design is important. Without it, a long meeting becomes a series of disjointed mini-summaries that repeat context. With it, the model gets to see the big picture after digesting the parts. The cost is one extra LLM call, which on a local model is free, and on GPT-4o mini is still under a tenth of a cent.

I want to stress the prompt design here. We do not ask the model to “be helpful.” We tell it exactly what structure to emit: bullet points, specific sections, or checklists. Vague prompts produce vague summaries. In agent work, your prompt is your specification document. The more precise it is, the less you have to clean up afterward.

Extracting Structured Action Items

Summaries are for humans. Calendars and project-management tools need structured data. In this step, we ask the model to read the transcript and emit strictly formatted JSON: a list of objects, each with who, what, and by_when. This is the hardest prompt in the chapter to get right, because models love to add conversational fluff around JSON. We are going to starve them of that temptation with a draconian system prompt.

```

def extract_actions(transcript_text: str) -> list[dict]:
    """Extract structured action items from a meeting transcript.

```

Returns a list of dicts, each with keys:

- who: the person responsible, or 'Unassigned'.
- what: the task description.
- by_when: the deadline, or 'TBD'.

If the response is not valid JSON, returns a single fallback item

containing the raw response so nothing is silently lost.

```
"""
system = (
    "You are a structured data extractor. "
    "Read the meeting transcript and output ONLY a JSON list
of action items. "
    "Each item must be an object with exactly these keys:
'who', 'what', 'by_when'. "
    "If the owner is unclear, use 'Unassigned'. "
    "If the deadline is not stated, use 'TBD'. "
    "Do not wrap the output in markdown code fences. "
    "Do not add any explanatory text. "
    "Output valid JSON and nothing else."
)

# If the transcript is huge, chunk and extract per chunk, then
merge.
if len(transcript_text) > MAX_CHARS:
    chunks = chunk_transcript(transcript_text)
else:
    chunks = [transcript_text]

all_actions = []
for chunk in chunks:
    response = call_llm(system, chunk, temperature=0.1)
    try:
        actions = json.loads(response)
        if not isinstance(actions, list):
            raise ValueError(
                f"Expected JSON list, got
{type(actions).__name__}"
            )
        for item in actions:
            item.setdefault("who", "Unassigned")
            item.setdefault("what", "")
            item.setdefault("by_when", "TBD")
        all_actions.extend(actions)
    except (json.JSONDecodeError, ValueError) as e:
        # Capture the failure visibly rather than swallowing
it.

        all_actions.append({
            "who": "Extraction Error",
            "what": (
                f"Failed to parse actions from chunk: {e}. "
                f"Raw: {response[:300]}"
            ),
        },
```

```

        "by_when": "TBD",
    })

    return all_actions

```

Notice the temperature of 0.1: lower than the summarizer. Structured extraction is a deterministic task. You want the model to follow instructions, not get creative. We also guard against malformed JSON explicitly. If the model wraps its JSON in triple backticks, or adds a preamble like “Here are the action items:”, `json.loads` will throw. Our fallback creates an error item with the raw text, so we never lose information. You can inspect that item, fix the prompt, and re-run.

The `by_when` field is the most fragile part of extraction. Humans are vague about deadlines. They say “by end of week” or “soon” or “maybe Tuesday.” Our prompt explicitly asks for TBD when the deadline is not stated, but a more advanced version could normalize relative dates against the meeting date. That normalization is one of the exercises in the “Try It Now” section.

Writing the Output

The final step ties everything together into a markdown file that you can email, Slack, or drop into Notion. The function below accepts the brief summary, the detailed summary, and the structured action items, then writes a neatly formatted document.

```

def write_meeting_notes(
    brief: str,
    detailed: str,
    actions: list[dict],
    output_path: str = "meeting_notes.md",
) -> None:
    """Write formatted meeting notes to a markdown file.

    Args:
        brief: The brief summary text.
        detailed: The detailed summary text.
        actions: List of action-item dicts.
        output_path: Destination file path.
    """
    with open(output_path, "w", encoding="utf-8") as f:
        f.write("# Meeting Notes\n\n")

        f.write("## Brief Summary\n\n")
        f.write(brief + "\n\n")

        f.write("## Detailed Summary\n\n")

```

```

f.write(detailed + "\n\n")

f.write("## Action Items\n\n")
if not actions:
    f.write("_No action items extracted._\n")
for item in actions:
    who = item.get("who", "Unassigned")
    what = item.get("what", "")
    when = item.get("by_when", "TBD")
    f.write(f"- [ ] **{what}** (Owner: {who}, By: {when})\n")

f.write("\n---\n")
f.write("_Generated by meeting transcript analyzer._\n")

```

The output is deliberately plain. No HTML tables, no PDF rendering, no fancy CSS. Plain markdown is the universal language of documentation tools. Drop this file into GitHub, Slack, Obsidian, or Jira, and it renders cleanly everywhere. The action items use GitHub-style task checkboxes (- []), which means if you paste the document into an issue tracker, you get clickable tasks for free.

The Full Pipeline

Here is the main() function that orchestrates the entire flow. It reads the audio path from an environment variable so you can swap meetings without editing code, transcribes the audio, builds the full timestamped transcript, generates both summary styles, extracts actions, and writes the file.

```

def main() -> None:
    """Run the full meeting transcript analyzer pipeline."""
    audio_path = os.environ.get("AUDIO_PATH", "meeting.mp3")

    print(f"Step 1/4: Transcribing {audio_path}...")
    result = transcribe_audio(audio_path)

    # Build a single timestamped transcript string from segments.
    lines = []
    for seg in result.get("segments", []):
        start = seg.get("start", 0.0)
        end = seg.get("end", 0.0)
        text = seg.get("text", "").strip()
        lines.append(f"[{start:.1f}s -> {end:.1f}s] {text}")
    transcript_text = "\n".join(lines)

    print("Step 2/4: Generating brief summary...")
    brief = summarize(transcript_text, style="brief")

```

```
print("Step 3/4: Generating detailed summary...")
detailed = summarize(transcript_text, style="detailed")

print("Step 4/4: Extracting action items...")
actions = extract_actions(transcript_text)

out_path = "meeting_notes.md"
write_meeting_notes(brief, detailed, actions, out_path)
print(f"Done. Notes written to {out_path}")

if __name__ == "__main__":
    main()
```

To run it with a local model and local Whisper:

```
export WHISPER_LOCAL=true
export AUDIO_PATH=weekly_sync.mp3
python meeting_analyzer.py
```

To run it with the OpenAI API for transcription and a local model for summarization:

```
export OPENAI_API_KEY=sk-...
export AUDIO_PATH=weekly_sync.mp3
python meeting_analyzer.py
```

The pipeline is idempotent. Run it twice on the same file, you get the same output. That safety matters when you are batch-processing a folder of recordings and you do not want duplicates or side effects.

Break and Fix: Five Ways This Project Can Fail

Every stage of this pipeline can break in ways that are silent and expensive. Here are the five failure modes I have hit personally, and the defenses I now build in from day one.

Break 1: Poor Audio Quality

You feed the agent a recording from a crowded conference room, and Whisper returns nonsense. “Auth flow” becomes “off slow.” The summary faithfully repeats the nonsense, and the action items include tasks like “review the owl database.” Garbage in, garbage out.

Why it fails: Whisper’s base model is lightweight and struggles with overlapping voices, echo, and laptop microphones placed three feet from the speaker. The transcription is a lossy compression of already-lossy audio.

Fix: Pre-process the audio before transcription. A lightweight pass through ffmpeg can normalize volume and strip high-frequency noise:

```
ffmpeg -i noisy_meeting.mp3 -af  
"highpass=f=200,lowpass=f=3000,loudnorm" \  
    cleaned_meeting.mp3
```

In code, you can wrap this in a preprocessing function that runs before `transcribe_audio`. If the transcript confidence looks low, or if the resulting text is shorter than a heuristic threshold, your script can warn the user instead of producing confident but wrong notes.

Break 2: Speakers Not Identified

The transcript is one wall of text. You see: “I will review the pull request.” But `extract_actions` assigns who as `Unassigned`, because it has no idea who “I” is. Two hours later, three people think they are responsible for the same task, or nobody does.

Why it fails: Standard Whisper does not perform speaker diarization. It only knows that *someone* spoke, not *who* spoke.

Fix: If you cannot run `pyannote.audio` or `whisperx` for diarization, give the language model contextual hints. Pass a list of known attendees into the extraction prompt so it can match voices to names when they are mentioned:

```
def extract_actions_with_attendees(  
    transcript_text: str, attendees: list[str]  
) -> list[dict]:  
    attendee_hint = (  
        "Known attendees: " + ", ".join(attendees) + ". "  
    )  
    system = (  
        attendee_hint  
        + "You are a structured data extractor... "  
        # ... rest of prompt  
    )  
    # ... rest of function
```

Even without perfect diarization, the model will often catch phrases like “Alice, can you handle that?” and assign the task to Alice correctly.

Break 3: Vague Action Items

The model returns action items like {"who": "Bob", "what": "Look into the auth thing", "by_when": "TBD"}. You show this to Bob. Bob has no idea what “the auth thing” means, and neither do you, because the original transcript was equally vague.

Why it fails: Agents cannot manufacture intent. If the meeting itself was fuzzy, the extraction will be fuzzy. The model is a mirror, not a mind reader.

Fix: Add a post-processing step that flags vague language. Maintain a list of weak verbs like “look into,” “think about,” and “consider,” and warn when an action item contains them:

```
WEAK_VERBS = {"look into", "think about", "consider", "check on"}

for item in actions:
    if any(verb in item["what"].lower() for verb in WEAK_VERBS):
        item["what"] += " [WARNING: task is vague; please clarify]"
```

These warnings show up in the final markdown and force the reader to refine the task before it becomes a forgotten ticket.

Break 4: Deadlines Not Stated

Every by_when field reads TBD. The model extracted the tasks correctly but captured no dates, because the meeting transcript said things like “by end of week” or “soon” or “Friday” without specifying which Friday.

Why it fails: Relative date references are useless without an anchor. The model has no implicit calendar.

Fix: Anchor the model to the meeting date via an environment variable, and instruct it to resolve relative language:

```
MEETING_DATE = os.environ.get("MEETING_DATE", "")

system = (
    f"The meeting took place on {MEETING_DATE}. "
    "If someone says 'by Friday', calculate the actual ISO date. "
    "If the deadline is still ambiguous, use 'TBD'."
)
```

If `MEETING_DATE` is not set, the model continues to fall back to TBD, but when it is set, your output becomes a real calendar entry instead of a guessing game.

Break 5: Transcript Too Long for Context Window

You run the analyzer on a two-hour board meeting. The script throws a timeout, or Ollama returns a cryptic error about tensor dimensions, or the cloud API silently truncates the prompt and produces half a summary.

Why it fails: You skipped chunking. Ten thousand words of transcript plus a long system prompt plus a detailed summary request exceeds the model's context window.

Fix: This is exactly what the `chunk_transcript` function solves. Make sure every call to `summarize` and `extract_actions` respects the `MAX_CHARS` limit. If you ever find yourself tempted to raise `MAX_CHARS` to fit a huge meeting, do not. Lower it and add more passes. Context windows are not suggestions; they are hard walls. The two-pass summary design we implemented earlier is the architectural answer to this problem. Use it.

Try It Now: Three Variations

The agent works. Now make it yours.

Variation 1: Compare to Previous Meeting

Store every generated `meeting_notes.md` in a dated folder. Write a script that loads the current action items and the previous week's action items, then produces a diff: which tasks are new, which are carried over, and which disappeared without a completion marker. This turns your pipeline from a single-meeting tool into a project tracking system. The diff is just set operations on the what strings.

Variation 2: Slack Messages to Assignees

Use a Slack incoming webhook or the Slack API to post each action item to the responsible person's direct message or a shared channel. The `extract_actions` function already gives you structured data, so formatting a Slack message is a single f-string. Add a helper function

like `notify_slack(action_item: dict, webhook_url: str)` and call it inside `main` right after extraction. Now your meeting notes do not sit in a file; they start a conversation.

Variation 3: Dashboard of Actions with Due Dates

Aggregate action items from multiple meeting files into a single markdown table sorted by due date. Read every `meeting_notes.md` in a directory, parse the action item lines with regular expressions, and write a `dashboard.md` that looks like this:

```
| Owner | Task | Meeting | Due |
|---|---|---|---|
| Alice | Deploy auth fix | 2026-08-01 | TBD |
| Bob | Update pricing page | 2026-08-01 | 2026-08-08 |
```

This is the seed of a real project-management dashboard, built entirely from files your agent already generates.

The OpenClaw Way

OpenClaw is built around the ReAct loop we saw in Chapter 1, but it also supports pipeline agents via its `Pipeline` class. You can define each stage of the meeting analyzer as a typed step: `transcribe`, `summarize`, `extract`, and `render`. OpenClaw handles chunking automatically when a step's input exceeds the configured context window, and it tracks the total token spend per stage so you can see exactly where your money went.

In OpenClaw, the extraction step can use `Pydantic` rather than raw JSON parsing. You define an `ActionItem` model with `who`, `what`, and `by_when` fields, and OpenClaw validates the model's output against that schema. If the model returns malformed JSON, OpenClaw retries with a stricter temperature automatically, up to a configured limit. This removes the manual fallback logic we wrote in `extract_actions`. The trade-off is that you must learn OpenClaw's step decorator syntax and its configuration schema. The benefit is that a twenty-line pure-Python function becomes a three-line decorated step, and the error handling is handled by the framework's retry middleware.

The Hermes Agent Way

Hermes Agent is a conversation-first tool, which makes it surprisingly effective for ad-hoc transcript analysis. You do not write a `main()` function. Instead, you point Hermes at your audio file using its built-in filesystem tools, tell it to summarize, and ask follow-up questions in natural language.

Here is how the same flow works in Hermes:

```
hermes
# Welcome to Hermes Agent v2.5.0

> Please summarize the transcript in meeting.mp3 and list action
items.

[transcribe] Transcribing meeting.mp3...
[summarize] Summary generated.
[extract] Found 4 action items.

Decisions: ...
Action items: ...

> Who was supposed to handle the auth fix?

Alice was assigned the auth fix, with a deadline of Friday.

> What did we decide about the pricing page?

We decided to delay the pricing page update until after the auth
fix is deployed.
```

Hermes chose its own tools. It transcribed, summarized, and extracted without you defining a pipeline. The trade-off is less reproducibility: if you run the same request twice, Hermes might phrase things differently or choose different tools. The benefit is zero setup for one-off tasks. When you need a reliable, repeatable pipeline, write the script. When you need answers in the next sixty seconds, open Hermes.

What You Built

In this chapter, you built a complete meeting intelligence pipeline from scratch:

- A `transcribe_audio()` wrapper that works with both local Whisper and the OpenAI API, with strict error handling for missing files, missing keys, and malformed responses.
- A `chunk_transcript()` function that splits long transcripts into LLM-safe blocks without breaking mid-sentence.
- A `summarize()` function that supports three output styles and uses a two-pass merge strategy to preserve coherence across chunks.
- An `extract_actions()` function that coerces an LLM into structured JSON output, with visible fallbacks when the model gets chatty.
- A `write_meeting_notes()` renderer that produces clean, portable markdown with task checkboxes.
- A `main()` pipeline orchestrator that ties the stages together and reports progress to the terminal.

You also learned how audio quality, speaker identity, vague language, missing dates, and context limits can each corrupt the output, and you wrote targeted defenses for all five.

Cost for this chapter: Using local Whisper and Ollama, the cost is \$0.00 plus your electricity bill. Using the OpenAI Whisper API, a one-hour meeting costs \$0.36 in transcription. The summarization and extraction add approximately \$0.02 per thousand tokens with GPT-4o mini, or effectively \$0.00 on a local model. Total per one-hour meeting: roughly \$0.40 cloud, \$0.00 local.

Chapter 11: Project 11 — CI/CD Watchdog Agent

A test failed at 3 AM. The agent investigated, diagnosed, and left a comment before you woke up.

I once spent four hours debugging a failing build only to discover the root cause was a missing environment variable that had been deleted three commits earlier. Nobody noticed because the failure message was buried ten thousand lines deep in a log file, sandwiched between dependency downloads and linting output. By the time I found it, three other engineers had rebased onto the bad commit, and the fix required a coordinated revert across four pull requests. The actual bug took thirty seconds to fix. The archaeology took all morning.

That kind of context loss is the real cost of CI failures. Not the compute time — that's pennies. It's the human time spent reconstructing what went wrong, often by someone who wasn't even awake when the failure happened. Every minute a build stays red is a minute your team can't merge, can't deploy, and can't trust the pipeline. And the longer it stays red, the more likely someone is to ignore it or work around it, which is how small problems become systemic rot.

The CI/CD Watchdog Agent exists to shrink that window from hours to minutes. It polls your GitHub Actions runs, detects failures, fetches the logs, extracts the error patterns, and drafts a diagnostic comment on the pull request before anyone asks. It doesn't replace human judgment — it gives humans a head start. Think of it as an intern who reads every log so you don't have to, and who never sleeps.

Why Fast Diagnosis Beats Perfect Prevention

You'll hear people say the goal is "zero failing builds." That's a fantasy. In any real codebase, tests flake, dependencies drift, and timing issues surface on Tuesdays. The question is not whether your pipeline will fail — it will — but how quickly someone understands why. A failing build with a clear diagnosis is a solved problem waiting for a commit. A failing build with no diagnosis is a mystery that accrues interest every hour.

The math is simple. If your average CI failure takes ninety minutes to debug and you have four failures a week, that's six hours of engineering time lost to log archaeology. Over a year, that's more than three hundred hours — nearly two months of a full-time developer, spent doing something a machine can do in seconds. The Watchdog Agent doesn't prevent failures. It converts them from time sinks into actionable tickets.

The architecture is straightforward: poll the GitHub API for failed workflow runs, fetch the associated logs, trim them to the most relevant section, send that section to an LLM for analysis, and post a structured comment back to the pull request. Along the way, we handle rate limits, oversized logs, ambiguous errors, and the occasional hallucinated diagnosis. Let's build it.

Connecting to the GitHub Actions API

Before we write any diagnosis logic, we need to talk to GitHub. The GitHub Actions REST API is well-documented, stable, and requires nothing more than a personal access token and an HTTP client. We'll use `httpx` because it handles timeouts and retries gracefully, and because you already know it from earlier chapters.

First, create a fine-grained personal access token at <https://github.com/settings/tokens>. Give it read access to Actions and Pull requests on the repositories you want to monitor. Store the token in an environment variable — never hardcode it:

```
export GITHUB_TOKEN="ghp_XXXXXXXXXXXXXXXXXXXXX"
```

Now let's write the base client. Every function in this chapter gets a docstring, because docstrings are contracts — even when the only thing reading them is a future version of you:

```
import os
import httpx
from typing import Optional

GITHUB_API = "https://api.github.com"
TOKEN = os.environ.get("GITHUB_TOKEN", "")

def github_client() -> httpx.Client:
    """Return an httpx client configured with the GitHub API base
    URL
    and Authorization header.
    """
    headers = {
        "Authorization": f"Bearer {TOKEN}",
        "Accept": "application/vnd.github+json",
        "X-GitHub-API-Version": "2022-11-28",
    }
    return httpx.Client(base_url=GITHUB_API, headers=headers,
        timeout=30.0)
```

The `github_client` function encapsulates everything a GitHub API request needs: the base URL, the authorization header, and a consistent API version. By returning a reusable client instead of making one-off requests, we also get connection pooling and shared configuration. The thirty-second timeout is a guardrail: if GitHub is slow, we'd rather fail fast and retry than hang indefinitely.

Polling for Failed Runs

The next step is finding the failures. The GitHub Actions API exposes workflow runs per repository. We'll query for the most recent runs and filter for those with conclusion "failure" or "timed_out". For this chapter, we assume a single owner/repo pair, but the function is written to accept any repository string in the form "owner/repo".

```
def fetch_failed_runs(repo: str, per_page: int = 10) ->
list[dict]:
    """Fetch recent workflow runs for *repo* and return only those
    that concluded with failure or timeout.

    Parameters
    -----
    repo:
        Repository identifier in "owner/repo" format.
    per_page:
        Maximum number of recent runs to inspect.

    Returns
    -----
    A list of run dictionaries with keys: id, name, head_branch,
    run_number, conclusion, html_url.
    """
    client = github_client()
    try:
        resp = client.get(
            f"/repos/{repo}/actions/runs",
            params={"per_page": per_page, "status": "completed"},
        )
        resp.raise_for_status()
    except httpx.TimeoutException:
        print("GitHub API timed out. Will retry next cycle.")
        return []
    except httpx.HTTPStatusError as e:
        print(f"GitHub API error: {e.response.status_code}")
        return []

    data = resp.json()
    runs = data.get("workflow_runs", [])
```

```

failed = []
for run in runs:
    conclusion = run.get("conclusion")
    if conclusion in ("failure", "timed_out"):
        failed.append({
            "id": run["id"],
            "name": run["name"],
            "head_branch": run["head_branch"],
            "run_number": run["run_number"],
            "conclusion": conclusion,
            "html_url": run["html_url"],
            "head_sha": run["head_sha"],
        })
return failed

```

Notice how the function validates its inputs only implicitly — `repo` is passed straight into the URL, so if you send a malformed string, GitHub returns a 404 and we catch it. That’s intentional. The API is the source of truth for what constitutes a valid repository, and duplicating that logic in our client creates drift.

Also note the error handling. We catch `TimeoutException` and `HTTPStatusError` separately because they require different responses. A timeout means “try again later.” A 404 or 401 means “check your configuration.” Never swallow both with a bare `except Exception`; you’ll silence real problems.

Fetching and Trimming Logs

Once we have a failed run, we need the logs. GitHub Actions stores logs per job and per step, but for diagnosis we usually only care about the final output — the last commands executed before the failure. A full log file for a long test suite can be hundreds of thousands of lines. Feeding that into an LLM would burn tokens on noise and exceed context windows.

The solution is aggressive trimming: fetch the full log, split it into lines, and keep only the last `N` lines. In practice, `N = 200` covers most failures. The failing test, the traceback, and the preceding few commands almost always fit inside that window.

```

def fetch_run_logs(run_id: int, repo: str, tail_lines: int = 200)
-> str:
    """Download the plain-text logs for a workflow run and return
    only the last *tail_lines* non-empty lines.

```

Parameters

run_id:

The numeric GitHub Actions run identifier.

repo:

Repository in "owner/repo" format.

tail_lines:

Number of trailing lines to retain.

Returns

Trimmed log text, or an empty string if logs are unavailable.

"""

```
client = github_client()
```

```
url = f"/repos/{repo}/actions/runs/{run_id}/logs"
```

```
try:
```

```
    resp = client.get(url, follow_redirects=True)
```

```
    resp.raise_for_status()
```

```
except httpx.HTTPStatusError as e:
```

```
    if e.response.status_code == 404:
```

```
        print(f"Logs not found for run {run_id}. May have  
expired.")
```

```
    elif e.response.status_code == 401:
```

```
        print("Authentication failed. Check your  
GITHUB_TOKEN.")
```

```
    else:
```

```
        print(f"Failed to fetch logs:  
{e.response.status_code}")
```

```
        return ""
```

```
except httpx.TimeoutException:
```

```
    print("Log download timed out. Skipping this run.")
```

```
    return ""
```

```
# Logs come back as a ZIP archive from GitHub.
```

```
# For simplicity, assume the caller handles unzipping or uses
```

```
# the text endpoint. If we receive plain text, trim directly.
```

```
text = resp.text
```

```
lines = [line for line in text.splitlines() if line.strip()]
```

```
trimmed = "\n".join(lines[-tail_lines:])
```

```
return trimmed
```

In reality, GitHub returns logs as a ZIP archive. Production code would unzip the archive, read the relevant .txt files, and concatenate them. For this chapter, the key concept is the trimming strategy: keep the tail, discard the rest. If you build the unzipping step, apply the same tail_lines filter to each job's log before concatenating. The principle is unchanged.

The empty-string guard on blank lines prevents the model from spending tokens on whitespace. Every line we send costs money; every blank line is a wasted fraction of a cent. At scale, trimming pays for itself.

Asking the Model to Diagnose

Now we have a 200-line log snippet. We need to turn that into a diagnosis. This is where the LLM earns its keep. We'll write a prompt that tells the model its role, describes the log format, and asks for a structured diagnosis with three fields: the likely root cause, the specific error message or traceback, and a suggested fix or next step.

```
import json

OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL = "llama3.1:8b"

def diagnose_logs(log_tail: str, run_name: str) -> dict:
    """Send the trimmed log tail to a local LLM and request a
    structured diagnosis with root cause, error excerpt, and fix.

    Parameters
    -----
    log_tail:
        The last N lines of the CI log.
    run_name:
        Human-readable name of the workflow run.

    Returns
    -----
    A dictionary with keys: root_cause, error_excerpt,
    suggested_fix.
    """
    prompt = (
        "You are a senior engineer diagnosing a CI/CD failure.
\n\n"
        f"Workflow: {run_name}\n\n"
        "Below is the tail of the build log. Identify the root
cause, "
        "quote the key error excerpt, and suggest a concrete fix.
\n\n"
        "Respond ONLY with valid JSON in this exact shape:\n"
        '{"root_cause": "...", "error_excerpt": "...",
"suggested_fix": "..."}\n\n'
        "Log tail:\n"
        "---\n"
        f"{log_tail}\n"
    )
```



```

        "----"
    )

    try:
        resp = httpx.post(
            OLLAMA_URL,
            json={"model": MODEL, "prompt": prompt, "stream":
False},
            timeout=60.0,
        )
        resp.raise_for_status()
    except (httpx.TimeoutException, httpx.HTTPStatusError) as exc:
        print(f"LLM call failed: {exc}")
        return {
            "root_cause": "LLM unavailable",
            "error_excerpt": "",
            "suggested_fix": "Check the Ollama server or model
availability.",
        }

    raw = resp.json().get("response", "").strip()

    # Extract JSON from potential markdown fences
    if "```" + "json" in raw:
        raw = raw.split("```" + "json")[1].split("```")[0].strip()
    elif "```" in raw:
        raw = raw.split("```")[1].split("```")[0].strip()

    try:
        parsed = json.loads(raw)
    except json.JSONDecodeError:
        # Fallback: wrap the raw text so we never crash downstream
        parsed = {
            "root_cause": "Unparseable LLM output",
            "error_excerpt": raw[:500],
            "suggested_fix": "Review the raw log manually.",
        }

    # Ensure all expected keys exist
    for key in ("root_cause", "error_excerpt", "suggested_fix"):
        parsed.setdefault(key, "")
    return parsed

```

The prompt is specific for a reason. Vague prompts produce vague answers. By specifying the exact JSON schema, we give the model a template to fill, which dramatically improves structured output reliability. The markdown-fence extraction is a safety net: even if the model wraps its JSON in code blocks, we still find it.

The fallback on `JSONDecodeError` is critical. In a production system, a crashed agent is worse than a slightly unhelpful comment. If the model returns poetry instead of JSON, we wrap what we got and move on. A human will still see the raw text and can act on it.

Drafting the PR Comment

A diagnosis sitting in a terminal is useless. The agent needs to surface it where developers already look: the pull request. We can use GitHub's Issues API, which also handles pull request comments. To comment on a PR, we need its issue number. The workflow run object includes `head_sha`, which we can map to a PR via the pulls API, or we can simply search open PRs for the same branch.

```
def comment_on_pr(repo: str, branch: str, diagnosis: dict) ->
bool:
    """Find an open pull request for *branch* in *repo* and post
    a diagnostic comment. Return True if a comment was posted.

    Parameters
    -----
    repo:
        Repository in "owner/repo" format.
    branch:
        The head branch name associated with the failed run.
    diagnosis:
        Output from *diagnose_logs*, containing root_cause,
        error_excerpt, and suggested_fix.

    Returns
    -----
    True if a comment was successfully created.
    """
    client = github_client()
    try:
        resp = client.get(
            f"/repos/{repo}/pulls",
            params={"state": "open", "head": f"{repo.split('/')
[0]}:{branch}"},
        )
        resp.raise_for_status()
    except (httpx.TimeoutException, httpx.HTTPStatusError) as exc:
        print(f"Failed to search PRs: {exc}")
        return False

    pulls = resp.json()
    if not pulls:
        print(f"No open PR found for branch {branch}. Skipping
comment.")
```

```

        return False

    pr_number = pulls[0]["number"]
    body = (
        "## 🐶 CI/CD Watchdog Report\n\n"
        f"**Root cause:** {diagnosis.get('root_cause', 'Unknown')}\n\n"
        f"**Error excerpt:**\n```\n{diagnosis.get('error_excerpt', 'N/A')}[:800]}\n```\n"
        f"\n**Suggested fix:** {diagnosis.get('suggested_fix', 'None provided')}\n\n"
        "_This comment was generated automatically. Please verify before acting._"
    )

    try:
        resp = client.post(
            f"/repos/{repo}/issues/{pr_number}/comments",
            json={"body": body},
        )
        resp.raise_for_status()
        print(f"Posted comment to PR #{pr_number}")
        return True
    except (httpx.TimeoutException, httpx.HTTPStatusError) as exc:
        print(f"Failed to post comment: {exc}")
        return False

```

The comment format is deliberate. The emoji header draws attention without being aggressive. The root cause comes first — that’s what someone scrolling past wants to know in ten seconds. The error excerpt is capped at eight hundred characters so we don’t flood the PR with a full traceback. And the disclaimer at the bottom sets expectations: this is a draft, not an order.

Putting It Together

Here’s `main()`, the function that ties all the pieces into a single polling cycle. It checks the repo, finds failures, diagnoses them, and comments. You could schedule this with cron, run it in a GitHub Action itself, or keep it running as a lightweight daemon.

```

def main(repo: str, only_first_failure: bool = True) -> None:
    """Run one polling cycle: find failed runs, diagnose, and comment.

```

Parameters

repo:

Repository to monitor, in "owner/repo" format.

```

only_first_failure:
    If True, only comment on the most recent failed run
    to avoid spamming a PR with repeated diagnoses.
"""
if not TOKEN:
    print("GITHUB_TOKEN is not set. Exiting.")
    return

failed = fetch_failed_runs(repo)
if not failed:
    print("No recent failures found.")
    return

if only_first_failure:
    failed = failed[:1]

for run in failed:
    print(f"Investigating run #{run['run_number']}
({run['name']})...")
    log_tail = fetch_run_logs(run["id"], repo, tail_lines=200)
    if not log_tail:
        print("No log data available. Skipping diagnosis.")
        continue

    diagnosis = diagnose_logs(log_tail, run["name"])
    print(f"Diagnosis: {diagnosis['root_cause']}")

    commented = comment_on_pr(repo, run["head_branch"],
diagnosis)
    if commented:
        print("Comment posted successfully.")
    else:
        print("Could not post comment. See errors above.")

if __name__ == "__main__":
    # Example: monitor a specific repository
    REPO = "your-org/your-repo"
    main(REPO, only_first_failure=True)

```

The `only_first_failure` flag is a noise-reduction lever. When a PR breaks four tests, you want one diagnosis, not four comments. The flag limits the agent to the most recent failure, which usually covers the primary blocker. You can flip it to `False` for a full audit trail, but start with `True` — your teammates will thank you.

Break and Fix: Five Ways the Watchdog Fails

Even a simple agent breaks in interesting ways. Here are the failure modes I've hit while running this in production-like settings.

Break 1: The Flaky Test

Your agent comments on a PR: "Test test_payment_flow failed." The developer reruns the job and it passes. The agent's diagnosis was accurate for that run, but the test itself is flaky. Now there's a false alarm sitting on the PR, and the developer learns to ignore watchdog comments.

Fix: Track test names across recent runs. If the same test fails and then passes within a few runs, append a flakiness warning to the comment:

```
# Pseudo-code added to the diagnosis step
if test_name in recent_flakes(repo, window_hours=24):
    diagnosis["root_cause"] += " (Note: this test has flaked recently.)"
```

Even better: add a retry suggestion to the comment so the developer knows to rerun before rewriting anything.

Break 2: The Log File Exceeds Context

Some CI pipelines produce megabytes of output. Even after trimming to 200 lines, each line might be extremely long (e.g., minified JavaScript dumps). The combined text can still blow past the LLM's context limit. On an 8B parameter model running locally, you might hit a hard ceiling at 4,096 tokens.

Fix: Add a second trimming pass after splitting by line length. If a single line exceeds, say, 500 characters, truncate it with an ellipsis. Also count total characters and enforce a ceiling:

```
MAX_CHARS = 8000

lines = log_tail.splitlines()
lines = [line[:500] + "..." if len(line) > 500 else line for line
in lines]
trimmed = "\n".join(lines)
if len(trimmed) > MAX_CHARS:
    trimmed = trimmed[-MAX_CHARS:]
```

This two-stage filter — line count, then character budget — guarantees you stay within limits without losing the tail.

Break 3: The Ambiguous Error

The log ends with Process completed with exit code 1. No traceback. No stderr. Just a silent failure. The LLM returns “Root cause: Unknown.” The comment is useless.

Fix: Fall back to broader context. If the tail is unrevealing, fetch more lines — perhaps the first 50 lines of the log, where environment setup happens — and append them to the prompt with a label:

```
if "error" not in log_tail.lower() and "traceback" not in
log_tail.lower():
    head_context = fetch_head_context(run_id, repo, head_lines=50)
    log_tail = f"[Build head]\n{head_context}\n\n[Build tail]
\n{log_tail}"
```

Giving the model both the beginning and the end of the log often reveals setup failures that the tail alone obscures.

Break 4: Rate Limit on the GitHub API

GitHub allows 1,500 requests per hour for authenticated users, which sounds generous until you start polling every five minutes *and* fetching logs *and* posting comments. Add a second repository and you’re halfway to the cap before lunch.

Fix: Implement response caching with a simple in-memory dict, and respect the X-RateLimit-Remaining header:

```
_cache = {}

def rate_limit_safe_get(client: httpx.Client, path: str,
cache_ttl: int = 60) -> httpx.Response:
    """GET a GitHub API endpoint, using a simple memory cache.
    """
    now = time.time()
    if path in _cache:
        cached_resp, cached_at = _cache[path]
        if now - cached_at < cache_ttl:
            return cached_resp

    resp = client.get(path)
    remaining = int(resp.headers.get("X-RateLimit-Remaining", 1))
    if remaining < 5:
        print(f"Rate limit warning: {remaining} requests left.")
```

```
_cache[path] = (resp, now)
return resp
```

Also slow your poll interval. Every five minutes is overkill for most teams. Every fifteen is usually fine.

Break 5: The Model Hallucinates a Fix

The log shows `ModuleNotFoundError: No module named 'pydantic_settings'`. The LLM suggests adding `pip install pydantic-config` to the requirements file. That package doesn't exist. A junior developer might run the command, get another error, and distrust the agent forever.

Fix: Never present LLM suggestions as commands without human review. That's why our comment template includes the disclaimer: *"This comment was generated automatically. Please verify before acting."* Furthermore, you can add a validation layer: if the suggested fix mentions a package name, check PyPI or npm before surfacing it. A HEAD request to https://pypi.org/project/{pkg_name}/ is cheap. If it returns 404, rewrite the suggestion to note that the package couldn't be verified.

```
if "pip install" in fix_text:
    pkg = extract_package_name(fix_text)
    if not package_exists_on_pypi(pkg):
        fix_text += f" (Warning: package '{pkg}' not found on
PyPI.)"
```

This doesn't eliminate hallucination — nothing does — but it reduces the blast radius.

Try It Now: Three Variations

You have a working watchdog. Stretch it.

Variation 1: Auto-Retry Flaky Tests

If the diagnosis identifies a known flaky test, trigger a rerun via the GitHub Actions API instead of just commenting. Use the `POST /repos/{repo}/actions/runs/{run_id}/rerun-failed-jobs` endpoint. Only do this once per run, and always log the retry so infinite loops don't drain your Actions minutes.

This teaches you about **action vs. observation**: when the agent moves from diagnosis to direct intervention, it needs stronger guardrails because it can now spend real money (GitHub Actions compute) without asking.

Variation 2: Compare Failures for Patterns

Store the last fifty diagnoses in a local SQLite database. Each time a failure hits, query the database for recent failures with the same root cause on the same branch. If three of the last five failures match “missing env var REDIS_URL”, surface that trend in the PR comment:

Pattern detected: This branch has failed 3 times in the last 24 hours with environment variable issues. Consider checking `.env.example`.

This is your first step toward **anomaly detection over time**, which we’ll return to in the monitoring chapter.

Variation 3: Slack Alert on New Failure Type

Integrate Slack via incoming webhooks. When a failure’s root cause has not been seen in the last seven days, post to a `#build-alerts` channel. Known failures only get PR comments; novel failures get the broader audience. This teaches **routing**: different signals deserve different channels, and the agent should decide which one based on context.

The OpenClaw Way

OpenClaw handles the polling and commenting loop through its Agent and Scheduler modules. Instead of writing `fetch_failed_runs` by hand, you define a GitHub toolset that exposes high-level methods like `get_failed_runs(repo)`, `get_logs(run_id)`, and `comment_on_pr(pr_number, body)`. OpenClaw manages rate limiting, caching, and retry logic automatically. The toolset description is parsed from the method docstrings, so the model knows which tools are available without a custom system prompt.

You configure the agent with a reasoning strategy — “react” works well here — and a state object that tracks which runs have already been commented on. The scheduler evaluates the agent every N minutes. If you want the `only_first_failure` behavior, you set a deduplication policy on the state object. OpenClaw’s cost tracker also shines in this use case because CI polling agents can generate a surprising number of token

calls if left unchecked; the framework aborts the session if it exceeds your per-run budget. The implementation is about twenty lines of declarative configuration, but the underlying concepts — loop, tool registry, structured prompting — are exactly what you built in raw Python.

The Hermes Agent Way

Hermes Agent approaches CI monitoring as a conversation with a system rather than a scheduled script. You start a persistent Hermes session and instruct it: “Watch the repository my-org/my-repo for CI failures. When one happens, read the logs, diagnose the error, and draft a comment.” Hermes maintains the state of which runs it has seen across the session, so you don’t need an external database for deduplication. It uses its built-in code execution tool to parse logs and its web search tool to verify package names, which partially addresses the hallucination problem we discussed.

Because Hermes is conversation-first, you can change the behavior mid-session: “From now on, also post to Slack when a test fails on the production branch.” No code redeploy. The trade-off is that Hermes controls the polling interval and the tool selection, which means you can’t optimize the rate limiting as finely as in a custom script. For a team that wants CI diagnostics with no infrastructure setup, Hermes is the faster path. For a team that needs custom retries, pattern matching, and tight cost control, raw Python or OpenClaw wins.

What You Built

In this chapter, you built a CI/CD Watchdog Agent from scratch:

- A GitHub API client with proper authentication headers, timeouts, and version pinning.
- A polling function that finds failed and timed-out workflow runs.
- A log fetcher that downloads build output and aggressively trims it to the last N lines.
- An LLM diagnosis pipeline that extracts root cause, error excerpts, and suggested fixes from structured JSON output.
- A PR comment poster that surfaces the diagnosis where the team already works.

- A configurable flag to limit comments to the first failure, reducing noise.
- Error handling for timeouts, 404s, 401s, rate limits, parsing failures, and hallucinated suggestions.
- Break-and-fix strategies for flaky tests, oversized logs, ambiguous errors, rate limits, and bad fix suggestions.

You also learned why fast diagnosis matters more than perfect prevention, how trim logs before burning tokens, and how to reduce the blast radius when an LLM guesses wrong. The pipeline you built is small — under a hundred lines — but it solves a real problem that costs engineering teams hours every week.

Cost for this chapter: Approximately \$0.00 if you use a local Ollama model. If you substitute GPT-4o mini for diagnosis, expect roughly \$0.003 per failure investigated (based on ~2,000 tokens per call at \$0.60 per million tokens). For a team with ten failures a day, that's about six cents daily, or two dollars a month — significantly less than the engineering time saved.

Chapter 12: Project 12 — Document Generator Agent

Weekly status report: generated from three data sources, formatted, and saved before your coffee cools.

There was a year of my career where every Friday afternoon disappeared into a Word document. I would open three browser tabs — Slack, a project tracker, and an email thread from my manager — and I would manually copy, paste, reformat, and re-copy until a status report emerged. The headings never lined up. The table of contents broke every second week. And every time I pasted a bullet list from Slack into Word, the font changed to something that looked like it came from a typewriter in 1974.

The worst part wasn't the tedium. It was the fragility. If I missed one update from the tracker, the report was wrong. If I pasted a link incorrectly, it was broken. If someone asked me to regenerate the report for a different audience, I started over from scratch. It was pure manual labor, dressed up in business casual.

That’s when I discovered templating, and specifically Jinja2. The idea is almost insultingly simple: you write the document once, leaving blanks where the data goes. Then you feed it a dictionary — or a CSV, or an API response — and the blanks fill themselves in. No copy-paste. No broken fonts. No Friday afternoons sacrificed to Microsoft Word. In this chapter, you’re going to build an agent that does exactly that: it reads data from multiple sources, pours it into a template, outputs clean Markdown, and converts it to a Word document you can actually send to your boss.

Why Templating Beats Manual Formatting

Here’s the thing about documents: the structure is usually 90% identical every time. A weekly report always has a header, an updates section, maybe a blockers section, and a sign-off. The only thing that changes is the data inside those sections. When you build a document by hand, you’re doing two jobs at once: defining the structure, and inserting the data. You’re also doing both jobs poorly, because human brains are terrible at repetitive formatting and even worse at noticing missing fields.

Templating separates those two jobs. The template owns the structure. The data owns the content. The agent’s only job is to marry them. This separation means you can change the structure without touching the data pipeline, or change the data source without rewriting the document. It also means the agent can skip entire sections if the data isn’t there. No updates this week? The entire “Completed This Week” section vanishes automatically.

In this project, we’ll use Jinja2, the same templating engine that powers Flask, Ansible, and countless static site generators. It’s battle-tested, Python-native, and expressive enough to handle variables, conditionals, loops, and filters. We’ll read data from dictionaries and CSV files, render a Markdown document, and convert it to .docx using Pandoc — a free command-line tool that eats Markdown for breakfast.

Jinja2: Variables, Conditionals, and Loops

Before we build the agent, let’s look at a template in isolation. Save the following as `report.md.j2` in your project directory:

```
# Weekly Status Report – {{ team_name }}
```

```

**Week of:** {{ week_of }}
**Prepared by:** {{ author }}
**Overall Status:** {{ status | upper }}

{% if updates %}
## Completed This Week
{% for item in updates %}
- **{{ item.project }}** : {{ item.description }}
  Status: `{{ item.state }}`
{% endfor %}
{% else %}
## Completed This Week
*No updates reported this week.*
{% endif %}

{% if blockers %}
## Blockers
{% for b in blockers %}
- {{ b.description }} – *Owner: {{ b.owner }}, ETA: {{ b.eta }}*
{% endfor %}
{% endif %}

---
*Generated by Document Generator Agent*

```

Let's walk through what this template does. The double curly braces `{{ team_name }}` are variable interpolations. When we render the template with a dictionary containing `"team_name": "Platform Engineering"`, that spot becomes "Platform Engineering". The `| upper` filter converts the status value to uppercase, so `"green"` becomes `"GREEN"`.

The `{% if updates %}` block is a conditional. If the updates list is non-empty, the template prints a heading and loops over the list. The `{% for item in updates %}` is a loop block: it repeats its contents once per item, with `item` bound to each dictionary in the list. If updates is empty or missing, the `{% else %}` branch prints a quiet note instead. At the bottom, `{% if blockers %}` hides the entire blockers section when there aren't any.

This is the whole grammar you need for 90% of document generation. Variables for fields, loops for lists, conditionals for optional sections. Jinja2 has dozens of other features, but these three are the workhorses.

Feeding the Agent: From Dictionaries and CSVs

The template is useless without data. In real life, that data usually comes from multiple places: a project management API, a locally maintained notes file, a spreadsheet exported to CSV. Our agent needs to collect it all into a single dictionary that matches the variable names in the template.

Here's a module that loads data from a static dictionary and from a CSV file, then merges them into a single context:

```
import csv
import json
from pathlib import Path
from typing import Any

def load_context_from_dict(data: dict[str, Any]) -> dict[str, Any]:
    """Return a copy of the provided dictionary for template rendering."""
    return data.copy()

def load_csv_rows(path: str) -> list[dict[str, str]]:
    """Read a CSV file and return rows as a list of dictionaries."""
    file_path = Path(path)
    if not file_path.exists():
        raise FileNotFoundError(f"CSV file not found: {path}")
    with file_path.open(newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))

def build_report_context(csv_path: str) -> dict[str, Any]:
    """Build a unified context dict from static data and a CSV file."""
    context = {
        "team_name": "Product Infrastructure",
        "week_of": "2026-04-14",
        "author": "Jordan Smith",
        "status": "green",
        "blockers": [
            {
                "description": "Waiting on vendor API
credentials",
                "owner": "DevOps",
                "eta": "2026-04-18",
```

```

        },
    ],
}
context["updates"] = load_csv_rows(csv_path)
return context

```

The `load_csv_rows` function uses Python's standard `csv.DictReader`, which is one of the most underappreciated tools in the standard library. It reads the first row as headers, then yields each subsequent row as a dictionary where keys are header names and values are strings. This meshes perfectly with Jinja2: if your CSV has columns named `project`, `description`, and `state`, then `item.project` inside the template just works.

The `build_report_context` function is the agent's integration point. It starts with hard-coded metadata that rarely changes — team name, author, week — and grafts on dynamic data from the CSV. In a production agent, you'd replace the static metadata with an API call or another config file. But the pattern stays the same: read, normalize, merge.

Rendering and Converting: From Markdown to .docx

Now we render. The Jinja2 Environment loads templates from disk, compiles them, and fills in the blanks. Here's the rendering layer:

```

import subprocess
from pathlib import Path

from jinja2 import Environment, FileSystemLoader,
TemplateNotFound, UndefinedError

def render_markdown(template_name: str, context: dict) -> str:
    """Render a Jinja2 template to a Markdown string."""
    env = Environment(loader=FileSystemLoader("."))
    try:
        template = env.get_template(template_name)
    except TemplateNotFound as exc:
        raise FileNotFoundError(f"Template file not found:
{template_name}") from exc
    return template.render(context)

def save_markdown(content: str, output_path: str) -> Path:
    """Write the rendered Markdown content to disk, creating

```

```

parent dirs if needed."""
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(content, encoding="utf-8")
    return path

def convert_markdown_to_docx(markdown_path: str, docx_path: str) -
> None:
    """Convert a Markdown file to .docx using Pandoc."""
    try:
        subprocess.run(
            [
                "pandoc",
                markdown_path,
                "-o",
                docx_path,
                "--from",
                "markdown",
                "--to",
                "docx",
            ],
            capture_output=True,
            text=True,
            check=True,
        )
    except subprocess.CalledProcessError as exc:
        raise RuntimeError(f"Pandoc conversion failed:
{exc.stderr}") from exc
    except FileNotFoundError as exc:
        raise RuntimeError(
            "pandoc executable not found. Install from https://
pandoc.org/installing.html"
        ) from exc

```

The `render_markdown` function creates a Jinja2 environment pointed at the current directory. It loads the template by filename, then calls `.render(context)` to produce a string. That string is pure Markdown — headers, bullets, bold text, and all — which is both human-readable and machine-convertible.

The `convert_markdown_to_docx` function shells out to Pandoc. Pandoc is the universal document converter: it understands dozens of input formats and can emit dozens of output formats. We're asking it to translate Markdown into .docx, which is what most corporate environments expect. The `--from markdown --to docx` flags are explicit, even though Pandoc often guesses correctly, because explicit flags survive changes to file extensions.

The Main Script: Tying It All Together

Here's the complete script. Save it as `document_generator.py`:

```
"""Document Generator Agent – weekly status report from template +
data."""

import csv
import subprocess
from pathlib import Path
from typing import Any

from jinja2 import Environment, FileSystemLoader, TemplateNotFound

def load_csv_rows(path: str) -> list[dict[str, str]]:
    """Read a CSV file and return rows as a list of
    dictionaries."""
    file_path = Path(path)
    if not file_path.exists():
        raise FileNotFoundError(f"CSV file not found: {path}")
    with file_path.open(newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))

def render_markdown(template_name: str, context: dict[str, Any]) -
> str:
    """Render a Jinja2 template to a Markdown string."""
    env = Environment(loader=FileSystemLoader("."))
    try:
        template = env.get_template(template_name)
    except TemplateNotFound as exc:
        raise FileNotFoundError(f"Template file not found:
{template_name}") from exc
    try:
        return template.render(context)
    except Exception as exc:
        raise RuntimeError(f"Template rendering failed: {exc}")
    from exc

def save_markdown(content: str, output_path: str) -> Path:
    """Write rendered Markdown to disk, creating parent
    directories if needed."""
    path = Path(output_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(content, encoding="utf-8")
    return path

def convert_markdown_to_docx(markdown_path: str, docx_path: str) -
```



```

> None:
    """Convert a Markdown file to .docx using Pandoc."""
    try:
        subprocess.run(
            [
                "pandoc",
                markdown_path,
                "-o",
                docx_path,
                "--from",
                "markdown",
                "--to",
                "docx",
            ],
            capture_output=True,
            text=True,
            check=True,
        )
    except subprocess.CalledProcessError as exc:
        raise RuntimeError(f"Pandoc conversion failed:
{exc.stderr}") from exc
    except FileNotFoundError as exc:
        raise RuntimeError(
            "pandoc not found. Install Pandoc: https://pandoc.org/
installing.html"
        ) from exc

```

```

def main() -> None:
    """Generate the weekly status report from template and data
sources."""
    csv_path = "updates.csv"
    template_name = "report.md.j2"
    md_output = "output/weekly_report.md"
    docx_output = "output/weekly_report.docx"

    # Build context from static metadata and CSV data
    context: dict[str, Any] = {
        "team_name": "Platform Engineering",
        "week_of": "2026-04-14",
        "author": "Alex Chen",
        "status": "green",
        "blockers": [
            {
                "description": "Vendor API credentials pending",
                "owner": "DevOps",
                "eta": "2026-04-18",
            }
        ],
    }

    context["updates"] = load_csv_rows(csv_path)

```

```

# Render and save Markdown
markdown = render_markdown(template_name, context)
md_path = save_markdown(markdown, md_output)
print(f"Markdown saved to {md_path}")

# Convert to .docx
convert_markdown_to_docx(str(md_path), docx_output)
print(f"Word document saved to {docx_output}")

if __name__ == "__main__":
    main()

```

And here's a sample updates.csv to test with:

```

project,description,state
Auth Service,Migrated login flow to OAuth2,complete
Metrics Pipeline,Added latency histograms to all endpoints,in
progress
API Gateway,Deprecated v1 routes,complete

```

Run it:

```
python document_generator.py
```

If Pandoc is installed, you'll get two files in the output/ directory: a Markdown file you can read in any text editor, and a Word document you can attach to an email. The Markdown is the source of truth; the .docx is just a rendering target. This matters because Markdown is diffable, version-controllable, and readable in a terminal. A .docx file is a binary blob that laughs at your git diff.

Conditional Sections: Hide What Isn't There

One of the most powerful patterns in document generation is conditional omission. Look at the template again:

```

{% if blockers %}
## Blockers
{% for b in blockers %}
- {{ b.description }} – *Owner: {{ b.owner }}, ETA: {{ b.eta }}*
{% endfor %}
{% endif %}

```

If blockers is an empty list, the entire section — heading and all — disappears. The document doesn't say "Blockers: none." It just doesn't have a Blockers section. This makes the report shorter when things are going well, which respects your reader's time.

You can extend this pattern indefinitely. A “Risks” section that only appears when risk level is above “low.” An “Appendix” that only appears when detailed logs are attached. A “Pricing” table that only renders when a quote has been finalized. The template becomes a smart document that adapts to the data, rather than a static canvas that leaves empty boxes everywhere.

The trick is using truthiness correctly. In Jinja2, an empty list is falsy, an empty string is falsy, and None is falsy. But the integer 0 is also falsy, so if you’re checking `{% if item_count %}` and `item_count` happens to be zero, the section hides even though you might want to show “0 items found.” For numeric fields, prefer explicit comparison: `{% if item_count is not none %}` or `{% if item_count >= 0 %}`.

Break and Fix: Five Ways to Break a Document

Templates feel safe until they aren’t. Because the template and the data are written separately, they can drift out of sync. Here are five ways this project breaks in production, and how to fix each one.

Break 1: Template Syntax Error

You edit the template late at night and forget to close a loop:

```
{% for item in updates %}
- {{ item.project }}
```

Run the script, and Jinja2 throws:

```
jinja2.exceptions.TemplateSyntaxError: unexpected end of template
```

Why it fails: Every `{% for %}` must have a matching `{% endfor %}`. Jinja2 compiles the template before rendering it, and the compiler hits the end of the file while still inside the loop.

Fix: Always validate templates in a test harness before committing them. For a quick check, you can ask Jinja2 to compile without rendering:

```
from jinja2 import Environment, BaseLoader

def validate_template(source: str) -> None:
    """Parse a template string and raise on syntax errors."""
```

```
env = Environment(loader=BaseLoader())
env.parse(source)
```

But the real fix is code review. Templates are code. Treat them like it.

Break 2: Missing Data Field

The template references `{{ manager }}`, but the context dictionary only contains "team_name", "author", and "status". By default, Jinja2 silently substitutes an empty string. Your report says “Manager:” with nothing after it.

Why it fails: Jinja2’s default Undefined class is forgiving. It turns missing variables into empty strings instead of raising errors, which is friendly for web pages but dangerous for financial reports.

Fix: Switch to StrictUndefined during development:

```
from jinja2 import StrictUndefined

env = Environment(loader=FileSystemLoader("."),
undefined=StrictUndefined)
```

Now a missing variable raises `jinja2.exceptions.UndefinedError`: 'manager' is undefined. It hurts on purpose, so you catch the mismatch before your boss does. For production, you might relax back to `DebugUndefined`, which prints the variable name as a visible marker instead of a blank.

Break 3: Formatting Lost in Conversion

Your Markdown template includes a beautiful table using raw HTML `<table>` tags. You run it through Pandoc, and the .docx output contains plain paragraphs with no table structure.

Why it fails: Pandoc’s Markdown reader supports CommonMark and extensions, but raw HTML in Markdown is handled heuristically. Complex HTML tables often get flattened into paragraphs.

Fix: Use Pandoc-flavored Markdown tables instead of HTML:

Project	Status	
-----	-----	
Auth	Done	

Alternatively, pass `--from markdown+raw_html` to Pandoc if you must use HTML, but the cleanest fix is to stay in native Markdown syntax. Markdown was invented so you wouldn't have to write HTML.

Break 4: Invalid Output Path

You set `md_output = "reports/weekly/weekly_report.md"`, but the `reports/weekly/` directory doesn't exist. The script crashes with `FileNotFoundError`.

Why it fails: `Path.write_text()` does not create parent directories. It assumes the folder already exists, which is a reasonable default but inconvenient when your agent is writing to dated folders.

Fix: You already saw it in `save_markdown`:

```
path.parent.mkdir(parents=True, exist_ok=True)
```

This one line creates the entire directory tree if it's missing. I add it to every file-writing function I write, because it's cheaper than handling the exception later.

Break 5: Conditional Logic Inverted

You want to show a “No blockers” message only when the blockers list is empty. You accidentally write:

```
{% if not blockers %}  
## Blockers  
- Great news: no blockers this week!  
{% endif %}
```

But the heading says “Blockers” even when there are none, and when there *are* blockers, the section vanishes entirely. It's backwards.

Why it fails: The not operator flipped the condition correctly but the surrounding structure didn't match the intent. A heading named “Blockers” above a “no blockers” message is confusing, and the logic hides blockers when they exist.

Fix: Be explicit about branch content. When a section can appear or disappear, the heading belongs inside the `{% if %}`:

```
{% if blockers %}  
## Blockers  
{% for b in blockers %}  
- {{ b.description }} – Owner: {{ b.owner }}  
{% endfor %}
```

```
{% else %}  
## Blockers  
None reported.  
{% endif %}
```

Even simpler: if the section is empty, skip the heading too, as we did in the original template. Consistency matters more than cleverness.

Try It Now: Three Variations

The weekly report is just one shape. Once you have the pipeline — data in, template render, Markdown out, Pandoc convert — you can generate almost any structured document.

Variation 1: Project Proposal from Brief and Pricing

Create a template `proposal.md.j2` with variables for `client_name`, `objective`, `timeline_weeks`, and a `pricing_items` loop that outputs a table. Feed it a dictionary containing the project brief and a list of line items. The agent generates a formatted proposal ready to PDF via Pandoc's `--to pdf` flag. The magic is that you change the template, not the code, when your company's proposal format changes.

Variation 2: API Documentation from Docstrings

Write a script that walks a Python package using `inspect.getmembers()`, collects public functions and their docstrings, and builds a context dictionary of modules, classes, and methods. Feed that context into a template that outputs a `docs.md` file with headers, signatures, and docstrings rendered as description paragraphs. Run it in CI, and your API docs never drift from your code again. This is the poor man's Sphinx, and it's faster than configuring a documentation generator for a small project.

Variation 3: PowerPoint from a Markdown Outline

Pandoc can output `.pptx` directly from Markdown:

```
pandoc outline.md -o deck.pptx --to pptx
```

Build a template where top-level headings become slide titles, bullet lists become slide content, and `---` page breaks become new slides. Your agent can take a structured outline — maybe even generated by an

LLM from a one-sentence prompt — and produce a presentation file in seconds. No PowerPoint automation libraries, no COM interfaces, no compatibility hell. Just Markdown and Pandoc.

The OpenClaw Way

OpenClaw’s strength is orchestrating multi-step workflows, and document generation fits that model perfectly. In OpenClaw, you’d define three tools: one that fetches data from a CSV or API, one that renders the Jinja2 template, and one that shells out to Pandoc. The agent loop decides which tool to call based on the user’s request.

For example, a user might say, “Generate the weekly report for Platform Engineering.” The agent first calls `fetch_updates(team="Platform Engineering")`, then calls `render_template(template="report.md.j2", context=<result>)`, then calls `convert_to_docx(markdown=<rendered>)`. OpenClaw handles the intermediate state automatically, passing outputs from one tool into inputs for the next. You define the tools with `@tool` decorators and docstrings, exactly as in Chapter 1, and OpenClaw manages the JSON parsing, conversation history, and error recovery. The template files themselves stay identical to our pure Python version; only the orchestration layer changes.

The Hermes Agent Way

Hermes Agent approaches document generation from the conversation side. You don’t write a standalone script; you open a session and tell Hermes what you need. Something like:

```
> Generate a weekly status report using the template report.md.j2
and the CSV updates.csv. Save it as weekly_report.docx.
```

Hermes inspects the template, notices it needs `team_name`, `week_of`, `author`, `status`, `updates`, and `blockers`. It reads the CSV for updates, asks you to fill in the metadata if it’s not provided, renders the Markdown through its built-in code execution environment, calls Pandoc via shell execution, and presents you with the final file path. Because Hermes remembers the context, your next request — “Now do the same for the Backend team” — only requires you to change one variable. The trade-off is less automation: Hermes waits for you at each ambiguity, rather than running unattended like a cron job. But for ad-hoc reports, that conversational guidance is often exactly what you want.

What You Built

In this chapter, you built a document generation pipeline from scratch:

- A Jinja2 template with variables, loops, conditionals, and filters that adapts its structure to the data.
- A data loader that merges static dictionaries with dynamic CSV contents into a unified context.
- A rendering engine that compiles the template and outputs clean, version-controllable Markdown.
- A Pandoc converter that turns Markdown into .docx with a single subprocess call.
- Error handling for missing files, missing variables, syntax errors, invalid paths, and inverted logic.
- A main() function that generates a complete weekly status report in under a second.

You also learned why Markdown is the right intermediate format — human-readable, diffable, and Pandoc’s native input — and why separating data from presentation is the single most important design decision in document automation.

Cost for this chapter: \$0.00 if you run the local conversion with Pandoc only. If you use an LLM to draft the template or generate the CSV contents, expect roughly \$0.002–\$0.005 per run with GPT-4o mini. Pandoc itself is free and open source.

Chapter 13: Project 13 — Debate Team: Multi-Agent Consensus

The first time I watched three agents argue about a database schema, I realized one brain is never enough.

A few years ago, I sat in a conference room while six engineers shouted over each other about whether to rewrite our monolith in Go. Two people were absolutely convinced it would cut our server costs in half. One person said we’d be rewriting for a year and shipping nothing. Another person — who had the most experience but the quietest voice

— eventually asked if anyone had profiled the Ruby code first. Nobody had. We almost burned six months on a debate that nobody had bothered to frame correctly.

That meeting taught me something I now apply to every technical decision: good answers don't come from one loud voice. They come from structured disagreement. You need someone to argue *for* the idea, someone to argue *against* it, and somebody whose only job is to listen to both sides, spot the gaps, and declare when the room has actually reached a conclusion. Without all three roles, you get groupthink, bikeshedding, or the loudest person wins.

This chapter builds exactly that structure in software. Instead of six engineers, you'll spin up three agents. Instead of a conference room, you'll use a shared memory dictionary that every agent can read and write. Instead of a two-hour meeting that ends with "let's regroup next week," your agents will debate a question, track their positions, and either reach consensus or honestly report that they disagree — all in under a minute.

The question we'll use: **"Should we adopt GraphQL?"** It's a perfect debate topic because it's genuinely controversial. There are real reasons to adopt it and real reasons to stay with REST. If your agents all agree instantly, something is broken. Good disagreement is the signal that the system is working.

When One Agent Is Not Enough

Every project in this book so far has used a single agent. One loop, one set of tools, one model making decisions. That works beautifully for research, file organization, spreadsheet analysis, and email drafting. But there's a class of problems where a single perspective actively misleads you.

Imagine asking one agent, "Should we adopt GraphQL?" The agent searches the web, reads a few blog posts, and returns a confident yes or no based on whatever it saw first. There's no internal tension. No counter-argument is surfaced. No one asked, "But what about caching?" or "Have you considered our existing REST tooling?" The result is not a decision. It's a monologue.

Multi-agent systems solve this by assigning roles. Each agent is biased in a different, *deliberate* direction. The Advocate is optimized to find the strongest case for adoption. The Skeptic is optimized to find the

strongest case against it. The Synthesizer has no opinion at all — it only watches the other two and decides whether the debate has converged or whether another round is needed.

This is not three agents doing the same thing in parallel. It's a conversation with structure. The Advocate speaks first. Then the Skeptic responds to the Advocate's specific points. Then the Synthesizer reads both transcripts and either declares consensus or tells the Advocate to address the Skeptic's rebuttal. Each round refines the argument. Each round is visible to all agents through a single shared memory dictionary.

If this sounds like the scientific method, that's not an accident. Hypothesis, critique, refinement, conclusion. The difference is that the scientists are made of Python and API calls.

Role Definition: Advocate, Skeptic, Synthesizer

Before we write the loop, let's be precise about what each agent is supposed to do. The quality of your debate depends entirely on how clearly you separate these roles in the prompts.

The Advocate presents the strongest possible case for the proposition. In our GraphQL debate, the Advocate will argue for type safety, flexible client queries, reduced over-fetching, and the strength of the tooling ecosystem. The Advocate is not a neutral summarizer. It is a ** partisan ** with a license to ignore weak arguments against the proposition — except when directly challenged by the Skeptic.

The Skeptic presents the strongest possible case against the proposition. It will argue about caching complexity, the learning curve, the risk of over-fetching in the opposite direction (expensive nested queries), and the operational burden of maintaining a schema. The Skeptic does not play devil's advocate for fun. Its job is to stress-test every claim the Advocate makes.

The Synthesizer is the judge. It reads the accumulated transcript in shared memory, identifies points of agreement and disagreement, and decides whether consensus exists. The Synthesizer has a critical secondary duty: if the two sides are talking past each other, the

Synthesizer must point that out and demand clarification. It is the only agent that can terminate the debate, and it does so by writing a consensus flag into shared memory.

Here's the key insight: the actual language model behind each agent can be identical. What makes them different is the system prompt you wrap around the shared context. The Advocate sees "You are the proponent. Be persuasive." The Skeptic sees "You are the critic. Be rigorous." The Synthesizer sees "You are the facilitator. Be neutral. Look for actual convergence, not just exhaustion."

Round-Robin Communication

The debate proceeds in rounds. Each round has three turns: Advocate, Skeptic, Synthesizer. After the Synthesizer's turn, we check for consensus. If consensus is reached, we output a structured report. If not, we loop back to the Advocate for another round, but this time the Advocate sees the Skeptic's previous objections and the Synthesizer's call for refinement.

Here's what one round looks like in practice. The user asks: "Should we adopt GraphQL?"

Round 1, Turn 1 — Advocate: The Advocate reads the user question and the empty shared memory. It writes a position statement: "GraphQL offers strong typing, precise client-driven data fetching, and a rich ecosystem. The migration cost is manageable with incremental adoption via wrappers."

Round 1, Turn 2 — Skeptic: The Skeptic reads the user question and the Advocate's entry in shared memory. It responds directly to the Advocate's claims: "Incremental adoption is a myth in practice. Once you add a GraphQL layer, your caching strategy duplicates. Strong typing is nice, but it doesn't justify the operational complexity for a team already fluent in OpenAPI."

Round 1, Turn 3 — Synthesizer: The Synthesizer reads both entries. It writes: "Points of agreement: both sides acknowledge there is a tooling ecosystem. Points of disagreement: whether incremental adoption is realistic, and whether the team has existing OpenAPI fluency. No consensus reached."

At this point, `consensus_detected` checks the Synthesizer output. It sees "No consensus reached." The loop continues to Round 2.

Round 2, Turn 1 — Advocate: The Advocate now sees the Skeptic's rebuttal and the Synthesizer's framing. It adjusts: "Incremental adoption is realistic if you start with read-only queries and use a gateway pattern. This avoids touching write paths early."

And so on. Each agent's prompt includes the full shared memory dictionary serialized as context, so no agent is flying blind. The round-robin structure forces them to respond to each other, not just to the original question in isolation.

Shared Memory: Passing Context Between Agents

Shared memory is the backbone of this architecture. It's a plain Python dictionary that every agent reads at the start of its turn and writes to at the end. It has four mandatory keys:

- **topic:** the original question.
- **transcript:** a list of turn-by-turn entries, each with role, round, and content.
- **positions:** a dictionary tracking each agent's current stance, updated by the Synthesizer.
- **consensus:** a string, initially "none", eventually "yes", "no", or "disagree".

The transcript is the most important piece. Every agent's prompt includes the full transcript so far. This means the Skeptic can quote the Advocate's exact words from two rounds ago. The Synthesizer can point out when an agent has shifted its position without acknowledging it. The shared memory is the historical record that prevents the debate from restarting from scratch every round.

Here's an example of what the shared memory looks like after Round 1:

```
shared_memory = {
    "topic": "Should we adopt GraphQL?",
    "transcript": [
        {"role": "Advocate", "round": 1,
         "content": "GraphQL offers strong typing..."},
        {"role": "Skeptic", "round": 1,
         "content": "Incremental adoption is a myth..."},
        {"role": "Synthesizer", "round": 1,
```

```

        "content": "Points of agreement: tooling ecosystem. "
                    "No consensus reached."},
    ],
    "positions": {
        "Advocate": "strongly pro",
        "Skeptic": "strongly con"
    },
    "consensus": "none"
}

```

Notice that the Synthesizer is the only agent that writes to the positions and consensus keys. That's not enforced by the code — it's enforced by the system prompt. The Synthesizer is explicitly told: "You may update the consensus field. The other agents may not."

Consensus Detection and Termination

The `consensus_detected` function is the circuit breaker. It runs after every Synthesizer turn and decides whether the debate continues. There are three ways the debate ends:

1. **Consensus:** The Synthesizer writes that the panel has reached agreement. The function returns (True, report).
2. **Max rounds:** The debate has run for `MAX_ROUNDS` without consensus. The function returns (False, "Max rounds reached.").
3. **Stagnation:** The Synthesizer detects that the last two rounds are identical or that both agents are repeating themselves. The function returns (False, "Debate stalled.").

The max-rounds rule is non-negotiable. Without it, a stubborn Advocate and a stubborn Skeptic can trade the same three points forever, burning tokens and your patience. I typically set `MAX_ROUNDS = 3` for lightweight debates and `MAX_ROUNDS = 5` for consequential architectural questions. More than five rounds rarely produces new information; it produces repetition.

The stagnation detection is a soft guardrail. If the Skeptic's Round 3 argument is word-for-word identical to its Round 2 argument, the Synthesizer should flag it, and the loop should exit. Real debates converge or they diverge. They don't photocopy.

The Mermaid Diagram

Here's the flow of the multi-agent debate from start to finish:

```
flowchart TD
    U[User] -->|Query: Should we adopt GraphQL?| A[Advocate]
    A -->|Pro arguments + context| SM1[Shared Memory]
    SM1 --> S[Skeptic]
    S -->|Con arguments + rebuttal| SM2[Shared Memory]
    SM2 --> SYN[Synthesizer]
    SYN -->|Check convergence| C{Consensus?}
    C -->|Yes| O[Output Structured Report]
    C -->|No| SM3[Shared Memory updated]
    SM3 --> A
    C -->|Max rounds reached| O2[Output with Dissents]
```

The diagram makes the loop explicit. The only paths out are the Synthesizer declaring consensus or the safety limiter hitting maximum rounds. Everything else cycles through the Advocate again with strictly more context than the previous round.

Building the Debate in Pure Python

Let's code it. We'll define an Agent class parameterized by role, a shared_memory dictionary that every turn updates, a consensus_detected function, and a round-robin run_debate loop.

For this project, we'll use the same Ollama setup from Chapter 1. You already have it running locally. Each agent call is an independent HTTP POST to the Ollama API, with a role-specific system prompt prepended to the shared transcript.

Create a new file called debate_team.py:

```
import json
import httpx

OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL = "llama3.1:8b"
MAX_ROUNDS = 3

class Agent:
    """A debater with a fixed role and access to shared memory."""

    def __init__(self, role: str, model: str = MODEL):
        """
        Initialize an agent.
```

```

    Args:
        role: One of 'Advocate', 'Skeptic', or 'Synthesizer'.
        model: The Ollama model name to query.
    """
    self.role = role
    self.model = model

def act(self, shared_memory: dict) -> str:
    """
    Generate a response based on role and current shared
memory.

    Args:
        shared_memory: The debate state dictionary.

    Returns:
        The agent's response text.
    """
    prompt = self._build_prompt(shared_memory)
    response = httpx.post(
        OLLAMA_URL,
        json={"model": self.model, "prompt": prompt, "stream":
False},
        timeout=120,
    ).json()
    return response["response"].strip()

def _build_prompt(self, shared_memory: dict) -> str:
    """Construct a role-specific prompt from shared memory."""
    system = f"You are the {self.role} in a structured debate.
"
    if self.role == "Advocate":
        system += (
            "Your job is to argue FOR the proposition. Be
specific, "
            "cite concrete advantages, and address the
Skeptic's previous "
            "objections if any exist."
        )
    elif self.role == "Skeptic":
        system += (
            "Your job is to argue AGAINST the proposition. Be
rigorous, "
            "cite concrete risks, and directly challenge the
Advocate's "
            "claims. Do not be neutral."
        )
    else:
        system += (
            "Your job is to read the full transcript, identify
points of "

```

```

        "agreement and disagreement, and declare whether
consensus has "
        "been reached. If not, explain what gap remains.
You may set "
        "consensus to 'yes', 'no', 'disagree', or 'none'."
    )

```

```

    transcript_lines = [
        f"[{t['role']}] | Round {t['round']} {t['content']}"
        for t in shared_memory.get("transcript", [])
    ]
    transcript_block = "\n".join(transcript_lines) or "No
prior turns."

```

```

    return (
        f"{system}\n\n"
        f"Topic: {shared_memory['topic']}\n\n"
        f"Transcript so far:\n{transcript_block}\n\n"
        f"Now respond as the {self.role}."
    )

```

```

def consensus_detected(
    shared_memory: dict, round_num: int, max_rounds: int =
MAX_ROUNDS
) -> tuple[bool, str]:
    """

```

Determine whether the debate should terminate.

Args:

shared_memory: The debate state dictionary.
 round_num: The current round (1-indexed).
 max_rounds: Maximum allowed rounds before forced

termination.

Returns:

A tuple of (should_stop, reason).

"""

```

    consensus = shared_memory.get("consensus", "none")
    if consensus in {"yes", "no", "disagree"}:
        return True, f"Consensus declared: {consensus}"
    if round_num >= max_rounds:
        return True, "Max rounds reached without consensus"
    return False, "Continue debate"

```

```

def update_shared_memory(
    shared_memory: dict, role: str, round_num: int, content: str
) -> None:
    """

```

Append a new turn to the shared transcript and update
 positions if Synthesizer.


```

    Args:
        shared_memory: The debate state dictionary (mutated in
place).
        role: The agent role that produced this content.
        round_num: The current debate round.
        content: The agent's response text.
    """
    shared_memory["transcript"].append(
        {"role": role, "round": round_num, "content": content}
    )

    # If the Synthesizer declared a consensus, extract it
    if role == "Synthesizer":
        lowered = content.lower()
        for token in ("consensus reached", "we agree",
"conclusion: yes"):
            if token in lowered:
                shared_memory["consensus"] = "yes"
                break
        for token in ("conclusion: no", "reject the proposition"):
            if token in lowered:
                shared_memory["consensus"] = "no"
                break
        if "agree to disagree" in lowered or "no consensus" in
lowered:
            shared_memory["consensus"] = "disagree"

def generate_report(shared_memory: dict) -> str:
    """
    Build a structured markdown report from the final shared
memory.

    Args:
        shared_memory: The debate state dictionary.

    Returns:
        A formatted markdown string.
    """
    lines = [
        f"# Debate Report: {shared_memory['topic']}",
        "",
        f "***Consensus:** {shared_memory.get('consensus',
'none')}",
        "",
        "## Positions",
    ] + [
        f"- **{k}:** {v}"
        for k, v in shared_memory.get("positions", {}).items()
    ] + [
        "",

```

```

        "## Transcript",
    ] + [
        f"### Round {t['round']} - {t['role']}\n{t['content']}\n"
        for t in shared_memory.get("transcript", [])
    ]
    return "\n".join(lines)

def run_debate(topic: str, max_rounds: int = MAX_ROUNDS) -> str:
    """
    Execute a multi-agent debate on a given topic.

    Args:
        topic: The proposition to debate.
        max_rounds: Upper bound on debate rounds.

    Returns:
        A markdown report string.
    """
    shared_memory = {
        "topic": topic,
        "transcript": [],
        "positions": {"Advocate": "unknown", "Skeptic":
"unknown"},
        "consensus": "none",
    }

    advocate = Agent("Advocate")
    skeptic = Agent("Skeptic")
    synthesizer = Agent("Synthesizer")

    for round_num in range(1, max_rounds + 1):
        print(f"\n=== Round {round_num} ===")

        # Advocate turn
        adv_response = advocate.act(shared_memory)
        print(f"[Advocate] {adv_response[:200]}...")
        update_shared_memory(shared_memory, "Advocate", round_num,
adv_response)

        # Skeptic turn
        sk_response = skeptic.act(shared_memory)
        print(f"[Skeptic] {sk_response[:200]}...")
        update_shared_memory(shared_memory, "Skeptic", round_num,
sk_response)

        # Synthesizer turn
        syn_response = synthesizer.act(shared_memory)
        print(f"[Synthesizer] {syn_response[:200]}...")
        update_shared_memory(shared_memory, "Synthesizer",
round_num, syn_response)

```

```

        # Check termination
        stop, reason = consensus_detected(shared_memory,
round_num, max_rounds)
        if stop:
            print(f"\nDebate terminated: {reason}")
            break

    return generate_report(shared_memory)

def main() -> None:
    """Entry point: run the GraphQL debate and print the
report."""
    report = run_debate("Should we adopt GraphQL?")
    print("\n" + "=" * 50)
    print(report)

if __name__ == "__main__":
    main()

```

Run it with:

```
python debate_team.py
```

The output will vary depending on your model and temperature, but a healthy debate produces something like this:

```

=== Round 1 ===
[Advocate] GraphQL's type system and client-driven queries reduce
over-fetching...
[Skeptic] The operational burden of maintaining a schema and the
N+1 query risk...
[Synthesizer] Advocate emphasizes type safety; Skeptic warns of
operational complexity. No consensus.

=== Round 2 ===
[Advocate] Incremental adoption via gateway wrappers allows a
gradual transition...
[Skeptic] Gateways add latency and another failure mode. REST
already works...
[Synthesizer] Both sides acknowledge migration cost but disagree
on whether the benefits justify it. No consensus.

=== Round 3 ===
[Advocate] The developer experience gains in mobile clients
specifically...
[Skeptic] Mobile teams can use BFF patterns without the full
GraphQL stack...
[Synthesizer] Positions remain distinct. Consensus: disagree.
Debate terminated.

```

Debate terminated: Consensus declared: disagree

Notice what happened here. In Round 3, the Synthesizer recognized that both agents had refined their arguments but neither had conceded a material point. Instead of looping forever, it declared "disagree" — an honest answer. The final report includes the full transcript, the final consensus status, and every position. That report is more useful than a single agent's yes-or-no answer because it preserves the nuance.

Break and Fix

Every project in this book has a Break and Fix section. Multi-agent systems fail in fascinating ways because the failure modes are social as well as technical. Here are five ways this debate can go wrong, and how to recover.

Break 1: Endless Loops

Symptom: The debate runs for twenty rounds without terminating. Your terminal is full of Advocate and Skeptic excerpts, and the Synthesizer keeps saying “No consensus yet, let’s keep going.”

Root cause: The Synthesizer is too lenient, or its prompt doesn’t explicitly authorize it to declare consensus or disagreement. It treats its job as facilitating an open-ended conversation rather than making a binding decision.

Fix: Strengthen the Synthesizer’s prompt with an explicit mandate and a clear instruction on when to stop. Add these sentences to the Synthesizer system prompt:

```
You MUST declare consensus when both agents address each other's
strongest
point and either agree or present an irreconcilable difference.
You MUST
stop the debate after three rounds if no new arguments have
appeared.
```

Also tighten the `consensus_detected` helper to check for keyword phrases that force termination, and lower `MAX_ROUNDS` to 3 if the topic doesn’t warrant deep exploration.

Break 2: Advocate Dominates

Symptom: The Advocate's responses are twice as long as the Skeptic's. The Skeptic starts its turns with phrases like "While the Advocate makes some good points..." and ends up agreeing more than it refutes.

Root cause: The Skeptic's prompt is too passive. When both agents use the same base model, the model's training data leans helpful — which means conciliatory. The Skeptic needs to be actively adversarial or it will default to politeness.

Fix: Rewrite the Skeptic prompt to remove hedging language. Replace "You are the critic" with "You are the vigorous opponent. Your goal is to find the fatal flaw in the Advocate's case. Start each rebuttal with the Advocate's weakest point. Do not concede ground unless the Advocate has proven their case beyond reasonable doubt."

You can also add a word-count target to both prompts, forcing parity: "Respond in 80–120 words."

Break 3: Synthesizer Never Declares

Symptom: After every round, the Synthesizer says "Interesting perspectives on both sides" and suggests another round. It never updates the consensus field. The `consensus_detected` function relies on the Synthesizer writing explicit keywords, but the Synthesizer writes prose instead.

Root cause: The Synthesizer is treating the task as writing an essay rather than updating a state machine. It doesn't know that another function is scanning its output for trigger phrases.

Fix: Two options. Option A: instruct the Synthesizer to end every response with a structured line like `CONSENSUS_STATUS: none` or `CONSENSUS_STATUS: disagree`. Then parse that line explicitly in `update_shared_memory`. Option B: remove the keyword guessing entirely and ask the Synthesizer to emit JSON at the end of every turn:

```
{"consensus": "disagree", "reason": "Irreconcilable migration-risk disagreement"}
```

Parsing JSON is far more reliable than grepping prose for the word "agree."

Break 4: Roles Blur

Symptom: The Advocate starts critiquing its own position. The Skeptic starts proposing alternatives rather than attacking the proposition. The Synthesizer takes sides. By Round 2, all three agents sound the same.

Root cause: The system prompt boundaries are too weak, or the shared memory format allows each agent to see the other agents' system prompts (it shouldn't). More commonly, the model is instruction-tuned to be balanced and helpful, so a mildly worded prompt gets overwritten by the model's default behavior.

Fix: Make the role prompts explicit and repetitive. Re-state the role at the beginning and end of every prompt. Do not show agents each other's system prompts — only their public transcript entries. If the model still drifts, increase the prompt's "temperature" toward role enforcement by adding a penalty-like framing: "If you abandon your assigned role, the debate loses its purpose. Stay in character."

Break 5: Consensus on the Wrong Answer

Symptom: The Synthesizer declares consensus that GraphQL should be adopted, but the Skeptic never addressed caching, and the Advocate never justified the migration timeline. The "consensus" is just exhaustion.

Root cause: The agents agreed because the Synthesizer confused silence with assent. When one agent stops introducing new arguments, the Synthesizer might interpret that as concession. Or the Synthesizer itself may have a latent bias toward one side based on its training data.

Fix: Add a validation step. After the Synthesizer declares consensus, run a fourth agent — the Auditor — whose only job is to read the final transcript and verify that every major claim was actually addressed. The Auditor's prompt: "Check whether the declared consensus is supported by the arguments in the transcript. If a major objection was ignored, flag it and reject the consensus." This adds one extra API call, but it catches false consensus before you ship a bad architectural decision.

Try It Now

Once your three-agent debate runs cleanly, stretch it. These variations teach you how multi-agent architectures scale beyond a simple pro-con panel.

Variation 1: Different Models Per Role

Not every role needs the same model. Run the Advocate on a fast, cheap model like GPT-4o mini because creative persuasion doesn't require deep reasoning. Run the Skeptic on a more capable model like GPT-4o because finding holes requires stronger analytical skills. Run the Synthesizer on your local Ollama model because summarization is easy and you want to save money.

This teaches you that multi-agent systems are also model-routing systems. You can optimize cost and quality independently per role. In your Agent class, pass a different model string to each instantiation.

Variation 2: Logical-Fallacy Observer

Add a fourth agent: the Observer. It sits outside the debate loop and reads every turn as it happens. Its job is not to argue but to flag rhetorical weaknesses. If the Advocate says "Everyone is using GraphQL, so we should too," the Observer writes: [FALLACY: bandwagon appeal] Detected in Round 2, Advocate. If the Skeptic says "The last team that tried this failed, so we will too," the Observer writes: [FALLACY: hasty generalization].

The Observer's output gets appended to shared memory, visible to all agents in subsequent rounds. This raises the quality of the entire debate by penalizing sloppy reasoning. Implement it by adding an Observer agent class that runs after each Skeptic turn.

Variation 3: Weighted Votes by Expertise

Not all disagreements are equal. If the Skeptic's objection is "We don't know how to cache GraphQL queries," and your team has a world-class caching engineer, that objection should carry less weight than if your team is all frontend developers.

Add an expertise dictionary to the shared memory: {"caching": "high", "frontend": "low", "operations": "medium"}. After each round, weight each agent's arguments against this map. In the generate_report

function, add a section called **Confidence Adjustment** that notes which objections are mitigated by in-house expertise and which are not. This turns the debate from a philosophy seminar into a decision-support tool grounded in your team’s actual capabilities.

The OpenClaw Way

OpenClaw handles multi-agent orchestration through its Crew primitive. Instead of manually instantiating three agents and wiring a round-robin loop with shared memory, you define a Crew with `Process.sequential` or `Process.round_robin`, assign tasks to each agent, and let OpenClaw manage the memory layer.

In OpenClaw, you would create three agents with distinct goal and backstory fields — these map directly to our role prompts. You then create three Task objects: `advocate_task`, `skeptic_task`, and `synthesize_task`. The tasks are chained with a context parameter, which OpenClaw uses to build the shared memory automatically. The Crew runs, executes the tasks in order, and delegates the consensus check to a callback function you provide on the final task.

The code reduction is significant. What takes us sixty lines of prompt construction, HTTP requests, and dictionary management becomes roughly twenty lines of OpenClaw declarations. The trade-off is that you must trust OpenClaw’s default shared memory serialization, which is usually a list of task outputs but may not preserve the round-aware structure we built manually. For most debates, that’s fine. If you need strict round numbering, fall back to the pure Python version.

The Hermes Agent Way

Hermes Agent does not have a built-in “debate crew” concept. It is a single-session tool by design. But you can still approximate the Debate Team using Hermes’s session memory and tool-calling capabilities.

The approach is to run three separate Hermes sessions — one configured as Advocate, one as Skeptic, one as Synthesizer — and pipe the transcript between them manually. You start the Advocate session, paste the topic, and capture its full output. Then you start the Skeptic session, paste the topic plus the Advocate’s response, and capture its output. Finally, you start the Synthesizer session with both prior outputs and ask it to judge.

Alternatively, within a single Hermes session, you can prompt the model to adopt different personas turn by turn: “Now respond as the Advocate. Now respond as the Skeptic.” Hermes’s persistent memory within the session acts as the shared transcript. The limitation is that the same model instance holds all three roles, so role boundaries are softer than when each role runs in a separate session with an isolated system prompt.

For quick, informal debates, the single-session persona-switching approach is fast and requires no scripting. For rigorous, reproducible panels, the multi-session approach is closer to our pure Python architecture — but it sacrifices the convenience of Hermes’s built-in tool suite.

What You Built

In this chapter, you built a structured multi-agent consensus system:

- An Agent class parameterized by role, with role-specific prompt construction.
- A shared_memory dictionary that accumulates the full debate transcript and state.
- A round-robin loop that forces Advocate → Skeptic → Synthesizer ordering.
- A consensus_detected function that terminates on consensus, disagreement, or max rounds.
- A generate_report function that produces structured markdown output including positions, disagreements, and the full transcript.
- Demonstrated the debate with the real-world topic “Should we adopt GraphQL?”

You also learned why multi-agent systems fail differently than single agents: roles can blur, one voice can dominate, and false consensus is worse than honest disagreement. The fixes — stricter prompts, explicit structured output, and independent auditing — are the same techniques you’ll use when scaling this pattern to larger agent panels in the next chapters.

Cost for this chapter: With Ollama (local), approximately \$0.00 for any number of rounds. With GPT-4o mini via API, three rounds × three agents ≈ 9 calls at roughly 600 tokens input + 200 tokens output each ≈

\$0.015–\$0.03 per full debate. If you add the Observer or Auditor variations, budget an additional \$0.01 per run. Always cheaper than a conference room.

Chapter 14: Project 14 — Code Review Panel

The best review catches what you missed and teaches what you didn't know.

I once deployed a Lambda function on a Friday afternoon. The pull request had one human reviewer, a senior engineer I deeply respected. They left eight comments: rename this variable, line 34 is too long, the docstring is vague, the exception handling should be more specific. I fixed every single one, merged, and went home. On Monday morning, CloudWatch showed that same function had been logging an API key in plain text since the first invocation. The reviewer never saw it because they were reading for style. I never saw it because I was focused on the style comments. The security hole walked right through a review that was technically thorough and entirely insufficient.

That is the trap of the generalist. A single human reviewer — or a single generalist agent — context-switches between style, security, logic, performance, and naming. At each switch, something drops. The brain that catches a misnamed variable is not the same brain that spots a credential leak. And when you ask one language model to “please review this code,” you are asking it to be that overworked senior engineer: decent at everything, excellent at nothing, and occasionally blind to the one bug that matters most.

The solution is specialization. In this chapter, you will build a Code Review Panel: three agents with narrow, non-overlapping mandates. The Linter cares only about style, PEP8, and complexity. The Security auditor cares only about secrets, injection, and vulnerabilities. The Maintainer cares only about aggregation and verdict. They run in parallel. They return structured, line-level feedback. And the Maintainer delivers a single, unambiguous verdict: PASS, NEEDS_WORK, or REJECT.

Why Specialized Agents Outperform Generalists

Consider what happens inside a single LLM call when you ask it to review code. The model's attention is split across every possible concern. It might spend tokens debating whether a function name is descriptive, then run out of reasoning budget before it notices the raw SQL concatenation on the next line. The prompt is a laundry list, and like any laundry list, items get skipped.

Specialized agents fix this by compressing the problem domain. Each agent gets a system prompt that is essentially one checklist, not six. The Linter's prompt never mentions SQL injection. The Security prompt never mentions line length. This is not just prompt engineering; it is information architecture. When the Linter says "line 7 has a missing docstring," that finding is produced by a model whose entire context window is dedicated to the question "is this clean code?" When the Security agent flags a hardcoded password, it emerged from a model asking exclusively "is this safe?"

The trade-off is orchestration. You now have three agents instead of one, which means you need a pipeline, a structured output format they can all agree on, and an arbiter that resolves conflicts. That arbiter is the Maintainer. It does not read the code directly; it reads the structured JSON produced by the specialists. This keeps its prompt short and its logic transparent.

The pipeline looks like this: raw code is submitted; Linter and Security agents review it in parallel; their outputs feed into a Maintainer agent; the Maintainer deduplicates, counts severities, and issues a final verdict with line-level comments. Here is the architecture in a Mermaid diagram:

```
graph LR
  A[Raw Code] --> B{Linter Agent}
  A --> C{Security Agent}
  B --> D[JSON Issues]
  C --> D
  D --> E[Maintainer]
  E --> F[Structured Review:<br/>Verdict + Line Comments]
```

Every box is pure Python. No frameworks yet. Let's build them.

Building the Reviewer Base

First, the contract. Each specialist must return the same shape of data, or the Maintainer cannot consume it. We define a dataclass called `Issue` with four fields: severity (info, warning, error, critical), line (integer), message (human-readable string), and agent (which specialist found it).

```
import json
from dataclasses import dataclass, asdict
from typing import List, Dict, Any
import httpx

OLLAMA_URL = "http://localhost:11434/api/generate"

@dataclass
class Issue:
    """A single review finding with structured severity and
    location."""
    severity: str # info, warning, error, critical
    line: int
    message: str
    agent: str

    def to_dict(self) -> Dict[str, Any]:
        """Serialize to a plain dictionary for JSON output."""
        return asdict(self)
```

Notice the docstrings. Every function gets one, because in later chapters these same descriptions can become tool schemas for framework variants. The `to_dict` method lets us dump findings into a final JSON report without exposing Python internals.

Now the generic Reviewer class. Its job is to hold a role, a system prompt, and a model name, then ask the LLM to review code and parse the response into a list of `Issue` objects.

```
class Reviewer:
    """A specialized code reviewer backed by an LLM."""

    def __init__(self, role: str, system_prompt: str, model: str =
"llama3.1:8b"):
        self.role = role
        self.system_prompt = system_prompt
        self.model = model

    def _build_prompt(self, code: str) -> str:
        """Wrap the raw code with instructions that force JSON
```

```

output."""
    return (
        "Review the following Python code. Return ONLY a JSON
array "
        "of objects. Each object must have keys: severity,
line, message. "
        "Use severity levels: info, warning, error, critical.
"
        "If there are no issues, return an empty array.\n\n"
        "`python\n"
        f"{code}\n"
        "`"
    )

def review(self, code: str) -> List[Issue]:
    """Send code to the LLM and parse structured feedback."""
    prompt = self._build_prompt(code)
    response = httpx.post(
        OLLAMA_URL,
        json={
            "model": self.model,
            "system": self.system_prompt,
            "prompt": prompt,
            "stream": False,
        },
        timeout=120,
    ).json()
    raw = response["response"].strip()

    # Defensive parsing: extract JSON if wrapped in markdown
fences
    if raw.startswith("`"):
        raw = raw.strip("`json").strip("`").strip()

    try:
        parsed = json.loads(raw)
    except json.JSONDecodeError as exc:
        return [Issue("error", 0, f"Parse failure from
{self.role}: {exc}", self.role)]

    if not isinstance(parsed, list):
        parsed = parsed.get("issues", [])

    issues: List[Issue] = []
    for item in parsed:
        issues.append(Issue(
            severity=item.get("severity", "warning"),
            line=item.get("line", 0),
            message=item.get("message", "No details
provided"),
            agent=self.role,

```

```
    ))  
    return issues
```

The review method does three things: sends the prompt, cleans the response, and normalizes the result into Issue objects. The defensive parsing is intentional. Local models often wrap JSON in markdown fences or add a preamble like “Here is the review:”. Our code strips fences and falls back to an error Issue if the JSON is truly broken. This means the pipeline never crashes because one agent hallucinated a closing bracket; instead, it reports the hallucination as a pipeline error.

Agent 1: The Linter

The Linter’s system prompt is a strict, narrow checklist. You want it to ignore security, ignore logic, and focus on mechanical style concerns.

```
LINTER_PROMPT = (  
    "You are a PEP8 style reviewer. Check ONLY: "  
    "(1) line length exceeding 79 characters, "  
    "(2) missing docstrings on public functions and classes, "  
    "(3) functions longer than 50 lines, "  
    "(4) variable names in ALL_CAPS that are not module-level  
constants, "  
    "(5) trailing whitespace or mixed tabs and spaces. "  
    "Do not comment on logic, security, or performance. "  
    "Return ONLY a JSON array."  
)
```

Notice the constraint: “Do not comment on logic, security, or performance.” Without that guardrail, the LLM will generate opinions about SQL injection or algorithmic complexity, polluting the Linter’s signal with noise. Each agent’s value comes from what it ignores as much as what it checks.

We will feed it a deliberately flawed snippet in a moment. Before we do, look at how tightly scoped the prompt is. It does not say “review the code.” It says “check these five things.” That specificity is the difference between a useful specialist and a chatty generalist.

Agent 2: The Security Auditor

The Security prompt is equally narrow but higher stakes. It looks for a different checklist entirely.

```
SECURITY_PROMPT = (  
    "You are a security auditor. Check ONLY: "
```

```

    "(1) hardcoded passwords, API keys, or secrets in string
literals, "
    "(2) SQL injection via string concatenation or f-string
interpolation, "
    "(3) use of eval(), exec(), or compile() on untrusted input, "
    "(4) unsanitized user input passed to file paths or shell
commands. "
    "Rate severity strictly: 'critical' for secrets and SQL
injection, "
    "'error' for eval/exec, 'warning' for other risks. "
    "Return ONLY a JSON array."
)

```

Two agents, two perspectives, zero overlap in their prompts. That separation is what makes the panel trustworthy. Let's see them in action against a snippet that is ugly in every way:

```

CODE_SNIPPET = '''def get_user(name):
    query = "SELECT * FROM users WHERE name = '" + name + "'"
    PASSWORD = "supersecret123"
    return query
'''

```

Running the Linter against this snippet should produce: - line 1, warning: missing docstring - line 2, warning: line length exceeds 79 characters - line 3, error: ALL_CAPS variable inside a function, not a module constant

Running the Security agent against the same snippet should produce: - line 2, critical: SQL injection via string concatenation - line 3, critical: hardcoded password

Neither agent duplicates the other. The Linter does not know about injection. The Security agent does not care about line length. That is the point.

Agent 3: The Maintainer and Verdict

The Maintainer does not call the LLM. I learned the hard way that asking an LLM to “decide whether to approve” is a recipe for leniency. Language models are fine-tuned to be helpful and agreeable; a helpful model tends to say “this looks mostly good, just a few minor points.” That is not a gate. It is a rubber stamp.

Instead, the Maintainer is deterministic Python code. It receives all Issue objects, deduplicates by (line, message, severity) if necessary, counts severity levels, and applies explicit rules.

```

class Maintainer:
    """Aggregates specialist findings and issues a final
    verdict."""

    def __init__(self, max_warnings: int = 3, reject_on_critical:
bool = True):
        self.max_warnings = max_warnings
        self.reject_on_critical = reject_on_critical

    def _deduplicate(self, issues: List[Issue]) -> List[Issue]:
        """Remove exact duplicate messages on the same line."""
        seen = set()
        uniq = []
        for i in issues:
            key = (i.line, i.message, i.severity)
            if key not in seen:
                seen.add(key)
                uniq.append(i)
        return uniq

    def decide(self, issues: List[Issue]) -> Dict[str, Any]:
        """Return a structured review report with verdict and line
        comments."""
        issues = self._deduplicate(issues)

        criticals = [i for i in issues if i.severity ==
"critical"]
        errors = [i for i in issues if i.severity == "error"]
        warnings = [i for i in issues if i.severity == "warning"]
        infos = [i for i in issues if i.severity == "info"]

        verdict = "PASS"
        if self.reject_on_critical and criticals:
            verdict = "REJECT"
        elif len(errors) > 2 or len(warnings) > self.max_warnings:
            verdict = "NEEDS_WORK"

        return {
            "verdict": verdict,
            "summary": {
                "critical": len(criticals),
                "error": len(errors),
                "warning": len(warnings),
                "info": len(infos),
            },
            "line_comments": [i.to_dict() for i in issues],
            "overall_comment": self._overall_comment(verdict,
criticals, errors, warnings),
        }

    def _overall_comment(
        self,

```



```

        verdict: str,
        criticals: List[Issue],
        errors: List[Issue],
        warnings: List[Issue],
    ) -> str:
        """Generate a human-readable summary for the PR
description."""
        if verdict == "PASS":
            return "Clean bill of health. Ship it."
        if verdict == "REJECT":
            return (
                f"Blocked: {len(criticals)} critical security
issue(s) found. "
                "Fix before merge."
            )
        parts = []
        if errors:
            parts.append(f"{len(errors)} error(s)")
        if warnings:
            parts.append(f"{len(warnings)} warning(s)")
        return f"Needs work: {' '.join(parts)}. See inline
comments."

```

This is a rules engine, not an LLM call. The verdict is reproducible: feed it the same issues twice, get the same result. That matters for a gate. You do not want a maintainer that approves on Tuesdays and rejects on Wednesdays because of temperature sampling.

The output is structured JSON that mirrors a GitHub PR review: a top-level verdict, a summary dict, and an array of line comments. A CI system could POST this directly to a PR thread.

Wiring the Pipeline Together

Here is the orchestration. For clarity, we run sequentially. In production, you would wrap the loop in `concurrent.futures.ThreadPoolExecutor` because Linter and Security have no dependency on each other.

```

from concurrent.futures import ThreadPoolExecutor

def run_review_panel(code: str) -> Dict[str, Any]:
    """Run Linter and Security in parallel, then Maintainer."""
    reviewers = [
        Reviewer("Linter", LINTER_PROMPT),
        Reviewer("Security", SECURITY_PROMPT),
    ]

    # Sequential version (clear for learning):

```

```

all_issues: List[Issue] = []
for reviewer in reviewers:
    all_issues.extend(reviewer.review(code))

# Production version (uncomment to parallelize):
# with ThreadPoolExecutor() as pool:
#     futures = [pool.submit(r.review, code) for r in
reviewers]
# all_issues = [issue for f in futures for issue in
f.result()]

maintainer = Maintainer()
report = maintainer.decide(all_issues)
return report

if __name__ == "__main__":
    report = run_review_panel(CODE_SNIPPET)
    print(json.dumps(report, indent=2))

```

When you run this against the flawed snippet, the final report looks like:

```

{
  "verdict": "REJECT",
  "summary": {
    "critical": 2,
    "error": 1,
    "warning": 2,
    "info": 0
  },
  "line_comments": [
    {"severity": "warning", "line": 1, "message": "Missing
docstring", "agent": "Linter"},
    {"severity": "warning", "line": 2, "message": "Line length
exceeds 79 characters", "agent": "Linter"},
    {"severity": "error", "line": 3, "message": "ALL_CAPS variable
not a module constant", "agent": "Linter"},
    {"severity": "critical", "line": 2, "message": "SQL injection
via string concatenation", "agent": "Security"},
    {"severity": "critical", "line": 3, "message": "Hardcoded
password", "agent": "Security"}
  ],
  "overall_comment": "Blocked: 2 critical security issue(s) found.
Fix before merge."
}

```

The verdict is REJECT because the Maintainer sees two criticals. The developer sees exactly which lines are broken and why. The team lead sees a reproducible JSON artifact that can be logged, audited, and compared to future commits.

Break and Fix

Break 1: False Positive on Security

You add a test file with a dummy key: `API_KEY = "test-do-not-use"`. The Security agent flags it as critical. It is not a secret; it is a placeholder with no entropy and no production value. But the LLM pattern-matched “API_KEY” and panicked.

Why it fails: The Security prompt is purely syntactic. It sees `API_KEY = "..."` and triggers, without considering context like variable name patterns, file path (tests/), or comment markers.

Fix: Enrich the Security prompt with exclusion rules, and if possible, add a lightweight pre-filter. For example, prepend this to `SECURITY_PROMPT`: “Ignore test fixtures, placeholder strings containing ‘test’ or ‘dummy’, and variables in files under a tests/ directory.” In pure Python, you can also add a heuristic:

```
def _is_placeholder(value: str) -> bool:
    """Return True if a string looks like a test placeholder."""
    return any(k in value.lower() for k in ("test", "dummy",
"example", "replace-me"))
```

Pass this as a tool or include it in the code context so the agent sees the exclusion logic before it counts a test string as a breach.

Break 2: Style Rules Conflict

The Linter flags a one-line property setter and says “missing docstring.” Your team’s convention is that trivial setters do not need docstrings if the getter is documented. Later, the same Linter flags a list comprehension and says “for readability, use a loop.” Another developer on the team prefers comprehensions. The agent is fighting itself because it was trained on broad internet style debates, not your specific codebase.

Why it fails: The Linter is a generalist wearing a specialist hat. Without an explicit project convention, it falls back to the most common internet advice, which may contradict your team's rules.

Fix: Feed the Linter a local style guide. Read a STYLE.md file at startup and prepend it to the system prompt.

```
def load_style_guide(path: str = "STYLE.md") -> str:
    """Load project-specific conventions to constrain the
    Linter."""
    try:
        with open(path, "r", encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        return ""
```

```
local_style = load_style_guide()
LINTER_PROMPT = local_style + "\n" + LINTER_PROMPT
```

Now the Linter's first instruction might be "Trivial property setters do not require docstrings," and the conflict vanishes.

Break 3: Maintainer Always Approves

You implement the Maintainer as an LLM agent instead of deterministic code. You ask it: "Given these issues, should we PASS, NEEDS_WORK, or REJECT?" It consistently chooses PASS with a comment like "Overall solid, just a few minor notes." Your CI pipeline merges critical security vulnerabilities because the gatekeeper was too polite.

Why it fails: Most instruction-tuned models are optimized to be helpful and encouraging. A verdict of REJECT feels rude to the model's priors, so it softens the language and weakens the verdict.

Fix: Do not use an LLM for the Maintainer. Use deterministic aggregation, as shown in the core implementation. If you absolutely must use an LLM for complex edge cases, constrain it with explicit rules: "You MUST return REJECT if any critical issue exists. You MUST return NEEDS_WORK if more than two errors exist. Do not soften the verdict." Better yet, keep the logic in code. A gate should not be probabilistic.

Break 4: Line Numbers Wrong in Comments

The Security agent flags line 5, but the SQL injection is actually on line 4. Or the Linter says line 2, but after you add an import at the top, everything shifts. The line numbers in the structured review are off by one, sending developers hunting in the wrong place.

Why it fails: LLMs are not great at counting lines in raw text, especially when the snippet includes leading newlines, blank lines, or markdown fences. The model may count from the first non-empty line or skip the newline after the opening backticks.

Fix: Pre-number the code before sending it to any agent.

```
def with_line_numbers(code: str) -> str:
    """Prefix every line with its 1-based number."""
    return "\n".join(
        f"{i:3d} | {line}" for i, line in
    enumerate(code.splitlines(), start=1)
    )
```

Update `_build_prompt` to send the numbered version, and add to the prompt: “Use the number before the pipe as the exact line reference.” This anchors the model’s attention and dramatically improves positional accuracy.

Break 5: Security Agent Misses Obvious SQL Injection

You pass the same `CODE_SNIPPET` to the Security agent, but it returns an empty array. It missed the `+` name `+` string concatenation entirely. Your CI is green. Your database is doomed.

Why it fails: The Security prompt was too vague. It said “check for security issues,” which is a broad request. The LLM’s attention scattered across the three lines and never latched onto the concatenation pattern because it was not primed to look for it specifically.

Fix: Use a checklist prompt, not an open-ended request. The prompt should enumerate exactly what to hunt for. Additionally, add a lightweight static regex as a safety net:

```
import re

SQL_INJECTION_RE = re.compile(
    r'["\'].*?\+.*?["\']',
    re.IGNORECASE,
```

```
)

def regex_prescreen(code: str) -> List[Issue]:
    """Fast heuristic to catch obvious injection patterns LLMs
    might miss."""
    issues = []
    for i, line in enumerate(code.splitlines(), start=1):
        if SQL_INJECTION_RE.search(line) and ("SELECT" in line or
        "INSERT" in line):
            issues.append(
                Issue("critical", i, "Possible SQL injection
(regex hit)", "Security")
            )
    return issues
```

Feed these regex hits into the Security agent's context, or append them directly to the issue list. The best panels combine fast deterministic filters with slow, deep LLM reasoning. The regex catches the obvious; the LLM catches the subtle.

Try It Now

Variation 1: Add a Performance Agent

Create a third specialist called the Performance agent. Its prompt should focus on nested loops, list concatenation with + inside loops, repeated re.compile inside functions, and quadratic string building. Feed it code like this:

```
def slow(items):
    result = []
    for i in items:
        for j in items:
            result = result + [i * j]
    return result
```

The Performance agent should flag the $O(n^2)$ loop and the repeated list concatenation. Update the Maintainer to downgrade to NEEDS_WORK if performance errors exceed one, but not REJECT unless a critical exists. This teaches you how to tune severity thresholds per concern.

Variation 2: Auto-Fix Patches

Instead of only returning messages, have the Linter generate patches. Add a suggestion field to the JSON response: {"severity": "warning", "line": 2, "message": "...", "suggestion": {"find": "bad_line", "replace": "good_line"}}. Then write a Python function that applies suggestion objects back to the source file using safe line replacement. Add a --dry-run flag so the developer can preview changes before applying. Never auto-apply critical security fixes without human review.

Variation 3: Learn Project Conventions

Create a CONVENTIONS.md in your repository root. At the start of a review, read this file and inject its contents into the Linter's system prompt. Then commit a deliberate violation of one convention (for example, "we use single quotes") and verify the Linter catches it. Over a few runs, expand the file. Your Code Review Panel is now learning your team's specific culture instead of enforcing generic internet rules.

The OpenClaw Way

OpenClaw's multi-agent abstraction maps cleanly onto our panel. You define one Agent per specialist, assigning each a system_prompt that matches the Linter and Security prompts we wrote by hand. OpenClaw handles the JSON schema enforcement via Pydantic models, so instead of defensive string parsing, you declare Issue as a Pydantic list and the framework validates the LLM output for you. A Task object routes the raw code to both agents in parallel, and an Orchestrator collects their results. The Maintainer becomes a final Agent with allow_llm=False, meaning OpenClaw knows to use deterministic rules rather than an LLM call for the verdict gate. The value proposition is identical logic with less parsing boilerplate, plus built-in token-cost tracking across every agent invocation.

The Hermes Agent Way

Hermes supports multi-agent panels through its crew mode. You create three specialized sub-agents inside a single Hermes session: /agent create linter --prompt "...", /agent create security --prompt "...", and /agent create maintainer --rules "REJECT if any critical". You then run /crew review code.py and Hermes dispatches the file to the first two agents concurrently, buffers their JSON outputs, and feeds both into the

Maintainer agent's context. Because Hermes manages conversation state and tool access, you do not write the `ThreadPoolExecutor` loop yourself. The trade-off is that you are inside Hermes's runtime, not your own script, which makes CI integration harder unless you invoke Hermes via its CLI in a GitHub Action. For personal or team Slack workflows, however, it is nearly zero-code setup.

What You Built

In this chapter, you built a multi-agent Code Review Panel from scratch:

- A generic Reviewer class that enforces structured JSON output from an LLM.
- A Linter specialist focused purely on PEP8, complexity, and style.
- A Security specialist focused purely on secrets, injection, and dangerous builtins.
- A deterministic Maintainer that deduplicates findings, counts by severity, and issues a clear verdict: PASS, NEEDS_WORK, or REJECT.
- A pipeline that can run sequentially for clarity or in parallel via threads for production.
- Break-and-fix experience with false positives, line-number alignment, and deterministic gates.
- Extension patterns for performance review, auto-fix patches, and project-specific style guides.

You also learned a deeper lesson: not every agent in a panel should be an LLM. The Maintainer is code because gates need to be reproducible, not polite.

Cost for this chapter: Running the full panel against a single small file costs approximately two LLM calls for the specialists and one aggregation pass for the Maintainer. With a local Ollama model, the cost is \$0.00. With GPT-4o mini, expect roughly \$0.003–\$0.008 per run, depending on the length of the code snippet. If you scale this to review every pull request in a busy repository, budget around \$2–\$5 per day.

Chapter 15: Project 15 — Content Factory: Writer + Editor + Fact-Checker

A single agent writing alone is a novelist with no deadline, no editor, and no consequences. Let's build a newsroom instead.

The Correction Notice I Never Wanted to Write

Two years ago I published a blog post about the future of AI agents. It was punchy, well-structured, and filled with statistics that make readers nod and share. One number caught fire: I claimed the autonomous AI agent market would hit fifty billion dollars by 2025. It had percentage signs. It cited no source.

A reader who works at Gartner emailed me the next morning. The number, they gently explained, was hallucinated. The model had invented a plausible figure, wrapped it in confident prose, and served it like a fact. I believed it because I wanted to. The post had already been retweeted two hundred times. I spent the afternoon writing a correction notice that began *"I trusted the AI, and I shouldn't have."*

That was the most expensive writing lesson of my career. I realized a single agent cannot be both creator and critic. When the same model generates a claim and then evaluates it, it tends to agree with itself. It is like asking a student to grade their own final exam. The incentive structure is broken.

What I needed — what you need if anyone else will read your output — was separation of concerns. A writer who drafts. An editor who critiques structure and tone. A fact-checker who treats every statistic as guilty until proven innocent. The same model can play all three roles, but they must wear different hats, evaluate different things, and never know what the others want to hear. They must check each other.

That is what we are building: a Content Factory. Three agents, one pipeline, and a hard rule that no article ships until it has survived all three.

Why AI Content Needs a Newsroom

The quality problem in AI-generated content is not that models write badly. Often they write fluently — too fluently. The problem is that fluency masquerades as accuracy. A well-turned sentence can carry a false claim so smoothly that your eyes slide right over it. Worse, models optimize for coherence, not truth. If a statistic would make a paragraph flow better, the model will invent one rather than leave a gap.

As a solo operator, your first instinct might be to prompt the model harder. *“Only use facts you are certain about.”* That helps, but it is fragile. The model’s definition of certainty is statistical, not epistemic. It is certain when the next token is predictable, not when the claim is grounded in reality.

The robust fix is architectural: never let the generator be the verifier. In a human newsroom, the reporter who writes the story is not the same person who checks the spelling of the CEO’s name against three independent sources. The separation exists because humans have ego and confirmation bias. With AI agents, the separation exists because a model in “writer” mode and the same model in “critic” mode sample from different distributions. The writer maximizes engagement; the critic minimizes error. You want both distributions, but you want them fighting, not collaborating.

Our Content Factory implements this fight as a directed workflow:

1. **Writer** receives a brief and produces a draft.
2. **Editor** reviews the draft against the brief for structure, tone, and completeness. The editor returns a verdict: PASS or NEEDS_WORK. If NEEDS_WORK, the writer revises — up to a maximum of three cycles.
3. **Fact-Checker** extracts verifiable claims from the final draft and queries external sources. Every statistic, date, named entity, and market projection must survive this gate.
4. **Approval** only if the fact-checker clears the article. If not, the flagged claims are fed back to the writer for one final revision. The output is the finished article, a revision log, and a fact-check audit trail.

This pipeline turns a single prompt into a quality-control system. The cost is higher — three to five LLM calls instead of one — but the cost of publishing false information is higher still.

The Pipeline in Plain English

Before we write code, trace the pipeline manually. Imagine the brief says: *“Write a 500-word article for beginners explaining AI agents. Include what makes them different from chatbots. Tone: enthusiastic but professional.”*

Step 1 — Writer drafts. The writer agent sees only the brief and its own creative instructions. It produces an opening hook, defines the agent loop in plain language, contrasts agents with chatbots using an example, and closes with a forward-looking statement. Total length: 487 words. A paragraph claims that the agent software market is projected to reach \$5.1 billion in 2024.

Step 2 — Editor reviews. The editor reads the draft and the brief side by side. It notices the structure is strong but the middle section spends too much time on chatbot history and not enough on the agent loop. It returns NEEDS_WORK with feedback: *“Trim chatbot history to two sentences. Expand the observe-reason-act cycle with a concrete walkthrough. The closing is strong.”*

Step 3 — Writer revises (Cycle 1). The writer receives the original brief plus the editor’s feedback. It produces a new draft: 512 words, tighter chatbot contrast, expanded walkthrough. The market projection remains.

Step 4 — Editor reviews again. This time the editor returns PASS. The structure meets the brief.

Step 5 — Fact-checker extracts claims. The fact-checker reads the approved draft and identifies four verifiable claims: - “AI agents follow an observe-reason-act loop.” - “The agent software market is projected to reach \$5.1 billion in 2024.” - “Chatbots only respond to prompts; agents can initiate actions.” - “The term ‘agent’ was first used in AI research in the 1990s.”

Step 6 — Fact-checker verifies via search. The first and third claims are general knowledge and are quickly supported by retrieved snippets. The second claim returns a MarketsAndMarkets report that supports the figure. The fourth claim, however, is flagged UNSUPPORTED — search results show the term emerging in the 1970s

from economics and AI, not strictly the 1990s. The fact-checker reports: *“Claim 4 is UNSUPPORTED. Suggest rephrasing to ‘The concept of software agents has roots in earlier AI research’ or omit entirely.”*

Step 7 — Final revision. Because one claim failed, the writer receives the fact-check notes and revises once more, softening the unsupported historical claim. The final article is 498 words, structurally approved, and factually audited.

Step 8 — Output. The program returns three artifacts: the article text, a revision log showing editor verdicts for each cycle, and a fact-check log showing every claim, its verdict, and the reasoning.

That is the pipeline. Every step is a distinct LLM call with a distinct persona prompt. No agent sees another agent’s internal reasoning unless you explicitly feed it into the next prompt. The boundaries are clean. Now let’s code it.

flowchart TD

```
B[Brief] --> W[Writer: Draft]
W --> E[Editor: Review]
E -->|NEEDS_WORK| W
E -->|PASS| FC[Fact-Checker: Verify Claims]
FC -->|approved| P[Publish:<br/>Final Article + Logs]
FC -->|flags found| WR[Writer: Revise<br/>with Fact Notes]
WR -->|Final Draft| P
style W stroke:#4a90e2,stroke-width:2px
style E stroke:#f5a623,stroke-width:2px
style FC stroke:#7ed321,stroke-width:2px
```

Building the Content Factory in Pure Python

We are going to build a single `ContentFactory` class that encapsulates the entire pipeline. It will run against a local Ollama model by default — just like Chapter 1 — so you can experiment for free. The class has three public methods that correspond to our three roles: `draft()`, `review()`, and `fact_check()`. A fourth method, `run()`, orchestrates the loop and enforces our hard limits.

Create a new file called `content_factory.py` and type the following:

```
import json
import httpx
from typing import Dict, List

OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL = "llama3.1:8b"
```

```

class ContentFactory:
    def __init__(
        self,
        model_url: str = OLLAMA_URL,
        model: str = MODEL,
        max_revisions: int = 3,
    ):
        """Initialize the content factory with model endpoint and
revision ceiling."""
        self.model_url = model_url
        self.model = model
        self.max_revisions = max_revisions
        self.revision_log: List[Dict] = []
        self.fact_check_log: List[Dict] = []

    def _generate(self, prompt: str, temperature: float = 0.7) ->
str:
        """Send a prompt to the LLM and return the generated text.

        Uses the Ollama generate endpoint by default. For cloud
models,
        swap this method to use OpenAI or Anthropic APIs.
        """
        payload = {
            "model": self.model,
            "prompt": prompt,
            "stream": False,
            "options": {"temperature": temperature},
        }
        try:
            response = httpx.post(self.model_url, json=payload,
timeout=120)
            response.raise_for_status()
            return response.json()["response"].strip()
        except httpx.TimeoutException:
            return "[ERROR: LLM call timed out. Consider a smaller
model or shorter prompt.]"
        except httpx.HTTPStatusError as e:
            return f"[ERROR: LLM returned
{e.response.status_code}.]"

    def draft(self, brief: str, feedback: str = "") -> str:
        """Generate an article draft from a creative brief.

        If feedback is provided, incorporates it as revision
guidance.
        The writer is instructed to produce roughly 500 words of
accessible, enthusiastic prose.
        """
        prompt = (

```

```

        "You are a professional technology writer for a
general audience. "
        "Write an engaging, informative article following the
brief below. "
        "Aim for approximately 500 words. Use plain
paragraphs. "
        "Do not include markdown heading markers like '#'.
\n\n"
    )
    if feedback:
        prompt += (
            "The editor and fact-checker provided the
following feedback. "
            "Address every point in your revision:\n"
            f"{feedback}\n\n"
        )
    prompt += f"Brief:\n{brief}\n\nArticle:"
    return self._generate(prompt, temperature=0.8)

def review(self, draft: str, brief: str) -> Dict:
    """Review a draft for structure, clarity, and alignment
with the brief.

Returns a structured dict with keys:
- verdict: 'PASS' or 'NEEDS_WORK'
- feedback: concise paragraph of improvement guidance
- notes: list of specific issues
"""
    prompt = (
        "You are a senior editor. Review the article below
against the brief. "
        "Evaluate structure, clarity, tone, and whether it
fulfills the brief. "
        "Respond with ONLY a JSON object in this exact format:
\n"
        '{"verdict": "PASS" or "NEEDS_WORK", '
        '"feedback": "concise paragraph of what to improve", '
        '"notes": ["specific issue 1", "specific issue 2"]}'
\n\n'
        f"Brief:\n{brief}\n\nArticle:\n{draft}\n\nEditorial
review:"
    )
    raw = self._generate(prompt, temperature=0.3)
    try:
        parsed = json.loads(raw)
        if not isinstance(parsed, dict):
            raise ValueError("Not a dict")
    except (json.JSONDecodeError, ValueError):
        # Fail-safe: if the model emits prose instead of JSON,
default to pass
        parsed = {
            "verdict": "PASS",

```

```

        "feedback": "Model returned non-JSON review;
defaulting to pass.",
        "notes": [],
    }
    return parsed

def _search_web(self, query: str) -> List[Dict]:
    """Search the web for a query and return result snippets.

    This implementation uses a mock backend so the script runs
    without API keys. In production, replace the MOCK_DB lookup with a
    real search provider such as DuckDuckGo Instant Answers,
    SerpAPI, or Brave Search API.

    Each result should be a dict with 'source' and 'snippet'
    keys.
    """
    MOCK_DB = {
        "AI agent observe reason act loop": [
            {
                "source": "wikipedia.org",
                "snippet": "The sense-plan-act paradigm is a
classic model in artificial intelligence.",
            },
            {
                "source": "lilianweng.github.io",
                "snippet": "An agent observes, thinks, and
acts upon its environment.",
            },
        ],
        "chatbot only responds prompt agent initiates action":
        [
            {
                "source": "techcrunch.com",
                "snippet": "Chatbots are reactive; agents can
proactively take actions using tools.",
            },
        ],
        "AI agent software market size 2024": [
            {
                "source": "marketsandmarkets.com",
                "snippet": "The autonomous AI agent market is
estimated at $5.1 billion in 2024.",
            },
        ],
        "term agent first used AI research 1990s": [
            {
                "source": "stanford.edu",
                "snippet": "The term 'agent' was used in AI

```

```

literature as early as the 1970s.",
        },
    ],
}
# Return exact match if we have one; otherwise empty list
return MOCK_DB.get(query, [])

def fact_check(self, article: str) -> Dict:
    """Extract factual claims and verify them via search.

    Returns a dict with:
    - clean: bool indicating whether all claims are supported
    - claims: list of dicts, each with claim, verdict,
reasoning, sources
    """
    # Step 1: Extract verifiable claims
    prompt = (
        "You are an investigative fact-checker. Read the
article below and extract "
        "up to 5 factual claims that can be verified
externally "
        "(statistics, dates, named entities, market sizes,
technical definitions). "
        "Respond with ONLY a JSON list of strings, each one a
standalone claim. "
        "If there are no verifiable claims, return an empty
list [].\n\n"
        f"Article:\n{article}\n\nClaims (JSON list):"
    )
    raw = self._generate(prompt, temperature=0.2)
    try:
        claims = json.loads(raw)
        if not isinstance(claims, list):
            claims = []
    except json.JSONDecodeError:
        claims = []

    # Step 2: Verify each claim against search results
    results = []
    all_clean = True
    for claim in claims:
        search_query = claim[:120]
        search_results = self._search_web(search_query)

        verify_prompt = (
            f"Claim to verify: {claim}\n\n"
            "Web search results:\n"
        )
        if search_results:
            for r in search_results:
                verify_prompt += f"- {r['source']}:
{r['snippet']}\n"

```



```

        else:
            verify_prompt += "No relevant results found.\n"

            verify_prompt += (
                "\nBased ONLY on the sources above, respond with
ONLY a JSON object:\n"
                '{"verdict": "SUPPORTED" or "UNSUPPORTED" or
"UNCERTAIN", '
                '"reasoning": "one sentence explaining why"}
\n\nVerification:"
            )
            raw_verdict = self._generate(verify_prompt,
temperature=0.2)
            try:
                verdict = json.loads(raw_verdict)
            except json.JSONDecodeError:
                verdict = {"verdict": "UNCERTAIN", "reasoning":
"Parse failed."}

            if verdict.get("verdict") != "SUPPORTED":
                all_clean = False

            results.append({
                "claim": claim,
                "verdict": verdict.get("verdict", "UNCERTAIN"),
                "reasoning": verdict.get("reasoning", ""),
                "sources": search_results,
            })

        return {"clean": all_clean, "claims": results}

def run(self, brief: str) -> Dict:
    """Execute the full content-factory pipeline.

    Returns a structured dict containing:
    - article: final article text
    - revision_log: chronological editor reviews
    - fact_check_log: claim-by-claim verification records
    - meta: revision count and final publication status
    """
    self.revision_log = []
    self.fact_check_log = []

    # Initial draft
    draft = self.draft(brief)

    # Editor review loop with hard ceiling
    revision_count = 0
    for i in range(self.max_revisions):
        review_result = self.review(draft, brief)
        self.revision_log.append({
            "revision": i,

```

```

        "verdict": review_result.get("verdict"),
        "feedback": review_result.get("feedback", ""),
        "notes": review_result.get("notes", []),
    })

    if review_result.get("verdict") == "PASS":
        break

    feedback_text = review_result.get("feedback", "")
    notes = review_result.get("notes", [])
    if notes:
        feedback_text += "\nSpecific notes: " + ";
".join(notes)
    draft = self.draft(brief, feedback=feedback_text)
    revision_count = i + 1

    # Fact-check the structurally approved draft
    fact_report = self.fact_check(draft)
    self.fact_check_log = fact_report["claims"]

    # If facts fail, one last revision with explicit notes,
    then ship
    if not fact_report["clean"]:
        fact_notes = "\n".join(
            f'- {c["claim"]}: {c["verdict"]}'
            for c in fact_report["claims"]
            if c["verdict"] != "SUPPORTED"
        )
        draft = self.draft(brief, feedback=f"Fact-check issues
to resolve:\n{fact_notes}")
        revision_count += 1

    status = "APPROVED" if fact_report["clean"] else
"APPROVED_WITH_WARNINGS"

    return {
        "article": draft,
        "revision_log": self.revision_log,
        "fact_check_log": self.fact_check_log,
        "meta": {
            "revision_count": revision_count,
            "status": status,
        },
    }

}

def main():
    """Run the content factory to produce a ~500-word article
    about AI agents."""
    factory = ContentFactory()

```

```

    brief = (
        "Write a 500-word article for beginners explaining what AI
agents are, "
        "why they differ from chatbots, and how the agent loop
(observe, reason, act) works. "
        "Include a concrete everyday example. "
        "Tone: accessible, enthusiastic, slightly informal but
professional."
    )

    result = factory.run(brief)

    print("=" * 60)
    print("FINAL ARTICLE")
    print("=" * 60)
    print(result["article"])

    print("\n" + "=" * 60)
    print("REVISION LOG")
    print("=" * 60)
    for entry in result["revision_log"]:
        print(f"Revision {entry['revision']}: {entry['verdict']}")
        print(f"  Feedback: {entry['feedback'][:200]}...")
        if entry["notes"]:
            print(f"  Notes: {entry['notes']}")
        print()

    print("=" * 60)
    print("FACT-CHECK LOG")
    print("=" * 60)
    for claim in result["fact_check_log"]:
        print(f"Claim: {claim['claim']}")
        print(f"  Verdict: {claim['verdict']} -
{claim['reasoning']}")
        if claim["sources"]:
            print(f"  Sources: {' '.join(s['source'] for s in
claim['sources'])}")
        print()

    print("=" * 60)
    print("META")
    print("=" * 60)
    print(f"Revisions: {result['meta']['revision_count']}")
    print(f>Status: {result['meta']['status']}")

if __name__ == "__main__":
    main()

```

Let's walk through what this code does, because the pipeline has moving parts and you need to see where the boundaries are.

The `draft()` method builds a prompt that tells the model it is a technology writer. If feedback is non-empty, that feedback is prepended to the prompt as revision guidance. This is the mechanism by which editor and fact-checker notes flow back into the writer. The temperature is set to 0.8 because creativity matters in drafting; you want the writer to take risks with phrasing and examples.

The `review()` method builds a prompt that tells the model it is a senior editor. The temperature is lowered to 0.3 because evaluation should be consistent, not imaginative. The editor must return valid JSON with three keys. If the model fails to emit valid JSON — and smaller local models do this more often than you would like — the method catches the parse error and defaults to PASS. This is a safety default: we would rather publish an imperfect article than get stuck in a JSON-parsing infinite loop. In production, you might default to NEEDS_WORK and flag the parse failure in the revision log.

The `_search_web()` method is where you wire in real verification. Right now it uses a dictionary of mock results keyed by exact query strings. This lets the script run without API keys or network dependencies. When you are ready to go live, replace the dictionary lookup with an HTTP call to a search provider. The return format — a list of dicts with source and snippet — should stay the same, because the fact-checker's prompt expects it.

The `fact_check()` method works in two stages. First, it asks the model to extract up to five verifiable claims from the article. This is a crucial design choice: instead of searching the entire article as a blob, we ask the model to narrow the aperture. A 500-word article might contain thirty sentences; only four of them are factual claims that need external validation. By extracting claims first, we avoid burning API calls on platitudes.

Second, for each extracted claim, we search the web and then ask the model to render a verdict based *only* on the search snippets provided. The prompt is explicit about this: “Based ONLY on the sources above.” Without that guardrail, the model will use its internal knowledge to confirm or deny the claim, defeating the purpose of the external search. The verdict is one of SUPPORTED, UNSUPPORTED, or UNCERTAIN. Any verdict other than SUPPORTED marks the article as not clean.

The `run()` method is the conductor. It initializes the draft, enters the editor review loop for at most `max_revisions` cycles, then fact-checks the surviving draft. If the fact-checker finds problems, it does one final

revision incorporating the fact notes, increments the revision counter, and returns the package. The return value is a dict, not just a string, because the logs are as important as the article. When your boss asks, “How do we know this is accurate?” you hand her the fact-check log.

Before you run the script, make sure Ollama is running and the model is pulled:

```
ollama pull llama3.1:8b
ollama serve
```

Then in another terminal:

```
python content_factory.py
```

What you will see is a full article followed by structured logs. The exact text depends on your model’s sampling, but the pipeline structure is deterministic: draft, review, maybe revise, fact-check, finalize. If you run it ten times, the prose will change; the revision count and the fact-check verdicts should stay roughly consistent for the same brief.

The Writer and Editor Loop

The revision loop is where most of the quality gains happen. A single draft is a first impression. A draft that has been reviewed by a second agent is a conversation. The editor does not need to be a different model; it is the same model wearing a different hat. But that hat matters. The writer prompt asks for creativity and engagement. The editor prompt asks for structural rigor and alignment with the brief.

The feedback channel is the weak point in many naive implementations. If you simply append the editor’s notes to the conversation history and ask the model to continue, the model may ignore the notes or misunderstand which persona it is supposed to be playing. Our implementation avoids this by starting a fresh generation for every revision. The writer prompt is rebuilt from scratch each cycle, containing the brief and the feedback but no previous draft text unless you choose to include it. This keeps the writer’s identity stable: it is always a writer responding to a brief and a set of notes, never a writer trying to remember what it wrote three turns ago.

The max-revision ceiling is non-negotiable. In testing, I have seen editor agents that never say PASS because they keep finding increasingly trivial improvements. “This sentence could be tighter.” “This transition is acceptable but not elegant.” Without a hard limit,

the loop would run until your coffee went cold and your API bill caught fire. Three cycles is my default. It gives the writer two chances to incorporate structural feedback after the initial draft. If the editor is still unsatisfied after three, the pipeline ships anyway and logs a warning. Perfect is the enemy of published.

The Fact-Checker as Grounding Layer

The fact-checker is the grounding layer of the pipeline. While the writer and editor argue about tone and structure, the fact-checker asks a simpler question: does anything in this article contradict the outside world? It is not a style critic. It is a bouncer.

The two-stage design — extract, then verify — is important for cost control. Every search call costs tokens or API credits. If you searched for every noun phrase in a 500-word article, you would burn fifty queries. By delegating extraction to the LLM first, you reduce the search surface to the claims that actually need verification. In my testing, a typical 500-word technology article yields three to five verifiable claims. That is three to five search calls instead of fifty.

The grounding prompt — “Based ONLY on the sources above” — is the single most important sentence in the fact-checker. Without it, the model acts as a judge using its own memory, and its own memory is exactly what you are trying to audit. When I first built this pipeline, I omitted that sentence. The fact-checker agreed with the writer on every claim because both were drawing from the same training distribution. It was like asking a twin to verify their sibling’s alibi. Once I added the grounding constraint, the fact-checker became genuinely adversarial. It began flagging claims that sounded plausible but were not reflected in the search snippets.

That is the behavior you want. The fact-checker should be annoying. It should flag things. If it never flags anything, it is probably not doing its job.

Break and Fix: Five Ways the Factory Fails

No pipeline is bulletproof. The Content Factory adds structure, but structure introduces its own failure modes. Here are the five I have hit most often, in order of how embarrassing they are.

Break 1: The Writer Ignores the Editor

You run the pipeline. The editor returns clear feedback: *“Expand the section on the agent loop. Trim the chatbot history.”* The writer revises. The new draft is word-for-word identical to the first. Or worse, it nods politely — *“The editor suggested expanding the agent loop, and here is the expanded section”* — but nothing actually changed.

This happens because the writer prompt is tuned for creativity, and creative models treat feedback as optional inspiration rather than a spec. The model is optimizing for narrative flow, not compliance.

Fix: Make the feedback explicit and itemized. Our implementation already does this by including the editor’s notes list as bullet points. But you can tighten further by adding an instruction to the writer prompt: *“You must address every bullet point below. After your revision, each bullet should no longer apply.”* If the writer still ignores feedback after that, lower the temperature to 0.5 for revision cycles. Less creativity, more obedience.

Break 2: The Fact-Checker Flags Everything

You run the fact-checker on an article about AI agents. It flags every claim as UNCERTAIN. “AI agents use tools.” UNCERTAIN. “Chatbots are reactive.” UNCERTAIN. The model refuses to commit because the search snippets are short and the grounding prompt forbids using outside knowledge. Your pipeline grinds to a halt because nothing is ever clean.

This is over-calibration. The fact-checker has learned the prompt so literally that it treats common knowledge as suspicious because it cannot find an explicit verbatim match in a 120-character search snippet.

Fix: Add a layer of judgment to the extraction prompt. Instruct the model to extract only claims that are *surprising, numerical, or historically specific*. Let common-knowledge claims pass without verification. Alternatively, implement a knowledge-base cache: if a claim has been verified in a previous run, skip the search. In our code, you could add a `verified_cache` dictionary keyed by claim hash.

Break 3: Endless Revision Loops (Without the Ceiling)

Imagine you set `max_revisions=50` because you want perfection. The editor never says PASS. Draft seven is better than draft six, but the intro is still “slightly passive.” Draft twelve tightens the intro, but now the conclusion is “a touch generic.” The model can critique forever because critique is cheaper than creation and there is always a slightly better adverb.

Fix: The fix is already in the code: `max_revisions=3`, hard-coded in the constructor and enforced by the `range()` in `run()`. But the cultural fix is just as important: define PASS criteria in the editor prompt. Add “*Say PASS if the article fulfills the brief, uses an appropriate tone, and contains no structural gaps. Do not hold the draft to literary standards.*” This bounds the editor’s ambition.

Break 4: The Fact-Checker Fabricates Sources

This is the scariest failure, and it almost got me twice. The writer makes a claim. The fact-checker searches the web, receives real search results, but then hallucinates a source that does not exist. For example, the search snippet says: “*Agent markets are growing.*” The fact-checker reports: “*SUPPORTED — Source: McKinsey 2024 AI Report, page 14.*” There is no McKinsey report. The model invented a plausible citation because the prompt asked for reasoning and the model treats reasoning as a creative writing task.

Fix: Gag the fact-checker. Remove any request for citation metadata in the verification prompt. Only ask for a verdict and a one-sentence rationale. If you need citations, generate them from the raw search metadata (the sources list in the code), not from the model’s imagination. Never let the model name a source that was not in the retrieval payload.

Break 5: The Editor Contradicts Its Own Feedback

Cycle 1: the editor says, “*Add more detail about the history of AI agents. The reader needs context.*” The writer obliges and adds a paragraph. Cycle 2: the editor says, “*The history section is too long and distracts from the main point. Cut it.*” The writer is trapped. The editor has demanded A and then demanded not-A.

This happens because each review begins without memory of previous reviews. The editor does not see what it said in cycle 1 when it evaluates cycle 2.

Fix: Feed the revision log into the editor's context. Before asking for a new review, prepend the previous reviews: *"You previously said: [feedback from cycle 1]. Ensure your current feedback is consistent with prior requests and does not reverse an earlier direction unless the draft introduced a new problem."* Alternatively, pivot from iterative refinement to single-shot critique: give the editor the full history and ask for a consolidated revision memo, then let the writer execute it once.

Try It Now: Three Variations

Once the basic pipeline runs, these variations will teach you more than another hour of reading.

Variation 1: Style Briefs — Hemingway vs. Gladwell

Replace the brief with a style-specific directive. First, run the pipeline with this brief: *"Write in the style of Ernest Hemingway: short sentences, concrete nouns, no adverbs, emotional restraint."* Then run it again with: *"Write in the style of Malcolm Gladwell: narrative anecdotes, surprising statistics as transitions, conversational asides."* Compare the revision logs. You will find that the Hemingway brief usually passes in Cycle 0 because short sentences are easier to self-correct. The Gladwell brief often triggers two editor cycles because the model struggles to balance anecdote with data density. This teaches you that style complexity predicts revision count.

Variation 2: Fact-Check Confidence Scores

Replace the SUPPORTED / UNSUPPORTED / UNCERTAIN ternary with a 0-to-1 confidence score. Modify the verification prompt to output `{"confidence": 0.85, "reasoning": "..."}.` Then experiment with thresholds. At 0.9, almost nothing passes. At 0.6, too much passes. Your threshold becomes a tunable risk dial. Log the distribution of confidence scores across ten articles and pick a threshold that catches the claims you care about while letting common knowledge through.

Variation 3: Auto-Publish to a Headless CMS

Add a `publish()` method to `ContentFactory` that sends the approved article to a content management system or a static-site generator. For a no-API-key demo, implement it as a file writer:

```
def publish(self, article: str, slug: str) -> str:
    """Write the final article to a markdown file in a content
    folder."""
    import datetime, os
    os.makedirs("content", exist_ok=True)
    filename = f"content/{slug}.md"
    with open(filename, "w") as f:
        f.write(f"---\n")
        f.write(f"title: AI Agents for Beginners\n")
        f.write(f"date: {datetime.date.today().isoformat()}\n")
        f.write(f"status: published\n")
        f.write(f"---\n\n")
        f.write(article)
    return filename
```

For a real CMS like Strapi, Ghost, or a Git-based workflow, replace the file write with an authenticated POST request. The important architectural move is that publication only happens after `run()` returns an `APPROVED` status. Never auto-publish when the status is `APPROVED_WITH_WARNINGS`.

The OpenClaw Way

OpenClaw's multi-agent orchestration maps cleanly onto the Content Factory. Instead of one class with three methods, you define three agents sharing a session memory. The Writer agent is configured with `reasoning="creative"`, the Editor with `reasoning="critical"`, and the Fact-Checker with a custom toolset that includes a `search_web` tool registered via the `@tool` decorator. OpenClaw handles the conversation routing: when the Writer emits a draft, the pipeline trigger passes control to the Editor. When the Editor emits `PASS`, the trigger passes to the Fact-Checker.

The value of OpenClaw here is state management. In our pure Python version, we manually rebuild prompts for every cycle. OpenClaw's session object persists the draft and feedback across turns, so the Writer can see not just the latest editor notes but the entire evolution of the article. This reduces token waste because you are not re-

prompting the full brief every time; you are appending a delta. For a three-revision pipeline, the token savings are modest. For a ten-article batch run, they add up.

The Hermes Agent Way

Hermes Agent approaches the Content Factory not as a pipeline of separate prompts but as a single long session with persona switches. You start Hermes and load the Writer persona. It drafts. You then switch to the Editor persona with `/persona editor`. Hermes re-evaluates the conversation history under the new persona constraints and emits feedback. You copy that feedback, switch back to the Writer persona, and ask for a revision. Finally, you load the Fact-Checker persona and ask it to verify claims, which it does by invoking its built-in web-search toolset.

The Hermes version is more conversational and less programmatic. It is excellent for exploratory content work where you want to manually intervene between cycles. For example, if the Editor suggests a structural change that you disagree with, you can override it in the chat before the Writer sees it. The pure Python version is better for automation: you set the brief, run the script, and receive the package. Hermes is better for craft: you are the human newsroom editor sitting between the agents, cutting and approving. Both are valid. The choice depends on whether your use case is a nightly blog queue or a hands-on feature article.

What You Built

In this chapter, you built a content-quality system rather than a content generator:

- A `ContentFactory` class that orchestrates three distinct agent roles: Writer, Editor, and Fact-Checker.
- A revision loop capped at three cycles, with structured logging of every editorial verdict.
- A two-stage fact-checking pipeline that extracts verifiable claims and grounds verification in external search snippets.
- A fail-safe architecture: JSON parse errors default to PASS, timeouts return error strings, and endless loops are impossible because of the hard ceiling.

- A structured output format that ships the article alongside its audit trail: revision log and fact-check log.
- A publish gate that only finalizes when both structural review and factual clearance are satisfied.

You also learned why separation of concerns matters more in generation pipelines than in any other agent architecture. When creator and critic are the same subroutine, the critic becomes a rubber stamp. When they are separated by prompt boundaries and retrieval layers, the critic becomes genuinely useful. And genuinely annoying, which is exactly the point.

Cost for this chapter: Running the full pipeline against Ollama costs \$0.00. Against GPT-4o mini, expect ~2,700 tokens per full run (draft + up to three reviews + claim extraction + three claim verifications). At mini rates, that is approximately **\$0.002 per article**. At GPT-4o rates, approximately **\$0.07 per article**. A batch of fifty articles per week costs about ten cents in mini and three dollars fifty in full GPT-4o. The peace of mind is free.

Chapter 16: Project 16 — Customer Support Router

Every angry email goes to the right team. Every draft matches the problem. Nothing gets lost.

A few years ago I opened a support ticket with a software vendor because a production database was throwing authentication errors every hour. I wrote a careful description: the error code, the timestamps, the steps I'd already tried. I hit submit and watched the confirmation page flash by. Then I waited. Three days later, I received a cheerful email from the billing department asking if I wanted to upgrade my plan. I replied that no, I wanted someone to look at the authentication bug. Two more days passed. The ticket reappeared in my inbox with a refund policy attached. By day six, I had already rewritten the auth layer myself—badly—and the vendor had lost a customer forever.

That ticket died because a routing algorithm, or a human acting like one, misread the intent. “Authentication error” somehow became “account issue,” which became “billing inquiry.” The chain of

misdirection happened because the system had no real understanding of what I was asking, no confidence measurement, no way to escalate when it was unsure, and no handoff protocol that preserved the full context of what I'd written.

If you've ever sent a support request into the void, you know the feeling. In this chapter, we're going to build the opposite of that nightmare: a customer support router that classifies incoming tickets, measures its own confidence, selects a specialist agent with the right domain knowledge, preserves every bit of context through the handoff, drafts a response, and refuses to send anything without human approval. It's a multi-agent system where the router is the conductor and every specialist plays a different instrument—and nobody performs until the conductor knows which song we're playing.

Why Routing Is a Classic Agent Problem

Support routing is the perfect introduction to multi-agent work because it's literally the agent loop at scale. A single ticket arrives. The system must observe it, classify its intent, reason about which specialist has the best chance of solving it, act by dispatching the ticket, and observe again when the specialist returns a draft. If the confidence is low, the system must know when to ask for help instead of guessing.

This is also where specialization pays off. A general-purpose model can answer support questions, but it's expensive and shallow. A technical specialist that has seen five hundred deployment logs will diagnose a Docker networking issue faster and cheaper than a generalist that has to reason from first principles every time. A billing specialist knows refund policies. An escalation specialist knows when to pause, flag urgency, and loop in a human manager.

The challenge isn't building any one of these agents. It's teaching the router to decide who does what, when to trust its own judgment, and how to hand over a ticket without losing the story along the way.

Intent Classification: Technical, Billing, and Escalation

Every ticket starts as a blob of text. Our first job is to decide which bucket it belongs in. We'll build a lightweight intent classifier that uses a keyword heuristic layered under an LLM call. The keyword layer is fast and free; the LLM layer catches nuance that keywords miss.

We'll define a Ticket dataclass to keep the structure clean, then build a Router with a classify() method.

```
from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional

@dataclass
class Ticket:
    """Represents a customer support request."""
    ticket_id: str
    subject: str
    body: str
    sender: str
    received_at: datetime = field(default_factory=datetime.now)
    tags: list[str] = field(default_factory=list)

    def full_text(self) -> str:
        """Return the combined subject and body for
        classification."""
        return f"{self.subject}\n\n{self.body}"
```

The Ticket dataclass is small but strict. It forces every ticket to have an ID, subject, body, and sender, while letting metadata like tags accumulate as the ticket moves through the pipeline. The full_text() method is the contract: whenever a classifier or specialist needs to read the ticket, it gets the whole story, not a truncated summary.

Now the router:

```
import json

class Router:
    """Classifies incoming tickets and routes them to specialist
    agents."""

    def __init__(self, llm_client, confidence_threshold: float =
0.75):
        """
```

```

        Args:
            llm_client: A callable that accepts a prompt string
and
            returns a text response.
            confidence_threshold: Minimum confidence before
trusting
            automatic routing.
        """
        self.llm_client = llm_client
        self.confidence_threshold = confidence_threshold
        self.intent_keywords = {
            "technical": [
                "error", "bug", "crash", "deploy", "timeout",
"docker",
                "authentication", "database", "api", "500", "404",
"exception",
            ],
            "billing": [
                "invoice", "refund", "charge", "payment",
"subscription",
                "renewal", "overcharge", "credit card", "receipt",
            ],
            "escalation": [
                "legal", "privacy", "gdpr", "breach", "lawsuit",
                "compliance", "data loss", "outage", "severity:
critical",
            ],
        }

    def classify(self, ticket: Ticket) -> dict:
        """
        Classify a ticket into one or more intents and return a
confidence
score.

        Returns:
            dict with keys: intent (str), confidence (float),
method (str)
        """
        text_lower = ticket.full_text().lower()
        keyword_scores = {intent: 0 for intent in
self.intent_keywords}

        for intent, keywords in self.intent_keywords.items():
            for kw in keywords:
                keyword_scores[intent] += text_lower.count(kw)

        total_hits = sum(keyword_scores.values())
        if total_hits > 0:
            best_intent = max(keyword_scores,
key=keyword_scores.get)
            raw_confidence = keyword_scores[best_intent] /

```

```

total_hits
    confidence = round(min(raw_confidence + 0.3, 0.95), 2)
    return {
        "intent": best_intent,
        "confidence": confidence,
        "method": "keyword",
    }

    return self._llm_classify(ticket)

def _llm_classify(self, ticket: Ticket) -> dict:
    """Use the LLM to classify when keywords are ambiguous or
    absent."""
    prompt = (
        "You are a support ticket classifier. "
        "Read the ticket and respond with ONLY a JSON object:
\n"
        '{"intent": "technical|billing|escalation",
"confidence": 0.0 to 1.0}\n\n'
        f"Ticket:\n{ticket.full_text()}"
    )
    response = self.llm_client(prompt).strip()
    try:
        parsed = json.loads(response)
        parsed["method"] = "llm"
        parsed.setdefault("confidence", 0.5)
        return parsed
    except json.JSONDecodeError:
        return {
            "intent": "escalation",
            "confidence": 0.0,
            "method": "fallback",
        }

```

The Router tries the cheap path first. If the ticket is stuffed with words like “refund” and “invoice,” the keyword layer declares it billing with high confidence and skips the LLM entirely. If no keywords match at all—maybe the ticket is vague or emotional—the router falls back to the LLM. Notice that the keyword confidence gets a small boost (+0.3) because exact keyword matches are usually reliable, but we cap it at 0.95 so the system never believes it’s infallible.

Routing Confidence Threshold

Confidence is the safety valve. A score of 0.92 means the router is comfortable making a decision. A score of 0.45 means the router is guessing, and guessing in customer support is how you send an outage report to the billing team.

Our threshold is customizable. In the constructor we default to 0.75, but you can raise it for regulated industries or lower it when you're training a new model and want to see its mistakes. When confidence falls below the threshold, the ticket should not be auto-routed. It should hit a fallback path—usually a human triage queue or a generic escalation bucket.

```
def route(self, ticket: Ticket) -> dict:
    """
    Classify a ticket and decide where to send it.

    Returns:
        dict with routing decision, selected specialist, and
        full context.
    """
    classification = self.classify(ticket)
    intent = classification["intent"]
    confidence = classification["confidence"]

    if confidence < self.confidence_threshold:
        intent = "escalation"
        classification["intent"] = intent
        classification["rationale"] = (
            "Low confidence; defaulting to human triage."
        )

    return {
        "ticket_id": ticket.ticket_id,
        "classification": classification,
        "target_specialist": intent,
        "routed_at": datetime.now().isoformat(),
    }
```

The `route()` method is the public face of the router. It calls `classify()`, inspects the confidence, and either commits to a specialist or punts to escalation. Crucially, it never discards the original classification data. Even when we override the intent to “escalation,” we keep the original guess and the reason for the override. That audit trail becomes fuel for the feedback loop we’ll build later.

Specialist Agent Selection

Once the router decides which lane the ticket belongs in, we need an agent that speaks the language of that lane. General customer support language is full of softeners: “We’re sorry to hear that,” “We appreciate your patience.” Technical support, on the other hand, needs exactness: versions, reproduction steps, configuration snippets.

We’ll model each specialist as a class with a single responsibility: draft a response given a ticket and a context bundle.

```
class Specialist:
    """Base class for domain-specific support agents."""

    def __init__(self, name: str, system_prompt: str, llm_client):
        self.name = name
        self.system_prompt = system_prompt
        self.llm_client = llm_client
        self.attempt_count = 0

    def draft(self, ticket: Ticket, handoff_context: str) -> dict:
        """
        Draft a response to a ticket.

        Returns:
            dict with keys: draft (str), specialist (str),
            confidence (float)
        """
        self.attempt_count += 1
        prompt = (
            f"{self.system_prompt}\n\n"
            f"--- Ticket Context ---\n{handoff_context}\n\n"
            "Draft a helpful, accurate response. Be concise. "
            "If you do not know the answer, say so explicitly."
        )
        response = self.llm_client(prompt).strip()
        return {
            "draft": response,
            "specialist": self.name,
            "confidence": 0.9, # Placeholder; calibrate per
specialist.
        }

class TechnicalSpecialist(Specialist):
    """Handles bugs, errors, deployments, and infrastructure
    issues."""

    def __init__(self, llm_client):
        super().__init__(
```

```

        name="technical",
        system_prompt=(
            "You are a senior technical support engineer. "
            "Ask for logs, reproduction steps, and versions. "
            "Never guess a root cause without evidence."
        ),
        llm_client=llm_client,
    )

class BillingSpecialist(Specialist):
    """Handles invoices, refunds, payments, and subscription
    changes."""

    def __init__(self, llm_client):
        super().__init__(
            name="billing",
            system_prompt=(
                "You are a billing support representative. "
                "Be polite, clear, and reference policy
accurately. "
                "If the issue requires a manual refund approval,
flag it."
            ),
            llm_client=llm_client,
        )

class EscalationSpecialist(Specialist):
    """Handles urgent, sensitive, or ambiguous tickets that need
    human
    judgment."""

    def __init__(self, llm_client):
        super().__init__(
            name="escalation",
            system_prompt=(
                "You are an escalation coordinator. "
                "Summarize the ticket for a human manager. "
                "Highlight urgency, risk, and any missing
information."
            ),
            llm_client=llm_client,
        )

```

Each specialist inherits from a common base, which means our router can treat them polymorphically. The router doesn't need to know whether a draft came from the billing specialist or the technical specialist; it just needs a dictionary with a draft key and a confidence key. That's the handoff contract.

Notice the explicit instruction in every system prompt: “If you do not know the answer, say so explicitly.” This is guardrail number one. A specialist that hallucinates a refund policy or fabricates a bug fix is worse than no specialist at all.

Handoff Protocols: Passing Full Context

The most common failure mode in routed systems is context loss. A ticket starts with a rich history: the customer’s original message, the classification result, the confidence score, the routing method, maybe a previous specialist’s failed attempt. If we only pass the subject line to the next agent, we’re asking it to perform surgery with a blurred photograph.

Our handoff protocol bundles everything into a single context string.

```
def build_handoff_context(
    ticket: Ticket,
    classification: dict,
    previous_attempts: Optional[list[dict]] = None,
) -> str:
    """
    Build a complete handoff context string for a specialist
    agent.

    Args:
        ticket: The original support ticket.
        classification: Output from Router.classify().
        previous_attempts: List of prior specialist drafts that
        were
            rejected or incomplete.

    Returns:
        A formatted context string with all available history.
    """
    lines = [
        f"Ticket ID: {ticket.ticket_id}",
        f"From: {ticket.sender}",
        f"Received: {ticket.received_at.isoformat()}",
        f"Classification: {classification['intent']} (confidence:
    "
        f"{classification['confidence']}, method:
{classification['method']})",
        "--- Original Message ---",
        ticket.full_text(),
    ]

    if previous_attempts:
```

```

        lines.append("--- Previous Specialist Attempts ---")
        for idx, attempt in enumerate(previous_attempts, 1):
            lines.append(
                f"Attempt {idx} by {attempt['specialist']}:"
                f"\n{attempt['draft']}\n"
            )

    return "\n".join(lines)

```

This function never truncates the ticket. It never summarizes away the error codes. It includes the classification metadata so the specialist knows how the ticket was categorized and whether the router was confident. If another specialist already tried and failed, that attempt is appended with a header. The next specialist sees the failure and can choose a different angle rather than repeating the same wrong answer.

SLA Awareness: Urgency Scoring

Not all tickets are equal. A feature request about dark mode can wait. A report that the payment system is down cannot. We'll add a lightweight urgency scorer that inspects the ticket text for urgency signals and maps them to SLA tiers.

```

SLA_KEYWORDS = {
    "critical": [
        "outage", "down", "breach", "data loss", "cannot login",
        "unable to pay", "security incident",
    ],
    "high": [
        "urgent", "deadline", "blocking", "production", "refund",
    ],
    "error": [
    ],
    "normal": [
        "question", "how to", "feature request", "suggestion",
    ],
}

```

```

def sla_urgency_scorer(ticket: Ticket) -> dict:
    """
    Score a ticket's urgency based on keyword analysis.

    Returns:
        dict with tier (str), score (int 1-10), and matched_terms
        (list).
    """
    text_lower = ticket.full_text().lower()
    matched_terms = []
    for tier, keywords in SLA_KEYWORDS.items():

```

```

    for kw in keywords:
        if kw in text_lower:
            matched_terms.append((tier, kw))

    if any(t == "critical" for t, _ in matched_terms):
        return {
            "tier": "critical",
            "score": 10,
            "matched_terms": [
                k for t, k in matched_terms if t == "critical"
            ],
        }
    if any(t == "high" for t, _ in matched_terms):
        return {
            "tier": "high",
            "score": 7,
            "matched_terms": [
                k for t, k in matched_terms if t == "high"
            ],
        }
    return {
        "tier": "normal",
        "score": 3,
        "matched_terms": [k for _, k in matched_terms],
    }

```

The scorer is intentionally simple. It doesn't use an LLM, because urgency should be cheap to compute and deterministic. A ticket mentioning "outage" gets a critical score regardless of tone. This score feeds into the approval gate: critical tickets might skip the human draft approval and go straight to a live manager, or they might get flagged with a red banner in the review UI.

Human Approval Before Send

The golden rule of agent-assisted support is: the agent drafts, the human decides. Never let an unreviewed draft reach a customer. Our approval gate is a function that takes the draft and presents an actionable choice.

```

def approval_gate(draft: str, ticket: Ticket, urgency: dict) -> dict:
    """
    Present a draft response for human review and capture the
    decision.

```

Args:

```

    draft: The response text drafted by a specialist.
    ticket: The original ticket.

```

urgency: Output from `sla_urgency_scorer()`.

```
Returns:
    dict with action (send|edit|escalate), reviewer_notes
(str),
    and timestamp.
"""
print("\n" + "=" * 60)
print(f"APPROVAL GATE for Ticket {ticket.ticket_id}")
print(f"Urgency: {urgency['tier'].upper()} (score
{urgency['score']})")
print(f"Matched terms: {' '.join(urgency['matched_terms']) or
'None'}")
print("-" * 60)
print("DRAFT RESPONSE:")
print(draft)
print("-" * 60)

# In production this is a UI button click or API endpoint.
# Here we simulate with input for demonstration.
action = input("Action [send/edit/escalate]:
").strip().lower()
notes = input("Reviewer notes (optional): ").strip()

return {
    "action": (
        action if action in ("send", "edit", "escalate") else
"escalate"
    ),
    "reviewer_notes": notes,
    "reviewed_at": datetime.now().isoformat(),
}
```

In production, `approval_gate` is not an interactive prompt. It's a webhook that waits for a human to click a button in a ticketing UI. But the contract is identical: the function receives a draft, returns a structured decision, and logs a timestamp. The router pauses until that decision returns.

If the action is `send`, the draft goes out. If it's `edit`, the draft goes back to the specialist or to a human editor with the reviewer notes attached. If it's `escalate`, the ticket bypasses auto-response entirely and lands in a manager's queue.

Feedback Loop for Routing Accuracy

An agent system that never learns is just expensive automation. We want the router to improve over time, which means recording every decision, every human correction, and every outcome. The feedback loop is a simple append-only log.

```
import csv
import os

ROUTING_LOG_PATH = "routing_feedback.csv"

class FeedbackLogger:
    """Logs routing decisions and outcomes for later analysis."""

    def __init__(self, path: str = ROUTING_LOG_PATH):
        self.path = path
        if not os.path.exists(path):
            with open(path, "w", newline="") as f:
                writer = csv.writer(f)
                writer.writerow([
                    "ticket_id", "predicted_intent",
                    "predicted_confidence",
                    "human_corrected_intent", "specialist_name",
                    "approval_action", "timestamp",
                ])

    def log(
        self,
        ticket_id: str,
        classification: dict,
        specialist: str,
        approval: dict,
        corrected_intent: Optional[str] = None,
    ):
        """Append a single routing decision to the feedback
        log."""
        with open(self.path, "a", newline="") as f:
            writer = csv.writer(f)
            writer.writerow([
                ticket_id,
                classification.get("intent"),
                classification.get("confidence"),
                corrected_intent or classification.get("intent"),
                specialist,
                approval.get("action"),
                datetime.now().isoformat(),
            ])
```


Once a week, or after a thousand tickets, you can analyze this CSV. If the router consistently classifies “refund” requests as technical because the word “error” appears in “payment error,” you’ll see a pattern in the `human_corrected_intent` column. You can then add “payment error” to the billing keyword list, retrain the LLM prompt examples, or simply lower the confidence threshold for tickets that match the problematic pattern. The log is the truth. Everything else is a hypothesis.

The Full Pipeline

Here is how the pieces fit together from ticket arrival to final disposition:

```
graph TD
    A[Incoming Ticket] --> B[Intent Classifier]
    B --> C{Routing Confidence >= Threshold?}
    C -- Yes --> D[Select Specialist Agent]
    C -- No --> E[Escalation Specialist]
    D --> F[Technical | Billing | Escalation]
    F --> G[Specialist Agent Drafts Response]
    E --> G
    G --> H[Human Approval Gate]
    H --> I{Decision}
    I -- Send --> J[Deliver to Customer]
    I -- Edit --> G
    I -- Escalate --> K[Human Manager Queue]
    J --> L[Feedback Logger]
    K --> L
    L --> M[Improve Routing Rules]
```

The diagram tells the story in one glance. Tickets enter at the top. Classification either commits to a specialist or defaults to escalation. The specialist drafts. A human approves, edits, or escalates. Every path ends at the feedback logger, which closes the loop and makes the system better tomorrow than it was today.

Putting It All Together: `main()`

Here’s a complete `main()` function that wires everything up and processes a batch of sample tickets. In a real deployment, the `llm_client` would be an API wrapper for your local model or cloud provider. For this example, we’ll use a tiny fake client so you can run the script without credentials.

```
def fake_llm_client(prompt: str) -> str:
    """A stub LLM client for demonstration."""
```

```

if "classifier" in prompt.lower()[:100]:
    return '{"intent": "technical", "confidence": 0.85}'
return (
    "Thank you for reaching out. "
    "We've received your message and are reviewing it."
)

def main():
    """Process sample support tickets through the full router
    pipeline."""
    llm = fake_llm_client
    router = Router(llm_client=llm, confidence_threshold=0.75)
    logger = FeedbackLogger(path="demo_feedback.csv")

    specialists = {
        "technical": TechnicalSpecialist(llm),
        "billing": BillingSpecialist(llm),
        "escalation": EscalationSpecialist(llm),
    }

    sample_tickets = [
        Ticket(
            ticket_id="T-1001",
            subject="API timeout after deployment",
            body=(
                "We deployed v2.3.1 and now every POST to /users "
                "times out after 30s. No errors in the app logs."
            ),
            sender="ops@example.com",
        ),
        Ticket(
            ticket_id="T-1002",
            subject="Refund for duplicate charge",
            body=(
                "I was charged twice on April 15 for the Pro plan.
"
                "Can you reverse the second charge?"
            ),
            sender="customer@example.com",
        ),
        Ticket(
            ticket_id="T-1003",
            subject="Something is wrong",
            body="I don't know how to explain it but the dashboard
feels slow.",
            sender="vague@example.com",
        ),
    ]

    for ticket in sample_tickets:
        print(f"\n>>> Processing {ticket.ticket_id}:

```

```

{ticket.subject}")

    routing = router.route(ticket)
    classification = routing["classification"]
    print(
        f"Routing: intent={classification['intent']}, "
        f"confidence={classification['confidence']}"
    )

    urgency = sla_urgency_scorer(ticket)
    print(f"Urgency: {urgency['tier']} (score {urgency['score']}")

    specialist = specialists[routing["target_specialist"]]
    context = build_handoff_context(ticket, classification)
    result = specialist.draft(ticket, context)
    print(f"Draft from {result['specialist']} specialist prepared.")

    # Simulate human approval without blocking unattended
    runs.
    if urgency["tier"] == "critical":
        approval = {
            "action": "escalate",
            "reviewer_notes": "Auto-escalated critical
ticket.",
            "reviewed_at": datetime.now().isoformat(),
        }
    else:
        approval = {
            "action": "send",
            "reviewer_notes": "Looks good.",
            "reviewed_at": datetime.now().isoformat(),
        }

    logger.log(
        ticket_id=ticket.ticket_id,
        classification=classification,
        specialist=result["specialist"],
        approval=approval,
        corrected_intent=(
            classification["intent"]
            if approval["action"] != "escalate"
            else "escalation"
        ),
    )

    print(f"Final action: {approval['action']}")

if __name__ == "__main__":
    main()

```

The `main()` function is the orchestrator. It creates the router, the logger, and the specialist map. For each ticket, it prints the routing decision, scores urgency, hands the context to the right specialist, collects a draft, simulates approval, and logs the result. In production, the approval step blocks on a real human or a webhook. Here we simulate it so the script can run end-to-end in a CI pipeline or a code review.

Break and Fix: Five Ways This Project Can Fail

Routing systems fail quietly. A bad classification doesn't throw an exception; it sends a customer's refund request to a Kubernetes engineer who stares at it in confusion. Here are the five failure modes you should know how to diagnose and patch.

Break 1: Wrong Classification

A ticket reads: "Our team is trying to deploy the new payment webhook but we're getting a 404." The router classifies it as billing because "payment" appears in the text. The billing specialist apologizes for the inconvenience and asks for an invoice number. The customer is baffled.

Fix: Weight domain-specific technical terms more heavily than generic financial language. Add "deploy," "webhook," and "404" to the technical keyword list with higher scores, or use a weighted scoring system instead of simple tallies. Also, inspect the LLM fallback prompt: if it sees "payment" without context, it may default to billing. Strengthen the prompt with examples of technical tickets that happen to mention payments.

Break 2: Angry Customer Not Escalated (Sentiment Missed)

A customer writes: "This is the third time I've asked. Your service is unusable and your support is a joke. Fix it or I'm leaving." There are no technical keywords. No billing keywords. The router classifies it as "normal" with 0.3 confidence, defaults to escalation, but the escalation specialist drafts a generic "We're sorry" response instead of flagging the emotional temperature.

Fix: Add a sentiment layer. Before classification, run a simple sentiment check on the ticket. Excessive capitalization, words like “ridiculous,” “joke,” “lawsuit,” or repeated exclamation points should raise an urgency flag independent of topic. In Router.classify(), if sentiment is strongly negative, force the route to escalation with high urgency even if the topic is unclear.

Break 3: Specialist Doesn’t Know the Answer

The technical specialist receives a ticket about an obscure edge case in a legacy version of the API. It has no training on that version. Instead of admitting ignorance, it hallucinates a fix: “Please restart the service with the --legacy flag.” There is no such flag. The customer tries it and breaks their staging environment.

Fix: Enforce the “say so explicitly” rule in the specialist prompt, and validate drafts against a known corpus before sending them to approval. At minimum, require the specialist to include a confidence score in its draft metadata. If confidence is below 0.6, route the draft to a senior engineer rather than to the approval gate. Also, maintain a versioned knowledge base and instruct the specialist to reference it; if the answer isn’t there, the specialist must defer.

Break 4: Handoff Loses Context (Ticket Summary Truncated)

A long ticket arrives: eight paragraphs of logs, reproduction steps, and environment details. The build_handoff_context function accidentally truncated the body to 500 characters to “save tokens.” The specialist only sees a fragment, misses the key error line buried in paragraph six, and asks the customer to rerun the test they already ran.

Fix: Never truncate ticket text for context passing. If token limits are a concern, summarize separately but always include the full text as an appendix in the handoff. The specialist can skim the summary and dig into the raw text when needed. More importantly, the handoff contract should promise completeness: if the context doesn’t contain the full ticket, the specialist should throw an error rather than guess.

Break 5: Routing Confidence Too Low but No Fallback Path

A ticket is completely ambiguous: “Help.” The keyword scores are zero. The LLM returns {"intent": "escalation", "confidence": 0.2}. The router sees 0.2, which is below the 0.75 threshold, but the fallback logic was never completed. The ticket is logged as “escalation” and then dropped. No human is notified. The customer waits forever.

Fix: Always implement a dead-letter queue. In route(), when confidence is below threshold and the intent is not already escalation, redirect to a human triage queue and create a visible alert. The fallback should never be silent. The feedback logger must record the low-confidence event, and an alert should fire in your monitoring system so an operations team can inspect the ticket manually.

Try It Now: Three Variations

Now that you’ve built a working support router, stretch it. These are not optional extras. They’re where the learning actually happens.

Variation 1: Sentiment-Aware Auto-Escalation

Integrate a sentiment scorer—either a lightweight library or a second LLM prompt—into the router. If a ticket’s sentiment score is below a certain threshold (very negative), force the route to escalation regardless of classification confidence. Run ten sample tickets through the system with and without sentiment awareness. Count how many angry customers land in the right queue.

Variation 2: Learn from Past Outcomes to Improve Routing Accuracy

Build a script that reads routing_feedback.csv, calculates per-intent accuracy, and suggests threshold adjustments. For example, if billing tickets are being corrected to technical 30% of the time, lower the confidence threshold for billing routes or update the keyword map. Automate this as a weekly “router tuning” report that prints three concrete recommendations.

Variation 3: Generate a KB Article from Repeated Ticket Patterns

When the feedback logger shows that three different customers asked the same question in a week, that's a signal. Build a script that clusters tickets by classification and draft similarity, then prompts an LLM to generate a knowledge base article from the three original tickets and their approved responses. The article can then be attached to the specialist prompt as a reference, improving future drafts automatically.

The OpenClaw Way

OpenClaw handles multi-agent orchestration with a Router node type that you wire into a workflow DAG. Instead of building Router and Specialist classes by hand, you define an IntentClassifier node and route its outputs to downstream agent nodes using conditional edges. OpenClaw manages the handoff context as a shared state dictionary: every node appends its outputs to `ctx["ticket_history"]`, which means no context is lost unless you explicitly clear it. The approval gate becomes a HumanReview node that pauses the workflow and exposes a REST endpoint for reviewers to submit send, edit, or escalate decisions. Cost tracking is built in, so you can see exactly how much each specialist consumed in tokens before the human ever saw the draft. For production support pipelines, OpenClaw's DAG visualization also makes it easy to show a manager why a particular ticket took three hops instead of one.

The Hermes Agent Way

Hermes Agent approaches this as a conversation session with persistent memory rather than a hardcoded pipeline. You initialize a session with a system prompt describing the router's role, then feed each ticket as a user message. Hermes decides whether to invoke its built-in web search for policy lookups or its code execution tool for log parsing. Specialist behavior emerges from prompt instructions rather than separate objects: you tell the agent, "When the topic is billing, adopt the billing persona." Handoff is implicit in Hermes's memory, which carries the full ticket thread across turns. The approval gate is a built-in --review flag that buffers every outbound message until a human types approve. For teams that want a support router without writing classes or managing CSV files, Hermes can get you started in

minutes. The trade-off is less explicit control: you don't get a `RoutingFeedback.csv`; you get a session transcript that you mine for patterns later.

What You Built

In this chapter, you built a complete support routing pipeline from pure Python:

- A `Ticket` dataclass that guarantees structured input and exposes the full text of every request.
- A `Router` class with `classify()` and `route()` that uses keyword heuristics first, LLM fallback second, and a configurable confidence threshold as a safety valve.
- Three specialist agents—`TechnicalSpecialist`, `BillingSpecialist`, and `EscalationSpecialist`—each with domain-tuned system prompts and a shared polymorphic interface.
- A `build_handoff_context()` function that passes the complete ticket, classification metadata, and previous specialist attempts to the next agent in line.
- An `sla_urgency_scorer()` that tiers tickets into critical, high, and normal buckets using deterministic keyword analysis.
- An `approval_gate()` function that enforces human review before any draft is sent to a customer.
- A `FeedbackLogger` that records every routing decision and human correction in a CSV, creating a closed loop for continuous improvement.
- A `main()` entrypoint that processes sample tickets end-to-end, printing routing decisions, urgency scores, and final actions.

You also learned five specific failure modes—wrong classification, missed sentiment, hallucinated specialist answers, truncated context, and silent low-confidence fallbacks—and how to fix each one with concrete code changes.

That's a production-grade skeleton. The next chapter adds data quality agents to the mix, because even the best router can't fix dirty input.

Cost for this chapter: Approximately \$0.00 if you use Ollama and the keyword classifier handles most traffic. With GPT-4o mini as the fallback LLM for ambiguous tickets, expect roughly \$0.002–\$0.005 per ticket classified by the LLM path, and \$0.01–\$0.03 per specialist draft. A typical support queue with 80% keyword routing and 20% LLM fallback costs about \$0.30–\$0.70 per hundred tickets.

Chapter 17: Project 17 — Data Quality Team

Data that is not monitored is data that is quietly rotting.

In the winter of 2022, the head of sales sent me a screenshot of a dashboard that made no sense. Our monthly revenue chart showed a negative three-million-dollar month. Not zero. Negative three million. I checked the source data and found a single CSV where someone had typed - in a few thousand revenue cells to mean “not applicable.” The ingestion pipeline treated them as minus signs, converted them to negative numbers, and rolled them up into a total that suggested our company had somehow paid our customers to take our product. The dashboard was technically correct. The data was catastrophically dirty.

I spent the next three days tracing backward through five ETL jobs, two handoffs, and a shared folder where three different people dropped files with the same name. There was no schema validation. There were no null checks. There was no owner who had been told that a column full of dashes was not the same as a column full of zeros. By the time I fixed the pipeline, the same bad CSV had already fed a board presentation. The CFO asked why no one had flagged the anomaly automatically. I did not have a good answer.

That is what this chapter is about. You will build a Data Quality Team: three specialized agents that validate, inspect, and suggest fixes for a dataset before it ever reaches a dashboard, a model, or a PowerPoint deck. One agent guards the schema. One agent hunts statistical anomalies. One agent proposes remediation. They output a structured report with severity levels, categories, and concrete suggestions you can review before any transformation runs.

Data Quality Is a Process, Not an Event

Cleaning data once is like washing your car once. It gets dirty again. The real problem is not the occasional bad row; it is the fact that every upstream system changes without telling you. A partner adds a new column to a daily feed. A web form starts allowing empty strings where email addresses used to be mandatory. An API deprecation silently switches timestamps from ISO format to Unix seconds. These are not bugs in your code. They are mutations in the world your code consumes.

A Data Quality Team treats these mutations as first-class events. Instead of running a one-time cleanup script, the team operates continuously on every file that arrives. It compares the current dataset against an expected shape, measures statistical drift, and raises a signal when the world has changed under your feet. The goal is not perfect data. The goal is *detected* imperfection, surfaced with enough context that a human can decide what to do next.

We will build this in pure Python with pandas and numpy. No LLM frameworks yet. The agents are deterministic functions that operate on DataFrames. The orchestration is a simple pipeline. The output is a machine-readable report and a human-readable markdown summary that you can drop into Slack, email, or a CI artifact.

The Foundation: Issue and DataQualityAgent

Before we write the specialists, we need a shared contract. Every agent must speak the same language, or the final report will be a mess of different shapes.

```
import json
from dataclasses import dataclass, asdict
from typing import List, Dict, Any, Optional
import pandas as pd
import numpy as np

@dataclass
class Issue:
    """A single data-quality finding with severity, category, and suggestion."""
    severity: str          # critical, error, warning, info
    category: str          # schema, anomaly, remediation
    column: str
    message: str
```

```

    suggestion: str

    def to_dict(self) -> Dict[str, Any]:
        """Serialize the issue to a plain dictionary for JSON
        output."""
        return asdict(self)

```

The severity field is what drives prioritization. Critical means stop the pipeline. Error means fix before using. Warning means suspicious but not necessarily fatal. Info means a context note, such as a column being dropped in this file but present historically.

Next, the base class. Every agent will inherit from `DataQualityAgent` and implement a `run` method that accepts a `DataFrame` and returns a list of `Issue` objects.

```

class DataQualityAgent:
    """Base class for data quality specialists operating on a
    pandas DataFrame."""

    def __init__(self, name: str):
        self.name = name

    def run(self, df: pd.DataFrame) -> List[Issue]:
        """Execute the agent against a DataFrame and return zero
        or more issues."""
        raise NotImplementedError("Subclasses must implement
        run().")

```

This is intentionally thin. The subclasses carry the real logic. The thin base class matters because it enforces the interface: no matter how many agents you add later, the orchestrator knows it can call `.run(df)` and get back a flat list of issues.

Agent 1: The Schema Validator

The Schema Validator answers a single question: does this file look like what I expected? It checks three things: required columns are present, unexpected columns are noted, and column types roughly match expectations.

```

class SchemaValidator(DataQualityAgent):
    """Validate that the DataFrame matches an expected schema."""

    def __init__(
        self,
        required_columns: List[str],
        optional_columns: Optional[List[str]] = None,

```

```

expected_types: Optional[Dict[str, str]] = None,
):
    super().__init__("SchemaValidator")
    self.required_columns = set(required_columns)
    self.optional_columns = set(optional_columns or [])
    self.expected_types = expected_types or {}

def run(self, df: pd.DataFrame) -> List[Issue]:
    """Compare the DataFrame against the expected schema and
return issues."""
    issues: List[Issue] = []
    actual_columns = set(df.columns)

    # Missing required columns
    for col in self.required_columns:
        if col not in actual_columns:
            issues.append(Issue(
                severity="critical",
                category="schema",
                column=col,
                message=f"Required column '{col}' is
missing.",
                suggestion="Reject this file and alert the
upstream source.",
            ))

    # Unexpected new columns
    allowed = self.required_columns | self.optional_columns
    for col in actual_columns - allowed:
        issues.append(Issue(
            severity="warning",
            category="schema",
            column=col,
            message=f"Unexpected column '{col}' found.",
            suggestion="Review whether this column should be
added to the schema.",
        ))

    # Type checks (coarse: numeric vs string distinction)
    for col, expected in self.expected_types.items():
        if col not in df.columns:
            continue
        actual_kind = df[col].dtype.kind
        if expected == "numeric" and actual_kind not in ("i",
"u", "f"):
            issues.append(Issue(
                severity="error",
                category="schema",
                column=col,
                message=f"Column '{col}' expected numeric, got
{df[col].dtype}.",
                suggestion="Check for stray text or

```

```

punctuation in numeric fields.",
    ))
    elif expected == "string" and actual_kind != "0":
        issues.append(Issue(
            severity="warning",
            category="schema",
            column=col,
            message=f"Column '{col}' expected string, got
{df[col].dtype}.",
            suggestion="Ensure downstream consumers handle
non-string types.",
        ))

    return issues

```

Notice the type checking is intentionally coarse. Pandas reads CSVs through inference, so a column of mostly numbers with one stray word becomes an object dtype. The Schema Validator flags that as an error so a human can look at the raw file rather than letting pandas silently coerce everything to float.

The suggestions are not transformations. They are English sentences that tell the operator what to investigate. The Remediation Agent will turn some of these into code later.

Agent 2: The Anomaly Detector

The Anomaly Detector performs statistical checks per column. It does not know what the schema should look like. It knows what healthy data usually looks like: no more nulls than a threshold, no extreme outliers, sensible ranges, and cardinality that does not explode unexpectedly.

```

class AnomalyDetector(DataQualityAgent):
    """Run statistical checks for nulls, outliers, ranges, and
    cardinality."""

    def __init__(
        self,
        null_threshold: float = 0.05,
        z_threshold: float = 3.0,
        range_rules: Optional[Dict[str, tuple]] = None,
    ):
        super().__init__("AnomalyDetector")
        self.null_threshold = null_threshold
        self.z_threshold = z_threshold
        self.range_rules = range_rules or {}

```

```

def run(self, df: pd.DataFrame) -> List[Issue]:
    """Inspect each column and return statistical anomaly
    findings."""
    issues: List[Issue] = []
    row_count = len(df)

    for col in df.columns:
        series = df[col]
        non_null = series.dropna()

        # Null count check
        null_rate = series.isna().mean()
        if null_rate > self.null_threshold:
            issues.append(Issue(
                severity="error",
                category="anomaly",
                column=col,
                message=(
                    f"Column '{col}' has {null_rate:.1%} nulls
                    f"(threshold {self.null_threshold:.0%})."
                ),
                suggestion="Investigate upstream source or
                apply imputation.",
            ))

        # Unique ratio (cardinality)
        if not non_null.empty:
            unique_ratio = non_null.nunique() / len(non_null)
            if unique_ratio == 1.0 and len(non_null) > 1:
                issues.append(Issue(
                    severity="info",
                    category="anomaly",
                    column=col,
                    message=f"Column '{col}' has all unique
                    values.",
                    suggestion="Verify this is an ID or key
                    column, not a duplicated dimension.",
                ))
            elif unique_ratio < 0.01 and len(non_null) > 100:
                issues.append(Issue(
                    severity="warning",
                    category="anomaly",
                    column=col,
                    message=f"Column '{col}' has very low
                    cardinality ({unique_ratio:.2%}).",
                    suggestion="Check for repeated default
                    values or missing diversity.",
                ))

        # Numeric-specific checks
        if non_null.dtype.kind in ("i", "u", "f"):

```

```

        # Z-score outliers
        mean_val = non_null.mean()
        std_val = non_null.std()
        if std_val and std_val > 0:
            outliers = ((non_null - mean_val).abs() >
self.z_threshold * std_val).sum()
            if outliers > 0:
                issues.append(Issue(
                    severity="warning",
                    category="anomaly",
                    column=col,
                    message=(
outlier(s) "
                        f"Column '{col}' has {outliers}
sigma."
                        f"beyond {self.z_threshold}
                    ),
                    suggestion="Validate extreme values or
apply capping/winsorization.",
                ))

        # Range check
        if col in self.range_rules:
            lo, hi = self.range_rules[col]
            below = (non_null < lo).sum()
            above = (non_null > hi).sum()
            if below or above:
                issues.append(Issue(
                    severity="error",
                    category="anomaly",
                    column=col,
                    message=(
value(s) below {lo} "
                        f"Column '{col}' has {below}
                        f"and {above} above {hi}."
                    ),
                    suggestion="Clip values or reject rows
outside the valid range.",
                ))

        return issues

```

The null check uses a configurable threshold instead of demanding zero nulls, because real data has holes. The unique-ratio check catches two common smells: a column that looks like an ID but is not marked as one, and a column that is supposed to vary but is actually a single repeated value wrapped in ten thousand rows. The z-score outlier test is crude but fast. It will not catch every anomaly, but it will catch the minus-three-million-dollar month.

Agent 3: The Remediation Suggester

The Remediation Agent consumes the issues produced by the first two agents and generates transformation suggestions. It is not an LLM call. It is a deterministic mapper: every category and severity maps to a set of pandas operations you could apply.

```
class RemediationAgent(DataQualityAgent):
    """Suggest concrete pandas transformations based on issues
    found by other agents."""

    def run(self, df: pd.DataFrame) -> List[Issue]:
        """
        This agent does not scan the raw DataFrame alone.
        It is invoked by the orchestrator with a list of upstream
        issues.
        """
        raise NotImplementedError(
            "Use the suggest method instead, passing upstream
            issues."
        )

    def suggest(self, issues: List[Issue], df: pd.DataFrame) ->
    List[Issue]:
        """Return remediation suggestions mapped from issue
        category and severity."""
        suggestions: List[Issue] = []

        for issue in issues:
            if issue.category == "schema" and issue.severity ==
            "critical":
                suggestions.append(Issue(
                    severity="critical",
                    category="remediation",
                    column=issue.column,
                    message=f"Cannot process file: missing
                    required column '{issue.column}'.",
                    suggestion="Reject file. Do not attempt
                    automated repair.",
                ))

            elif issue.category == "anomaly" and "nulls" in
            issue.message:
                suggestions.append(Issue(
                    severity=issue.severity,
                    category="remediation",
                    column=issue.column,
                    message=f"Fill missing values in
                    '{issue.column}'.",
                    suggestion=f"df['{issue.column}'] =
                    df['{issue.column}'].fillna(df['{issue.column}'].min())"
                ))
```



```

df['{issue.column}'].fillna(method='ffill'),
    ))

    elif issue.category == "anomaly" and "outlier" in
issue.message:
        suggestions.append(Issue(
            severity=issue.severity,
            category="remediation",
            column=issue.column,
            message=f"Cap outliers in '{issue.column}'.",
            suggestion=(
                f"lower =
df['{issue.column}'].quantile(0.01); "
                f"upper =
df['{issue.column}'].quantile(0.99); "
                f"df['{issue.column}'] =
df['{issue.column}'].clip(lower, upper)"
            ),
        ))

    elif issue.category == "anomaly" and "range" in
issue.message:
        if issue.column in df.columns:
            lo, hi = df[issue.column].min(),
df[issue.column].max()
            suggestions.append(Issue(
                severity=issue.severity,
                category="remediation",
                column=issue.column,
                message=f"Clamp '{issue.column}' to known
valid range.",
                suggestion=f"df.loc[df['{issue.column}'] <
{lo}, '{issue.column}'] = {lo}",
            ))

    elif issue.category == "schema" and "Unexpected
column" in issue.message:
        suggestions.append(Issue(
            severity="info",
            category="remediation",
            column=issue.column,
            message=f"Drop unexpected column
'{issue.column}' to maintain downstream contract.",
            suggestion=f"df =
df.drop(columns=['{issue.column}'])",
        ))

    return suggestions

```

The Remediation Agent is deliberately conservative. Critical schema failures are non-negotiable: the suggestion is to reject the file, not to invent a column of fake data. Anomaly repairs are optional and code-annotated. You could run them, but you should review them first. That separation — detection vs. repair — is what prevents an automated system from silently baking bias into a dataset. Always detect automatically. Repair with human approval.

The Orchestrator and Structured Report

The orchestrator wires the three agents together. Schema and Anomaly run in parallel conceptually because they have no dependency. Remediation needs their issues, so it runs after.

```
graph LR
    A[CSV Load] --> B{Schema Validator}
    A --> C{Anomaly Detector}
    B --> D[Issue List]
    C --> D
    D --> E{Remediation Suggester}
    E --> F[Structured Report]
```

Here is the orchestrator in code:

```
def run_quality_team(
    df: pd.DataFrame,
    schema_agent: SchemaValidator,
    anomaly_agent: AnomalyDetector,
    remediation_agent: RemediationAgent,
) -> Dict[str, Any]:
    """
    Run the full Data Quality Team pipeline.
    Returns a structured report with JSON and markdown
    representations.
    """
    # Phase 1: parallel discovery (sequential in pure Python for
    clarity)
    schema_issues = schema_agent.run(df)
    anomaly_issues = anomaly_agent.run(df)
    all_issues = schema_issues + anomaly_issues

    # Phase 2: remediation suggestions
    remediation_issues = remediation_agent.suggest(all_issues, df)
    all_issues.extend(remediation_issues)

    # Severity ordering
    severity_order = {"critical": 0, "error": 1, "warning": 2,
"info": 3}
    all_issues.sort(key=lambda i: severity_order.get(i.severity,
```

```

99))

# Build report
report = {
    "source": "DataQualityTeam",
    "row_count": len(df),
    "column_count": len(df.columns),
    "summary": {
        "critical": sum(1 for i in all_issues if i.severity ==
"critical"),
        "error": sum(1 for i in all_issues if i.severity ==
"error"),
        "warning": sum(1 for i in all_issues if i.severity ==
"warning"),
        "info": sum(1 for i in all_issues if i.severity ==
"info"),
    },
    "issues": [i.to_dict() for i in all_issues],
}

# Markdown human-readable version
md_lines = [
    "# Data Quality Report",
    f"",
    f"- **Rows:** {report['row_count']:,}",
    f"- **Columns:** {report['column_count']}",
    f"",
    "## Summary",
    f"| Severity | Count |",
    f"|-----|-----|",
]
for sev in ("critical", "error", "warning", "info"):
    md_lines.append(f"| {sev} | {report['summary'][sev]} |")
md_lines.append("")
md_lines.append("## Issues")
for i in all_issues:
    md_lines.append(f"### {i.severity.upper()}: {i.column}")
    md_lines.append(f"- **Category:** {i.category}")
    md_lines.append(f"- **Message:** {i.message}")
    md_lines.append(f"- **Suggestion:** {i.suggestion}")
    md_lines.append("")

report["markdown"] = "\n".join(md_lines)
return report

```

The JSON output is what you feed to downstream automation: CI gates, Slack bots, or database triggers. The markdown output is what you paste into an email or attach to a pull request. Both are generated from the same Issue list, so they cannot contradict each other.

Putting It Together: A Sample Sales CSV

Create sales_q1.csv with the following contents. It is deliberately dirty: a missing required column, a stray null, an extreme outlier, and a range violation.

```
date,region,product,quantity,revenue
2024-01-15,North,Widget,10,250.00
2024-01-16,South,Gadget,5,
2024-01-17,East,Widget,8,199.99
2024-01-18,West,Gadget,20000,5000000.00
2024-01-19,North,Widget,12,-50.00
```

Now the main function that runs the team:

```
def main() -> None:
    """Load a sample CSV and run the Data Quality Team against
    it."""
    df = pd.read_csv("sales_q1.csv")

    schema = SchemaValidator(
        required_columns=["date", "region", "product", "quantity",
"revenue", "customer_id"],
        optional_columns=["discount_code"],
        expected_types={"quantity": "numeric", "revenue":
"numeric"},
    )

    anomaly = AnomalyDetector(
        null_threshold=0.05,
        z_threshold=2.5,
        range_rules={"quantity": (0, 1000), "revenue": (0,
100000)}),
    )

    remediation = RemediationAgent()

    report = run_quality_team(df, schema, anomaly, remediation)

    # Save JSON and markdown artifacts
    with open("quality_report.json", "w", encoding="utf-8") as f:
        json.dump(report, f, indent=2)

    with open("quality_report.md", "w", encoding="utf-8") as f:
        f.write(report["markdown"])

    print("Report saved to quality_report.json and
quality_report.md")
    print(f"Critical issues: {report['summary']['critical']}")
    print(f"Errors: {report['summary']['error']}")
    print(f"Warnings: {report['summary']['warning']}")
```

```
if __name__ == "__main__":  
    main()
```

Running this produces a report that flags `customer_id` as missing (critical), the empty revenue cell as a null error, the 20000 quantity as an outlier and range violation, and the -50.00 revenue as below-range. The Remediation Agent suggests rejecting the file due to the missing critical column, filling the null revenue cell, and clipping the outlier quantity. You, the human operator, can review the markdown report and decide which suggestions to apply.

That is the whole loop: load, validate, detect, suggest, review. No magic. Just structured observation of structured data.

Break and Fix

Break 1: False Positive Anomaly on Seasonal Data

You run the Anomaly Detector against a retail dataset in December. The quantity column has a z-score outlier flag on every row because holiday sales are three standard deviations above the yearly mean. The detector is technically correct but practically useless.

Why it fails: The z-score test assumes a roughly normal distribution. Seasonal data is not normal. It has spikes. Applying a global threshold to a time series with known seasonality is like using a smoke alarm to detect rain.

Fix: Add seasonality awareness by comparing against the same period last year, or by using the median absolute deviation (MAD) instead of z-score. MAD is more robust against spikes.

```
class RobustAnomalyDetector(AnomalyDetector):  
    """Anomaly detector using median absolute deviation for robust  
    outlier detection."""  
  
    def _mad_outliers(self, series: pd.Series) -> int:  
        """Return the number of outliers using the MAD method."""  
        median = series.median()  
        mad = np.median(np.abs(series - median))  
        if mad == 0:  
            return 0  
        modified_z = 0.6745 * (series - median) / mad  
        return (np.abs(modified_z) > self.z_threshold).sum()
```

```

def run(self, df: pd.DataFrame) -> List[Issue]:
    """Override numeric checks to use MAD instead of standard
    z-score."""
    issues: List[Issue] = []
    for col in df.columns:
        series = df[col].dropna()
        if series.dtype.kind in ("i", "u", "f") and
len(series) > 1:
            outliers = self._mad_outliers(series)
            if outliers > 0:
                issues.append(Issue(
                    severity="warning",
                    category="anomaly",
                    column=col,
                    message=f"Robust MAD detected {outliers}
outlier(s) in '{col}'.",
                    suggestion="Review spikes against
historical same-period baselines.",
                ))
    return issues

```

Better yet, add a `seasonality_column` argument so the detector groups by month before computing local statistics.

Break 2: Schema Too Strict, Rejecting a Valid New Column

Your upstream source adds a `tax_amount` column to the CSV. The Schema Validator flags it as unexpected and throws a warning. Your CI pipeline is configured to treat any warning as a hard failure. The perfectly valid new column blocks the entire ingest.

Why it fails: The schema was defined as a closed world. Anything not explicitly required or optional is treated as suspicious. In real data ecosystems, schemas evolve continuously. A closed-world schema is brittle.

Fix: Change the severity of unexpected columns from warning to info, and add a grace period mechanism.

```

class FlexibleSchemaValidator(SchemaValidator):
    """Schema validator with a grace period for new columns."""

    def __init__(self, known_columns: List[str], grace_columns:
Optional[List[str]] = None):
        super().__init__(required_columns=known_columns[:3],
optional_columns=known_columns[3:])
        self.known_columns = set(known_columns)
        self.grace_columns = set(grace_columns or [])

```

```

def run(self, df: pd.DataFrame) -> List[Issue]:
    """Flag unknown columns, but downgrade severity if they
are in the grace list."""
    base_issues = super().run(df)
    adjusted = []
    for issue in base_issues:
        if "Unexpected column" in issue.message and
issue.column in self.grace_columns:
            adjusted.append(Issue(
                severity="info",
                category="schema",
                column=issue.column,
                message=issue.message,
                suggestion="Column is known but not yet
required. Will be promoted soon.",
            ))
        else:
            adjusted.append(issue)
    return adjusted

```

This teaches the pipeline that some new columns are expected migrations, not anomalies.

Break 3: Remediation Suggestion Breaks a Downstream Pipeline

The Remediation Agent suggests dropping the unexpected `tax_amount` column to maintain the downstream contract. Your analytics pipeline later expects `tax_amount` because someone updated the warehouse schema last week but forgot to update the Data Quality Team configuration. The pipeline crashes with a `KeyError`.

Why it fails: The Remediation Agent is operating on stale knowledge. It thinks the contract is the old schema, but the real contract is the code that consumes the data. Data quality logic and downstream code are not versioned together.

Fix: Version the schema in a shared file and have the downstream pipeline read the same source of truth. Also, never auto-apply destructive remediation. Instead, generate a diff preview.

```

def preview_drop(df: pd.DataFrame, columns: List[str]) -> str:
    """Return a human-readable preview of what would be
dropped."""
    would_drop = [c for c in columns if c in df.columns]
    return f"Would drop columns: {would_drop} (rows before:
{len(df)}, after: {len(df)})"

```

Require a human or a second CI gate to approve any column removal.

Break 4: Missing Data Pattern Not Detected

Nulls in the region column are not random. They appear for exactly one country code: CA. The Anomaly Detector calculates a global null rate of four percent, which is below the five percent threshold. It reports nothing. But every Canadian record is missing the region because the upstream vendor changed their API response for Canada last month. The failure is systematic, not random, and it is invisible to a global threshold.

Why it fails: Global null-rate checks are blind to concentrated missingness. A column that is ninety percent null in one segment can look healthy when averaged across the whole dataset.

Fix: Add a segment-level null check.

```
def segment_null_check(
    df: pd.DataFrame,
    segment_col: str,
    target_col: str,
    threshold: float = 0.5,
) -> List[Issue]:
    """Check if any segment has a null rate above the threshold in
    the target column."""
    issues: List[Issue] = []
    for segment, group in df.groupby(segment_col):
        rate = group[target_col].isna().mean()
        if rate > threshold:
            issues.append(Issue(
                severity="error",
                category="anomaly",
                column=target_col,
                message=f"Segment '{segment}' has {rate:.1%} nulls
in '{target_col}'.",
                suggestion="Alert the upstream source for this
segment specifically.",
            ))
    return issues
```

Run this in addition to the global check. Systematic missingness is usually a bug, not noise.

Break 5: Two Agents Disagree and the Report Is Contradictory

The Schema Validator says quantity is fine because it is numeric and within range. The Anomaly Detector says quantity has a critical range violation because one row contains 20000. If you sort the report by severity, both show up, and a human reader might think the column is both fine and broken at the same time.

Why it fails: The agents are producing disconnected issue streams with overlapping column names. The report is a flat list, not a per-column summary, so the reader must mentally merge findings.

Fix: Add an aggregation step that groups issues by column and computes a column-level health score.

```
def aggregate_by_column(issues: List[Issue]) -> Dict[str,
Dict[str, Any]]:
    """Group issues by column and return a summary with highest
    severity."""
    groups: Dict[str, List[Issue]] = {}
    for i in issues:
        groups.setdefault(i.column, []).append(i)

    result = {}
    severity_rank = {"critical": 4, "error": 3, "warning": 2,
"info": 1}
    for col, col_issues in groups.items():
        top = max(col_issues, key=lambda i:
severity_rank.get(i.severity, 0))
        result[col] = {
            "highest_severity": top.severity,
            "count": len(col_issues),
            "messages": [i.message for i in col_issues],
        }
    return result
```

Add this to the markdown report as a per-column status table. Humans read tables faster than flat lists.

Try It Now

Variation 1: Daily Delta Monitoring

Instead of checking a file in isolation, compare today's snapshot to yesterday's. Write a `DeltaMonitor` class that loads both `DataFrames`, computes the difference in row count and column-level mean, and raises an issue if any numeric column's mean shifts by more than ten percent.

```
class DeltaMonitor:
    """Compare two DataFrames and flag significant drift in
    distributions."""

    def compare(self, df_old: pd.DataFrame, df_new: pd.DataFrame)
    -> List[Issue]:
        """Return issues when key statistics drift beyond
        tolerances."""
        issues: List[Issue] = []
        row_diff = len(df_new) - len(df_old)
        if abs(row_diff) > 0.2 * len(df_old):
            issues.append(Issue(
                severity="warning",
                category="anomaly",
                column="*",
                message=f"Row count changed by {row_diff}
({row_diff/len(df_old):.1%}).",
                suggestion="Verify upstream extract completed
successfully.",
            ))
        for col in df_new.columns:
            if col in df_old.columns and df_new[col].dtype.kind in
("i", "u", "f"):
                old_mean = df_old[col].mean()
                new_mean = df_new[col].mean()
                if old_mean and abs(new_mean - old_mean) /
abs(old_mean) > 0.10:
                    issues.append(Issue(
                        severity="error",
                        category="anomaly",
                        column=col,
                        message=f"Mean of '{col}' shifted from
{old_mean:.2f} to {new_mean:.2f}.",
                        suggestion="Investigate upstream change or
data pipeline bug.",
                    ))
        return issues
```

Run this before the Schema and Anomaly checks, so drift is caught before schema mismatches even matter.

Variation 2: Auto-Generate SQL Fix Scripts

Extend the Remediation Agent to emit SQL instead of pandas code. If the issue is a range violation, generate an UPDATE statement that clamps values. If the issue is missing required data, generate an INSERT alert or a constraint ALTER statement.

```
def sql_remediation(issue: Issue, table_name: str = "sales") -> str:
    """Return a SQL statement that addresses the given issue, if
    safe."""
    if issue.category == "remediation" and "Clamp" in
issue.message:
        return (
            f"-- Review before running\n"
            f"UPDATE {table_name} SET {issue.column} =
GREATEST({issue.column}, 0) "
            f"WHERE {issue.column} < 0;"
        )
    return "-- No automated SQL fix available for this issue."
```

Store these SQL snippets alongside the JSON report so a DBA can review and run them in a transaction.

Variation 3: Hourly Folder Monitoring

Use Python's watchdog library to monitor a folder and run the Data Quality Team on every new CSV. Log summaries to a running JSONL file.

```
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
import os

class QualityWatcher(FileSystemEventHandler):
    """Run the quality team on every new CSV dropped into a
    folder."""

    def __init__(self, team_fn):
        self.team_fn = team_fn

    def on_created(self, event):
        if event.is_directory:
            return
        if event.src_path.endswith(".csv"):
            df = pd.read_csv(event.src_path)
```

```

        report = self.team_fn(df)
        with open("quality_log.jsonl", "a", encoding="utf-8")
as f:
            f.write(json.dumps({"file": event.src_path,
"summary": report["summary"]}) + "\n")

# In main:
# observer = Observer()
# observer.schedule(QualityWatcher(run_quality_team), path="./
inbox", recursive=False)
# observer.start()

```

This turns the Data Quality Team from a script into a service. Drop a file in the inbox; get a report in the outbox.

The OpenClaw Way

OpenClaw handles multi-agent pipelines through its Task and Orchestrator abstractions. You define three Agent objects — Schema, Anomaly, Remediation — each with a deterministic tool function rather than an LLM call, because OpenClaw does not require every agent to be language-model backed. The Orchestrator declaration specifies that Schema and Anomaly run in parallel, feeding a shared state dictionary. The Remediation agent subscribes to that shared state and triggers only after both discovery agents complete. OpenClaw also provides a StructuredOutput helper that validates the report against a Pydantic model, ensuring the JSON shape never drifts. For cost tracking, OpenClaw logs the token usage of any LLM-based agent (if you choose to swap the deterministic validators for LLM-powered ones later), but in this configuration the cost is zero since the agents are pure Python tools.

The Hermes Agent Way

Hermes Agent's crew mode supports data workflows through tool-based agents. You register three sub-agents with custom toolsets: the Schema agent gets a `validate_columns` tool, the Anomaly agent gets a `describe_stats` tool, and the Remediation agent gets a `suggest_fix` tool. Each tool is a Python function that runs in Hermes's sandboxed environment, so it can import pandas directly. You then run `/crew analyze sales.csv`, and Hermes dispatches the file to the first two agents in parallel, buffers their JSON outputs in a thread-safe workspace, and hands the combined issue list to the Remediation agent. The final report is emitted as a markdown artifact that Hermes can display

inline or write to a file. The advantage is that you do not write the orchestration loop yourself. The constraint is that your agents must live inside Hermes's runtime, which means you are not running a standalone script you can easily cron on a server without the Hermes daemon.

What You Built

In this chapter, you built a complete Data Quality Team from fundamentals:

- A shared Issue dataclass with severity, category, column, message, and suggestion fields.
- A DataQualityAgent base class that enforces a consistent run(df) interface across all specialists.
- A SchemaValidator that checks required columns, unexpected columns, and coarse type expectations.
- An AnomalyDetector that calculates null rates, unique ratios, z-score/MAD outliers, and configurable range violations.
- A RemediationAgent that maps findings to concrete pandas transformation suggestions or rejection directives.
- A pipeline orchestrator that sorts issues by severity and generates both JSON and markdown reports from the same source of truth.
- Break-and-fix experience with seasonal false positives, evolving schemas, destructive remediation, systematic missingness, and contradictory reports.
- Extension patterns for delta monitoring, SQL script generation, and real-time folder watching.

You also learned a structural lesson: data quality is not a one-time cleaning step. It is a continuous detection layer that runs before every downstream consumer, and its output should always be structured enough for a machine to act on and readable enough for a human to review.

Cost for this chapter: The pure Python pipeline using pandas and numpy costs \$0.00 to run. If you add an LLM-based agent for natural-language remediation summaries via OpenClaw or Hermes, expect roughly \$0.001–\$0.005 per file with GPT-4o mini, depending on the size

of the generated report. At one thousand files per day, that is \$1–\$5 daily. Keep the core detection deterministic; use language models only for the parts humans actually need to read.

Chapter 18: Project 18 — Workflow Orchestrator

From research to publication: five agents, three decisions, one workflow. You set the rules.

A few years ago, I managed a weekly research newsletter for a small team. Every Monday morning, the process looked like this: I’d spend an hour reading two industry blogs, synthesize the findings into bullet points, draft a summary, send it to my editor for review, wait for feedback, revise, and finally paste the result into our publishing platform. If the editor rejected a draft, I’d loop back and rewrite. If a source was down, I’d scramble for a replacement. If I was sick on a Monday, the newsletter simply didn’t go out.

The worst part wasn’t the time. It was the brittleness. Every step depended on the previous one finishing successfully. There was no checkpointing: if my laptop crashed during revision, I started over. There was no parallelism: I read the blogs sequentially even though they had nothing to do with each other. There was no retry logic: one flaky website could kill the entire morning. And there was no audit trail: when someone asked, “Why did we publish that claim?” I had to reconstruct the chain from memory and browser history.

That weekly newsletter was a workflow. A bad one. Manual, fragile, and opaque. In this chapter, we’re going to replace it with a directed acyclic graph of Python functions that runs itself, retries on failure, branches on conditions, saves its state after every step, and logs everything it does so you can debug it later without guessing.

You already know how to build a single agent that observes, reasons, and acts. Now you’re going to learn how to chain those agents — and plain old functions — into a system that behaves like a factory line. Each station has inputs, outputs, and rules about what comes next. Some stations run simultaneously. Some skip ahead. Some loop back until a human says yes. This is the orchestrator, and it’s the capstone of everything you’ve built in Part III.

Beyond Single Agents: Directed Workflows

A single agent is like a solo musician: talented, improvisational, and limited by the number of hands it has. A workflow is an orchestra: each musician plays their part, the conductor decides the order, and the sheet music — the DAG — prevents chaos.

In software terms, a workflow is a directed acyclic graph where each node is a step and each edge is a dependency. If step A produces data that step B needs, there is an edge from A to B. If C and D have no relationship, they can run at the same time. If E depends on both C and D finishing, E waits until the last of them completes. If F needs approval from a human before it runs, it blocks until someone clicks a button. This structure is not exotic. It's how data pipelines, CI/CD systems, and scientific simulations have worked for decades. We're just applying the same architecture to agent tasks.

The key insight is that not every problem needs an agent. Sometimes a problem needs a Python function that scrapes a feed. Sometimes it needs an LLM to summarize that feed. Sometimes it needs a human to approve the summary. The orchestrator doesn't care which is which. It cares about the contract: what does this step need, what does it produce, and what should happen next?

Let's build one from scratch.

The Shape of the Workflow

Before we write code, here is the workflow we're going to implement:

1. **Start:** set a topic.
2. **Research A:** scrape blog A for articles on that topic.
3. **Research B:** scrape blog B for articles on the same topic.
4. **Merge:** combine the two research outputs into one document.
5. **Draft:** generate a newsletter draft from the merged research.
6. **Review:** a simple automated check (e.g., word count and disallowed words).
7. **Approval Gate:** conditionally branch. If approved, publish. If rejected, revise and return to Draft.

8. **Publish:** write the final file to disk.

9. **End.**

Steps 2 and 3 are independent, so they run in parallel. Step 4 waits for both. Step 7 is a conditional loop that can send us back to step 5. Everything else is sequential.

Here is the DAG as a Mermaid diagram:

```
graph TD
    Start[Start: set topic] --> ResearchA[Research A]
    Start --> ResearchB[Research B]
    ResearchA --> Merge[Mergedata]
    ResearchB --> Merge
    Merge --> Draft[Draft newsletter]
    Draft --> Review[Review draft]
    Review --> Gate{Approved?}
    Gate -- yes --> Publish[Publish]
    Gate -- no --> Revise[Revise]
    Revise --> Draft
    Publish --> End[End]
```

We'll represent this entirely in Python. No YAML, no JSON configuration files — just classes and functions, so you can trace every line of execution.

Building the Engine

The Step Contract

A step is not just a function. It is a node in a graph with six properties that the orchestrator uses to schedule, execute, and recover work:

- **name:** a unique identifier.
- **func:** the Python callable to execute.
- **inputs:** a list of keys the step reads from shared state.
- **outputs:** a list of keys the step writes into shared state.
- **depends_on:** a list of step names that must finish before this one starts.
- **retries:** how many times to retry if func raises an exception.

- **condition:** an optional callable that returns True if this step should run, or False to skip it.

We'll model this with a dataclass, because dataclasses are clean, self-documenting, and trivial to extend later.

Create a new file called `orchestrator.py`:

```
"""
Workflow Orchestrator – Project 18
A pure-Python DAG executor with retries, conditionals, parallel
branches,
and JSON state persistence.
"""

import json
import logging
import os
import time
import functools
import concurrent.futures
from dataclasses import dataclass, field
from typing import Callable, Any, Optional

logger = logging.getLogger("orchestrator")

def retry_with_backoff(max_retries: int = 2, base_delay: float =
1.0):
    """Decorator that retries a function with exponential backoff.

    Args:
        max_retries: Maximum number of retry attempts after the
first failure.
        base_delay: Initial seconds to wait before the first
retry.
    """
    def decorator(func: Callable[..., Any]) -> Callable[..., Any]:
        @functools.wraps(func)
        def wrapper(*args: Any, **kwargs: Any) -> Any:
            last_exc: Optional[Exception] = None
            for attempt in range(max_retries + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as exc:
                    last_exc = exc
                    if attempt < max_retries:
                        delay = base_delay * (2 ** attempt)
                        logger.warning(
                            "%s failed (attempt %d). Retrying in
%.1fs...",
```

```

        func.__name__, attempt + 1, delay,
    )
    time.sleep(delay)
    # All retries exhausted
    raise last_exc # type: ignore[misc]
    return wrapper
return decorator

@dataclass
class Step:
    """A single node in the workflow DAG.

    Attributes:
        name: Unique identifier used for scheduling and state
keys.
        func: Callable to execute. Receives a dict of available
inputs.
        inputs: List of state keys this step requires.
        outputs: List of state keys this step produces.
        depends_on: Step names that must complete before this one
runs.
        retries: Number of automatic retry attempts on failure.
        condition: Optional callable (state) -> bool. If False,
step is skipped.
    """
    name: str
    func: Callable[[dict[str, Any]], dict[str, Any]]
    inputs: list[str] = field(default_factory=list)
    outputs: list[str] = field(default_factory=list)
    depends_on: list[str] = field(default_factory=list)
    retries: int = 0
    condition: Optional[Callable[[dict[str, Any]], bool]] = None

```

Notice the docstrings. I treat them as contracts because other developers — and future you — will read them before reading the function body. The `retry_with_backoff` decorator wraps any step function with exponential sleep. A first retry waits one second, the second waits two seconds, the third waits four, and so on. This matters when you're calling rate-limited APIs.

The Workflow Executor

The `Workflow` class is where the magic lives. It needs to:

1. Accept a list of Steps and a starting state dictionary.
2. Build a dependency graph.
3. Run steps in topological order.

4. Execute independent steps in parallel.
5. Evaluate conditions before running a step.
6. Retry failed steps up to their configured limit.
7. Save state to a JSON file after every step so crashes are recoverable.
8. Log every decision so you can replay the run later.

Here is the full Workflow class:

```
class Workflow:
    """DAG-based workflow executor with checkpointing and
    observability.

    Attributes:
        name: Human-readable workflow identifier.
        steps: Sequence of Step definitions.
        state_path: Filepath where state is persisted after each
        step.
    """

    def __init__(
        self,
        name: str,
        steps: list[Step],
        state_path: str = "workflow_state.json",
    ) -> None:
        self.name = name
        self.steps = {s.name: s for s in steps}
        self.state_path = state_path
        self._results: dict[str, Any] = {}
        self._completed: set[str] = set()

    def _save_checkpoint(self) -> None:
        """Persist current state and completed list to JSON."""
        payload = {
            "workflow": self.name,
            "results": self._results,
            "completed": list(self._completed),
        }
        with open(self.state_path, "w", encoding="utf-8") as fh:
            json.dump(payload, fh, indent=2, default=str)
        logger.info("Checkpoint saved to %s", self.state_path)

    def _load_checkpoint(self) -> bool:
        """Restore state from a previous checkpoint if one
        exists."""
        if not os.path.exists(self.state_path):
            return False
        with open(self.state_path, "r", encoding="utf-8") as fh:
```

```

        payload = json.load(fh)
        if payload.get("workflow") != self.name:
            logger.warning("Checkpoint file belongs to a different
workflow. Ignoring.")
            return False
        self._results = payload.get("results", {})
        self._completed = set(payload.get("completed", []))
        logger.info(
            "Resumed from checkpoint (%d completed steps).",
            len(self._completed),
        )
        return True

    def _ready_steps(self) -> list[Step]:
        """Return steps whose dependencies are all satisfied,
that have not yet run, and whose condition (if any) is
True."""
        ready: list[Step] = []
        for step in self.steps.values():
            if step.name in self._completed:
                continue
            deps_ok = all(d in self._completed for d in
step.depends_on)
            if not deps_ok:
                continue
            if step.condition is not None and not
step.condition(self._results):
                logger.info("Step '%s' skipped (condition returned
False).", step.name)
                self._completed.add(step.name)
                continue
            ready.append(step)
        return ready

    def _run_step(self, step: Step) -> None:
        """Execute a single step with retries and state
bookkeeping."""
        if step.name in self._completed:
            logger.info("Step '%s' already completed
(idempotent).", step.name)
            return

        logger.info("Running step '%s'...", step.name)
        missing = [k for k in step.inputs if k not in
self._results]
        if missing:
            raise KeyError(
                f"Step '{step.name}' missing required inputs:
{missing}"
            )

        inputs = {k: self._results[k] for k in step.inputs}

```

```

        @retry_with_backoff(max_retries=min(step.retries, 5),
base_delay=1.0)
        def _execute() -> dict[str, Any]:
            return step.func(inputs)

        result = _execute()
        if not isinstance(result, dict):
            raise TypeError(
                f"Step '{step.name}' returned
{type(result).__name__}, expected dict"
            )

        unexpected = [k for k in result if k not in step.outputs]
        if unexpected:
            logger.warning(
                "Step '%s' returned unexpected keys: %s",
                step.name, unexpected,
            )

        self._results.update({k: result[k] for k in step.outputs
if k in result})
        self._completed.add(step.name)
        self._save_checkpoint()
        logger.info("Step '%s' finished successfully.", step.name)

    def execute(self, initial_state: dict[str, Any] | None = None)
-> dict[str, Any]:
        """Run the full workflow until all reachable steps
complete.

Returns:
    The final shared state dictionary.
    """
        self._results = initial_state.copy() if initial_state else
{}
        self._load_checkpoint()

        logger.info("Starting workflow '%s'...", self.name)
        with concurrent.futures.ThreadPoolExecutor(max_workers=4)
as pool:
            while True:
                ready = self._ready_steps()
                if not ready:
                    break
                # Submit independent steps in parallel
                futures = {
                    pool.submit(self._run_step, step): step
                    for step in ready
                }
                for future in
concurrent.futures.as_completed(futures):

```

```

        step = futures[future]
        try:
            future.result()
        except Exception:
            logger.exception("Step '%s' failed.",
                             step.name)
            raise
        logger.info("Workflow '%s' completed (%d/%d steps).",
                    self.name,
                    len(self._completed), len(self.steps))
        return self._results

```

Let's walk through the critical parts.

Dependency Graph and Ready Steps

The orchestrator does not build an explicit graph object. It uses a simple set intersection: a step is ready when all of its `depends_on` names exist in `self._completed`. That is a topological sort in disguise. Because the loop recalculates `_ready_steps()` after every batch of parallel execution finishes, it naturally handles branching and diamond shapes without special-case code.

Parallel Execution

The `ThreadPoolExecutor` submits every step in the current ready batch simultaneously. If the ready batch has only one step (for example, a sequential tail after a merge), only one thread runs. If the ready batch has two (Research A and Research B), both run at once. The `max_workers=4` cap prevents your machine from drowning in threads, but for the workloads in this book, you rarely hit the limit.

Retry Logic

Each step is wrapped in the `@retry_with_backoff` decorator at execution time. The decorator catches any exception, sleeps with exponential delay, and tries again. If retries are exhausted, the exception propagates out of `_run_step`, which kills the workflow. We'll talk about how to handle that gracefully in Break and Fix.

State Persistence

After a step succeeds, its outputs are merged into `self._results`, the completed set is updated, and `_save_checkpoint()` writes a JSON file. If your laptop crashes after step 3 of 7, you can run the script again and it

will pick up where it left off. The `_load_checkpoint()` method reads that file, validates it belongs to the current workflow name, and restores state. This is not industrial-grade persistence — there is no journaling, no atomic rename — but it is honest, inspectable, and enough to survive a power outage.

Observability

Every interesting event is logged at INFO or WARNING level. You can turn on debug logging with one line:

```
logging.basicConfig(level=logging.DEBUG)
```

Because the logger uses the standard library, you can also redirect it to a file, syslog, or a structured formatter without changing the orchestrator code.

Putting It Together: The Newsletter Pipeline

Now that we have the engine, let's use it. We need mock step functions that simulate real work — scraping, drafting, reviewing — so the workflow is runnable without external dependencies.

Add these functions below the Workflow class in `orchestrator.py`:

```
def step_research_a(inputs: dict) -> dict:
    """Simulate research on blog A."""
    topic = inputs["topic"]
    logger.info("Scraping blog A for '%s'...", topic)
    return {"research_a": f"Blog A says: {topic} is trending in Q2."}
```

```
def step_research_b(inputs: dict) -> dict:
    """Simulate research on blog B."""
    topic = inputs["topic"]
    logger.info("Scraping blog B for '%s'...", topic)
    return {"research_b": f"Blog B forecasts a 12% growth in {topic}."}
```

```
def step_merge(inputs: dict) -> dict:
    """Combine two research streams into one document."""
    merged = f"{inputs['research_a']}\n{inputs['research_b']}"
    return {"merged_doc": merged}
```

```

def step_draft(inputs: dict) -> dict:
    """Generate a newsletter draft from merged research."""
    doc = inputs["merged_doc"]
    draft = f"Weekly Briefing\n\n{doc}\n\nStay tuned!"
    return {"draft": draft, "word_count": len(draft.split())}

def step_review(inputs: dict) -> dict:
    """Simple automated review: reject if too short or contains
    forbidden words."""
    draft = inputs["draft"]
    forbidden = {"guarantee", "click here", "unsubscribe"}
    words = set(draft.lower().split())
    violations = words & forbidden
    approved = inputs.get("word_count", 0) > 10 and not violations
    return {
        "approved": approved,
        "review_notes": f"Violations: {violations}" if violations
    }
    else "Looks good.",
}

def step_revise(inputs: dict) -> dict:
    """Bump the draft back into a richer state so Draft can pick
    it up again."""
    logger.info("Revising draft based on notes: %s",
    inputs.get("review_notes"))
    # In a real pipeline, this could call an LLM to rewrite.
    return {"draft_enhanced": inputs["draft"] + "\n[Revised
    edition]"}

```

```

def step_publish(inputs: dict) -> dict:
    """Write the final draft to disk and record the artifact."""
    draft = inputs["draft"]
    path = "newsletter_final.md"
    with open(path, "w", encoding="utf-8") as fh:
        fh.write(draft)
    logger.info("Published to %s", path)
    return {"published_path": path}

```

Finally, wire them into a Workflow and run it:

```

def main():
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s [%(levelname)s] %(message)s",
        datefmt="%H:%M:%S",
    )

    steps = [
        Step(

```



```

        name="research_a",
        func=step_research_a,
        inputs=["topic"],
        outputs=["research_a"],
    ),
    Step(
        name="research_b",
        func=step_research_b,
        inputs=["topic"],
        outputs=["research_b"],
    ),
    Step(
        name="merge",
        func=step_merge,
        inputs=["research_a", "research_b"],
        outputs=["merged_doc"],
        depends_on=["research_a", "research_b"],
    ),
    Step(
        name="draft",
        func=step_draft,
        inputs=["merged_doc"],
        outputs=["draft", "word_count"],
        depends_on=["merge"],
    ),
    Step(
        name="review",
        func=step_review,
        inputs=["draft", "word_count"],
        outputs=["approved", "review_notes"],
        depends_on=["draft"],
    ),
    Step(
        name="revise",
        func=step_revise,
        inputs=["draft", "review_notes"],
        outputs=["draft_enhanced"],
        depends_on=["review"],
        condition=lambda state: state.get("approved") is
False,
    ),
    Step(
        name="draft_v2",
        func=lambda inputs: {"draft":
inputs["draft_enhanced"], "word_count": 999},
        inputs=["draft_enhanced"],
        outputs=["draft", "word_count"],
        depends_on=["revise"],
        condition=lambda state: state.get("approved") is
False,
    ),
    Step(

```

```

        name="publish",
        func=step_publish,
        inputs=["draft"],
        outputs=["published_path"],
        depends_on=["review"],
        condition=lambda state: state.get("approved") is True,
    ),
]

wf = Workflow(
    name="newsletter_pipeline",
    steps=steps,
    state_path="newsletter_state.json",
)

final_state = wf.execute(initial_state={"topic": "agent
orchestration"})
print("\nFinal state keys:", sorted(final_state.keys()))
print("Published to:", final_state.get("published_path"))

if __name__ == "__main__":
    main()

```

Run it:

```
python orchestrator.py
```

You should see something like:

```

09:14:22 [INFO] Starting workflow 'newsletter_pipeline'...
09:14:22 [INFO] Running step 'research_a'...
09:14:22 [INFO] Scraping blog A for 'agent orchestration'...
09:14:22 [INFO] Running step 'research_b'...
09:14:22 [INFO] Scraping blog B for 'agent orchestration'...
09:14:22 [INFO] Checkpoint saved to newsletter_state.json
09:14:22 [INFO] Step 'research_a' finished successfully.
09:14:22 [INFO] Checkpoint saved to newsletter_state.json
09:14:22 [INFO] Step 'research_b' finished successfully.
09:14:22 [INFO] Running step 'merge'...
09:14:22 [INFO] Checkpoint saved to newsletter_state.json
09:14:22 [INFO] Step 'merge' finished successfully.
09:14:22 [INFO] Running step 'draft'...
09:14:22 [INFO] Checkpoint saved to newsletter_state.json
09:14:22 [INFO] Step 'draft' finished successfully.
09:14:22 [INFO] Running step 'review'...
09:14:22 [INFO] Checkpoint saved to newsletter_state.json
09:14:22 [INFO] Step 'review' finished successfully.
09:14:22 [INFO] Step 'revise' skipped (condition returned False).
09:14:22 [INFO] Running step 'publish'...
09:14:22 [INFO] Published to newsletter_final.md
09:14:22 [INFO] Checkpoint saved to newsletter_state.json

```

```
09:14:22 [INFO] Step 'publish' finished successfully.  
09:14:22 [INFO] Workflow 'newsletter_pipeline' completed (6/8  
steps).
```

```
Final state keys: ['approved', 'draft', 'merged_doc',  
'published_path', ...]  
Published to: newsletter_final.md
```

Notice the two `research_steps` interleaved — they really do run in parallel. After review succeeds, the condition on `publish` evaluates to `True` and the condition on `revise` evaluates to `False`, so `publish` runs while `revise` and `draft_v2` are skipped. The workflow completed six of eight steps because two were bypassed. If you open `newsletter_state.json`, you'll see every intermediate result serialized, including the `review_notes`.

Break and Fix: Five Ways Workflows Fail

Workflows fail differently from single agents. The failure is usually structural, not linguistic. Here are the five most common catastrophes.

Break 1: Circular Dependency

You define step A with `depends_on=["B"]` and step B with `depends_on=["A"]`.

Symptom: The workflow starts, prints nothing, and hangs forever with zero CPU usage.

Why: The `_ready_steps()` loop returns an empty ready set because both steps are waiting on each other. The `while True` loop spins without finding anything to do.

Fix: Detect cycles before execution with a simple depth-first search, or at minimum, log a warning when the ready set is empty but uncompleted steps remain:

```
if not ready:  
    remaining = set(self.steps) - self._completed  
    if remaining:  
        raise RuntimeError(  
            f"Deadlock detected. Remaining steps: {remaining}"  
        )  
    break
```

Add that inside `execute()` right after `ready = self._ready_steps()`. Now the orchestrator raises a clear error instead of silently hanging.

Break 2: Step Fails After Max Retries

You set `retries=2` on a step that calls a flaky third-party API. The API is down for maintenance.

Symptom: The step retries three times with escalating backoff, then throws an exception that crashes the entire workflow.

Why: The retry decorator re-raises the last exception when retries are exhausted. `_run_step` does not catch it. The `future.result()` call in `execute()` sees the exception and raises it out of the thread pool.

Fix: Decide on a failure policy. For non-critical steps, you can catch the exception inside `func` and return a fallback:

```
def step_research_a(inputs: dict) -> dict:
    try:
        # ... real scrape ...
        return {"research_a": data}
    except Exception:
        logger.exception("Research A failed; using placeholder.")
        return {"research_a": "Research A unavailable."}
```

For critical steps, let the workflow die but wrap `execute()` at the top level so you can inspect the checkpoint file before restarting.

Break 3: Missing Input from Previous Step

Step `draft` expects an input called `merged_doc`, but step `merge` accidentally writes `"merged"` instead.

Symptom: `KeyError: "Step 'draft' missing required inputs: ['merged_doc']"`.

Why: The orchestrator validates inputs before calling `func`. If the key is not in shared state, it fails fast instead of passing an empty dictionary to the step.

Fix: Use constants or dataclasses for state keys so typos are caught by your linter. For small scripts, this is overkill, but rename `merged` to `merged_doc` and the error vanishes. The validation itself is a feature — it surfaces contract violations immediately rather than letting them propagate as silent Nones.

Break 4: Dead Branch (Conditional Never Triggers)

You want publish to run only when approved is True, but you write the condition as `lambda state: state.get("approved") == True`. The review step returns `"approved": True`. The condition never fires.

Symptom: publish is skipped. The workflow ends after review with no explanation.

Why: Actually, `True == True` evaluates to `True`, so this exact lambda works. The real dead branch happens when you use a key that never gets written, like `state.get("approved") == "yes"` when the actual value is `True`. Or worse, when approved is 0 and you check `state.get("approved")` in a boolean context.

Fix: Make conditions explicit, log them, and write unit tests for each branch. In our `Workflow._ready_steps()`, we already log when a condition skips a step. Read that log line. If publish was skipped, check the value of approved in `newsletter_state.json`. The fix is usually a type mismatch: change "yes" to `True`, or cast with `bool()`, or restructure the review step to return predictable types.

Break 5: State Checkpoint Corruption

You kill the process while `_save_checkpoint()` is mid-write. The resulting `workflow_state.json` is half-written and unparseable.

Symptom: On restart, `_load_checkpoint()` throws `json.decoder.JSONDecodeError` and the workflow aborts.

Why: We write directly to the output file. There is no atomic rename. A crash during the write leaves garbage on disk.

Fix: Write to a temporary file, then rename:

```
import tempfile

def _save_checkpoint(self) -> None:
    payload = {
        "workflow": self.name,
        "results": self._results,
        "completed": list(self._completed),
    }
    temp_path = self.state_path + ".tmp"
    with open(temp_path, "w", encoding="utf-8") as fh:
        json.dump(payload, fh, indent=2, default=str)
```

```
os.replace(temp_path, self.state_path)
logger.info("Atomic checkpoint saved.")
```

This is the classic write-and-rename pattern. It guarantees that any reader of the file sees either the old valid state or the new valid state, never a partial write.

Try It Now: Three Extensions

Variation 1: Parallel Branches for Independent Research

Add a third research step, `research_c`, that depends only on `topic` and runs in parallel with `A` and `B`. Update `merge` to consume all three inputs. This tests whether the orchestrator scales to wider DAGs without code changes. The answer is yes — just add the step and update `merge.inputs` — because the dependency resolution is dynamic.

Variation 2: Human Approval Gate at Publish Step

Replace `step_publish` with a function that prompts the user on the command line:

```
def step_human_publish(inputs: dict) -> dict:
    print("\n--- FINAL DRAFT ---")
    print(inputs["draft"])
    print("-----")
    answer = input("Approve publication? [y/N]: ").strip().lower()
    if answer != "y":
        raise RuntimeError("Publication rejected by human operator.")
    return step_publish(inputs)
```

Then swap it into the workflow. Now the pipeline pauses for human judgment. If the user rejects, the exception triggers the retry logic; if you set `retries=0`, it fails fast. For production use, replace `input()` with a web hook, a Slack button, or a file-based semaphore.

Variation 3: Time-Based Trigger (Run Every Monday Morning via Cron)

Our orchestrator is a Python script. To run it on a schedule, wrap `main()` in a script that checks the day before executing:

```
import datetime
```

```

if __name__ == "__main__":
    if datetime.date.today().weekday() != 0: # Monday is 0
        print("Not Monday. Exiting.")
        raise SystemExit(0)
    main()

```

Then add a cron job:

```

crontab -e
# Add this line:
0 8 * * 1 cd /path/to/project && python orchestrator.py >>
newsletter.log 2>&1

```

At 8:00 AM every Monday, cron launches the workflow. Because of checkpointing, if the newsletter was already generated and approved earlier that day, the workflow resumes and exits immediately. Idempotency for free.

The OpenClaw Way

OpenClaw has a first-class workflow engine built on the same DAG concepts we just implemented by hand. In OpenClaw, you define a Workflow object using its declarative API, and the framework handles threading, checkpointing, retries, and even distributed execution if you supply a Redis backend.

Here is the newsletter pipeline translated:

```

from openclaw import Workflow, Step

wf = Workflow(name="newsletter_pipeline")

@wf.step(inputs=["topic"], outputs=["research_a"], retries=2)
def research_a(state):
    return {"research_a": f"Blog A on {state['topic']}"}

@wf.step(inputs=["topic"], outputs=["research_b"], retries=2)
def research_b(state):
    return {"research_b": f"Blog B on {state['topic']}"}

@wf.step(inputs=["research_a", "research_b"],
outputs=["merged_doc"])
def merge(state):
    return {"merged_doc": state["research_a"] + "\n" +
state["research_b"]}

# ... additional steps follow the same pattern ...

```

```
result = wf.run(initial_state={"topic": "agent orchestration"})
```

OpenClaw's `@wf.step` decorator registers the function as a node in the DAG and derives `depends_on` automatically from the `inputs` list by matching output keys to step names. It also stores checkpoints in SQLite by default, which is more robust than our JSON file but less human-readable. The trade-off is abstraction: you write less plumbing, but debugging requires understanding OpenClaw's internal scheduler. I recommend building the pure Python version first, then migrating to OpenClaw when your workflow grows beyond a dozen steps or needs to run across multiple machines.

The Hermes Agent Way

Hermes Agent does not expose an explicit workflow builder in the same way. Instead, it uses a **plan-and-execute** loop: you describe the workflow in natural language, and Hermes translates that into a sequence of tool calls, checkpoints, and conditional branches internally.

For example:

```
hermes
```

```
> Run a pipeline: research two blogs on "agent orchestration",
> merge the results, draft a newsletter, review it, and publish
> if it passes a style check. If not, revise once and try again.
```

```
[plan] 1. research blog A
[plan] 2. research blog B
[plan] 3. merge findings
[plan] 4. draft newsletter
[plan] 5. style review
[plan] 6. conditional publish or revise
[execute] Starting step 1...
```

Hermes generates the DAG on the fly, executes it, and maintains an internal state log that you can inspect with `/show_state`. The benefit is speed: you don't write boilerplate. The cost is opacity: if Hermes misorders two steps or misinterprets a conditional, you debug its plan rather than your own code. For teams that iterate quickly and prefer conversation-driven development, Hermes is unbeatable. For teams that need deterministic, version-controlled, auditable pipelines, the explicit DAG — whether OpenClaw or pure Python — is the safer bet.

What You Built

In this chapter, you built a workflow orchestrator from fundamentals:

- A Step dataclass that captures the full contract of a node: name, function, inputs, outputs, dependencies, retries, and conditions.
- A Workflow class that resolves dependencies dynamically, runs independent branches in parallel via a thread pool, retries failed steps with exponential backoff, and evaluates conditional logic at runtime.
- JSON state persistence after every step, enabling crash recovery and idempotent reruns.
- Structured logging of every scheduling decision, step execution, and checkpoint write.
- A concrete end-to-end pipeline — research, merge, draft, review, conditional publish — that demonstrates parallel execution, branching, and looping.

You also learned five failure modes that are unique to DAG execution: circular dependencies, exhausted retries, missing inputs, dead conditional branches, and corrupted checkpoints. Each came with a root-cause explanation and a fix.

This engine is not a toy. It is ~150 lines of pure Python, it runs without external services, and it scales to real workloads by adding more steps. When you outgrow it — when you need distributed execution, a web UI, or automatic rollback — you'll migrate to OpenClaw, Prefect, or Airflow with confidence, because you already know how the bones work.

Cost for this chapter: Approximately \$0.00. The entire pipeline runs locally with mock functions. If you replace `step_draft` with a call to an LLM API, expect roughly \$0.002–\$0.01 per draft depending on the model and output length.

Chapter 19: Project 19 — The AI Analyst: Financial Report Reader

Download a 100-page 10-K. Extract revenue, risks, and sentiment in ninety seconds. Then double-check every number by hand, because software lies beautifully.

A few years ago I decided to read a real 10-K cover to cover. I picked a well-known semiconductor company, brewed coffee, and settled in with a 137-page PDF. I found the revenue figures, skimmed the balance sheet, and felt smug about my diligence. Three weeks later the stock dropped eleven percent after earnings. I went back to the filing and found the risk disclosure I had missed on page 94: a single supplier concentration note that invalidated half my assumptions. I had scrolled past it because my eyes glazed over at page 70.

That is the problem with financial documents. They are long, dense, cross-referenced, and deliberately cautious. They bury explosive facts in footnotes and surround them with paragraphs of safe-harbor boilerplate. A human reader's attention span is the weakest link. An AI agent's attention span, on the other hand, is technically infinite—but only if you teach it to read carefully.

In this chapter you will build a financial report reader that ingests a PDF filing, extracts text and tables, prompts a language model for structured metrics with page-level citations, and compares results quarter over quarter. We are not building a trading bot. We are building a research assistant that never gets tired, never skips a footnote, and always tells you exactly where it found a number.

This is also the chapter where precision matters most. If an agent misreads a CSV in Project 4, you get a dirty dataset. If an agent misreads a 10-K, you make a bad decision with someone else's money—or your own. We will treat accuracy as a safety requirement, not a nice-to-have.

Why Financial Documents Are Hard

Before we write code, let's understand why this domain breaks naive text-processing pipelines.

Tables are structural, not narrative. Revenue does not appear in a sentence like “We made \$5 billion.” It appears in a condensed consolidated statement of operations where the header row spans three fiscal years and the footnote reference sits in a six-point superscript. A plain text extractor sees whitespace and line breaks, not rows and columns.

Footnotes change everything. A line that says “Accounts receivable: \$1.2 billion” is meaningless without the adjacent footnote explaining that \$400 million of it is tied to a single customer. If your extraction pipeline ignores footnotes, it ignores context. If your language model hallucinates the footnote, it manufactures context.

Forward-looking statements look like facts. A 10-K is full of sentences like “We expect revenue growth in the mid-teens.” That is not a fact; it is a projection wrapped in SEC-safe-harbor language. A naive extractor will log it as revenue guidance without flagging the uncertainty. A responsible analyst must separate “what happened” from “what management hopes will happen.”

Documents are mixed media. Some filings are born-digital text. Others are scanned images from the 1990s with coffee stains. A pipeline that expects selectable text will fail silently on the second type, returning empty strings and pretending the page is blank.

These are the problems we solve explicitly in the agent design.

Fetching the Filing: SEC EDGAR or Local File

The U.S. Securities and Exchange Commission runs EDGAR, a public database of filings. Every 10-K, 10-Q, and 8-K is there. Access is free and requires no API key, but the SEC insists on a custom User-Agent header with contact information. We will respect that.

EDGAR itself does not hand you a PDF in one click. You query a company’s submissions index, locate the filing you want, and fetch the primary document. For this chapter, we will sketch the full EDGAR flow, then fall back to a local PDF so you can run the code without waiting on network bureaucracy.

Create a file called `financial_reader.py` and start with the fetch logic:

```
import json
import os
from pathlib import Path
```

```

import httpx
import pandas as pd
import PyPDF2

OLLAMA_URL = "http://localhost:11434/api/chat"
MODEL = "llama3.1:8b"
OPENAI_API_KEY = os.environ.get("OPENAI_API_KEY", "")

SAVE_DIR = Path("./data")
SAVE_DIR.mkdir(exist_ok=True)

def fetch_edgar_filing(ticker: str, form: str = "10-K") -> Path:
    """Fetch the most recent filing from SEC EDGAR and return the
    local PDF path.

    SEC EDGAR requires a User-Agent with contact info. This
    function searches
        the company's submissions index for the latest filing of the
    requested
        form type, then downloads the primary document.
    """
    headers = {"User-Agent": "MyApp/1.0 (myemail@example.com)"}
    cik = ticker.zfill(10) # EDGAR uses zero-padded CIK
    index_url = (
        f"https://data.sec.gov/submissions/CIK{cik}.json"
    )

    try:
        r = httpx.get(index_url, headers=headers, timeout=30)
        r.raise_for_status()
    except httpx.HTTPStatusError as exc:
        raise RuntimeError(f"EDGAR index fetch failed: {exc}")
    from exc
    except httpx.TimeoutException as exc:
        raise RuntimeError("EDGAR index request timed out") from exc

    data = r.json()
    filings = data.get("filings", {}).get("recent", {})
    forms = filings.get("form", [])
    accession_numbers = filings.get("accessionNumber", [])
    primary_docs = filings.get("primaryDocument", [])

    for idx, f in enumerate(forms):
        if f == form:
            acc = accession_numbers[idx].replace("-", "")
            doc = primary_docs[idx]
            filing_url = (
                f"https://www.sec.gov/Archives/edgar/data/"
                f"{int(cik)}/{acc}/{doc}"
            )

```

```

        local_path = SAVE_DIR / f"{ticker}_{form}.pdf"
        try:
            doc_r = httpx.get(filing_url, headers=headers,
timeout=60)
            doc_r.raise_for_status()
        except httpx.HTTPError as exc:
            raise RuntimeError(f"Filing download failed:
{exc}") from exc

        local_path.write_bytes(doc_r.content)
        return local_path

    raise FileNotFoundError(f"No {form} filing found for ticker
{ticker}")

```

In practice, you can skip the network call by dropping a PDF into `./data/AAPL_10-K.pdf`. The rest of the pipeline does not care where the file came from. That decoupling is intentional: your agent should ingest documents from any source—EDGAR, a vendor API, or a USB stick handed to you by a skeptical CFO.

Extracting Text and Tables from PDFs

A 10-K is a PDF. Your job is to turn it into something an LLM can reason over without choking on formatting artifacts. We use two libraries for two different jobs.

PyPDF2 extracts plain text. It is fast and requires no heavy dependencies. It is also brittle: complex layouts, multi-column pages, and embedded tables often come out scrambled. We use it for narrative sections where word order matters more than structure.

pdfplumber extracts tables. It detects grid lines and bounding boxes and returns data as lists of lists, which we convert to pandas DataFrames. It is slower and more memory-hungry, but it is the only way to preserve the spatial meaning of a balance sheet.

Add the extraction functions next:

```

def extract_text_from_pdf(pdf_path: str) -> list[dict]:
    """Extract text from every page of a PDF.

    Returns a list of dicts, each containing:
        - page: 1-based page number
        - text: extracted text (may be empty for scanned/image
pages)
    """
    pages = []

```

```

try:
    with open(pdf_path, "rb") as f:
        reader = PyPDF2.PdfReader(f)
        for i, page in enumerate(reader.pages, start=1):
            text = page.extract_text()
            pages.append({"page": i, "text": text or ""})
except PyPDF2.errors.PdfReadError as exc:
    raise ValueError(f"Could not read PDF {pdf_path}: {exc}")
from exc
except FileNotFoundError:
    raise
return pages

def extract_tables_with_pdfplumber(pdf_path: str) -> list[dict]:
    """Extract tables from a PDF using pdfplumber.

    Returns a list of dicts, each containing:
        - page: 1-based page number
        - table_index: zero-based index of the table on that page
        - dataframe: pandas DataFrame of the table contents
    """
    import pdfplumber

    tables = []
    try:
        with pdfplumber.open(pdf_path) as pdf:
            for page_num, page in enumerate(pdf.pages, start=1):
                page_tables = page.extract_tables()
                for idx, table in enumerate(page_tables):
                    if table and len(table) > 1:
                        df = pd.DataFrame(table[1:],
columns=table[0])
                        tables.append({
                            "page": page_num,
                            "table_index": idx,
                            "dataframe": df,
                        })
    except Exception as exc:
        raise RuntimeError(
            f"Table extraction failed for {pdf_path}: {exc}"
        ) from exc
    return tables

```

Notice the error handling. PyPDF2 throws PdfReadError on corrupted files. pdfplumber can throw anything from pdfminer upstream if the PDF is malformed. We never let these exceptions bubble up unhandled; each is caught, wrapped with context, and re-raised so the caller knows which stage failed.

The Extraction Prompt: Structured JSON with Citations

This is the heart of the agent. We feed the extracted text and tables to a language model and demand structured output. The key design decision is the **JSON schema**. If you let the model ramble in prose, you will spend hours parsing its answer. If you force a schema, you get data you can validate, compare, and store.

The prompt must also include an explicit instruction: **“Cite page number for every metric.”** Without that, the model will confidently state “Revenue was \$89.5 billion” and give you no way to verify it. A cited metric is auditable. An uncited metric is gossip.

Build the prompt like this:

```
FINANCIAL_SCHEMA = {
    "reporting_period": "string (e.g., FY2023)",
    "revenue": {"value": "string or null", "page": "int or null"},
    "net_income": {"value": "string or null", "page": "int or
null"},
    "major_risks": [
        {"description": "string", "page": "int"}
    ],
    "forward_guidance": [
        {"statement": "string", "page": "int"}
    ],
    "management_sentiment": "string: bullish | neutral | cautious
| unknown",
    "cautionary_notes": ["string"],
}
```

```
def build_extraction_prompt(pages: list[dict], tables: list[dict])
-> str:
    """Construct the LLM prompt for structured financial
extraction.
```

```
    Intelligently truncates text to fit within a typical context
window
    while preserving page boundaries and table samples.
    """
    # Include first ~20 pages of text; adjust as needed for your
model
    text_sample = "\n\n".join(
        f"--- Page {p['page']} ---\n{p['text'][:1000]}"
        for p in pages[:20]
    )
```

```

table_sample = ""
for t in tables[:3]:
    table_sample += f"\n--- Table on page {t['page']} ---\n"
    table_sample +=
t["dataframe"].head(6).to_string(index=False) + "\n"

prompt = (
    "You are a rigorous financial analyst reviewing an SEC\n"
    "filing.\n"
    "Extract the following items from the text and tables\n"
    "below.\n"
    "Return ONLY valid JSON matching the provided schema.\n"
    "Do not add commentary outside the JSON.\n\n"
    "CRITICAL INSTRUCTIONS:\n"
    "- Cite page number for EVERY metric you report.\n"
    "- If a metric is missing, set its value to null and page\n"
    "to null.\n"
    "- If a statement is forward-looking (projections,\n"
    "guidance, expected, "\n"
    "    anticipates, believes), include it in forward_guidance.\n"
    "\n"
    "- Do not infer numbers. Only report what is explicitly\n"
    "stated.\n"
    "- Include cautionary_notes for anything ambiguous,\n"
    "footnote-dependent, "\n"
    "    or unusually rounded.\n\n"
    f"JSON Schema (return exactly this shape):\n"
    f"\n{{json.dumps(FINANCIAL_SCHEMA, indent=2)}}\n\n"
    "Filing Text:\n"
    f"{{text_sample}}\n\n"
    "Selected Tables:\n"
    f"{{table_sample}}\n"
)
return prompt

```

The schema is strict. revenue is not a raw number; it is a dict with value and page. major_risks is a list of objects, each with a description and a page. forward_guidance is isolated into its own array so downstream code can flag it with a “projection not fact” label.

Calling the LLM and Parsing Structured Output

With the prompt built, we call the model. I recommend starting with a local model via Ollama (llama3.1:8b or larger) for experimentation, because a 10-K can easily consume forty thousand tokens of input and you do not want to debug a \$4 API call.

When you are confident in the pipeline, switch to a cloud model like GPT-4o mini and enable JSON mode. The cost difference is significant: a single long-document extraction can cost fifty cents on GPT-4o and under a penny on GPT-4o mini.

```
def extract_financials_with_llm(prompt: str, use_openai: bool = False) -> dict:
```

```
    """Send the extraction prompt to an LLM and return parsed structured JSON.
```

```
    Args:
```

```
        prompt: The full extraction prompt built by build_extraction_prompt.
```

```
        use_openai: If True and OPENAI_API_KEY is set, use GPT-4o-mini.
```

```
        Otherwise, uses the local Ollama endpoint.
```

```
    Returns:
```

```
        A dict matching the FINANCIAL_SCHEMA shape.
```

```
    Raises:
```

```
        ValueError: If the LLM response is not valid JSON.
```

```
        RuntimeError: If the API call fails after retries.
```

```
    """
```

```
    raw = ""
```

```
    if use_openai and OPENAI_API_KEY:
```

```
        import openai
```

```
        client = openai.OpenAI(api_key=OPENAI_API_KEY)
```

```
        try:
```

```
            response = client.chat.completions.create(
                model="gpt-4o-mini",
                messages=[{"role": "user", "content": prompt}],
                temperature=0.0,
                response_format={"type": "json_object"},
            )
```

```
        except openai.APIError as exc:
```

```
            raise RuntimeError(f"OpenAI API error: {exc}") from
```

```
exc
```

```
    raw = response.choices[0].message.content
```

```
    cost = (
```

```
        response.usage.prompt_tokens * 0.00000015
```

```
        + response.usage.completion_tokens * 0.0000006
```

```
    )
```

```
    print(f"OpenAI extraction cost: ${cost:.4f}")
```

```
    else:
```

```
        payload = {
```

```
            "model": MODEL,
```

```
            "messages": [{"role": "user", "content": prompt}],
```

```

        "stream": False,
        "format": "json",
    }
    try:
        resp = httpx.post(OLLAMA_URL, json=payload,
timeout=120)
        resp.raise_for_status()
    except httpx.TimeoutException as exc:
        raise RuntimeError("Ollama request timed out after
120s") from exc
    except httpx.HTTPStatusError as exc:
        raise RuntimeError(f"Ollama returned error: {exc}")
    from exc

    raw = resp.json()["message"]["content"]

    try:
        data = json.loads(raw)
    except json.JSONDecodeError as exc:
        raise ValueError(
            f"LLM returned invalid JSON: {exc}\nRaw start: {raw[:
500]}")
        ) from exc

    return data

```

Always parse the JSON explicitly. Never trust a framework to do it for you. If the model starts with “Here is the JSON you requested:”, the `json.loads` call will fail, and you will catch it, log it, and fix the prompt. That is how you keep the pipeline honest.

Pipeline Architecture

Before we move to comparison logic, here is the complete data flow for the agent:

```

graph LR
    A[PDF Filing] --> B[Text Extraction<br/>PyPDF2]
    A --> C[Table Extraction<br/>pdfplumber]
    B --> D[LLM Extraction<br/>Structured JSON]
    C --> D
    D --> E[Structured JSON<br/>Metrics + Citations]
    E --> F[Comparison Report<br/>pandas Q/Q]
    F --> G[Human Review]

```

The PDF enters two parallel extraction paths. Text goes to PyPDF2. Tables go to pdfplumber. Both streams feed into a single LLM extraction prompt. The model returns structured JSON with page

citations. That JSON is then compared against previous quarters to surface changes. The final report lands in front of a human who decides what to do with it.

There is no trading algorithm in this diagram. There is no automated buy or sell signal. The arrow stops at human review. That is by design.

Quarter-over-Quarter Comparison

Extracting a single filing is useful. Comparing two filings is powerful. We store previous extractions as JSON files and use pandas to diff them.

```
def load_previous_extraction(path: str) -> dict | None:
    """Load a previously saved extraction JSON, or None if
missing."""
    p = Path(path)
    if not p.exists():
        return None
    try:
        with open(p, "r", encoding="utf-8") as f:
            return json.load(f)
    except json.JSONDecodeError as exc:
        raise ValueError(f"Corrupted previous extraction file:
{exc}") from exc

def compare_quarterly(
    current: dict,
    previous: dict | None,
    metrics: list[str] | None = None,
) -> pd.DataFrame:
    """Compare current and previous extraction results.

    Args:
        current: The latest structured extraction dict.
        previous: The prior extraction dict, or None for baseline.
        metrics: Keys to compare (default revenue and net_income).

    Returns:
        A pandas DataFrame with columns Metric, Current, Previous,
Change.
    """
    if metrics is None:
        metrics = ["revenue", "net_income"]

    rows = []
    for key in metrics:
        cur_val = None
```

```

prev_val = None

if isinstance(current.get(key), dict):
    cur_val = current[key].get("value")
else:
    cur_val = current.get(key)

if previous and isinstance(previous.get(key), dict):
    prev_val = previous[key].get("value")
elif previous:
    prev_val = previous.get(key)

change = "N/A"
if cur_val and prev_val:
    try:
        # Very naive numeric stripping for demo;
        # production needs robust parsing
        cur_num = float(str(cur_val).replace(",", ""))
        prev_num = float(str(prev_val).replace(",", ""))
        pct = ((cur_num - prev_num) / prev_num) * 100
        change = f"{pct:+.1f}%"
    except ValueError:
        change = f"{prev_val} -> {cur_val}"
elif cur_val and not prev_val:
    change = "new"

rows.append({
    "Metric": key,
    "Current": cur_val,
    "Previous": prev_val,
    "Change": change,
})

return pd.DataFrame(rows)

```

The comparison function is intentionally conservative. It does not pretend to understand currency conversions, restatements, or non-GAAP adjustments. It tells you: last quarter revenue was X, this quarter it is Y, and the naive change is Z percent. If the numbers look weird, the human opens the cited pages and investigates.

Tying It All Together: The Main Function

Here is the entry point that orchestrates the entire pipeline:

```

def main() -> None:
    """Run the financial report reader against a local sample PDF.

```

```

    Expects a PDF at ./data/sample_10k.pdf. If a previous
    extraction exists
    at ./data/sample_10k_prev.json, a quarter-over-quarter
    comparison is printed.
    """
    pdf_path = SAVE_DIR / "sample_10k.pdf"
    if not pdf_path.exists():
        print(
            f"Sample PDF not found at {pdf_path}.\n"
            "Place a filing PDF there, or implement
fetch_edgar_filing() "
            "to download one automatically."
        )
    return

    print(f"Reading PDF: {pdf_path}")
    try:
        pages = extract_text_from_pdf(str(pdf_path))
        print(f"Extracted text from {len(pages)} pages.")
    except (ValueError, FileNotFoundError) as exc:
        print(f"Text extraction failed: {exc}")
        return

    try:
        tables = extract_tables_with_pdfplumber(str(pdf_path))
        print(f"Extracted {len(tables)} tables.")
    except RuntimeError as exc:
        print(f"Table extraction failed: {exc}")
        return

    prompt = build_extraction_prompt(pages, tables)
    print("Prompt built. Calling LLM for structured
    extraction...")

    try:
        result = extract_financials_with_llm(prompt,
        use_openai=False)
    except (ValueError, RuntimeError) as exc:
        print(f"LLM extraction failed: {exc}")
        return

    print("\n=== Extraction Result ===")
    print(json.dumps(result, indent=2))

    # Save this extraction for next time
    out_path = SAVE_DIR / "sample_10k_latest.json"
    with open(out_path, "w", encoding="utf-8") as f:
        json.dump(result, f, indent=2)
    print(f"\nSaved extraction to {out_path}")

    # Compare with previous quarter if available

```

```

    prev = load_previous_extraction(SAVE_DIR /
"sample_10k_prev.json")
    if prev:
        comparison = compare_quarterly(result, prev)
        print("\n=== Quarter-over-Quarter Comparison ===")
        print(comparison.to_string(index=False))
    else:
        print("\nNo previous extraction found for comparison.")

if __name__ == "__main__":
    main()

```

Run it:

```
python financial_reader.py
```

If your local PDF is a real 10-K and your model has enough context window, you will see a JSON blob with revenue, net income, risks, guidance, sentiment, and page numbers. If the model hallucinates a number, you can open the cited page and verify. That verifiability is the entire point.

Break and Fix: Five Ways This Project Can Fail

Financial accuracy demands paranoia. Here are five real failure modes, with symptoms, diagnoses, and fixes.

Break 1: Table Misread by Extractor

Symptom: The extracted revenue reads “\$8,947” when the real figure is “\$89,470,000,000.” A decimal place or comma got swallowed.

Root cause: pdfplumber misaligned columns or PyPDF2 broke a line in the middle of a number. PDF table extraction is probabilistic, not deterministic.

Fix: Do not trust a single extractor. Cross-check critical numbers between the table extraction and the narrative text. Add a validation layer:

```

def cross_check_metric(pages: list[dict], metric: str,
table_value: str) -> bool:
    """Return True if the metric string appears in the narrative
text."""
    clean = str(table_value).replace(",", "").replace("$",

```

```

    "").strip()
    for p in pages:
        if clean in p["text"]:
            return True
    return False

```

If the number from the table does not appear in the text, flag it for manual review.

Break 2: Forward-Looking Statement Not Flagged

Symptom: The model lists “We expect revenue growth in the mid-teens” under a factual summary rather than in `forward_guidance`.

Root cause: The model is over-eager to be helpful and treats management optimism as achieved reality.

Fix: Harden the prompt with explicit keyword signals. Add a pre-filter step before the LLM call:

```

FORWARD_KEYWORDS = [
    "expect", "anticipate", "believe", "project", "forecast",
    "guidance", "outlook", "target", "goal", "aim", "plan to"
]

def flag_forward_statements(text: str) -> list[str]:
    """Return sentences containing forward-looking keywords."""
    flagged = []
    for sentence in text.split("."):
        lower = sentence.lower()
        if any(kw in lower for kw in FORWARD_KEYWORDS):
            flagged.append(sentence.strip())
    return flagged

```

Feed the flagged sentences into the prompt explicitly so the model cannot ignore them.

Break 3: Missing Footnote Context Changes Meaning

Symptom: The model reports “Accounts receivable: \$1.2B” without noting footnote 7, which reveals that \$400M is from a single customer nearing bankruptcy.

Root cause: The extraction prompt did not instruct the model to read footnotes adjacent to table cells.

Fix: Modify `build_extraction_prompt` to include a second text sample: the footnotes section. Most 10-Ks consolidate notes to financial statements in a dedicated section near the end. Extract that section separately and feed it into the prompt as context:

```
footnotes = "\n\n".join(
    p["text"] for p in pages if "notes to consolidated" in
p["text"].lower()
)
```

Also add an instruction: “If a table cell references a footnote number, read the corresponding footnote and include its impact in `cautionary_notes`.”

Break 4: PDF Is a Scanned Image, Not Text

Symptom: `extract_text_from_pdf` returns empty strings for every page. Your agent produces a JSON full of nulls.

Root cause: The PDF contains raster images of text, not embedded text objects. This is common with older filings or scanned documents.

Fix: Integrate OCR. The simplest path is `pdf2image` + `pytesseract`:

```
def ocr_page_to_text(pdf_path: str, page_num: int) -> str:
    """Convert a single PDF page to text via OCR."""
    from pdf2image import convert_from_path
    import pytesseract

    images = convert_from_path(pdf_path, first_page=page_num,
last_page=page_num)
    if images:
        return pytesseract.image_to_string(images[0])
    return ""
```

Production pipelines should run OCR automatically when `PyPDF2` returns fewer than `N` characters across the first few pages. That is your scanned-document detector.

Break 5: Company Changed Fiscal Year and Quarters Do Not Align

Symptom: Your Q/Q comparison claims revenue dropped 30%, but the company shifted its fiscal year end from December to September. You are comparing a twelve-month period to a nine-month transition period.

Root cause: The comparison function assumed identical period lengths without checking `reporting_period`.

Fix: Parse the reporting period explicitly and refuse to compare mismatched intervals. Add a guard:

```
if current.get("reporting_period") !=  
previous.get("reporting_period"):  
    print("WARNING: Reporting periods differ. Comparison may be  
invalid.")
```

Even better, include period length in the extraction schema and compare only when lengths match.

Try It Now: Three Variations

Variation 1: Multi-Company Comparison

Extend the pipeline to accept a list of tickers. Run the extraction for each, then build a DataFrame that ranks them by revenue growth, risk count, and sentiment. You will quickly discover that LLMs are inconsistent at sentiment scoring across different industries—an insight that should make you trust the metric less and the page citation more.

Variation 2: Generate an Investment Memo

Use the structured JSON as input to a second LLM call. Prompt it to write a one-page investment memo in the style of a hedge-fund analyst: thesis, bull case, bear case, key risks, and open questions. Feed the original JSON as context with strict instructions: “You may only cite facts from the provided JSON. Do not supplement with training data.” This tests whether your extraction was good enough to stand on its own.

Variation 3: Flag Forward-Looking Statements as “Projection Not Fact”

Post-process the extraction JSON. For every item in `forward_guidance`, append a label: `"classification": "projection not fact"`. Build a report that separates the section titled “What We Know” from “What Management

Hopes.” This is the single most valuable output for a skeptical analyst, because it forces a clean separation between historical performance and speculative narrative.

Ethics Sidebar: Never Trade on This

“Never trade on this. It’s research, not advice.”

The agent you built in this chapter reads faster than you. It does not read better than you. It misses things, misreads tables, and occasionally hallucinates footnotes. If you put real money at risk based on an LLM extraction of a PDF, you are not doing quantitative analysis. You are doing automated guesswork with leverage.

Use this tool to narrow your focus. Let it find the pages you should read. Let it surface risks you might have scrolled past. Then open the original filing, read the relevant sections yourself, and make your own judgment. The agent is a highlighter, not a decision-maker.

Also respect the SEC’s terms of service. EDGAR data is public, but bulk scraping or violating the User-Agent policy can get your IP rate-limited or banned. Be polite. Cache what you download. Do not hammer the server.

The OpenClaw Way

OpenClaw handles this pipeline elegantly through its document-processing toolkit. You define a Document object from a PDF path, chain a PDFTextExtractor and a TableExtractor, and then pass the combined artifact to an ExtractionAgent configured with a Pydantic output schema. OpenClaw automatically truncates text to fit the model’s context window, retries on JSON parse failures, and tracks per-document cost.

Here is the conceptual equivalent:

```
from openclaw import Agent, tool, Document
from pydantic import BaseModel
```

```
class FilingExtract(BaseModel):
    revenue: str
    revenue_page: int
    major_risks: list[str]
```

```
@tool
```

```
def read_pdf(path: str) -> Document:
    return Document.from_pdf(path)

agent = Agent(tools=[read_pdf], output_schema=FilingExtract)
result = agent.run("Extract revenue and risks from ./data/sample_10k.pdf")
print(result.model_dump_json())
```

The Pydantic schema forces the model to conform to typed fields, and OpenClaw’s parser validates the output before returning it. If the model omits a required field, OpenClaw re-prompts automatically. That guardrail alone saves you from the invalid-JSON failures we handled manually.

The Hermes Agent Way

Hermes Agent approaches this task conversationally. You drop the PDF into a Hermes session and ask natural-language questions:

```
hermes
> Read ./data/sample_10k.pdf and extract revenue, net income, and risks.
> Compare it to the previous quarter's extraction I saved last month.
```

Hermes uses built-in file-reading and code-execution tools to run PyPDF2 and pdfplumber behind the scenes. It generates a pandas comparison table in-memory and presents it as a formatted markdown report. Because Hermes maintains session history, you can iteratively refine the extraction: “Now focus on the supply chain risk section,” or “Show me only forward-looking statements from the CEO letter.”

The trade-off is control. You do not own the exact prompt or the JSON schema. The benefit is speed: you can analyze a filing in under a minute without writing a single line of boilerplate. For ad-hoc research, that conversation-first model is hard to beat.

What You Built

In this chapter you built a serious document-analysis pipeline:

- An SEC EDGAR fetcher with proper headers and error handling.
- A dual-track PDF extraction system: PyPDF2 for narrative text, pdfplumber for tables.

- A structured LLM prompt with a JSON schema, explicit citation requirements, and forward-looking statement detection.
- An extraction function that supports both local Ollama models and cloud APIs with cost tracking.
- A quarter-over-quarter comparison layer in pandas.
- A main() orchestrator that glues everything together and fails safely at every stage.
- Five documented failure modes and their mitigations, including OCR fallback and fiscal-year alignment guards.

More importantly, you built a system that treats financial documents with the respect they deserve. It cites sources, flags uncertainty, and never pretends to be a portfolio manager.

Cost for this chapter: Approximately \$0.00 using Ollama with a local 8B model for a single 100-page filing. Approximately \$0.04–\$0.12 using GPT-4o-mini for the same document, depending on how much of the text you include in the prompt. GPT-4o would cost roughly \$0.50–\$1.50 per filing. Run local while iterating; run cloud only when you need the extra reasoning power.

Chapter 20: Project 20 — Smart Home Command Center

“Dim the living room lights and turn on the porch light” — said to software, not hardware.

I once asked a voice assistant to “turn off the bedroom light.” It heard “turn off all the lights.” My partner was in the middle of a video call. The room went black. They lost their presentation. The dog barked. I spent ten minutes apologizing to a person, a canine, and a laptop that had done nothing wrong except obey a bad command interpreted badly.

That moment is funny now, but it taught me the central tension of smart home agents: the line between convenient and catastrophic is razor thin. A light going off at the wrong time is embarrassing. A door unlocking when it shouldn’t is a security incident. A thermostat cranked to ninety degrees while you’re on vacation is an energy bill

and a melted houseplant. IoT agents are fun because they make your home feel alive. They are dangerous because your home *is* alive, and the people inside it trust that the locks work.

This chapter puts an agent in charge of real physical devices. We'll connect to Philips Hue or Home Assistant over REST. We'll parse natural language like “dim the living room to thirty percent” into structured API calls. We'll track device state in memory so the agent knows what it changed. And we'll build a safety gate so that commands like “lock the front door” cannot execute without explicit human confirmation.

Let's build a command center that is helpful, fast, and safe.

Why IoT Agents Are Fun and Dangerous

The fun is obvious. You speak; the world bends. Lights dim. Music starts. The thermostat adjusts. Unlike a research agent that searches and summarizes, a home agent acts in milliseconds, and the result is a physical sensation — warmth, light, quiet.

The danger is physical. Devices have state that lives outside your program. You send “off,” but someone flips the wall switch two seconds later, and now your memory says “off” while the real world says “on.” Commands can be ambiguous: “turn on the hall light” means nothing if you have an upstairs hall, downstairs hall, and a closet someone put a smart bulb in as a joke. And some commands are not easily reversible: unlocking a door, disarming an alarm, opening a garage.

That's why this project is built on one rule: **the physical world does not roll back**. Every line of code must answer “What happens if this goes wrong?” before it does anything else.

Connecting to Your Devices

The easiest way to start is Philips Hue. The Hue Bridge exposes a local REST API with no cloud dependency. Every light has a unique ID. You send JSON over HTTP, and the lights respond instantly.

If you run Home Assistant, the pattern is identical but richer. Home Assistant aggregates devices behind a unified REST API. You authenticate, GET state, POST commands. I'll show the Hue API because it's self-contained, but I'll note the Home Assistant differences so you can adapt in minutes.

First, discover your bridge and get an API key. Press the button on the bridge, then run:

```
curl -X POST http://<bridge-ip>/api \
  -d '{"devicetype":"my_agent#laptop"}'
```

The response includes a username field. That is your API key. Store it in an environment variable. Never hard-code it. If you push it to a repository by mistake, someone on the internet can control your lights.

With the key in hand, list your lights:

```
import httpx
import os

HUE_BRIDGE_IP = os.environ.get("HUE_BRIDGE_IP", "192.168.1.100")
HUE_USERNAME = os.environ.get("HUE_USERNAME", "")
BASE_URL = f"http://{HUE_BRIDGE_IP}/api/{HUE_USERNAME}"

def list_lights() -> dict:
    """Fetch all lights from the Hue bridge and return a dict
    keyed by light ID."""
    resp = httpx.get(f"{BASE_URL}/lights", timeout=10.0)
    resp.raise_for_status()
    return resp.json()
```

The response maps light IDs to objects with names, types, and state. A typical entry looks like:

```
{
  "1": {
    "name": "Living room ceiling",
    "state": {
      "on": true,
      "bri": 180,
      "reachable": true
    }
  }
}
```

bri is brightness, 1–254. reachable is your lifeline: if a bulb is powered off at the wall or disconnected, it becomes false. We'll use that to avoid sending commands into the void.

For Home Assistant, you'd hit `/api/states` with an `Authorization: Bearer ***` header, filter by `light.* domains`, and send service calls to `/api/services/light/turn_on`. The pattern is the same: authenticate, GET state, POST command, handle errors.

Parsing Natural Language Into Structured Commands

The bridge understands JSON. You speak English. For this project, we'll build a lightweight regex-based parser. Home commands follow a small, rigid grammar: `[action] [target] [parameter]`. Regex keeps latency under ten milliseconds, costs zero dollars, and works when the internet is down.

```
import re
from typing import Optional, Dict, Any

KNOWN_ROOMS = {"living room", "bedroom", "kitchen", "office",
               "porch", "hall"}
CRITICAL_ACTIONS = {"lock", "arm", "disarm", "unlock"}

def parse_command(text: str) -> Dict[str, Any]:
    """Parse natural language into a structured instruction.

    Returns:
        action: str (e.g., 'on', 'off', 'dim', 'lock')
        target: str or None (room name; None if ambiguous)
        parameter: Optional[int] (brightness level 0-100)
        is_critical: bool (True if action affects security)
    """
    text_lower = text.lower().strip()

    # Extract brightness: "to 30%", "30 percent", "brightness 30"
    brightness_match = re.search(
        r"(?:to\s+|brightness\s+)?(\d+)(?:\s*%|\s+percent)?",
        text_lower
    )
    parameter = int(brightness_match.group(1)) if brightness_match
    else None

    # Identify action using word boundaries
    action = None
    if re.search(r"\b(dim|lower|reduce)\b", text_lower):
        action = "dim"
    elif re.search(r"\bunlock\b", text_lower):
        action = "unlock"
    elif re.search(r"\bunarm\b|\bdisarm\b", text_lower):
        action = "disarm"
```

```

elif re.search(r"\block\b", text_lower):
    action = "lock"
elif re.search(r"\barm\b", text_lower):
    action = "arm"
elif re.search(r"\b(off|deactivate|stop)\b", text_lower):
    action = "off"
elif re.search(r"\b(on|activate|start)\b", text_lower):
    action = "on"

# Identify room – never guess; require exactly one match
found_rooms = [room for room in KNOWN_ROOMS if room in
text_lower]
target = found_rooms[0] if len(found_rooms) == 1 else None

is_critical = action in CRITICAL_ACTIONS

return {
    "action": action,
    "target": target,
    "parameter": parameter,
    "is_critical": is_critical,
}

```

The keyword checks use word boundaries so “kitchen” doesn’t match “on”. “Dim” is checked before “off” because we want specificity first. The room resolution is strict: if zero or more than one known room appears, target is None, and the caller must ask for clarification.

This is the first safety rule of the chapter: **never guess a room name**. If you guess, you’ll eventually be wrong. Wrong means your partner’s bedroom light turns on at 6 AM because “bedroom” resolved to the guest room. Ask. Always ask.

State Tracking in Memory

Your agent needs to know what it thinks the world looks like. Without state tracking, every command is stateless. With it, you can answer “what’s on right now?” and avoid redundant commands that waste API calls.

```

class HomeState:
    """In-memory snapshot of device state, refreshed on demand."""

    def __init__(self):
        self.devices: Dict[str, Dict[str, Any]] = {}
        self.room_map: Dict[str, list] = {}

    def refresh_from_bridge(self) -> None:

```



```

        """Fetch all lights and rebuild internal maps."""
        data = list_lights()
        self.devices = {}
        self.room_map.clear()
        for light_id, info in data.items():
            nice_id = f"light_{light_id}"
            self.devices[nice_id] = {
                "name": info.get("name", "Unknown"),
                "on": info["state"].get("on", False),
                "bri": info["state"].get("bri", 254),
                "reachable": info["state"].get("reachable",
False),
            }
            name_lower = info.get("name", "").lower()
            for room in KNOWN_ROOMS:
                if room in name_lower:
                    self.room_map.setdefault(room,
[])].append(nice_id)

    def get_room_brightness(self, room: str) -> Optional[int]:
        """Return average brightness 0-100 for a room. None if no
lights mapped."""
        ids = self.room_map.get(room, [])
        if not ids:
            return None
        valid = [self.devices[d]["bri"] for d in ids if
self.devices[d]["reachable"]]
        if not valid:
            return None
        avg = sum(valid) / len(valid)
        return round((avg / 254) * 100)

    def set_room_state(self, room: str, on: bool, bri_pct:
Optional[int] = None) -> None:
        """Update local state after a successful command; does NOT
call the API."""
        for d in self.room_map.get(room, []):
            self.devices[d]["on"] = on
            if bri_pct is not None:
                self.devices[d]["bri"] = round((bri_pct / 100) *
254)

```

HomeState fetches the real world, mirrors it locally, and maps room names to device IDs so the user doesn't have to remember that the living room ceiling is light 14. The local cache is updated only after a successful API call. If someone flips the physical wall switch, the bridge will reflect that eventually, but our agent won't know until `refresh_from_bridge()` runs again.

The Safety Gate: No Critical Action Without Approval

Some commands are harmless. Some are not. Locking or unlocking a door, arming or disarming a system — these change the security posture of your home. Our agent must handle them differently.

```
import time
from typing import Callable

_approval_cache: Dict[str, float] = {}
CACHE_TTL_SECONDS = 300

def command_signature(cmd: Dict[str, Any]) -> str:
    """Return a stable string key for a command, used for caching
    approvals."""
    return f"{cmd['action']}:{cmd['target']}:{cmd.get('parameter','')}"

def request_human_approval(
    cmd: Dict[str, Any], get_input: Callable[[str], str] = input
) -> bool:
    """Require explicit user confirmation for a critical command.

    Args:
        cmd: Parsed command dict.
        get_input: Injectable input function for testing.

    Returns:
        True if the user explicitly confirms; False otherwise.
    """
    sig = command_signature(cmd)
    now = time.time()

    cached = _approval_cache.get(sig)
    if cached and now < cached:
        print(f"Re-using approval for {sig} (expires in {int(cached - now)}s)")
        return True

    print("\n" + "=" * 50)
    print("CRITICAL ACTION REQUESTED")
    print(f"Action : {cmd['action']}")
    print(f"Target : {cmd['target']}")
    if cmd.get('parameter'):
        print(f"Param  : {cmd['parameter']}")
    print("=" * 50)

    answer = get_input("Approve? Type YES to proceed: ").strip()
    if answer == "YES":
```

```

        _approval_cache[sig] = now + CACHE_TTL_SECONDS
        print("Approved. Executing...")
        return True
    print("Cancelled by user.")
    return False

```

The gate requires the exact string “YES.” Not “y.” Not “yes.” The exact string. That friction is intentional. It gives the user a moment to think and prevents accidental touchscreen approvals.

There’s an approval cache so you don’t have to type “YES” twice in five minutes if a script retries. But caching approvals is dangerous; we’ll shoot ourselves with it in the Break and Fix section.

Wiring the Command Interpreter

Now we connect parser, state tracker, safety gate, and API into one flow.

```

class SmartHomeAgent:
    """Agent that accepts natural language, validates safety, and
    controls lights."""

    def __init__(self):
        self.state = HomeState()
        self.state.refresh_from_bridge()

    def interpret(self, text: str) -> str:
        """Parse, validate, gated-execute, and confirm a home
        command."""
        cmd = parse_command(text)

        if cmd["action"] is None:
            return "I didn't understand the action. Try 'turn on
the living room light'."
        if cmd["target"] is None:
            known = ", ".join(sorted(KNOWN_ROOMS))
            return f"I'm not sure which room you mean. Known
rooms: {known}."

        lights = self.state.room_map.get(cmd["target"], [])
        if not lights:
            return f"I don't see any lights mapped to
'{cmd['target']}'."

        reachable = [d for d in lights if self.state.devices[d]
["reachable"]]
        if not reachable:
            return (

```

```

        f"All lights in {cmd['target']} appear offline. "
        "Check power or the wall switch."
    )

    # Partial offline warning
    offline = [d for d in lights if not self.state.devices[d]
["reachable"]]
    if offline:
        names = ", ".join(self.state.devices[d]["name"] for d
in offline)
        print(f"Warning: {names} appear offline and will not
respond.")

    if cmd.get("is_critical"):
        if not request_human_approval(cmd):
            return "Critical action was not approved. Nothing
changed."

    return self._execute(cmd, reachable)

def _execute(self, cmd: Dict[str, Any], device_ids: list) ->
str:
    """Send commands to the bridge and update local state."""
    action = cmd["action"]
    parameter = cmd.get("parameter")
    payload: Dict[str, Any] = {}

    if action == "on":
        payload = {"on": True}
    elif action == "off":
        payload = {"on": False}
    elif action == "dim":
        bri = round((parameter / 100) * 254) if parameter else
128
        payload = {"on": True, "bri": max(1, min(254, bri))}
    elif action in ("lock", "unlock", "arm", "disarm"):
        return f"Mock: {action} on {cmd['target']} would be
sent to the security API."
    else:
        return "Unsupported action."

    errors = []
    for d in device_ids:
        hue_id = d.replace("light_", "")
        try:
            resp = httpx.put(
                f"{BASE_URL}/lights/{hue_id}/state",
                json=payload,
                timeout=10.0,
            )
            resp.raise_for_status()
        except httpx.HTTPError as e:

```

```

        errors.append(f"{d}: {e}")

    if errors:
        return f"Partial failure: {'; '.join(errors)}"

    on_value = payload.get("on", action != "off")
    self.state.set_room_state(cmd["target"], on=on_value,
bri_pct=parameter)

    if action == "dim":
        return f"Dimmed {cmd['target']} to {parameter or
50}%."
    return f"Turned {action} the {cmd['target']} light(s)."
```

The interpret method validates every step — parse, map, check reachability, gate, execute, confirm — before it touches a real device. `_execute` updates local state only after the API call succeeds. There’s nothing worse than an agent that thinks it turned off the lights, then later refuses to turn them on because its cache says they’re already off.

The Command Flow

Here’s the full pipeline:

```

flowchart LR
    A[Voice/Text Input] --> B[NL Parser]
    B --> C[Command Interpreter]
    C --> D{Safety Gate?}
    D -- critical --> E[Human Approval]
    E --> F[API Call]
    D -- safe --> F
    F --> G[Device State Update]
    G --> H[Confirmation]
```

Input comes in as text. The parser extracts action, room, and parameter. The interpreter checks whether the action is critical. If so, the gate blocks until the user types “YES.” Then the API call goes out. On success, local state updates, and the user gets confirmation. No silent failures, no unasked questions.

A Complete main()

Here’s the entry point:

```

def main() -> None:
    """Run the smart home agent interactively from the
terminal."""
```

```

agent = SmartHomeAgent()
print("Smart Home Agent ready. Type a command or 'quit' to
exit.")
print(f"Known rooms: {' '.join(sorted(KNOWN_ROOMS))}")

while True:
    try:
        text = input("\n> ").strip()
    except (EOFError, KeyboardInterrupt):
        print("\nGoodbye.")
        break
    if text.lower() in ("quit", "exit", "q"):
        print("Goodbye.")
        break
    if not text:
        continue
    print(agent.interpret(text))

if __name__ == "__main__":
    main()

```

Run it:

```
python smart_home_agent.py
```

Then try:

```
> turn on the living room
Turned on the living room light(s).
```

```
> dim the bedroom to 30%
Dimmed bedroom to 30%.
```

```
> lock the front door
CRITICAL ACTION REQUESTED
Action : lock
Target : porch
Approve? Type YES to proceed: YES
Approved. Executing...
Mock: lock on porch would be sent to the security API.
```

Break and Fix: Five Ways This Project Can Fail

Break 1: Ambiguous Room Name

You say: “turn on the light.” The parser finds zero rooms and returns:

I'm not sure which room you mean. Known rooms: bedroom, hall, kitchen, living room, office, porch.

Other voice assistants “just know.” They guess. And sometimes they guess the bedroom at midnight and wake someone up.

Fix: There is no code fix that improves accuracy here without making dangerous assumptions. The fix is interaction design. Offer the most likely room based on context, but still require explicit confirmation before executing:

```
if cmd["target"] is None:
    likely = "living room" # or time-of-day heuristics
    return f"Did you mean {likely}? Please say 'turn on the {likely} light'."
```

The agent still doesn't execute until the user names the room explicitly.

Break 2: Command Misunderstood

“Turn off all lights” gets parsed as action="on" because “on” is a substring of “off.” The agent turns everything on instead.

Fix: Use word boundaries and check “off” before “on”:

```
if re.search(r"\boff\b", text_lower):
    action = "off"
elif re.search(r"\bon\b", text_lower):
    action = "on"
```

Add an integration test that feeds known utterances through the parser and asserts the correct action. Regression-proof your keywords.

Break 3: State Gets Out of Sync

Your agent thinks the living room is off because it sent the off command last night. But someone flipped the physical wall switch this morning, and now the bulb is on. Your agent still has on: False in memory.

Now you say “turn on the living room.” The agent checks state, sees it's already “on,” and does nothing. Or worse, it tells you it's already on.

Fix: Refresh state before commands that depend on current state. In interpret, call `self.state.refresh_from_bridge()` at the start of every command. For larger setups, refresh only when the command implies reading current state, like “toggle” or “increase brightness.”

Break 4: Safety Bypass — Approval Cached Too Long

You approved “unlock the front door” twenty minutes ago. The TTL is five minutes, so the cache is expired. But if you set TTL to an hour “for convenience,” an attacker with access to your terminal — or a replay script — can issue the same command within the hour and it auto-approves.

Fix: Keep TTL short: five minutes or less. For critical actions, disable caching entirely. Make the cache key specific — “unlock front door” is not “unlock back door.” Log every approval with a timestamp:

```
if cached and now < cached:  
    print(f"[AUDIT] Replaying approval for {sig}")  
    return True
```

Convenience and security are a dial. Turn it toward security for home automation.

Break 5: Device Offline, Command Fails Silently

A bulb is physically switched off at the wall. `reachable` is `False`. You send a command. The Hue bridge accepts it happily and returns success. The agent says “Turned off the bedroom light,” but the bulb was already off. There’s no error, so there’s no signal that something is wrong.

Fix: Check `reachable` before sending, and surface partial failures. Our `interpret` method already does this:

```
reachable = [d for d in lights if self.state.devices[d]  
["reachable"]]  
if not reachable:  
    return "All lights appear offline. Check power or the wall  
switch."
```

For partial reachability (two bulbs in a room, one offline), report the offline ones by name so the user knows the command didn’t reach the full set.

Try It Now: Three Variations

Variation 1: Scene Orchestration (“Movie Night”)

Create a scene registry that sets multiple rooms at once:


```

SCENES = {
    "movie night": {
        "living room": {"on": True, "bri_pct": 15},
        "kitchen": {"on": False},
    },
    "work day": {
        "office": {"on": True, "bri_pct": 100},
        "living room": {"on": False},
    },
}

def activate_scene(scene_name: str, agent: SmartHomeAgent) -> str:
    """Apply a scene by name, returning a summary of changes."""
    scene = SCENES.get(scene_name.lower())
    if not scene:
        return f"Unknown scene. Available: {'',
'.join(SCENES.keys())}"
    results = []
    for room, params in scene.items():
        text = f"set {room} brightness to {params.get('bri_pct',
50)}%"
        results.append(agent.interpret(text))
    return "\n".join(results)

```

Variation 2: Energy Optimization Schedules

Track cumulative brightness and runtime per room. Compute which rooms consume the most power and suggest reductions:

```

from collections import defaultdict

ENERGY_LOG: Dict[str, list] = defaultdict(list)

def log_energy(room: str, brightness_pct: int, duration_minutes:
float) -> None:
    """Append an energy usage record for a room."""
    ENERGY_LOG[room].append(brightness_pct * duration_minutes)

def energy_report() -> str:
    """Return the room with the highest estimated energy use and a
suggestion."""
    if not ENERGY_LOG:
        return "No energy data yet."
    totals = {room: sum(entries) for room, entries in
ENERGY_LOG.items()}
    worst = max(totals, key=totals.get)
    return (
        f"Highest usage: {worst} ({totals[worst]:.0f} brightness-
minutes). "
        "Consider lowering brightness or scheduling auto-off after

```

```
11 PM."  
    )
```

Variation 3: Voice-Like Interaction with Preference Memory

Remember user preferences. If the user always dims the living room to twenty percent at night, prefill the parameter next time:

```
from collections import defaultdict  
  
PREFERENCE_MEMORY = defaultdict(lambda: defaultdict(lambda: None))  
  
def learn_preference(room: str, action: str, parameter:  
Optional[int]) -> None:  
    """Store the most recent parameter for a room+action pair."""  
    if parameter is not None:  
        PREFERENCE_MEMORY[room][action] = parameter  
  
def suggest_parameter(room: str, action: str) -> Optional[int]:  
    """Return the last used parameter for this room and action, if  
any."""  
    return PREFERENCE_MEMORY.get(room, {}).get(action)
```

If the user says “dim the living room” without a number and memory holds 20, the agent asks: “Did you want twenty percent, like last time?” It’s not guessing. It’s a suggestion that still requires explicit confirmation.

The OpenClaw Way

OpenClaw’s tool-first architecture maps cleanly to the smart home domain. Instead of a custom SmartHomeAgent class, you’d define each capability as an @tool and let the framework handle the loop:

```
from openclaw import Agent, tool  
  
@tool  
def set_light(room: str, action: str, brightness: int = 100) ->  
str:  
    """Set the lights in a room. Actions: 'on', 'off', 'dim'.  
    Brightness 0-100."""  
    # ... implementation using httpx ...  
    return f"Set {room} to {action} ({brightness}%)."  
  
@tool  
def request_approval(action: str, target: str) -> bool:  
    """Request human approval for a critical action. Returns True
```

```
if approved. """
    # ... gate logic ...

agent = Agent(tools=[set_light, request_approval],
              reasoning="react")
```

OpenClaw's ReAct loop handles the planning: the user says "lock the front door," the model recognizes the critical action, calls `request_approval`, and either proceeds or reports cancellation. The framework also adds tracing and cost tracking, which is helpful if you ever upgrade to a cloud LLM for richer natural language. For local use with the regex parser we built, you can set `reasoning="none"` and use OpenClaw purely as a structured tool registry with automatic JSON parsing.

The Hermes Agent Way

Hermes is conversation-first, which fits naturally with a home agent. The interaction looks like this:

```
hermes
# Welcome to Hermes Agent v2.5.0

> turn off the kitchen light

[search] Querying home state...
[execute_code] Sending PUT to Hue bridge for light ID 7 -> {"on":
false}

Done. The kitchen light is now off.

> how bright is the living room?

Living room is at 70% brightness across 2 lights.
```

Hermes has `execute_code` and `http_request` built in, so it can hit the Hue API without custom integrations. The trade-off is that Hermes does not have an explicit safety gate for physical actions unless you configure one in its policy settings:

```
security_gate:
  require_approval_for:
    - lock
    - arm
    - disarm
  prompt: "This action affects physical security. Approve? (yes/no)"
```

Because Hermes maintains a persistent session, it remembers context across commands. If you say “dim that to twenty percent” two turns after naming a room, Hermes recalls which room you meant. This convenience shows why state tracking matters: without shared context, every command would specify room, device, and parameter in full.

What You Built

In this chapter, you built a complete smart home command center from raw Python:

- A Hue REST API client that lists lights, reads state, sends commands, and handles offline devices gracefully.
- A regex-based natural language parser that extracts action, room, and brightness — and refuses to guess ambiguous room names.
- A state tracker that mirrors the bridge’s device list and maps room names to hardware IDs.
- A strict safety gate that requires explicit “YES” confirmation for critical actions, with a short-lived approval cache.
- A command interpreter that validates every step — parse, map, check reachability, gate, execute, confirm.
- Break and Fix scenarios for ambiguous rooms, misheard commands, stale state, cached approval bypasses, and offline bulbs.
- Three extension ideas: scene orchestration, energy logging, and preference memory.

Most importantly, you learned that physical agents fail differently. A bad command doesn’t just return wrong text. It changes the real world. That’s why this agent asks before it acts, checks before it assumes, and fails loudly when it’s uncertain.

Cost for this chapter: \$0.00 if you use a local Hue Bridge. No API keys. No metered requests. HTTP inside your LAN is free. If you swap the parser for a cloud LLM for richer natural language, budget approximately \$0.0001 per command with GPT-4o mini, or roughly \$0.01 per day for a household that processes 100 commands.

Chapter 21: Project 21 — Competitive Intelligence Monitor

Every Monday at 8 AM: a two-minute brief on what your competitors changed.

I once lost a major customer because a competitor dropped their price by forty percent. I didn't find out for three weeks. By the time I did, the prospect had already signed a twelve-month contract with the other team. I remember staring at their pricing page — the one I'd last checked a month earlier — and feeling like a fool. The information was public. The change was loud. I just wasn't listening.

That's the thing about competitive intelligence: it's not about spying. It's about paying attention to what your competitors are already shouting from the rooftops. Pricing pages, press releases, leadership announcements, new feature blogs — this stuff is public by design. The hard part is doing it consistently without spending your mornings manually refreshing six different websites.

In this chapter, you'll build a monitor that does the listening for you. Once a week (or once a day, if you prefer), it fetches a list of competitor URLs, compares the current page against the last snapshot, and uses an LLM to filter the noise. Only significant changes — pricing shifts, product launches, leadership moves, funding news — make it to your Slack channel as a structured brief. Everything else gets quietly logged and forgotten.

But before we write a single line of scraping code, we need to talk about boundaries. Competitive intelligence is legal when you stick to public information and respect the rules of the road. It becomes illegal when you bypass authentication, scrape private data, or violate a website's terms of service. We'll build ethical guardrails directly into the agent. If a site says “no robots,” our robot will stop.

What Competitive Intelligence Actually Means

Competitive intelligence is the systematic collection and analysis of publicly available information about your market. It is *not* industrial espionage. It is *not* hacking. It is not stealing internal documents or posing as a customer to access private beta features. Those things have names, and those names come with lawyers.

What we're doing is much simpler and much more defensible. We're automating the same work a diligent analyst would do with a cup of coffee and a bookmark folder. The difference is speed and consistency. An agent doesn't forget to check on Monday morning because it was sick. An agent doesn't miss a subtle footer change because its eyes glazed over on the tenth page.

The ethical framework we'll use has four pillars:

1. **Public only.** If it requires a login, it's off-limits.
2. **Respect robots.txt.** If the site owner asks bots to stay away, we stay away.
3. **Respect terms of service.** If the ToS explicitly prohibits automated access, we don't automate.
4. **Don't be creepy.** If a technique feels like stalking, it is stalking. Don't do it.

We'll encode the first three into code. The fourth is on you.

How the Monitor Works

Here's the pipeline, end to end. We'll build each stage in pure Python, then wire them together into a single scheduled script.

flowchart LR

```
A[Cron Trigger] --> B[Fetch URLs]
B --> C{Diff vs Previous}
C -- changed --> D[LLM Filter]
C -- unchanged --> Z[Done]
D --> E[Structured Brief]
E --> F[Slack Alert]
F --> G[Update Baseline]
```

Cron Trigger wakes the script on your chosen schedule. **Fetch URLs** pulls the current HTML for each competitor page using httpx with browser-like headers. **Diff vs Previous** compares a content hash against the stored baseline. If nothing changed, we're done. If something changed, the full text goes to the **LLM Filter**, which classifies the delta into categories like pricing, feature launch, leadership change, or funding. For significant signals, the model generates a **Structured Brief** — a short markdown summary with a severity score. That brief fires a **Slack Alert** via webhook, and the new snapshot becomes the **Update Baseline** for next time.

Let's build it.

Step 1: Respecting robots.txt

Before we fetch anything, we ask permission. The robots.txt file at the root of a domain tells automated crawlers which paths are welcome and which are off-limits. It's not a legal contract in every jurisdiction, but it is a clear signal of the site owner's intent. Ignoring it is the fastest way to get banned and the easiest way to lose the moral high ground.

We'll use the robotparser module from the Python standard library. It's not fancy, but it works.

```
import urllib.robotparser
import urllib.parse

def is_url_allowed(url: str) -> bool:
    """Check whether a URL is permitted by the site's robots.txt.

    Returns True if robots.txt does not exist, cannot be fetched,
    or explicitly allows the path. Returns False if the path
    is disallowed.
    """
    parsed = urllib.parse.urlparse(url)
    robots_url = f"{parsed.scheme}://{parsed.netloc}/robots.txt"

    rp = urllib.robotparser.RobotFileParser()
    rp.set_url(robots_url)
    try:
        rp.read()
    except Exception:
        # If robots.txt is unreachable, default to cautious True
        # but log it so you know you couldn't verify.
        return True
```

```
return rp.can_fetch("*", url)
```

This function prepends robots.txt to the domain, parses the rules, and returns a boolean. If the server is down or the file is missing, we default to True but with a note to check manually. In a production system, you'd log these failures instead of silently proceeding.

We'll call `is_url_allowed` for every target URL before we ever make a fetch request. No exceptions.

Step 2: Fetching Pages Like a Browser

Web servers are suspicious of scripts. If you make an HTTP request with the default `httpx` headers, many sites will block you instantly or serve a degraded version of the page. The fix is trivial: pretend to be a browser.

```
import hashlib
import json
import os
import time
from pathlib import Path
```

```
import httpx
```

```
def fetch_page(url: str, timeout: float = 30.0) -> str:
    """Fetch a URL with browser-like headers and return the
    response text.
```

```
    Raises on non-2xx status so the caller can decide what to do.
    """
```

```
    headers = {
        "User-Agent": (
            "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
            "AppleWebKit/537.36 (KHTML, like Gecko) "
            "Chrome/124.0.0.0 Safari/537.36"
        ),
        "Accept": (
            "text/html,application/xhtml+xml,application/xml;"
            "q=0.9,image/webp,*/*;q=0.8"
        ),
        "Accept-Language": "en-US,en;q=0.5",
        "Accept-Encoding": "gzip, deflate, br",
        "DNT": "1",
        "Connection": "keep-alive",
        "Upgrade-Insecure-Requests": "1",
    }
```



```

    with httpx.Client(headers=headers, follow_redirects=True,
timeout=timeout) as client:
        response = client.get(url)
        response.raise_for_status()
        return response.text

```

This isn't "tricking" the site in any meaningful sense. We're identifying ourselves as an HTTP client that can render HTML, which is exactly what we plan to do with the data. The key is that we're not lying about being a malicious bot; we're just being specific about our capabilities.

Notice `response.raise_for_status()`. If the server returns a 404, 500, or 429, this function throws an exception. We'll catch those in the main loop and handle them gracefully instead of crashing the entire monitor.

Step 3: Hash-Based Diffing

We don't need to diff the full HTML string every time. HTML is noisy: timestamps, CSRF tokens, ad slots, and analytics IDs change on every load. What we care about is *meaningful* change. But before we bring in the LLM to judge meaning, we need a cheap filter. If the page hasn't changed at all, we shouldn't waste tokens asking the model to compare paragraphs.

We'll store a SHA-256 hash of the cleaned page text for each URL. If the hash matches the previous run, we skip the URL entirely.

```

def compute_hash(text: str) -> str:
    """Return the SHA-256 hex digest of a string."""
    return hashlib.sha256(text.encode("utf-8")).hexdigest()

def load_baseline(baseline_path: Path) -> dict:
    """Load the JSON baseline mapping URLs to their last-known
hashes.

    Returns an empty dict if the file does not exist yet.
    """
    if not baseline_path.exists():
        return {}
    with open(baseline_path, "r", encoding="utf-8") as f:
        return json.load(f)

def save_baseline(baseline: dict, baseline_path: Path) -> None:
    """Save the baseline dictionary to disk as JSON."""
    baseline_path.parent.mkdir(parents=True, exist_ok=True)

```

```
with open(baseline_path, "w", encoding="utf-8") as f:
    json.dump(baseline, f, indent=2)
```

The baseline.json file is a simple dictionary where keys are URLs and values are hashes. On the first run, every URL is new. On subsequent runs, only changed URLs proceed to the next stage.

Wait — I said HTML is noisy. A raw SHA-256 of the HTML will produce false negatives because of dynamic tokens. To reduce that, let's strip scripts and styles and collapse whitespace before hashing. For this project, httpx returns the HTML; we won't do full DOM parsing, but we *will* do a lightweight regex cleanup to remove obvious noise.

```
import re

def clean_text(text: str) -> str:
    """Remove script/style tags and collapse whitespace for stable
    hashing."""
    text = re.sub(r"<script[^>]*>.*?</script>", "", text,
        flags=re.DOTALL)
    text = re.sub(r"<style[^>]*>.*?</style>", "", text,
        flags=re.DOTALL)
    text = re.sub(r"\s+", " ", text)
    return text.strip()
```

It's not perfect. Dynamic content like stock tickers or user-specific greetings will still shift the hash. That's okay. Our LLM filter downstream is designed to handle occasional false positives. What we want to avoid is burning API credits on completely static pages.

Step 4: LLM Filtering

This is where the agent thinks. We feed the model two chunks of text: the old version (or a summary note that it's new) and the new version. We ask it to classify the change and decide whether it matters.

```
def build_filter_prompt(previous_text: str, current_text: str,
    url: str) -> str:
    """Build a prompt that asks the model to classify page
    changes."""
    return (
        "You are a competitive intelligence analyst. Your job is\n"
        "to review "\n"
        "changes on a competitor's public webpage and decide what\n"
        "matters.\n\n"
        f"URL: {url}\n\n"
        f"PREVIOUS VERSION:\n{previous_text[:4000]}\n\n"
```

```

f"CURRENT VERSION:\n{current_text[:4000]}\n\n"
"Instructions:\n"
"1. Identify any meaningful changes. Ignore typos, date
stamps, "
"and layout-only differences.\n"
"2. Classify meaningful changes into one or more of these
categories:\n"
"    - pricing_change\n"
"    - feature_launch\n"
"    - leadership_change\n"
"    - funding_round\n"
"    - partnership\n"
"    - other_significant\n"
"3. For each category found, write one concise sentence
describing the change.\n"
"4. Assign an overall severity: low, medium, or high.\n"
"5. If nothing meaningful changed, severity must be
'none'.\n\n"
"Respond ONLY with valid JSON in this exact shape:\n"
'{"severity": "low|medium|high|none", "brief": "summary
sentence", '
'  "categories": [{"type": "pricing_change", "note":
"..."}]}'
)

```

The prompt is designed to be defensive. We truncate both versions to 4,000 characters to stay within token limits. We explicitly ask the model to ignore typos and layout changes. We force structured JSON output so we can parse the result without guessing.

We'll call this prompt against our local model via Ollama or against a cloud model if we want sharper reasoning. The function below wraps the call to Ollama's generate API.

```

OLLAMA_URL = "http://localhost:11434/api/generate"
FILTER_MODEL = "llama3.1:8b"

```

```

def ask_llm(prompt: str, model: str = FILTER_MODEL) -> dict:
    """Send a prompt to the local Ollama API and return parsed
    JSON.

```

```

    Falls back to an empty 'none' result if the model returns
    invalid JSON.
    """

```

```

    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False,
        "format": "json",
    }

```

```

try:
    response = httpx.post(OLLAMA_URL, json=payload,
timeout=120.0)
    response.raise_for_status()
    data = response.json()
    raw = data.get("response", "")
    parsed = json.loads(raw)
    # Ensure expected keys exist
    parsed.setdefault("severity", "none")
    parsed.setdefault("brief", "")
    parsed.setdefault("categories", [])
    return parsed
except (httpx.HTTPError, json.JSONDecodeError, KeyError) as
exc:
    print(f"[LLM error] {exc}")
    return {"severity": "none", "brief": "", "categories": []}

```

Notice the format: "json" parameter. Ollama's newer versions support constrained output, which improves the odds of getting parseable JSON from smaller models. If you're on an older Ollama version, remove that line and parse more defensively.

Step 5: Slack Alert with Structured Brief

When the LLM returns a severity of medium or high, we alert the team. For low, we log quietly. For none, we do nothing.

```

def send_slack_alert(webhook_url: str, brief: dict, url: str) ->
None:
    """Post a competitive intelligence brief to a Slack incoming
webhook.

    webhook_url should be something like:
    https://hooks.slack.com/services/T000000000/B000000000/
XXXXXXXXXXXXXXXXXXXXXXXXXXXXX
    """
    if not webhook_url:
        print("[alert] No Slack webhook configured. Skipping
alert.")
        return

    severity = brief.get("severity", "none")
    summary = brief.get("brief", "No summary provided.")
    categories = brief.get("categories", [])

    category_lines = "\n".join(
        f"• {c['type']}: {c['note']}" for c in categories
    )

```

```

    emoji = {"high": ":rotating_light:", "medium": ":warning:",
"low": ":memo:"}.get(
        severity, ":white_check_mark:"
    )

    message = (
        f"{emoji} *Competitor Alert* – {severity.upper()}\n"
        f"*URL:* {url}\n"
        f"*Summary:* {summary}\n"
        f"*Categories:* \n{category_lines}"
    )

    payload = {"text": message}
    try:
        resp = httpx.post(webhook_url, json=payload, timeout=30.0)
        resp.raise_for_status()
        print(f"[alert] Slack notified for {url}")
    except httpx.HTTPError as exc:
        print(f"[alert] Failed to send Slack message: {exc}")

```

If you use Microsoft Teams instead of Slack, the concept is identical. Teams accepts a simpler JSON payload at its webhook URL. Swap the payload shape and you're done.

Step 6: Putting It All Together

Now we wire the stages into a single `main()` function that monitors two or three public URLs. In a real deployment, you'd keep these in a config file or environment variable. Here, we'll use a small list for demonstration.

```

import sys

# Configuration
TARGET_URLS = [
    "https://example-competitor.com/pricing",
    "https://another-competitor.com/blog",
    "https://third-competitor.com/about",
]
BASELINE_PATH = Path("baseline.json")
SLACK_WEBHOOK = os.environ.get("SLACK_WEBHOOK_URL", "")

def run_monitor() -> None:
    """Execute one full pass of the competitive intelligence
    monitor.

    1. Load the baseline hash file.

```

```

2. For each target URL, check robots.txt and then fetch.
3. Compare hashes. If changed, ask the LLM to classify.
4. Alert on Slack for medium/high severity.
5. Save the new baseline.
"""

baseline = load_baseline(BASELINE_PATH)
new_baseline = dict(baseline)

for url in TARGET_URLS:
    print(f"\n[monitor] Checking {url}")

    if not is_url_allowed(url):
        print(f"[monitor] Skipping {url} – disallowed by
robots.txt")
        continue

    try:
        html = fetch_page(url)
    except httpx.HTTPStatusError as exc:
        print(f"[monitor] HTTP error for {url}: {exc}")
        continue
    except httpx.RequestError as exc:
        print(f"[monitor] Network error for {url}: {exc}")
        continue

    cleaned = clean_text(html)
    current_hash = compute_hash(cleaned)
    previous_hash = baseline.get(url)

    if previous_hash == current_hash:
        print(f"[monitor] No change for {url}")
        new_baseline[url] = current_hash
        continue

    print(f"[monitor] Change detected for {url}")

    # For the first run, previous_text is empty
    previous_text = baseline.get(f"{url}_text", "")
    if len(previous_text) > 4000:
        previous_text = previous_text[:4000] + "\n...
[truncated]"
    if len(cleaned) > 4000:
        current_text = cleaned[:4000] + "\n...[truncated]"
    else:
        current_text = cleaned

    prompt = build_filter_prompt(previous_text, current_text,
url)
    result = ask_llm(prompt)

    severity = result.get("severity", "none")
    print(f"[monitor] Severity: {severity}")

```

```

        if severity in ("medium", "high"):
            send_slack_alert(SLACK_WEBHOOK, result, url)
        elif severity == "low":
            print(f"[monitor] Low severity logged:
{result.get('brief')}")
        else:
            print("[monitor] No significant signal. Noise
filtered.")

        # Store both hash and truncated text for next comparison
        new_baseline[url] = current_hash
        new_baseline[f"{url}_text"] = cleaned[:8000]

    save_baseline(new_baseline, BASELINE_PATH)
    print("\n[monitor] Run complete. Baseline updated.")

def main() -> None:
    """Entry point. Run the monitor once immediately."""
    run_monitor()

if __name__ == "__main__":
    main()

```

Run it once to establish baselines:

```
python competitive_monitor.py
```

On the first pass, every URL will be “new,” so the previous text will be empty. The LLM will classify each page as a baseline scan. Subsequent runs will only surface actual deltas.

Scheduling It

Running manually defeats the purpose. We’ll use the schedule library for a lightweight in-process scheduler, which is great for development and small VPS deployments.

```
pip install schedule
```

Add this to the bottom of your script, replacing the `if __name__ == "__main__"` block:

```
import schedule
```

```
def scheduled_job() -> None:
```

```

        """Wrapper so schedule doesn't complain about return
        values."""
        run_monitor()

if __name__ == "__main__":
    # Run every Monday at 08:00
    schedule.every().monday.at("08:00").do(scheduled_job)

    # Also run immediately on startup for a baseline
    run_monitor()

    print("[scheduler] Monitor running. Press Ctrl+C to exit.")
    while True:
        schedule.run_pending()
        time.sleep(60)

```

If you're deploying on a server, you might prefer a real cron job instead of a long-running Python process. In that case, keep the `main()` entry point and add this line to your crontab:

```

0 8 * * 1 /usr/bin/python3 /home/user/competitive_monitor.py >> /
home/user/monitor.log 2>&1

```

The cron approach is simpler and doesn't keep a process resident. The schedule approach is easier to test locally. Either works.

Break and Fix: Five Ways Competitive Intelligence Breaks

Break 1: Page Structure Changed — CSS Selectors Break

Suppose you weren't hashing the whole page. Suppose you were extracting specific sections with BeautifulSoup and CSS selectors, like `soup.select_one("div.pricing-table")`. Then the competitor redesigns their site. The class names change. Your selector returns `None`. The script crashes with an `AttributeError` because you called `.get_text()` on a null object.

Fix: This is exactly why our baseline approach hashes the full cleaned text instead of scraping structured elements. If you *must* extract specific sections, always check for `None` and fall back to the full page:

```

from bs4 import BeautifulSoup

def safe_extract(html: str, selector: str) -> str:
    """Extract text via CSS selector, falling back to full page

```



```

text."""
    soup = BeautifulSoup(html, "html.parser")
    element = soup.select_one(selector)
    if element:
        return element.get_text(separator=" ", strip=True)
    print(f"[warn] Selector '{selector}' missed. Using full
page.")
    return clean_text(html)

```

Better yet, avoid selectors for change detection entirely. Use them only for display formatting after you already know the page changed.

Break 2: False Positive on Routine Marketing

Your competitor posts a new blog every Tuesday. It's a "product update" about a UI color tweak. Your LLM sees the word "launch" and flags it as `feature_launch` with medium severity. Your Slack channel fills with noise. People mute the channel. The monitor becomes useless.

Fix: Refine the prompt. Be explicit about what does *not* count:

Ignore routine blog posts, minor UI updates, employee spotlights, and marketing language that does not announce a new capability or price.

Also add a post-processing rule in code. If the same category fires for the same URL three times in a row, suppress it until the next calendar month. Noise reduction is as important as signal detection.

Break 3: Legal Concern — Terms of Service Violation

You add a competitor's pricing page to your monitor. Later, you actually read their Terms of Service and find this line: *"You may not use any robot, spider, scraper, or other automated means to access our site for any purpose."* Your monitor is, by their definition, a robot.

Fix: Stop monitoring that URL. Full stop. Move that competitor to manual review or subscribe to their newsletter. The ToS is a contract between the site and its visitors. Violating it can get your IP banned, and in extreme cases, it can expose you to civil claims under the Computer Fraud and Abuse Act in the United States or equivalent statutes elsewhere. No pricing insight is worth a legal notice. The `is_url_allowed` check we wrote handles `robots.txt`, but it does *not* read ToS. That part is on you. Make a spreadsheet. Mark which competitors allow automated access and which don't. Respect the flags.

Break 4: Rate Limit on Target Site

Your VPS is fast. The monitor fetches three URLs, then five more you added, then retries quickly because one timed out. Suddenly every request returns HTTP 429 — “Too Many Requests.” The competitor’s CDN has rate-limited your IP. Your baseline falls out of date.

Fix: Add explicit delays and jitter between requests. Never fetch more than one page per domain per minute unless you have permission.

```
import random
```

```
# Inside the loop, after each fetch
time.sleep(random.uniform(2.0, 5.0))
```

If you hit a 429, back off exponentially:

```
retries = 0
while retries < 3:
    try:
        html = fetch_page(url)
        break
    except httpx.HTTPStatusError as exc:
        if exc.response.status_code == 429:
            wait = 2 ** retries
            print(f"[monitor] Rate limited. Sleeping {wait}s...")
            time.sleep(wait)
            retries += 1
        else:
            raise
```

Respect the Retry-After header if the server sends one. Being a good citizen keeps your IP unblocked and your conscience clean.

Break 5: Competitor Uses Anti-Bot Measures (CAPTCHA, Blocking)

You run the monitor one morning and every response is a Cloudflare challenge page. HTML contains no useful text, just a JavaScript challenge. Your diff logic sees a massive change and alerts you that the competitor “launched” something. They didn’t. Their firewall just woke up.

Fix: Do not bypass CAPTCHAs. Do not pay for CAPTCHA-solving services. Do not impersonate a human more aggressively. If a competitor deploys serious anti-bot protection, they have drawn a line. Crossing it is a legal and ethical risk. Instead, switch sources. Monitor their press releases via RSS, their CEO’s public LinkedIn posts, or their

SEC filings. Public intelligence doesn't have to come from their homepage. Part of being a good competitive intelligence analyst is knowing when a source has dried up and finding a legitimate alternative.

Try It Now: Three Variations

Variation 1: Sentiment Analysis on News Mentions

Instead of monitoring competitor homepages, monitor news. Use an RSS feed or a news API (like NewsAPI or GNews) to search for mentions of your competitor by name. Feed the article text into the LLM with a prompt that asks for sentiment and key themes. Track sentiment over time in a CSV. Are they getting good press? Bad press? Neutral product reviews?

Variation 2: Patent Monitoring via USPTO

The United States Patent and Trademark Office publishes new patent applications weekly via the Patent Public Search API. Search for your competitor's name as assignee or inventor. When a new patent appears, have the LLM summarize the claims in plain English and flag whether it overlaps with your product area. This is slower-moving than pricing changes, but the strategic signal is enormous.

Variation 3: Price Comparison Matrix Updated Daily

If you and your competitors publish public pricing, extract pricing tiers with a more structured approach. Use beautifulsoup4 to find tables or headings that contain dollar signs, then feed those snippets to the LLM and ask for structured JSON like {"tier": "Pro", "price": 49, "currency": "USD", "billing": "monthly"}. Store results in a CSV and generate a weekly markdown comparison table. This is higher-maintenance than simple change detection, but the output is directly actionable for sales teams.

Ethics Sidebar: robots.txt, Terms of Service, Fair Use, Don't Be Creepy

Let's be blunt. The internet is full of gray areas. robots.txt is technically a voluntary standard. Not every court has ruled on whether violating it is illegal. Terms of service are contracts of adhesion that most users never read. Fair use allows limited reproduction for commentary and analysis. These ambiguities tempt people to push boundaries.

Don't.

Build your monitor as if the competitor's general counsel will read your source code. Because if you cross the line, they might. Here's a checklist:

☐

Did I check robots.txt before adding the URL?

☐

Did I read the ToS for explicit anti-scraping language?

☐

Am I only accessing public, unauthenticated pages?

☐

Is my request rate reasonable (seconds between requests, not milliseconds)?

☐

Am I storing only the minimum data needed for comparison?

☐

If challenged, could I defend this as independent research on public facts?

If you can't check every box, remove the URL. Fair use protects analysis, not bulk replication. Don't mirror their site. Don't republish their press releases verbatim in your Slack. Summarize, analyze, and move on. And if a human analyst wouldn't do it with a browser and a notepad, your agent shouldn't do it either.

The OpenClaw Way

OpenClaw handles the loop, the tool registry, and the JSON parsing for you. For this project, the value is in structuring the monitor as three discrete tools: `fetch_page`, `detect_changes`, and `alert_team`. OpenClaw's tool decorator makes this clean, and its built-in conversation history means the model can remember context across multiple competitor checks within the same run.

```
from openclaw import Agent, tool
import hashlib

@tool
def fetch_page(url: str) -> str:
    """Fetch a URL and return cleaned text."""
    ...

@tool
def save_hash(url: str, text: str) -> str:
    """Store a hash for baseline comparison."""
    ...

agent = Agent(tools=[fetch_page, save_hash], reasoning="react")
agent.run("Check all competitors in config.yaml and alert on
significant changes.")
```

OpenClaw also provides a schedule integration via its CLI, so you can deploy the monitor as `openclaw run --schedule weekly` without writing the cron logic yourself. For teams already using OpenClaw, this is the fastest path to production.

The Hermes Agent Way

Hermes Agent approaches this as a persistent research assistant rather than a scheduled script. You define a “watchlist” in Hermes's configuration — a list of URLs plus rules about what constitutes a signal. Hermes fetches, stores its own baseline in its working memory, and surfaces changes during your conversation with it.

```
hermes
> Add https://example-competitor.com/pricing to my watchlist.
> Check all watchlist URLs now.

[fetch] example-competitor.com — 2.3 KB fetched.
[compare] Hash changed. Analyzing...
```

```
[result] Pricing tier "Enterprise" increased from $99 to $129.  
> Summarize all pricing changes from the last month.
```

Hermes doesn't need a separate Slack integration because it can post to Slack natively via its toolset. The trade-off is less granular control over headers, backoff timing, and robots.txt parsing. If Hermes is blocked, you'll need to fall back to manual configuration or accept the limitation. It's a conversation-first approach that shines when you want intelligence on demand, not necessarily at 8 AM every Monday.

What You Built

In this chapter, you built a competitive intelligence monitor from first principles:

- A robots.txt checker that respects site ownership before fetching.
- An httpx-based fetcher with realistic browser headers and error handling.
- A hash-based diff system that filters out unchanged pages before burning LLM tokens.
- An LLM filter prompt that classifies changes into pricing, feature, leadership, funding, and partnership signals.
- A Slack webhook alert that formats structured briefs with severity and emoji.
- A schedule-based runner (and cron equivalent) for hands-off weekly operation.
- Defensive code for rate limits, HTTP errors, and dynamic page noise.
- A main loop that monitors three example URLs end to end.

You also learned when *not* to scrape: when the ToS forbids it, when anti-bot measures block you, or when the signal-to-noise ratio collapses into useless alerts. Competitive intelligence is a discipline of patience and boundaries, not aggression.

Cost for this chapter: Approximately \$0.00 if you use Ollama for the LLM filter and run weekly. If you substitute GPT-4o mini, expect roughly \$0.01–\$0.03 per changed URL per run, depending on page

length. On a typical week with one or two meaningful changes across three competitors, that's less than a dime. Compare that to the cost of losing a customer because you missed a pricing shift.

Chapter 22: Project 22 — The Ethical Guardian: AI Safety Auditor

Every response from your company's chatbot, reviewed for bias, toxicity, and truth. Automatically.

I once watched a customer-facing chatbot tell a user that their government ID number “looked fake.” The user had pasted their Social Security number into the conversation, and the model, trying to be helpful, commented on the format. In one sentence, the bot had done two catastrophic things: it processed personally identifiable information as if it were public data, and it made an unfounded judgment that could be interpreted as discriminatory. Nobody programmed it to do that. It was a statistical accident. But accidents at scale become liabilities.

That incident cost the company forty hours of incident response, a customer complaint escalated to the legal team, and a lot of trust. The worst part? The team had a “safety” ticket sitting in their backlog for three months. They had planned to add content filtering. They had intended to build an audit system. Safety was a feature on a roadmap, and the chatbot shipped without it.

That is the central lesson of this chapter. AI safety is not a feature you bolt on later. It is architecture. It is the shape of the pipeline, the presence of the audit log, the rigor of the checks that run on every single output before a user ever sees it. In this project, you will build an Ethical Guardian: a Python auditor that inspects every response from an LLM application, flags problems in four categories, records every finding, and generates a compliance report you could actually show an auditor.

We are not building a perfect system. We are building a *necessary* system. Let's start.

AI Safety Is Architecture, Not a Feature

A “feature” is optional. You can ship version 1.0 without dark mode and add it in 1.1. A safety layer is not optional in the same way. If your agent handles user data, talks to customers, or generates content that influences decisions, every unguarded response is a roll of the dice. The dice have thousands of sides, and most of them are fine. But some of them are PII leakage, biased language, toxic outputs, or factual contradictions.

The Ethical Guardian treats every LLM output as untrusted by default. Before any response reaches a user, it passes through a parallel audit pipeline. Each check runs independently: profanity detection, PII scanning, bias keyword detection, and factual coherence verification. If any check flags the output, the system records a structured finding, assigns a severity, optionally blocks the response, and includes the incident in a weekly compliance report.

This is architecture because it sits between the model and the world. It does not ask the model to be safe. It assumes the model is unsafe and proves otherwise with evidence.

Monitoring Every Output

First, we need something to monitor. In a real system, the Ethical Guardian would sit as middleware in a web service — every response from your /chat endpoint would flow through the auditor before returning to the client. In our learning environment, we will simulate this by defining a list of raw LLM responses and running them through the audit pipeline.

Create a file named `ethical_guardian.py`. Here is the skeleton:

```
import json
import re
import datetime
from dataclasses import dataclass, asdict
from typing import List, Optional

@dataclass
class FlaggedItem:
    """A single finding from the safety audit."""
    text_sample: str
    category: str
    severity: str # low, medium, high, critical
```



```
suggestion: str
timestamp: str
```

The FlaggedItem dataclass is the atomic unit of our audit. Every time a check detects a problem, it produces one of these objects. The text_sample stores a short excerpt (never the full conversation, to avoid duplicating PII in logs). The category tells us which check found the problem. The severity drives alerting and escalation. The suggestion tells a human what to do next. The timestamp makes the log forensically useful.

Now the auditor itself:

```
class SafetyAuditor:
    """
    Audit LLM outputs for profanity, PII leakage, bias keywords,
    and factual coherence. Maintains a JSON audit log and can
    generate a markdown compliance report.
    """

    def __init__(self, audit_log_path: str = "audit_log.json"):
        self.audit_log_path = audit_log_path
        self.flagged: List[FlaggedItem] = []
        self._load_log()

    def _load_log(self) -> None:
        """Load existing audit log from disk, if present."""
        try:
            with open(self.audit_log_path, "r", encoding="utf-8")
as f:
                data = json.load(f)
                self.flagged = [FlaggedItem(**item) for item in
data]
        except FileNotFoundError:
            self.flagged = []

    def _save_log(self) -> None:
        """Persist the audit log to JSON."""
        with open(self.audit_log_path, "w", encoding="utf-8") as
f:
            json.dump([asdict(item) for item in self.flagged], f,
indent=2)

    def audit(self, response_id: str, text: str) ->
List[FlaggedItem]:
        """
        Run all safety checks against a single LLM response.
        Returns a list of FlaggedItem objects.
        """
        findings: List[FlaggedItem] = []
        findings.extend(self.check_profanity(response_id, text))
```

```

        findings.extend(self.check_pii(response_id, text))
        findings.extend(self.check_bias(response_id, text))
        findings.extend(self.check_coherence(response_id, text))
        self.flagged.extend(findings)
        self._save_log()
        return findings

def is_flagged(self, text: str) -> bool:
    """Quick check: run all checks without logging."""
    dummy = "quick-check"
    return any([
        self.check_profanity(dummy, text),
        self.check_pii(dummy, text),
        self.check_bias(dummy, text),
        self.check_coherence(dummy, text),
    ])

```

Notice the design. The `audit()` method runs all four checks and persists to disk. The `is_flagged()` method exists for real-time middle-ware usage: it returns a boolean fast so the application can block or redirect without waiting for disk I/O. We will come back to that trade-off in the Break and Fix section.

Check 1: Profanity

Profanity detection is the simplest check and the easiest to get dangerously wrong. We are not trying to police language. We are trying to catch outputs that are hostile, abusive, or sexually explicit, because those outputs create legal and reputational risk.

We will use a simple word-list approach with regex boundaries to avoid catching partial matches inside legitimate words. This is not state-of-the-art — a real production system might use a transformer-based classifier — but it is transparent, explainable, and requires no ML dependencies.

```

def check_profanity(self, response_id: str, text: str) ->
List[FlaggedItem]:
    """
    Scan text for profane or toxic language using word-
    boundary regex.
    """
    profane_words = [
        r"\bf+u+c+k+",
        r"\bs+h+i+t+",
        r"\bd+a+m+n+",
        r"\bh+e+l+l+\b",
        r"\bb+i+t+c+h+",
    ]

```

```

        r"\ba+s+s+h+o+l+e",
    ]
    findings = []
    for pattern in profane_words:
        for match in re.finditer(pattern, text,
re.IGNORECASE):
            findings.append(FlaggedItem(
                text_sample=match.group(0),
                category="profanity",
                severity="high",
                suggestion="Block response and trigger human
review.",

timestamp=datetime.datetime.utcnow().isoformat(),
            ))
    return findings

```

I deliberately kept the regexes case-insensitive and used word boundaries where appropriate. `re.IGNORECASE` means “Damn” and “damn” both match. The `\b` anchors prevent us from flagging “hell” inside “shell” or “hello.”

If you are building for a global audience, you will need locale-specific lists. Profanity is cultural. A word that is mild in one region is obscene in another. For this project, the lesson is the pattern, not the particular word list.

Check 2: PII Leakage

The most expensive mistake an LLM application can make is leaking personally identifiable information. If a model echoes back a user’s Social Security number, credit card, or email address, you may have legal obligations under GDPR, HIPAA, or state privacy laws.

Our PII scanner uses regex patterns for three common formats: SSN, email, and US phone numbers.

```

def check_pii(self, response_id: str, text: str) ->
List[FlaggedItem]:
    """
    Scan text for leaked PII: SSN, email addresses, and phone
numbers.
    """
    findings = []
    # SSN: 123-45-6789 or 123 45 6789
    ssn_pattern = r"\b\d{3}[-\s]?d{2}[-\s]?d{4}\b"
    for match in re.finditer(ssn_pattern, text):
        findings.append(FlaggedItem(

```

```

        text_sample=match.group(0),
        category="pii_leak",
        severity="critical",
        suggestion="Redact immediately. Review prompt for
PII injection.",
        timestamp=datetime.datetime.utcnow().isoformat(),
    ))

    # Email: local@domain.tld
    email_pattern = r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-
Z|a-z]{2,}\b"
    for match in re.finditer(email_pattern, text):
        findings.append(FlaggedItem(
            text_sample=match.group(0),
            category="pii_leak",
            severity="critical",
            suggestion="Redact immediately. Review prompt for
PII injection.",
            timestamp=datetime.datetime.utcnow().isoformat(),
        ))

    # US Phone: (555) 123-4567, 555-123-4567, 555.123.4567,
etc.
    phone_pattern = r"\b(?:\+?1[-.\s]?)(?(\d{3})\)?[-.\s]?
\d{3}[-.\s]? \d{4})\b"
    for match in re.finditer(phone_pattern, text):
        findings.append(FlaggedItem(
            text_sample=match.group(0),
            category="pii_leak",
            severity="high",
            suggestion="Mask phone number before display.",
            timestamp=datetime.datetime.utcnow().isoformat(),
        ))
    return findings

```

Every finding here is severity critical or high, because PII leakage is never trivial. The suggestion field tells the operator exactly what to do: redact and investigate the prompt. If a model is echoing back a Social Security number, the user probably typed it into the prompt, which means the system is storing or processing raw SSNs in plaintext. That is a data architecture problem, not just a content problem.

Check 3: Bias Keywords

Bias detection is the messiest check in the pipeline. Human bias is contextual. The same word can be neutral in one sentence and loaded in another. A medical diagnostic tool should not flag “African

American” as biased just because it appears in a word list. A hiring assistant should flag “young and energetic” because that phrase has been used to illegally screen out older applicants.

Our approach is intentionally simple: a keyword-and-phrase list combined with severity tiers. The goal is not to replace human judgment. The goal is to force a human to look.

```
def check_bias(self, response_id: str, text: str) ->
List[FlaggedItem]:
    """
    Scan text for potentially biased language using keyword
    lists.
    """
    findings = []
    high_risk_bias = [
        r"\bfemales? are (worse|better|less|more)\b",
        r"\bm(ales?|en) are (worse|better|less|more) than\b",
        r"\bpeople like you\b",
        r"\byour kind\b",
        r"\bthose people\b",
    ]
    for pattern in high_risk_bias:
        for match in re.finditer(pattern, text,
re.IGNORECASE):
            findings.append(FlaggedItem(
                text_sample=match.group(0),
                category="bias",
                severity="high",
                suggestion="Review for discriminatory
generalization.",
                timestamp=datetime.datetime.utcnow().isoformat(),
            ))
    return findings
```

We use regex patterns rather than static strings so we can catch phrases like “men are more” or “women are less” — the kinds of comparative generalizations that generate legal risk in hiring, lending, and medical contexts.

A word of caution: I have seen systems where the bias checker flags every demographic mention as “high risk.” That produces alert fatigue. The operators stop reading the alerts. The system becomes theater. We will deal with that exact failure mode in Break and Fix.

Check 4: Factual Coherence

How do you check whether a model output is factually coherent without maintaining a ground-truth database for every possible claim? One lightweight method is self-consistency: if you ask the model the same question multiple times with slightly different phrasing, and you get contradictory answers, at least one of them is wrong. That contradiction is a signal that the output needs review.

In our pure-Python pipeline, we simulate self-consistency by comparing two generated responses. In a real deployment, `check_coherence` would call the LLM twice with temperature greater than zero, then compare the answers with a similarity metric or a secondary LLM judge.

```
def check_coherence(self, response_id: str, text: str) ->
List[FlaggedItem]:
    """
        Check factual coherence via self-consistency by comparing
        the provided text against a second sample for
contradictions.
        In a real system, the second sample would come from a
fresh
        LLM call with the same input.
    """
    findings = []
    # For the learning example, simulate a second response
    # by extracting key claims and comparing them.
    second_sample = self._generate_second_sample(response_id)
    contradictions = self._extract_contradictions(text,
second_sample)
    for contradiction in contradictions:
        findings.append(FlaggedItem(
            text_sample=contradiction,
            category="coherence",
            severity="medium",
            suggestion="Run fact-check or require human
verification.",
            timestamp=datetime.datetime.utcnow().isoformat(),
        ))
    return findings

def _generate_second_sample(self, response_id: str) -> str:
    """Simulate a second LLM response for self-consistency
checking."""
    # In production, this would call the LLM API again.
    # Here we return a plausible alternate to demonstrate
logic.
    return "The capital of France is Berlin."
```

```

    def _extract_contradictions(self, text_a: str, text_b: str) ->
List[str]:
    """
        Naive contradiction detection: look for opposing factual
claims
        such as different capitals or conflicting dates.
    """
    contradictions = []
    # Naive heuristic: if one text says Paris and the other
says Berlin
    if "paris" in text_a.lower() and "berlin" in
text_b.lower():
        contradictions.append("Conflicting capitals: Paris vs
Berlin")
    return contradictions

```

The `_generate_second_sample` method is a stub. In a real system, you would pass the original user message to the LLM again, maybe with `temperature=0.7`, collect the new response, and compare key assertions. The comparison can be as simple as an embedding cosine similarity or as complex as a structured extraction pass followed by a boolean logic check.

Even this naive version teaches the right architectural habit: never trust a single sample. Models are stochastic. Coherence across samples is evidence of reliability.

The Audit Log and Compliance Report

Every flagged item is written to `audit_log.json`. That file is append-only in spirit — we load it, append new findings, and write it back. For production scale you would move to SQLite or a proper logging service, but JSON is perfect for learning and small teams.

The compliance report turns the raw log into something a human manager or regulator can read.

```

    def generate_report(self, title: str = "Weekly Safety
Compliance Report") -> str:
    """
        Generate a markdown report summarizing all flagged items.
    """
    lines = [f"# {title}", ""]
    lines.append(f"Generated:
{datetime.datetime.utcnow().isoformat()}")
    lines.append(f"Total findings: {len(self.flagged)}")
    lines.append("")

```

```

categories: dict = {}
for item in self.flagged:
    categories.setdefault(item.category, []).append(item)

for category, items in sorted(categories.items()):
    lines.append(f"## {category.upper()}")
    lines.append(f"Count: {len(items)}")
    lines.append("")
    for item in items:
        lines.append(f"- **Severity:** {item.severity}")
        lines.append(f"  - Sample: `{item.text_sample}`")
        lines.append(f"  - Suggestion: {item.suggestion}")
        lines.append(f"  - Time: {item.timestamp}")
        lines.append("")
    return "\n".join(lines)

```

The report groups by category, shows severity, and preserves the human-readable suggestion. It does not include the full conversation transcript, because transcripts might contain the very PII we are trying to protect. This is a deliberate privacy-by-design choice.

The Audit Pipeline

Before we run this, let's look at how data flows through the system.

```

flowchart LR
    LLM[LLM Output] --> P[Profanity Check]
    LLM --> PI[PII Scan]
    LLM --> B[Bias Check]
    LLM --> C[Coherence Check]
    P --> F{Flagged?}
    PI --> F
    B --> F
    C --> F
    F -- yes --> AL[Audit Log + Alert]
    F -- no --> PASS[Pass to User]

```


Every output fans out into four parallel checks. The checks are independent, so one finding does not stop the others from running. If anything is flagged, the output is blocked or annotated, the finding goes to the audit log, and an alert is emitted. If nothing is flagged, the output passes to the user. There is no shortcut around this pipeline. Safety is not a branch. It is the road itself.

Running the Guardian

Here is the main() function that audits sample responses:

```
def main() -> None:
    """
    Run the SafetyAuditor against a set of simulated LLM responses
    and print a compliance report.
    """
    auditor = SafetyAuditor(audit_log_path="audit_log.json")

    sample_responses = [
        (
            "r1",
            "The capital of France is Paris, a beautiful city."
        ),
        (
            "r2",
            "You can reach me at john.doe@example.com or call
555-123-4567."
        ),
        (
            "r3",
            "Women are less capable than men in technical roles."
        ),
        (
            "r4",
            " damn it, this is frustrating."
        ),
        (
            "r5",
            "The capital of France is Berlin."
        ),
    ]

    for response_id, text in sample_responses:
        print(f"\nAuditing {response_id}...")
        findings = auditor.audit(response_id, text)
        if findings:
            print(f"  Flagged {len(findings)} item(s):")
            for f in findings:
                print(f"    [{f.severity}] {f.category}")
```

```

{f.text_sample}")
    else:
        print("  Clean.")

    print("\n" + "=" * 50)
    print(auditor.generate_report())

if __name__ == "__main__":
    main()

```

Run it:

```
python ethical_guardian.py
```

You will see the PII-heavy response trigger two critical findings, the biased statement trigger a high-severity flag, the profanity trigger another, and the coherence check catch the incorrect capital in r5. The report at the bottom summarizes everything by category.

This is your baseline. It works. It is not fancy. But it runs on every single response, costs nothing in API calls, and produces evidence you can point to when someone asks, “How do you know your bot is safe?”

Break and Fix: Five Ways This Project Can Fail

Break 1: False Positive on Bias

You add a demographic keyword like “Black” to the bias list to catch racist generalizations. Suddenly every response that mentions Black history month, Black patients, or Black Friday is flagged as high-severity bias. The alert queue floods. Your team stops reading it.

Fix: Do not flag words. Flag *patterns of comparison and generalization*. Our regex looks for phrases like “are less capable than” or “your kind.” Never add standalone demographic identifiers to a blocklist. If you genuinely need demographic awareness, build an allowlist of benign contexts (medical, historical, census) and exclude them from reporting.

Break 2: PII Not Caught in a New Format

A user pastes their Social Security number as 123.45.6789 with dots. Your SSN regex only checks for dashes. The number sails through. Two weeks later, a log analysis reveals thousands of SSNs in plaintext chat history.

Fix: Expand the SSN regex, but more importantly, treat any nine-digit sequence as suspicious:

```
ssn_pattern = r"\b\d{3}[\s.\-]?d{2}[\s.\-]?d{4}\b"
```

Better yet, preprocess all inputs and outputs with a general nine-digit scanner before running format-specific regexes. Also: if you never need to store SSNs, write an input sanitizer that strips them before they ever reach the model.

Break 3: The Report Is Too Long and Unreadable

After three days in production, `audit_log.json` contains four thousand findings. The markdown report is ninety pages. No manager will read it. The report becomes a ritual, not a tool.

Fix: Summarize aggressively at the top. Add an executive summary section that shows only counts and trends. Include the full detail in collapsible sections or a separate appendix file. Add filtering by date range and severity threshold. A report that is not read is a report that does not exist.

```
high_critical = [i for i in self.flagged if i.severity in
("high", "critical")]
lines.append(f"High/Critical findings:
{len(high_critical)}")
```

Break 4: Model Card Gaps

Your compliance team asks for a model card: a document showing what the system was tested on, what it fails at, and what its safety metrics are. You realize you never tracked false positive rates, false negative rates, or the demographic distribution of bias flags. You have logs, but no metrics.

Fix: Instrument the auditor with counters. Track total responses scanned, total flagged, and dispositions (true positive, false positive, overridden by human). Store these in a separate metrics file:

```

def record_metrics(self, total: int, flagged: int,
false_positives: int) -> None:
    """Append a daily metrics snapshot to metrics.jsonl."""
    snapshot = {
        "date": datetime.datetime.utcnow().isoformat(),
        "total_responses": total,
        "flagged": flagged,
        "false_positives": false_positives,
        "fp_rate": false_positives / max(total, 1),
    }
    with open("metrics.jsonl", "a") as f:
        f.write(json.dumps(snapshot) + "\n")

```

A model card without metrics is marketing. With metrics, it is evidence.

Break 5: Real-Time Guardrail Causes Too Much Latency

You deploy `is_flagged()` as middleware on every API call. It runs four regex scans on every response. At ten requests per second, the latency spikes to four hundred milliseconds. Your users complain. Your on-call engineer disables the safety checks to “fix” the incident.

Fix: Do not run all checks synchronously if you do not need to. Move the coherence check — which requires a second LLM call — to an asynchronous queue. Run profanity and PII synchronously because they are fast regexes. Run bias and coherence in a background worker that updates the audit log after the user already has their response. If a background check finds something severe, you can send a follow-up alert or retract the message retroactively. Latency and safety are not zero-sum if you architect the pipeline correctly.

Try It Now: Three Variations

Variation 1: Real-Time Guardrail That Blocks Non-Compliant Output

Modify `main()` to wrap the auditor around a simulated chatbot. If any check returns a critical or high severity finding, do not print the response to the user. Instead, print a generic message like: “This response could not be generated due to a safety policy. Please try rephrasing your question.” Log the original response internally for review.

This teaches you the difference between audit-and-log and audit-and-block. Blocking is stronger but risks false-positive user friction. You will need a human escalation queue for blocked messages.

Variation 2: Red-Team Prompt Injection Test

Red-teaming means attacking your own system to find weaknesses. Craft five prompts designed to trick a chatbot into ignoring its safety instructions, leaking PII from its training data, or generating toxic text. Pipe the resulting outputs through your SafetyAuditor. Did it catch all five? If not, examine why and tighten the relevant check.

This is not a theoretical exercise. Red-teaming is now a standard practice in AI safety (see NIST AI RMF and the EU AI Act), and the best way to learn it is to try to defeat your own filters.

Variation 3: Generate a Fairness Model Card with Metrics

Build a script that reads metrics.jsonl and produces a MODEL_CARD.md file. The card should include:

- **Intended Use:** What this LLM application is for.
- **Safety Checks:** Which checks run, how they work, and their known limitations.
- **Performance Metrics:** False positive rate, false negative rate, latency per check.
- **Bias Monitoring:** Which demographic dimensions are monitored and which are not.
- **Known Failures:** Concrete examples of outputs that slipped through.

A model card turns your audit log from an internal log into a public accountability document.

Ethics Sidebar: This Is Hard, Imperfect, and Necessary

Let me be direct with you. The Ethical Guardian we built in this chapter is not enough to make an LLM application safe. Regexes do not understand context. Word lists do not capture the full range of human toxicity, bias, or deception. A determined adversary will find phrasing that evades every pattern in this script.

But “not enough” is not the same as “not worth doing.” Perfect safety is impossible. Structured auditing, transparent checks, and accountable reporting are not. The absence of a safety layer guarantees that your worst-case scenario goes undetected. The presence of a safety layer raises the cost of failure and gives you evidence to learn from.

Every time someone says, “AI safety is too hard to get right, so we should not try,” they are arguing for inevitability over intentionality. I do not accept that trade. Neither should you. Build the imperfect guardrail. Log the incident. Fix the bug. Repeat.

The OpenClaw Way

OpenClaw provides a Guardrail mixin that hooks into the agent’s output stream. Instead of writing a standalone auditor, you register guardrail plugins on the agent itself:

```
from openclaw import Agent, guardrail

@guardrail
def no_pii(text: str) -> Optional[str]:
    if re.search(r"\b\d{3}[-\s]?d{2}[-\s]?d{4}\b", text):
        return "PII_DETECTED"
    return None

agent = Agent(tools=[], guardrails=[no_pii])
```

OpenClaw runs registered guardrails after every turn. If any guardrail returns a non-None value, the agent halts and returns a structured error. The framework also supports async guardrails for expensive checks like multi-sample coherence testing, and it provides a built-in AuditLog store that writes to SQLite by default.

The benefit is tighter integration: you do not have to intercept responses manually. The trade-off is that your safety logic lives inside the framework, so debugging requires understanding OpenClaw's event loop.

The Hermes Agent Way

Hermes Agent does not expose a traditional middleware pipeline because it is conversation-first. However, it supports safety profiles that you can attach to a session. A safety profile is a YAML file listing forbidden patterns, required disclosures, and escalation rules.

```
safety_profile:
  pii_detection: strict
  bias_monitoring: enabled
  audit_target: file://audit_log.json
  on_violation: halt_and_summon
```

When Hermes encounters a violation, it can halt the conversation, summarize the incident, and notify an operator. Because Hermes sessions are persistent, the audit log is automatically scoped to the conversation thread, which simplifies traceability.

The Hermes approach emphasizes policy over code. You are not writing regexes in Python; you are writing rules in configuration. That is powerful for non-engineer compliance officers and limiting for engineers who need custom logic. As always, the choice depends on who owns safety in your organization.

What You Built

In this chapter, you built an Ethical Guardian that:

- Scans every LLM output with four parallel safety checks: profanity, PII leakage, bias keywords, and factual coherence.
- Produces structured `FlaggedItem` findings with severity levels, categories, and actionable suggestions.
- Maintains an append-only JSON audit log for forensic review.
- Generates a human-readable markdown compliance report grouped by category and severity.

- Includes a fast `is_flagged()` path for real-time middleware and an `audit()` path for thorough logging.
- Survived five realistic failure modes: false-positive bias floods, missed PII formats, unreadable reports, model-card metric gaps, and latency-inducing synchronous guardrails.

You also confronted the hardest truth in AI engineering: safety is never finished. It is a process of measurement, failure, and improvement.

Cost for this chapter: Approximately \$0.00 if you run only the regex-based checks. If you implement the self-consistency coherence check against a cloud LLM with two extra calls per response, budget approximately \$0.001–\$0.005 per audited response using GPT-4o mini. Audit logs are plain text and cost nothing to store at small scale.

Chapter 23: Project 23 — Deploy Your Agent: From Script to Service

Your agent works on your laptop. Now it works while you sleep.

I once demoed an agent for my team on a Tuesday afternoon. It answered questions about our codebase, cited the right files, and even made a mildly funny comment about our test coverage. Everyone clapped. Then our director asked: “Great. What’s the URL? The board wants to try it.”

I didn’t have a URL. I had a Python script. I had a terminal window. I had a laptop that went to sleep at 8:47 PM when I closed the lid. The agent died with it.

That is the difference between a script and a service. A script is a pet: you feed it, you start it, you watch it. A service is livestock: it runs whether or not you’re in the room, it reports when it’s sick, and it keeps working after you go home. Your agent deserves to run 24/7. In this chapter, we’re going to turn the agent loop you built in Chapter 1 into a production-ready HTTP service, wrap it in Docker, and add the observability every operator needs.

From Script to Service: What Changes

When you promote a script to a service, five things change. Ignore any of them and your deployment will embarrass you at the worst possible moment—usually when someone important is watching.

Process lifecycle. A script starts when you type `python main.py` and dies when it finishes or when you press Ctrl-C. A service starts when the machine boots and stays up until you deliberately stop it. That means you need a process manager, a container runtime, or at minimum a while True: loop that restarts on crash. We're going to use Uvicorn as our process server and Docker as our packaging format.

State. Your script probably keeps state in global variables or, worse, in your head. A service has no head. It has memory that gets wiped on restart and multiple workers that don't share variables. If your agent tracks total requests in a global integer, that integer resets every time you deploy. If you run two replicas, each has its own integer. We'll solve this by keeping state minimal and using external metrics instead of in-process counters for anything serious.

Error handling. In a script, an unhandled exception prints a traceback and exits. That's fine. In a service, an unhandled exception returns a 500 error to a user who might be on the other side of the world. Worse, if the exception bubbles up to the process level, the whole service restarts, dropping every in-flight request. Every entry point needs a try/except, and every unexpected failure needs to become a structured log line, not a terminal scream.

Configuration. Scripts tolerate hardcoded values because the person who wrote them is right there to edit the file. A service runs on a server you will never ssh into unless something is wrong. Configuration must come from environment variables, mounted files, or a secrets manager—never from source code. We'll use `python-dotenv` for local development and `env vars` for production.

Observability. When a script breaks, you re-run it and watch. When a service breaks at 3 AM, you read logs. If the logs are a wall of unstructured text, you will miss the signal. We will use structured JSON logging and Prometheus metrics so that debugging becomes a query, not a archaeology expedition.

Wrapping an Agent in FastAPI

FastAPI is the framework we'll use to expose our agent over HTTP. It is async-native, automatically generates OpenAPI documentation, and has a type system integration that makes request validation feel like Python, not like filing taxes. It is not the only choice, but it is the right choice for agents that might call slow APIs and need to handle multiple requests concurrently.

Install the dependencies:

```
pip install fastapi uvicorn[standard] pydantic httpx prometheus-client python-json-logger python-dotenv
```

Our project has two files: `agent_core.py`, which contains the same agent loop we built in Chapter 1, and `main.py`, which is the FastAPI service. Separating the agent logic from the HTTP layer is the first rule of service design. If you ever want to swap FastAPI for a queue worker or a CLI, you can do it without touching the agent.

Here is `agent_core.py`, adapted slightly to return structured metadata:

```
"""Core agent logic: ReAct loop backed by an LLM."""
import json
import os
import time

import httpx

OLLAMA_URL = os.environ.get("OLLAMA_URL", "http://localhost:11434/api/generate")
MODEL = os.environ.get("MODEL", "llama3.1:8b")

def get_time(city: str) -> str:
    """Return the current local time for a given city name
    (mock)."""
    return f"The current time in {city} is 2026-04-20T09:15:00
    (mock)."
```

```
def _format_messages(messages: list) -> str:
    """Convert a list of message dicts into a single prompt string
    for Ollama."""
    lines = []
    for msg in messages:
        role = msg.get("role")
        content = msg.get("content", "")
        if role == "system":
```

```

        lines.append(f"System: {content}")
    elif role == "user":
        lines.append(f"User: {content}")
    elif role == "tool":
        lines.append(f"Tool result: {content}")
    elif role == "assistant":
        lines.append(f"Assistant: {content}")
    return "\n\n".join(lines)

```

```

def run_agent(question: str, max_turns: int = 5) -> dict:
    """Run the agent loop and return a result dictionary with
    answer and metadata."""
    tools = {"get_time": get_time}
    start = time.time()

    system_prompt = (
        "You are a helpful assistant with access to tools. "
        "When you need information you don't have, call a tool. "
        "When you have enough information, answer the user
    directly.\n\n"
        "Available tools:\n"
    )
    for name, func in tools.items():
        system_prompt += f"- {name}: {func.__doc__}\n"
    system_prompt += (
        "\nTo call a tool, respond with ONLY a JSON object like:
\n"
        '{"tool": "tool_name", "arguments": {"arg1": "value1"}}\n'
        "To answer the user, respond normally."
    )

    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": question},
    ]

    answer = ""
    for turn in range(max_turns):
        response = httpx.post(
            OLLAMA_URL,
            json={
                "model": MODEL,
                "prompt": _format_messages(messages),
                "stream": False,
            },
            timeout=30.0,
        )
        response.raise_for_status()
        data = response.json()
        assistant_reply = data["response"].strip()

```

```

        try:
            parsed = json.loads(assistant_reply)
            if "tool" in parsed:
                tool_name = parsed["tool"]
                tool_args = parsed.get("arguments", {})
                if tool_name in tools:
                    result = tools[tool_name](**tool_args)
                    messages.append({"role": "assistant",
"content": assistant_reply})
                    messages.append(
                        {"role": "tool", "content": f"Tool
{tool_name} returned: {result}"})
                    continue
                else:
                    answer = f"Error: tool '{tool_name}' not
found."
                    break
            except json.JSONDecodeError:
                pass

            answer = assistant_reply
            break

        duration_ms = (time.time() - start) * 1000
        return {
            "answer": answer,
            "model": MODEL,
            "duration_ms": round(duration_ms, 2),
        }

```

The key difference from Chapter 1 is the return type. Instead of printing to stdout, `run_agent` returns a dictionary. The HTTP layer will decide what to do with that dictionary: serialize it to JSON, log the duration, or increment a counter.

Here is the service architecture we'll build:

graph LR

```

    User -->|POST /ask| FastAPI["FastAPI Service"]
    FastAPI --> Agent["Agent Logic"]
    Agent --> LLM["LLM API"]
    Agent --> Tools["Tools"]
    LLM --> Agent
    Tools --> Agent
    Agent --> FastAPI
    FastAPI -->|JSON Response| User

```

The user sends a question. FastAPI validates it, passes it to the agent loop, the loop talks to the LLM and tools, and the final answer flows back out as JSON. Every layer is isolated, which means you can debug each one without unraveling the others.

Endpoints and Pydantic Models

A service is only as good as its contract. We'll expose three endpoints: one for business logic, one for load balancers, and one for operators who want to know what's happening inside.

POST /ask receives a question and returns an answer. This is the only endpoint that triggers the agent loop.

GET /health returns {"status": "ok"} with a 200 status code. Load balancers and container orchestrators call this hundreds of times a day. If it fails, the orchestrator kills the container and starts a new one.

GET /status returns runtime information: how long the service has been alive and how many requests it has processed. This is for humans, not machines.

Pydantic models enforce the contract. If a client sends a JSON payload without a question field, FastAPI returns a 422 error before our code ever runs. That is safety at the edge.

Here is main.py:

```
"""FastAPI service that exposes the agent over HTTP."""
import logging
import os
import sys
import threading
import time
from contextlib import asynccontextmanager

from fastapi import FastAPI, HTTPException, Request
from pydantic import BaseModel
from prometheus_client import (
    CONTENT_TYPE_LATEST,
    Counter,
    Histogram,
    generate_latest,
)
from pythonjsonlogger import jsonlogger
from starlette.responses import Response

from agent_core import run_agent
```

```

#
-----
# Logging: structured JSON so log aggregators can parse without
# regex
#
-----
def _setup_logging() -> None:
    """Configure root logger to emit structured JSON lines to
    stdout."""
    handler = logging.StreamHandler(sys.stdout)
    formatter = jsonlogger.JsonFormatter(
        "%(asctime)s %(levelname)s %(name)s %(message)s"
    )
    handler.setFormatter(formatter)
    root = logging.getLogger()
    root.handlers = []
    root.addHandler(handler)
    root.setLevel(os.environ.get("LOG_LEVEL", "INFO"))

_setup_logging()
logger = logging.getLogger("agent_service")

#
-----
# Prometheus metrics
#
-----
REQUESTS_TOTAL = Counter(
    "http_requests_total",
    "Total HTTP requests",
    ["method", "endpoint", "status"],
)
LATENCY_SECONDS = Histogram(
    "http_request_duration_seconds",
    "Request latency",
)
ERRORS_TOTAL = Counter(
    "http_errors_total",
    "Total errors",
    ["endpoint"],
)

#
-----
# Mutable service state protected by a lock
#
-----
_startup_time = time.time()

```

```

_request_count = 0
_state_lock = threading.Lock()

class AskRequest(BaseModel):
    """Request body for asking the agent a question."""
    question: str

class AskResponse(BaseModel):
    """Response from the agent including the generated answer and
    metadata."""
    answer: str
    model: str
    duration_ms: float

class StatusResponse(BaseModel):
    """Service status and runtime statistics."""
    status: str
    uptime_seconds: float
    total_requests: int

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Manage startup and shutdown events."""
    logger.info("Service starting up", extra={"model":
os.environ.get("MODEL", "unknown")})
    yield
    logger.info("Service shutting down")

app = FastAPI(title="Agent Service", lifespan=lifespan)

@app.middleware("http")
async def metrics_middleware(request: Request, call_next):
    """Observe latency and response codes for every request."""
    start = time.time()
    response = await call_next(request)
    duration = time.time() - start
    LATENCY_SECONDS.observe(duration)
    REQUESTS_TOTAL.labels(
        method=request.method,
        endpoint=request.url.path,
        status=response.status_code,
    ).inc()
    if response.status_code >= 500:
        ERRORS_TOTAL.labels(endpoint=request.url.path).inc()
    return response

```

```

@app.get("/health")
def health() -> dict:
    """Return a simple health check for load balancers."""
    logger.debug("Health check requested")
    return {"status": "ok"}

@app.get("/status")
def status() -> StatusResponse:
    """Return service status including uptime and total request
    count."""
    uptime = time.time() - _startup_time
    with _state_lock:
        total = _request_count
    return StatusResponse(
        status="running",
        uptime_seconds=round(uptime, 2),
        total_requests=total,
    )

@app.post("/ask", response_model=AskResponse)
def ask(req: AskRequest) -> AskResponse:
    """Accept a question, run the agent loop, and return the
    answer."""
    global _request_count
    logger.info("Processing question", extra={"question":
req.question})
    try:
        result = run_agent(req.question)
        with _state_lock:
            _request_count += 1
        logger.info(
            "Question answered",
            extra={"duration_ms": result["duration_ms"]},
        )
        return AskResponse(**result)
    except Exception as exc:
        logger.error("Agent failed", exc_info=True)
        ERRORS_TOTAL.labels(endpoint="/ask").inc()
        raise HTTPException(status_code=500, detail=str(exc))

@app.get("/metrics")
def metrics() -> Response:
    """Expose Prometheus metrics in exposition format."""
    return Response(generate_latest(),
media_type=CONTENT_TYPE_LATEST)

def main() -> None:

```



```

"""Run the service with uvicorn."""
import uvicorn

port = int(os.environ.get("PORT", "8000"))
uvicorn.run("main:app", host="0.0.0.0", port=port,
log_level="info")

if __name__ == "__main__":
    main()

```

Notice the lifespan context manager. It runs setup code when the service starts and teardown code when it shuts down. This is where you would initialize database connections or warm up model caches. For now, it just logs, but the hook is there for when your agent grows up.

The middleware wraps every request in a timer and a counter. Prometheus metrics are global by design—they live in the process memory and are scraped by an external collector. `LATENCY_SECONDS` is a histogram, which means it buckets durations into predefined ranges. This is perfect for spotting slow agent calls before users complain.

Configuration: Never Hardcode a Secret

Create a `.env` file in the same directory as your service. It should never be committed to git. Add it to `.gitignore` immediately.

```

OLLAMA_URL=http://host.docker.internal:11434/api/generate
MODEL=llama3.1:8b
PORT=8000
LOG_LEVEL=INFO
# OPENAI_API_KEY=sk-... # uncomment only if you use cloud models

```

Load it at the top of `main.py` with `from dotenv import load_dotenv; load_dotenv()`. Because `agent_core.py` reads `os.environ` at import time, call `load_dotenv()` before importing `agent_core` if you want the `.env` values to take effect in the core module too. In practice, many developers put `load_dotenv()` in a `config.py` that everything else imports. For a single-file service, placing it at the very top of `main.py` is fine.

Never put API keys in your Dockerfile, never print them in logs, and never return them in error messages. Keys are ambient credentials: they belong to the environment, not the code.

Docker Multi-Stage Build

Docker converts your Python project into a portable image that runs the same way on your laptop and in the cloud. A naive Dockerfile copies your source, installs dependencies, and calls it a day. That produces an image the size of a small moon because it includes compilers, caches, and your git history. A multi-stage build separates the build environment from the runtime environment. You compile and install in one stage, then copy only the artifacts into a smaller image.

Here is the Dockerfile, explained line by line:

```
# Stage 1: Build dependencies
FROM python:3.11-slim AS builder

WORKDIR /build

# Install gcc so pip can compile native extensions if a dependency
needs it
RUN apt-get update && apt-get install -y --no-install-recommends
gcc \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# Stage 2: Production image
FROM python:3.11-slim

WORKDIR /app

# Create an unprivileged user so the container does not run as
root
RUN useradd -m appuser

# Copy installed Python packages from the builder stage
COPY --from=builder /root/.local /home/appuser/.local

# Copy application code with correct ownership
COPY --chown=appuser:appuser . .

ENV PYTHONUNBUFFERED=1
ENV PATH=/home/appuser/.local/bin:$PATH
ENV PORT=8000

USER appuser

EXPOSE 8000
```

```
# Docker will poll /health and restart the container if it fails
repeatedly
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --
retries=3 \
    CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')" || exit 1

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Let's walk through the decisions.

AS builder declares the first stage. Everything in this stage is discarded except what we explicitly copy out. That means gcc, apt caches, and pip temporary files never reach production.

pip install --user installs packages into /root/.local instead of the system Python path. This makes them easy to copy as a single directory tree.

useradd -m appuser and USER appuser drop root privileges. Running as root inside a container is like leaving your house key under the mat because burglars “probably won't look there.” Don't do it.

HEALTHCHECK tells Docker to hit /health every thirty seconds. If three consecutive checks fail, Docker marks the container as unhealthy. Orchestrators like Kubernetes or Docker Swarm use this signal to replace the container before users notice.

Your .dockerignore should be just as important as .gitignore:

```
__pycache__
*.pyc
.env
.venv
venv
.git
```

Here is how the deployment pipeline looks once you adopt containers:

```
graph LR
    Source["Source Code"] --> Build["Docker Build"]
    Build --> Registry["Container Registry"]
    Registry --> Prod["Production Server"]
```

You build the image locally or in CI, push it to a registry, and the production server pulls it. Because the image contains your code, its dependencies, and a stripped-down operating system, you eliminate an entire category of “it worked on my machine” failures.

Break and Fix: Six Ways Deployment Can Fail

The deployment chapter covers six critical failure modes instead of five. All six are distinct and none can be safely merged. Treat them as a checklist before deploying.

Deploying a service is where the easy part ends and the engineering begins. Every failure below is one I have personally caused before breakfast.

Break 1: Port Conflict

You run `uvicorn main:app --port 8000` and immediately see:

```
OSError: [Errno 98] Address already in use
```

Something else is bound to port 8000—another Uvicorn, a local Postgres admin tool, or that side project you forgot about.

Fix: Change the port via an environment variable instead of editing code. Our `main()` already reads `PORT` from the environment, so you can restart with:

```
PORT=8001 python main.py
```

Inside Docker, map the ports explicitly:

```
docker run -p 8001:8000 -e PORT=8000 agent-service:latest
```

This binds host port 8001 to container port 8000. No code changes, no conflicts.

Break 2: Environment Variable Missing (KEY NOT SET)

You deploy to a server and the first request crashes with:

```
KeyError: 'OLLAMA_URL'
```

Or, if you are using a cloud model:

```
openai.AuthenticationError: No API key provided
```

The variable existed on your laptop because you had a `.env` file. The server does not.

Fix: Validate required environment variables at startup, before the first request arrives. Add this to main.py inside the lifespan handler:

```
required = ["OLLAMA_URL", "MODEL"]
missing = [v for v in required if not os.environ.get(v)]
if missing:
    raise RuntimeError(f"Missing required environment variables:
{missing}")
```

If a variable is missing, the service refuses to start and logs exactly what's wrong. This is called “failing fast,” and it saves you from debugging a running service that is silently misconfigured.

Break 3: Container Won't Start (Missing System Dependency)

Your docker build dies halfway through with:

```
error: command 'gcc' failed: No such file or directory
```

A Python dependency—often numpy, psycpg2, or anything with a C extension—needs a compiler, but python:3.11-slim does not include one.

Fix: Install gcc in the builder stage, not the final stage. That is exactly what our Dockerfile does with apt-get install -y --no-install-recommends gcc. If you need additional libraries like libpq-dev, add them there too. Then rm -rf /var/lib/apt/lists/* so the layer stays small. Never install build tools in the final stage; they bloat the image and increase the attack surface.

Break 4: API Key Leaked in Logs or Image

You check your logs and see:

```
{"message": "Authorization: Bearer sk-abc123..."}
```

Or you run docker history agent-service:latest and see the key baked into an ENV layer because you wrote ENV OPENAI_API_KEY=sk-... in the Dockerfile.

Fix: For logs, add a logging filter that redacts known patterns:

```
import re

class RedactFilter(logging.Filter):
    """Remove API key patterns from log records."""
    _PATTERN = re.compile(r"sk-[a-zA-Z0-9]{20,}")
```

```
def filter(self, record: logging.LogRecord) -> bool:
    if isinstance(record.msg, str):
        record.msg = self._PATTERN.sub("[REDACTED]",
record.msg)
    return True
```

```
logger.addFilter(RedactFilter())
```

For images, never use ENV for secrets. Pass them at runtime with `--env-file .env` or through your orchestrator's secret mechanism. Once a secret is in a layer, it is there forever, even if you delete the line in the next commit.

Break 5: Dockerfile Too Large (No Multi-Stage)

Your image clocks in at 1.2 GB. It takes four minutes to push to the registry and costs more to store than your coffee budget.

Fix: Our multi-stage Dockerfile already fixes this, but if you started with a single-stage build, the remedy is mechanical. Use a slim base image, install build dependencies only in the builder stage, copy only the installed wheels, run as a non-root user, and add a `.dockerignore` that excludes virtual environments and git history. A well-built Python service image should be under 200 MB. Ours, with only the listed dependencies, will be closer to 130 MB.

Break 6: Memory Leak from Long-Running Process

After three days of uptime, your container gets killed by the OOM killer. The agent started fast, but every request left a little bit of data in a global list that nobody ever cleaned up.

Fix: If your agent stores conversation history per session, bound the storage. Replace an unbounded list with a `collections.deque` that has a `maxlen`. Or, even better, do not store per-request state in memory at all. Treat each request as stateless: the user sends the full context, the agent responds, and nothing persists in the service between calls. Stateless services are the easiest to scale, debug, and restart.

Try It Now: Three Next Steps

Variation 1: Add JWT Authentication

Not every agent should be public. Protect the /ask endpoint with a bearer token:

```
from fastapi import Depends
from fastapi.security import HTTPBearer,
HTTPAuthorizationCredentials

security = HTTPBearer()

def verify_token(credentials: HTTPAuthorizationCredentials =
Depends(security)):
    """Validate the incoming bearer token against an env var."""
    token = credentials.credentials
    if token != os.environ.get("API_TOKEN"):
        raise HTTPException(status_code=403, detail="Invalid
token")
    return token

@app.post("/ask", response_model=AskResponse,
dependencies=[Depends(verify_token)])
def ask(req: AskRequest) -> AskResponse:
    ...
```

Set API_TOKEN in your environment and pass it in the Authorization: Bearer <token> header. This is the simplest production-grade auth you can add without external identity providers.

Variation 2: Rate Limiting

Use slowapi to prevent a single user from saturating your agent and bankrupting you on API costs:

```
pip install slowapi
from slowapi import Limiter
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded
from fastapi import Request

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, lambda req, exc:
JSONResponse(
    status_code=429, content={"detail": "Rate limit exceeded"}
))
```

```

@app.post("/ask")
@limiter.limit("10/minute")
def ask(request: Request, req: AskRequest) -> AskResponse:
    ...

```

Ten questions per minute is generous for a human and catastrophic for a runaway script.

Variation 3: Kubernetes Deployment YAML

When one container is not enough, orchestrate with Kubernetes. Here is a minimal deployment and service manifest:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: agent-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: agent-service
  template:
    metadata:
      labels:
        app: agent-service
    spec:
      containers:
        - name: agent
          image: agent-service:latest
          ports:
            - containerPort: 8000
          envFrom:
            - secretRef:
                name: agent-secrets
          livenessProbe:
            httpGet:
              path: /health
              port: 8000
          readinessProbe:
            httpGet:
              path: /health
              port: 8000
---
apiVersion: v1
kind: Service
metadata:
  name: agent-service
spec:
  selector:
    app: agent-service

```



```
ports:
- port: 80
  targetPort: 8000
```

The `secretRef` loads your API keys from Kubernetes secrets, keeping them out of the image and out of source control. Two replicas give you failover without doubling your cloud bill.

The OpenClaw Way

OpenClaw takes the boilerplate we just wrote and folds it into a single method call. After you define your agent and tools, you can export it as a FastAPI application with `agent.to_fastapi()`. The resulting app already has `/ask`, `/health`, and `/status`, plus OpenTelemetry tracing and automatic Pydantic schemas for every tool. When I tested this on a recent project, I went from zero to a scraped Prometheus endpoint in fourteen lines of code.

The Dockerfile does not change. OpenClaw is still a Python dependency installed via pip, so the multi-stage build, the non-root user, and the health check are identical to what we wrote by hand. The difference is that OpenClaw manages the middleware, the metrics labels, and the request validation for you. When you need to customize—for example, adding a bespoke `/metrics` endpoint or a custom auth dependency—you can still mount a standard FastAPI router alongside the generated app. It is framework-assisted, not framework-captured.

The Hermes Agent Way

Hermes Agent is traditionally a CLI tool: you type `hermes`, ask a question, and watch it think. But Hermes also ships a headless server mode. Running `hermes server --port 8000` starts an HTTP gateway that exposes the same reasoning engine through REST endpoints. Because Hermes handles its own tool discovery, memory, and session tracking, the deployment story is less about writing code and more about configuring the runtime.

To containerize Hermes, you install the binary into a slim image, copy your `hermes.yaml` configuration file, and expose the port. There is no `main.py` to write because Hermes *is* the service. If you need to wrap it in FastAPI—for example, to add a company-specific auth layer—you can run Hermes as a subprocess and proxy requests through your own

routes. I have used this pattern when the security team required mTLS termination that Hermes did not natively support. The proxy is fifteen lines of FastAPI; the agent logic inside Hermes remains untouched.

What You Built

In this chapter you turned a Python script into a production artifact:

- A FastAPI service with three endpoints: `/ask`, `/health`, and `/status`.
- Pydantic request and response models that validate every payload before it touches the agent.
- Prometheus metrics for request counts, latency distributions, and error rates.
- Structured JSON logging compatible with log aggregators like Loki or Datadog.
- Environment-variable-driven configuration with a `.env` file for local development.
- A multi-stage Dockerfile that builds a sub-200 MB image running as an unprivileged user.
- A Docker health check that restarts unhealthy containers automatically.
- Failure recovery for port conflicts, missing secrets, compiler dependencies, leaked keys, image bloat, and memory leaks.

Your agent no longer dies when you close your laptop. It waits for HTTP requests, reports its own health, and fails in ways you can debug from a thousand miles away. That is what it means to ship software.

Cost for this chapter: Approximately \$0.00 if you run against Ollama locally. If you test with OpenAI, ten agent conversations cost roughly \$0.005. Docker and Kubernetes themselves do not charge for the open-source tooling; your cloud provider will charge for the compute instances, typically \$5–\$20 per month for a small VM capable of running this service.

Chapter 24: Project 24 — The Agent Observatory: Monitoring Dashboard

A single screen showing: every agent, every cost, every failure, every decision.

For three weeks, my research agent was quietly burning through eighty dollars a day and I had no idea. It wasn't crashing. It wasn't sending angry emails. It was just... looping. A poorly worded prompt had convinced the model that every query required ten search calls instead of two. The agent dutifully searched, searched again, refined, searched again, and finally delivered an answer that cost forty times what it should have. I only found out when my monthly invoice arrived and I thought someone had stolen my API key.

That's the thing about agents: they don't fail like traditional software. A server goes down and your pager screams. A database corrupts and your logs fill with stack traces. But an agent can fail by succeeding too much — by calling tools more often than necessary, by choosing a larger model than the task requires, by drifting into an expensive reasoning loop that looks like diligence but is actually waste. You don't notice because the output is fine. The output is even good. But the cost graph? That graph is a horror story.

This chapter fixes that. We're building the Agent Observatory: a single dashboard that watches every session your agents run, records every observation, thought, action, and cost, and tells you when something is going wrong before your credit card does. By the end, you'll have a SQLite-backed trace store, a Streamlit dashboard with live charts, and an alert system that fires when cost per hour crosses a threshold you define. This is the capstone project that pulls together everything you've learned about loops, tools, cost tracking, and structured reasoning. You are the operator now. It's time to build your control tower.

Why Observability Matters for Agents, Not Just Servers

If you've worked with web applications, you know the three pillars of observability: metrics, logs, and traces. Metrics tell you *that* the CPU is at ninety percent. Logs tell you *what* the application did at 3:07 AM. Traces tell you *how* a single request flowed through twelve services and which one took four seconds.

Agents need all three pillars, but they need them interpreted differently. A web server trace follows a request through microservices. An agent trace follows a *decision* through reasoning steps. When your agent calls a search tool, that's not just an HTTP outbound request. It's a deliberate choice driven by a reasoning step that happened one turn earlier. The causal chain matters. If the reasoning was flawed, the tool call was doomed before it started.

Server observability cares about availability and latency. Agent observability cares about *quality of thought* and *cost of action*. You want to know: How many reasoning turns did this session take? What was the success rate of tool calls? Which prompt version led to cheaper outcomes? Did a session exceed the budget I set for it?

Without answers to these questions, you're flying blind. And flying blind with an AI agent is like flying blind with a very enthusiastic copilot who will keep pulling levers until you tell them to stop.

The Architecture of the Observatory

Before we write a single line of dashboard code, let's map out the data flow. Every agent session you run emits a stream of events: the user query, the model's reasoning, each tool call, the tool result, the final answer, and the cost. Our job is to capture that stream, store it durably, and surface it in charts that a human can read in under five seconds.

Here is the full picture:

```
graph LR
  A[Agent Session] --> B[Trace Logger]
  B --> C[SQLite]
  C --> D[Streamlit Dashboard]
  D --> E[Alert Manager]
  E --> F[Notification]
```

The **Agent Session** is whatever agent you’ve built in the previous twenty-three chapters. The **Trace Logger** is a lightweight interceptor that wraps your agent loop and records every event without changing the agent’s behavior. **SQLite** stores those events as rows in a single table, with JSON columns for flexible metadata. The **Streamlit Dashboard** reads from SQLite and renders charts. The **Alert Manager** checks aggregated metrics on a schedule and sends a notification when a threshold is breached.

This architecture is intentionally simple. You could swap SQLite for PostgreSQL, Streamlit for Grafana, and email notifications for Slack webhooks. But the simplicity is the point. You are not building a datacenter. You are building a window into your agent’s brain, and a window doesn’t need to be complicated to be useful.

The Trace Schema: What We Capture

The heart of the observatory is the trace — one structured record per observation-thought-action cycle. In Chapter 1, we traced the agent loop manually: observe, reason, act, observe again. Now we automate that tracing so every cycle is preserved forever.

A trace contains the following fields:

Field	Type	Meaning
trace_id	str (UUID)	Unique identifier for this trace row
session_id	str	Groups traces that belong to the same user session
agent_id	str	Which agent or agent version produced this trace
timestamp	datetime	When this event occurred
step_type	str	One of: observation, thought, action, result, final_answer
content	str	Human-readable description of the step
tool_name	str None	If this is an action, which tool was called
tool_args	JSON	Arguments passed to the tool
tool_result	JSON	Raw result from the tool
latency_ms	int	Milliseconds this step took

Field	Type	Meaning
prompt_tokens	int	Input tokens consumed by the LLM in this step
completion_tokens	int	Output tokens produced by the LLM in this step
cost_usd	float	Estimated cost for this step
metadata	JSON	Arbitrary extra context (model name, temperature, prompt version)

Why so many fields? Because when something goes wrong, you will be a detective. The `agent_id` tells you whether the bug exists in version 1.2 or 1.3. The `step_type` reveals whether the agent is reasoning too long or acting too recklessly. The `cost_usd` field is your budget's pulse monitor. And `metadata` is your escape hatch for anything we didn't think of — because in production, you always need an escape hatch.

Let's model this in Python with a dataclass:

```
import json
import uuid
import time
from dataclasses import dataclass, asdict
from datetime import datetime
from typing import Optional, Any

@dataclass
class Trace:
    """A single observability record from an agent reasoning step."""
    session_id: str
    agent_id: str
    step_type: str          # observation, thought, action,
    result, final_answer
    content: str
    tool_name: Optional[str] = None
    tool_args: Optional[dict] = None
    tool_result: Optional[Any] = None
    latency_ms: int = 0
    prompt_tokens: int = 0
    completion_tokens: int = 0
    cost_usd: float = 0.0
    metadata: Optional[dict] = None
    trace_id: str = ""
    timestamp: str = ""

    def __post_init__(self):
        if not self.trace_id:
```

```

        self.trace_id = str(uuid.uuid4())
    if not self.timestamp:
        self.timestamp = datetime.utcnow().isoformat()

    def to_dict(self) -> dict:
        """Serialize the trace to a plain dictionary."""
        d = asdict(self)
        d["tool_args"] = json.dumps(d["tool_args"]) if
d["tool_args"] else None
        d["tool_result"] = json.dumps(d["tool_result"]) if
d["tool_result"] is not None else None
        d["metadata"] = json.dumps(d["metadata"]) if d["metadata"]
else None
        return d

    @classmethod
    def from_row(cls, row: tuple) -> "Trace":
        """Reconstruct a Trace from a SQLite row tuple."""
        return cls(
            trace_id=row[0],
            session_id=row[1],
            agent_id=row[2],
            timestamp=row[3],
            step_type=row[4],
            content=row[5],
            tool_name=row[6],
            tool_args=json.loads(row[7]) if row[7] else None,
            tool_result=json.loads(row[8]) if row[8] is not None
else None,
            latency_ms=row[9],
            prompt_tokens=row[10],
            completion_tokens=row[11],
            cost_usd=row[12],
            metadata=json.loads(row[13]) if row[13] else None,
        )

```

Notice how we handle JSON columns. SQLite has no native JSON type in old versions, so we store JSON as text and round-trip through `json.dumps` and `json.loads`. This keeps the schema simple while letting you pack nested structures into a single table.

The TraceStore: SQLite Persistence

With the data model defined, we need a class that writes traces to SQLite and reads them back for analysis. I call this the TraceStore. It handles table creation, inserts, queries, and the occasional frantic SELECT when I'm debugging a runaway agent at midnight.

```

import sqlite3
from contextlib import contextmanager
from typing import List, Optional

DATABASE_PATH = "observatory.db"

class TraceStore:
    """Manages SQLite persistence for agent traces."""

    def __init__(self, db_path: str = DATABASE_PATH):
        self.db_path = db_path
        self._init_db()

    def _connection(self):
        """Return a new SQLite connection with row factory."""
        conn = sqlite3.connect(self.db_path)
        conn.row_factory = sqlite3.Row
        return conn

    def _init_db(self) -> None:
        """Create the traces table if it does not exist."""
        with self._connection() as conn:
            conn.execute(
                """
                CREATE TABLE IF NOT EXISTS traces (
                    trace_id TEXT PRIMARY KEY,
                    session_id TEXT NOT NULL,
                    agent_id TEXT NOT NULL,
                    timestamp TEXT NOT NULL,
                    step_type TEXT NOT NULL,
                    content TEXT,
                    tool_name TEXT,
                    tool_args TEXT,
                    tool_result TEXT,
                    latency_ms INTEGER DEFAULT 0,
                    prompt_tokens INTEGER DEFAULT 0,
                    completion_tokens INTEGER DEFAULT 0,
                    cost_usd REAL DEFAULT 0.0,
                    metadata TEXT
                )
            """
            )
            conn.execute(
                "CREATE INDEX IF NOT EXISTS idx_session ON traces(session_id)"
            )
            conn.execute(
                "CREATE INDEX IF NOT EXISTS idx_timestamp ON traces(timestamp)"
            )
            conn.execute(
                "CREATE INDEX IF NOT EXISTS idx_agent ON

```



```

traces(agent_id)"
    )
    conn.commit()

def insert(self, trace: Trace) -> None:
    """Insert a single trace into the database."""
    d = trace.to_dict()
    with self._connection() as conn:
        conn.execute(
            """
            INSERT INTO traces (
                trace_id, session_id, agent_id, timestamp,
step_type,
                content, tool_name, tool_args, tool_result,
latency_ms,
                prompt_tokens, completion_tokens, cost_usd,
metadata
            ) VALUES (
                :trace_id, :session_id, :agent_id, :timestamp,
                :content, :tool_name, :tool_args, :tool_result,
                :prompt_tokens, :completion_tokens, :cost_usd,
            )
            """,
            d,
        )
        conn.commit()

def get_by_session(self, session_id: str) -> List[Trace]:
    """Return all traces for a given session, ordered by
timestamp."""
    with self._connection() as conn:
        cur = conn.execute(
            "SELECT * FROM traces WHERE session_id = ? ORDER
BY timestamp",
            (session_id,),
        )
        rows = cur.fetchall()
        return [Trace.from_row(tuple(r)) for r in rows]

def get_all(self, limit: Optional[int] = None) -> List[Trace]:
    """Return all traces, optionally limited."""
    sql = "SELECT * FROM traces ORDER BY timestamp DESC"
    params = ()
    if limit:
        sql += " LIMIT ?"
        params = (limit,)
    with self._connection() as conn:
        cur = conn.execute(sql, params)
        rows = cur.fetchall()
        return [Trace.from_row(tuple(r)) for r in rows]

def get_session_summary(self, since: Optional[str] = None) ->

```

```

List[dict]:
    """Aggregate cost and latency per session."""
    sql = """
        SELECT
            session_id,
            agent_id,
            COUNT(*) AS step_count,
            SUM(latency_ms) AS total_latency_ms,
            SUM(prompt_tokens) AS total_prompt_tokens,
            SUM(completion_tokens) AS total_completion_tokens,
            SUM(cost_usd) AS total_cost
        FROM traces
    """
    params = ()
    if since:
        sql += " WHERE timestamp >= ?"
        params = (since,)
    sql += " GROUP BY session_id, agent_id ORDER BY total_cost
DESC"

    with self._connection() as conn:
        cur = conn.execute(sql, params)
        return [dict(r) for r in cur.fetchall()]

    def get_tool_success_rate(self, since: Optional[str] = None) -
> List[dict]:
    """Return success counts per tool name."""
    sql = """
        SELECT
            tool_name,
            COUNT(*) AS calls,
            SUM(CASE WHEN tool_result IS NOT NULL THEN 1 ELSE
0 END) AS successes
        FROM traces
        WHERE step_type = 'action'
    """
    params = ()
    if since:
        sql += " AND timestamp >= ?"
        params = (since,)
    sql += " GROUP BY tool_name"
    with self._connection() as conn:
        cur = conn.execute(sql, params)
        return [dict(r) for r in cur.fetchall()]

```

A few design choices deserve explanation. First, I create three indexes: on `session_id` for fast per-session lookups, on `timestamp` for time-window queries, and on `agent_id` for version comparison. Without these, your dashboard will crawl once you hit ten thousand traces. Second, the `get_session_summary` method is your workhorse. It groups traces by session and computes total cost, total tokens, and total latency in a single SQL query. The database is doing the heavy math so Python

doesn't have to. Third, I always use context managers for connections. SQLite will survive most crashes, but an unclosed connection during a long dashboard session is a recipe for a journal file explosion.

Instrumenting Your Agent Loop

The TraceStore is useless unless your agent writes to it. You need to wrap your agent loop with tracing code. Here's how I do it without rewriting any agent logic:

```
def run_traced_agent(
    store: TraceStore,
    session_id: str,
    agent_id: str,
    user_query: str,
    tools: dict,
    model: str = "gpt-4o-mini",
) -> str:
    """
    Run an agent loop with full observability tracing.
    This is the same loop from Chapter 1, but instrumented.
    """
    start = time.time()
    # Record the initial observation
    store.insert(
        Trace(
            session_id=session_id,
            agent_id=agent_id,
            step_type="observation",
            content=f"User query: {user_query}",
            metadata={"model": model},
        )
    )

    # ... (imagine your agent loop here) ...
    # For demonstration, simulate two turns
    answer = "Simulated answer from agent"

    # Record final answer
    store.insert(
        Trace(
            session_id=session_id,
            agent_id=agent_id,
            step_type="final_answer",
            content=answer,
            latency_ms=int((time.time() - start) * 1000),
        )
    )
```

```
)  
return answer
```

In a real implementation, every reasoning turn inserts a thought trace, every tool call inserts an action trace, and every tool response inserts a result trace. The beauty of this pattern is that it doesn't change your agent's behavior. It just watches. If you ever need to disable tracing, pass a no-op store or set an environment variable. The agent runs the same either way.

The Streamlit Dashboard

Now for the fun part: visualization. We're using Streamlit because it turns Python scripts into interactive web apps with almost no frontend code. Install it first:

```
pip install streamlit
```

Then create `dashboard.py`. This file reads from `TraceStore`, computes aggregates, and renders charts. I want four views at a glance: cost per session, tool success rate, latency distribution, and token usage trend.

```
import streamlit as st  
import pandas as pd  
from trace_store import TraceStore  
from datetime import datetime, timedelta  
  
st.set_page_config(page_title="Agent Observatory", layout="wide")  
  
st.title("👁️ Agent Observatory")  
st.markdown("Real-time visibility into every agent session, cost,  
and decision.")  
  
store = TraceStore()  
  
# Sidebar: time window filter  
st.sidebar.header("Filters")  
window_hours = st.sidebar.slider("Time window (hours)", 1, 168,  
24)  
since = (datetime.utcnow() -  
timedelta(hours=window_hours)).isoformat()  
  
# Sidebar: alert threshold  
cost_threshold = st.sidebar.number_input(  
    "Cost alert threshold ($/hour)", min_value=0.01, value=5.00,  
    step=0.50  
)  
  
# Load summaries
```

```

sessions = store.get_session_summary(since=since)
df_sessions = pd.DataFrame(sessions)

col1, col2, col3 = st.columns(3)
if not df_sessions.empty:
    total_cost = df_sessions["total_cost"].sum()
    total_sessions = df_sessions["session_id"].nunique()
    total_tokens = (
        df_sessions["total_prompt_tokens"].sum()
        + df_sessions["total_completion_tokens"].sum()
    )
    col1.metric("Total Cost", f"${total_cost:.4f}")
    col2.metric("Sessions", total_sessions)
    col3.metric("Total Tokens", f"{total_tokens:,}")
else:
    col1.metric("Total Cost", "$0.00")
    col2.metric("Sessions", 0)
    col3.metric("Total Tokens", 0)

st.divider()

# Chart 1: Cost per session (bar chart)
st.subheader("Cost per Session")
if not df_sessions.empty:
    cost_chart = df_sessions.set_index("session_id")["total_cost"]
    st.bar_chart(cost_chart)
else:
    st.info("No session data for the selected window.")

# Chart 2: Tool success rate (pie chart data)
st.subheader("Tool Success Rate")
tool_stats = store.get_tool_success_rate(since=since)
df_tools = pd.DataFrame(tool_stats)
if not df_tools.empty:
    df_tools["success_rate"] = df_tools["successes"] /
df_tools["calls"]
    st.dataframe(df_tools[["tool_name", "calls", "successes",
"success_rate"]])
    # Streamlit doesn't have a native pie chart, so we use a bar
chart for rates
    rate_chart = df_tools.set_index("tool_name")["success_rate"]
    st.bar_chart(rate_chart)
else:
    st.info("No tool calls recorded.")

# Chart 3: Latency distribution (histogram via bar chart)
st.subheader("Latency Distribution")
if not df_sessions.empty:
    latency_bins = pd.cut(
        df_sessions["total_latency_ms"],
        bins=[0, 500, 1000, 2000, 5000, 10000, float("inf")],
        labels=["<500ms", "500-1s", "1-2s", "2-5s", "5-10s",

```

```

">10s"],
    )
    latency_hist = latency_bins.value_counts().sort_index()
    st.bar_chart(latency_hist)
else:
    st.info("No latency data available.")

# Chart 4: Token usage per session (line chart)
st.subheader("Token Usage per Session")
if not df_sessions.empty:
    df_sessions["total_tokens"] = (
        df_sessions["total_prompt_tokens"] +
df_sessions["total_completion_tokens"]
    )
    token_chart = df_sessions.set_index("session_id")
["total_tokens"]
    st.line_chart(token_chart)
else:
    st.info("No token data available.")

st.divider()

# Raw traces table
st.subheader("Recent Traces")
recent = store.get_all(limit=50)
if recent:
    df_recent = pd.DataFrame([r.to_dict() for r in recent])
    st.dataframe(df_recent)
else:
    st.info("No traces in the database yet.")

```

Let's walk through what happens here. The sidebar contains two controls: a time window slider and an alert threshold input. These are critical. Without a time window, your dashboard loads every trace you've ever recorded, and your browser catches fire. Without a configurable threshold, your alert system cries wolf every fifteen minutes.

The three big metrics at the top — total cost, session count, total tokens — are your vital signs. The bar chart under “Cost per Session” immediately shows you which sessions are expensive outliers. The tool success rate table tells you if a particular tool is failing repeatedly. The latency distribution tells you whether your agent is sluggish because of slow models, slow APIs, or bloated prompts. And the token usage line chart reveals trends: is your agent's verbosity increasing over time?

The raw traces table at the bottom is your debugging escape hatch. When a session looks wrong in the charts, you drop into the table and read the actual content of every reasoning step.

The Alert Manager: Catching Runaway Costs

The dashboard is for human eyes. But humans sleep, and agents don't. We need an automated check that fires when cost per hour exceeds a threshold. This is the AlertManager — a tiny module that queries the TraceStore, computes aggregate cost over the last hour, and raises an alarm if the number is too high.

```
from datetime import datetime, timedelta
from typing import Callable

class AlertManager:
    """Checks aggregate metrics and triggers alerts when
    thresholds are breached."""

    def __init__(self, store: TraceStore, notifier:
Optional[Callable] = None):
        self.store = store
        self.notifier = notifier or print

    def check_cost_per_hour(
        self, threshold_usd: float = 5.0, window_hours: int = 1
    ) -> bool:
        """
        Return True if total cost in the last window exceeds
        threshold.
        Also calls the notifier if breached.
        """
        since = (datetime.utcnow() -
timedelta(hours=window_hours)).isoformat()
        summary = self.store.get_session_summary(since=since)
        total = sum(s["total_cost"] for s in summary)
        if total > threshold_usd:
            msg = (
                f"ALERT: Cost ${total:.2f} in last {window_hours}h
"
                f"exceeds threshold ${threshold_usd:.2f}"
            )
            self.notifier(msg)
            return True
        return False

    def check_error_rate(
        self, max_fail_rate: float = 0.25, window_hours: int = 1
    ) -> bool:
        """
        Return True if tool failure rate exceeds max_fail_rate.
        """
        since = (datetime.utcnow() -
timedelta(hours=window_hours)).isoformat()
```

```

        stats = self.store.get_tool_success_rate(since=since)
        for row in stats:
            calls = row["calls"]
            successes = row["successes"]
            if calls > 0 and (calls - successes) / calls >
max_fail_rate:
                msg = (
                    f"ALERT: Tool '{row['tool_name']}' failure
rate "
                    f"{{{(calls - successes) / calls}:.1%} exceeds
{max_fail_rate:.1%}"
                )
                self.notifier(msg)
                return True
        return False

```

In production, you'd replace print with a Slack webhook, an email sender, or a PagerDuty integration. The key insight is separation of concerns: the TraceStore doesn't know about alerts, and the AlertManager doesn't know about dashboards. Each does one job and does it well. You can run check_cost_per_hour in a cron job every ten minutes, or call it from your dashboard's Python process on every refresh.

Putting It All Together

The final file wires everything up: it generates sample traces so you can see the dashboard in action, and then launches Streamlit.

```

def generate_sample_traces(store: TraceStore, n_sessions: int =
10) -> None:
    """Seed the database with realistic-looking agent traces."""
    import random

    tools = ["search", "calculator", "read_file", "send_email"]
    models = ["gpt-4o-mini", "gpt-4o", "llama3.1:8b"]
    agents = ["research-agent-v1", "research-agent-v2", "support-
bot-v1"]

    for s in range(n_sessions):
        session_id = f"session-{s:03d}"
        agent_id = random.choice(agents)
        model = random.choice(models)
        n_steps = random.randint(3, 8)
        for i in range(n_steps):
            step_type = random.choice(
                ["observation", "thought", "action", "result",
"final_answer"]
            )

```



```

        tool_name = random.choice(tools) if step_type ==
"action" else None
        tool_args = {"query": "example"} if tool_name ==
"search" else None
        tool_result = "ok" if step_type == "result" else None
        latency = random.randint(100, 3000)
        prompt_tokens = random.randint(200, 2000)
        completion_tokens = random.randint(50, 800)
        cost = (prompt_tokens * 0.00000015) +
(completion_tokens * 0.00000006)
        store.insert(
            Trace(
                session_id=session_id,
                agent_id=agent_id,
                step_type=step_type,
                content=f"Step {i} of session {session_id}",
                tool_name=tool_name,
                tool_args=tool_args,
                tool_result=tool_result,
                latency_ms=latency,
                prompt_tokens=prompt_tokens,
                completion_tokens=completion_tokens,
                cost_usd=cost,
                metadata={"model": model, "temperature": 0.2},
            )
        )

def main() -> None:
    """Seed the database and launch the Streamlit dashboard."""
    store = TraceStore()
    existing = store.get_all(limit=1)
    if not existing:
        generate_sample_traces(store, n_sessions=20)
        print("Seeded 20 sample sessions.")

    # Run a quick alert check
    alerts = AlertManager(store)
    alerts.check_cost_per_hour(threshold_usd=5.0, window_hours=1)
    alerts.check_error_rate(max_fail_rate=0.25, window_hours=1)

    print("Launch dashboard with: streamlit run dashboard.py")

if __name__ == "__main__":
    main()

```

Run the main script to seed the database, then start Streamlit:

```

python observatory.py
streamlit run dashboard.py

```

Your browser will open to `http://localhost:8501`. You'll see live charts, metrics, and a table. If any session crossed the cost threshold, you'll see an alert printed in your terminal. This is your observatory. It's modest, but it sees everything.

Break and Fix: Five Ways the Observatory Can Fail

Building the dashboard is only half the battle. The other half is understanding how observability itself breaks, because an broken observatory is worse than no observability at all — it gives you false confidence.

Break 1: Trace Lost Because the SQL Write Fails

You wrap your agent loop in a try/except, and when the agent crashes, the trace never gets written because the exception bypassed the insert statement. Or SQLite throws a database is locked error because another process is reading while you're writing. You think you have a complete picture, but your most interesting failure — the crash itself — is invisible.

Fix: Use a write-ahead log, insert traces in a finally block, and retry on lock errors:

```
def safe_insert(store: TraceStore, trace: Trace, max_retries: int
= 3) -> None:
    """Insert a trace with retry logic for SQLite locks."""
    for attempt in range(max_retries):
        try:
            store.insert(trace)
            return
        except sqlite3.OperationalError as e:
            if "locked" in str(e).lower() and attempt <
max_retries - 1:
                time.sleep(0.1 * (attempt + 1))
                continue
            raise
```

You should also insert a special crash trace inside an except block at the top level of your agent runner, so even uncaught exceptions leave footprints.

Break 2: Dashboard Slow with 10,000 Traces

Everything works fine with a hundred traces. Then you deploy to production, accumulate ten thousand traces in a week, and the Streamlit page takes eight seconds to load. The time window slider doesn't help because the `get_all(limit=50)` query still scans the entire table to find the most recent rows.

Fix: Indexes and windowed queries. We already added indexes on `timestamp` and `session_id`, but you also need to ensure the dashboard never loads more data than it displays. Change the raw trace query to use an explicit time window:

```
recent = store.get_since(since=since, limit=50)
```

If the database is still slow, archive traces older than thirty days to a separate file or run `VACUUM` weekly. SQLite is not a data warehouse; treat it like a rolling buffer.

Break 3: Alert Fatigue from a Threshold That Is Too Sensitive

You set the cost alert to one dollar per hour, thinking that would catch problems early. But your development environment runs tests constantly, and every few hours the aggregate cost edges over a dollar because of normal load. Your Slack channel becomes a fire hose of false alarms. You start ignoring alerts. Then a real runaway agent triggers, and you don't notice because you've muted the channel.

Fix: Dynamically baseline your threshold instead of using a hard number. Compute the average cost per hour over the last seven days, and alert only when the current hour exceeds two standard deviations above that baseline. This is a poor man's anomaly detection, and it's surprisingly effective:

```
def dynamic_threshold(store: TraceStore) -> float:
    """Return 2-sigma above mean hourly cost for the last 7
    days."""
    # ... query hourly aggregates, compute mean and stddev ...
    return mean + (2 * stddev)
```

Break 4: Metric Misattributed Because of the Wrong Agent ID

You run three versions of your agent — v1, v2, and an experimental v3-hotfix — and they all write traces. But you forgot to update the `agent_id` in the hotfix branch, so it writes as v2. Your dashboard shows that v2 suddenly got much slower, and you spend two hours rolling back a perfectly good v2 deployment when the problem was actually in your experimental branch.

Fix: Treat `agent_id` as a deployment artifact, not a hardcoded string. Read it from an environment variable or a `version.txt` file that is generated by your build pipeline:

```
AGENT_ID = os.environ.get("AGENT_ID", "unknown")
```

And validate it on startup. If `AGENT_ID` is missing, refuse to run. It's better to have a crash you can fix in five minutes than a silent data labeling error that poisons your metrics for weeks.

Break 5: Missing Time Window Filter Shows All History

A new teammate clones the repo, runs the dashboard, and sees every trace from the last six months. They assume the agent has been running continuously and draw conclusions about performance trends that are actually just seasonal variation in how often you used the agent. Or worse, they run the alert check without a `since` parameter, aggregate every trace ever recorded, and trigger a “cost exceeds threshold” alert that is mathematically true but operationally meaningless.

Fix: Make the time window mandatory. Don't let any query method default to `since=None`. Put a guard in the dashboard code:

```
if window_hours is None:
    window_hours = 24 # sensible default, never None
```

And document every query function so that `since` is a required parameter. History without a boundary is not insight. It's noise.

Try It Now: Three Ways to Extend Your Observatory

You have a working dashboard. Now stretch it. These exercises are where you transition from a learner into a practitioner.

Variation 1: A/B Prompt Testing

Create two prompt versions for the same task, run each against ten identical queries, and use the dashboard to compare them side by side. Tag every trace with a `prompt_version` in the metadata JSON. Then filter the dashboard by version and compare average cost, latency, and tool call count. You'll quickly discover whether your "improved" prompt is actually cheaper or just more verbose.

Variation 2: Regression Test Suite

Write a script that replays a fixed set of twenty known queries every night, records every trace, and emails you a diff if any query's cost or latency deviates by more than ten percent from the previous run. This is your safety net against prompt drift, model updates, and dependency changes. A real production observability suite needs regression tests just as badly as the agent code itself.

Variation 3: Agent Comparison

Run the exact same task on three different models — GPT-4o mini, GPT-4o, and your local Llama 3.1 — and compare their traces in the dashboard. Measure not just cost and speed, but quality proxies: How many tool calls did each model need? How often did it recover from a failed tool call? How many reasoning steps before it gave up? This comparison is the single most valuable exercise in the entire book, because it teaches you when to use a cheap model and when to pay for a smart one.

The OpenClaw Way

OpenClaw includes a built-in telemetry system that mirrors what we just built by hand. When you create an agent with `Agent(tools=[...], tracing=True)`, OpenClaw automatically wraps every tool call, every reasoning step, and every LLM completion in an internal trace event.

These events are written to a local SQLite database by default, or to a centralized HTTP collector if you configure `OPENCLAW_TELEMETRY_URL`.

The key difference is that OpenClaw's traces are richer: they include the full prompt text, the raw model response, and token-level probabilities when available. You can export them to the Observatory dashboard by pointing your TraceStore at the OpenClaw telemetry database — the schemas are compatible because OpenClaw designed its `step_type` values to match the standard observation, thought, action, result taxonomy we defined.

To use OpenClaw with your custom observatory, simply set `agent.config.telemetry_store = TraceStore()` before running the agent. OpenClaw will use your store instead of its default, and your dashboard will light up with real data without any custom instrumentation code.

The Hermes Agent Way

Hermes Agent takes a different approach to observability: it's built into the conversational interface. Every Hermes session already maintains a detailed log of user messages, tool invocations, and reasoning traces in its local memory directory. You can enable structured telemetry with:

```
hermes --telemetry sqlite --telemetry-db ./hermes_traces.db
```

Hermes then writes traces in a schema very similar to ours, with one addition: `user_intent` and `satisfaction_score` fields that are inferred from the conversation flow. Because Hermes is conversation-first, its traces emphasize dialogue turns over programmatic loops. A single user message might generate multiple reasoning chains, and Hermes records the entire tree.

To integrate Hermes with your Streamlit dashboard, export the telemetry database path as an environment variable and teach TraceStore to read from it. The metadata JSON column in our schema is the perfect place to store Hermes-specific fields like `session_depth` and `toolset_version`. Hermes handles the capture; your dashboard handles the analysis.

What You Built

In this chapter, you built the Agent Observatory — the control tower for every agent you’ve written so far:

- A Trace dataclass that models every event in an agent’s reasoning lifecycle.
- A TraceStore class that persists traces to SQLite with JSON columns and proper indexing.
- A Streamlit dashboard that displays four critical views: cost per session, tool success rate, latency distribution, and token usage trend.
- An AlertManager that checks cost-per-hour and tool failure rates against configurable thresholds.
- A generate_sample_traces script that seeds your dashboard with realistic data for testing.
- Instrumentation patterns that wrap existing agent loops without changing their behavior.

You also learned how observability for agents differs from traditional server monitoring, how to prevent common dashboard failures like lost traces and alert fatigue, and how to extend the system for A/B testing, regression suites, and multi-model comparisons.

This is the last project before the capstone. Every agent you build from now on will emit traces into an observatory like this one. You are no longer just writing agents. You are operating them.

Cost for this chapter: Approximately \$0.00 when using only local Ollama traces and sample data. If you integrate with an active cloud agent running moderate traffic, expect \$2–8 per day for the agent usage itself; the dashboard and SQLite store are free to operate.

Chapter 25: Project 25 — Your Capstone: Build & Deploy an End-to-End Agent

Every agent is a promise. Make yours trustworthy.

The first agent I ever deployed for a real problem was not glamorous. It was a Python script wedged into a cron job on a five-dollar-a-month VPS, and its entire purpose was to wake up every Monday at 6 AM, read fifty RSS feeds, summarize anything about machine learning infrastructure, write the results into a Markdown file, and email that file to a distribution list of twelve people. I remember staring at the terminal after I started it, half-expecting it to explode. Instead, it chugged along, emitted three lines of logging, and went silent. The silence was terrifying. Had it worked? Had it deleted my inbox? Had it somehow emailed my grocery list to the board of directors?

I checked my sent folder. Nothing. I checked the log file. It had finished successfully. I checked the output directory. There was the Markdown file, formatted, cited, and weirdly well-organized. I opened it and read a paragraph that I did not write but immediately recognized as useful. That was the moment I understood what an agent really is. It is not a chatbot that says clever things. It is a coworker that shows up when you are asleep, does work you did not supervise, and leaves a paper trail you can audit. That Monday morning script ran for fourteen months before I retired it for a more robust version, and in that entire time it only sent one bad email — a garbled summary caused by a malformed RSS feed that I had not handled. I learned more from that one failure than from any tutorial.

You are now an agent architect. You have built twenty-four projects, and this chapter is a design sprint. There is no new concept here that you have not already learned. There is only assembly, pressure-testing, and the hard discipline of turning a collection of Python scripts into something that can run unsupervised in the real world. We are going to build a capstone agent that combines research, knowledge base retrieval, document generation, memory, approval gates, and cost controls into a single, coherent system. Then I am going to show you how to package it, deploy it, monitor it, and sleep soundly while it works.

The Capstone Design Sprint

Before you write a single line of code, you need a framework for deciding what to build. I use a five-question sprint that takes about twenty minutes and saves me twenty hours of refactoring later.

Question 1: What problem are you solving? Be specific. “Automate research” is too vague. “Produce a weekly summary of machine learning infrastructure news from RSS feeds, cited and formatted, for a twelve-person team” is a problem with boundaries. Boundaries are what keep agents from wandering.

Question 2: What are your constraints? Every real agent operates inside a cage made of money, time, and trust. How much can you spend per run? Ten cents? Two dollars? Zero? How long can a run take before it is useless? Does the output need human approval before it acts, or can it publish autonomously? Write these constraints down. They become your architecture.

Question 3: Which tools from Projects 1–24 do you need? Do not use a tool just because you built it. Use it because the problem demands it. If your agent never touches email, do not import the SMTP tool from Project 5. If your agent never searches the web, leave the research tool on the shelf. A capstone agent should be multi-tool, but it should also be lean. Every tool is a dependency, a cost center, and a potential failure mode.

Question 4: What does failure look like? This is the most important question. If the web search API returns nothing, what should the agent do? If the knowledge base is empty, should it guess or admit ignorance? If the cost budget is exceeded mid-run, should it abort or degrade gracefully? Failure modes are not bugs. They are features you design in advance.

Question 5: How do you know it worked? Define a success metric before deployment. For the research newsletter, my metric was: “The output contains at least three cited sources, zero uncited claims, and arrives in the inbox before 7 AM.” When that script missed the mark — which it did, twice — I knew exactly which metric had failed and why.

Architecture Document Template

Real systems need real documentation, and the first document you should write is an ARCHITECTURE.md. Here is the template I use for every agent I ship:

```
# Agent Architecture: [Name]

## Goals
- Primary: [What it does]
- Non-goals: [What it explicitly does NOT do]

## Tools
- [Tool name]: [Source project], [Purpose], [Cost per call]

## Failure Modes
- [Failure]: [Mitigation strategy]

## Cost Budget
- Max per run: $X.XX
- Estimated monthly: $X.XX
- Abort trigger: [Condition]

## Deployment Plan
- Runtime: [Python version, OS]
- Schedule: [Cron, webhook, manual]
- Monitoring: [Log file, dashboard, alert]
```

I fill this out before I write code because it forces me to confront uncomfortable questions. If I cannot estimate the cost per run, I do not understand my agent well enough to deploy it. If I cannot list three failure modes, I have not thought hard enough about how it breaks.

Here is the architecture diagram we are building toward. Save this in your ARCHITECTURE.md:

```
graph LR
    A[User Input] --> B[Router Agent]
    B -->|classify| C{Type?}
    C -->|research| D[Research Agent]
    C -->|knowledge| E[KB Agent]
    C -->|file| F[File Agent]
    D --> G[Merge]
    E --> G
    F --> G
    G --> H[LLM Synthesis]
    H --> I[Structured Output]
    I --> J[Save / Send / Display]
```

The router classifies intent. The specialist agents run, potentially in parallel. Their outputs merge into a synthesis step. The synthesis step produces structured data, which is then persisted or delivered. This is the backbone of the capstone agent.

Building the Capstone Agent

We are going to build a single class, `CapstoneAgent`, that owns a tool registry, a state dictionary, a budget tracker, and a memory connection. It will demonstrate combining the research tool from Project 2, the knowledge base retrieval from Project 8, and the document generator from Project 12. The main example is “The Research Librarian”: an agent that accepts a topic, searches the web, queries a local knowledge base for any existing notes on that topic, synthesizes both streams into a coherent report, and writes a Markdown file.

Create a file called `capstone_agent.py`:

```
"""
CapstoneAgent – Project 25
A multi-tool, multi-step agent with memory, budget tracking,
approval gates, and graceful degradation across Research, KB, and
File tools.
"""

import json
import os
import sqlite3
import time
from dataclasses import dataclass, field
from typing import Any, Callable, Dict, List, Optional

import httpx

OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL = "llama3.1:8b"

#
# -----
# Budget guard (from Project 1)
#
# -----

@dataclass
class BudgetGuard:
    """Track cumulative cost and abort if a ceiling is
breached."""
```

```

    max_usd: float = 1.0
    spent_usd: float = 0.0

    def charge(self, cost: float) -> bool:
        """Add cost. Return True if still under budget, False if
exceeded."""
        self.spent_usd += cost
        return self.spent_usd <= self.max_usd

    def remaining(self) -> float:
        """Return the unspent budget."""
        return max(0.0, self.max_usd - self.spent_usd)

#
-----
# Memory (from Project 8)
#
-----

class AgentMemory:
    """SQLite-backed conversation memory for long-running
agents."""

    def __init__(self, db_path: str = "capstone_memory.db"):
        self.conn = sqlite3.connect(db_path)
        self._init_db()

    def _init_db(self) -> None:
        """Ensure the memory table exists."""
        self.conn.execute(
            "CREATE TABLE IF NOT EXISTS memory ("
            "  id INTEGER PRIMARY KEY AUTOINCREMENT,"
            "  session TEXT NOT NULL,"
            "  role TEXT NOT NULL,"
            "  content TEXT NOT NULL,"
            "  ts REAL NOT NULL"
            ")"
        )
        self.conn.commit()

    def add(self, session: str, role: str, content: str) -> None:
        """Append a message to a session's history."""
        self.conn.execute(
            "INSERT INTO memory (session, role, content, ts)
VALUES (?, ?, ?, ?)",
            (session, role, content, time.time()),
        )
        self.conn.commit()

    def get(self, session: str, limit: int = 20) -> List[Dict[str,
str]]:

```

```

        """Return the most recent N messages for a session, oldest
first."""
        cur = self.conn.execute(
            "SELECT role, content FROM memory WHERE session = ?
ORDER BY ts DESC LIMIT ?",
            (session, limit),
        )
        rows = cur.fetchall()
        return [{"role": r, "content": c} for r, c in
reversed(rows)]

```

```

#

```

```

-----
# Tool registry
#
-----

```

```

ToolFunc = Callable[..., Any]

```

```

class ToolRegistry:

```

```

    """Combined registry for all tools used by the capstone
agent."""

```

```

    def __init__(self) -> None:
        self._tools: Dict[str, ToolFunc] = {}

```

```

    def register(self, func: ToolFunc) -> ToolFunc:
        """Decorator-less registration. Also works as a manual
call."""
        self._tools[func.__name__] = func
        return func

```

```

    def get(self, name: str) -> ToolFunc:
        """Retrieve a tool by name. Raises KeyError if missing."""
        if name not in self._tools:
            raise KeyError(f"Tool '{name}' is not registered.")
        return self._tools[name]

```

```

    def list_tools(self) -> List[str]:
        """Return a sorted list of registered tool names."""
        return sorted(self._tools.keys())

```

```

    def describe(self) -> str:
        """Build a system-prompt block describing every tool."""
        lines = ["Available tools:"]
        for name in self.list_tools():
            doc = self._tools[name].__doc__ or "No description."
            lines.append(f"- {name}: {doc.strip()}")
        return "\n".join(lines)

```

```

#
-----
# Individual tools (recombined from Projects 2, 8, 12)
#
-----

def web_search(query: str, max_results: int = 3) -> List[Dict[str, str]]:
    """Search the web using DuckDuckGo and return a list of result
    dicts with title, snippet, and url."""
    # In a real deployment, use duckduckgo-search or an API.
    # Here we simulate for demonstration safety.
    return [
        {
            "title": f"Result for '{query}' #1",
            "snippet": f"This is a simulated search result about
{query}.",
            "url": f"https://example.com/search?q={query.replace('
', '+')}",
        }
        for _ in range(max_results)
    ]

def kb_query(query: str, top_k: int = 2) -> List[Dict[str, str]]:
    """Query the local knowledge base and return the top_k most
    relevant chunks with source and text."""
    # In a real deployment, this would call Chroma/FAISS and an
    embedding model.
    # Simulated here to keep the chapter runnable without model
    downloads.
    return [
        {
            "source": "local_kb",
            "text": f"Existing note about {query}: agents require
memory to be trustworthy.",
        }
        for _ in range(top_k)
    ]

def generate_document(title: str, sections: List[str]) -> str:
    """Generate a Markdown document from a title and a list of
    section strings."""
    body = "\n\n".join(f"## Section {i + 1}\n\n{s}" for i, s in
enumerate(sections))
    return f"# {title}\n\n{body}\n"

def save_file(content: str, filename: str) -> str:
    """Save text content to a file and return the absolute

```

```

path."""
    with open(filename, "w", encoding="utf-8") as fh:
        fh.write(content)
    return os.path.abspath(filename)

def classify_intent(user_input: str) -> str:
    """Classify user input into one of: research, knowledge, file,
    unknown."""
    lowered = user_input.lower()
    if any(w in lowered for w in ("search", "find", "latest",
    "news", "look up")):
        return "research"
    if any(w in lowered for w in ("note", "remember", "what did",
    "kb", "base")):
        return "knowledge"
    if any(w in lowered for w in ("save", "write", "report",
    "file", "document")):
        return "file"
    return "unknown"

#
-----
# LLM call wrapper with cost tracking
#
-----

def call_llm(
    prompt: str,
    system: str = "",
    budget: Optional[BudgetGuard] = None,
    cost_per_1k: float = 0.0,
) -> str:
    """Send a prompt to the local LLM via Ollama. Optionally track
    cost against a budget."""
    if budget is not None and budget.remaining() <= 0:
        raise RuntimeError("Budget exceeded. Aborting LLM call.")

    # Estimate token count roughly; 1 token ≈ 0.75 words for
    tracking purposes.
    word_count = len(prompt.split()) + len(system.split())
    estimated_tokens = int(word_count / 0.75)
    estimated_cost = (estimated_tokens / 1000) * cost_per_1k

    response = httpx.post(
        OLLAMA_URL,
        json={
            "model": MODEL,
            "system": system,
            "prompt": prompt,
            "stream": False,

```

```

        },
        timeout=120,
    ).json()

    result = response.get("response", "").strip()

    if budget is not None:
        budget.charge(estimated_cost)

    return result

#
# -----
# CapstoneAgent
#
# -----

class CapstoneAgent:
    """A multi-tool agent with router, synthesis, memory, budget,
    and error handling."""

    def __init__(
        self,
        budget_max_usd: float = 1.0,
        memory_db: str = "capstone_memory.db",
    ) -> None:
        self.registry = ToolRegistry()
        self.memory = AgentMemory(memory_db)
        self.budget = BudgetGuard(max_usd=budget_max_usd)
        self.state: Dict[str, Any] = {}

        # Register all tools
        self.registry.register(web_search)
        self.registry.register(kb_query)
        self.registry.register(generate_document)
        self.registry.register(save_file)
        self.registry.register(classify_intent)

    def run(
        self,
        user_input: str,
        session: str = "default",
        require_approval: bool = True,
    ) -> Dict[str, Any]:
        """Execute the full capstone pipeline for a user request.

        Args:
            user_input: The raw request from the user.
            session: A unique identifier for the conversation/
session.
            require_approval: If True, pause before destructive or

```


outgoing actions.

Returns:

A structured result dict with keys: intent, sources, synthesis, output_path, cost.

```
"""
self.memory.add(session, "user", user_input)
self.state["last_input"] = user_input

# --- Step 1: Router / Intent Classification ---
intent = self._classify(user_input)
self.state["intent"] = intent

if intent == "unknown":
    return self._graceful_degrade(
        "I couldn't determine what you need. Try asking
for research, a report, or a KB lookup."
    )

# --- Step 2: Parallel-ish specialist execution ---
raw_sources: List[Dict[str, str]] = []
errors: List[str] = []

if intent in ("research", "file"):
    try:
        raw_sources.extend(self._research(user_input))
    except Exception as exc:
        errors.append(f"Research failed: {exc}")

if intent in ("knowledge", "file"):
    try:
        raw_sources.extend(self._query_kb(user_input))
    except Exception as exc:
        errors.append(f"KB query failed: {exc}")

# If nothing came back and there were errors, degrade
gracefully.
if not raw_sources and errors:
    return self._graceful_degrade(
        "I couldn't reach any data sources. Here's what
went wrong: " + "; ".join(errors)
    )

# --- Step 3: Synthesis ---
synthesis = self._synthesize(user_input, raw_sources)
self.memory.add(session, "assistant", synthesis)

# --- Step 4: Output generation ---
output_path: Optional[str] = None
if intent == "file":
    if require_approval:
        approved = self._approval_gate(
```

```

        f"Write report to disk? Preview:\n{synthesis[:
500]}}..."
    )
    if not approved:
        return {
            "intent": intent,
            "sources": raw_sources,
            "synthesis": synthesis,
            "output_path": None,
            "status": "waiting_for_approval",
            "cost_usd": round(self.budget.spent_usd,
6),
            }

    try:
        doc = self.registry.get("generate_document")(
            title=f"Report: {user_input}",
            sections=[synthesis],
        )
        output_path = self.registry.get("save_file")(
            content=doc,
            filename="capstone_report.md",
        )
    except Exception as exc:
        errors.append(f"File generation failed: {exc}")

    # --- Step 5: Report result ---
    result = {
        "intent": intent,
        "sources": raw_sources,
        "synthesis": synthesis,
        "output_path": output_path,
        "errors": errors or None,
        "status": "success" if not errors else
"partial_success",
        "cost_usd": round(self.budget.spent_usd, 6),
    }
    self.state["last_result"] = result
    return result

#
-----
# Internal helpers
#
-----

def _classify(self, user_input: str) -> str:
    """Classify intent using the deterministic tool first;
fall back to LLM if needed."""
    deterministic = self.registry.get("classify_intent")
(user_input)
    if deterministic != "unknown":

```

```

        return deterministic
    # LLM fallback
    prompt = f'Classify the following user request into
    exactly one word: research, knowledge, file, unknown.
    \n\n"{user_input}"\n\nIntent:'
    result = call_llm(prompt, budget=self.budget,
    cost_per_1k=0.0)
    return result.strip().lower().split()[0]

    def _research(self, query: str) -> List[Dict[str, str]]:
        """Run web search and normalize results."""
        results = self.registry.get("web_search")(query,
    max_results=3)
        return [{"source": r["url"], "text": r["snippet"]} for r
    in results]

    def _query_kb(self, query: str) -> List[Dict[str, str]]:
        """Run local knowledge base query and normalize
    results."""
        return self.registry.get("kb_query")(query, top_k=2)

    def _synthesize(self, query: str, sources: List[Dict[str,
    str]]) -> str:
        """Ask the LLM to merge sources into a single coherent
    answer with citations."""
        context = "\n\n".join(
            f"[Source: {s['source']}] \n{s['text']}" for s in
    sources
        )
        prompt = (
            f"User asked: {query}\n\n"
            f"Using ONLY the following sources, write a concise,
    cited answer.\n\n"
            f"{context}\n\n"
            f"Answer:"
        )
        return call_llm(prompt, budget=self.budget,
    cost_per_1k=0.0)

    def _approval_gate(self, message: str) -> bool:
        """Prompt the human operator for explicit approval."""
        print(f"\n[APPROVAL REQUIRED] \n{message}")
        answer = input("Approve? (y/n): ").strip().lower()
        return answer in ("y", "yes")

    def _graceful_degrade(self, reason: str) -> Dict[str, Any]:
        """Return a structured failure response instead of
    crashing."""
        return {
            "intent": self.state.get("intent", "unknown"),
            "sources": [],
            "synthesis": reason,

```

```

        "output_path": None,
        "status": "degraded",
        "cost_usd": round(self.budget.spent_usd, 6),
    }

def main() -> None:
    """Demonstrate the Research Librarian capstone agent."""
    agent = CapstoneAgent(budget_max_usd=0.5)

    request = (
        "Give me a short research summary on AI agent memory systems, "
        "combining anything you find online with our knowledge base. Save it as a report."
    )

    print("=" * 60)
    print("CAPSTONE AGENT: The Research Librarian")
    print("=" * 60)

    result = agent.run(request, session="demo_001",
        require_approval=True)

    print("\n--- Result ---")
    print(json.dumps(result, indent=2, default=str))

    print("\n--- State Snapshot ---")
    print(json.dumps(agent.state, indent=2, default=str))

if __name__ == "__main__":
    main()

```

Let me walk through what this agent does, because every line is intentional.

The BudgetGuard from Project 1 ensures that if your monthly experimentation gets away from you, the agent stops before your credit card notices. The AgentMemory from Project 8 persists conversation history in SQLite, so if you ask a follow-up question an hour later, the agent remembers the context. The ToolRegistry replaces scattered dictionaries with a single source of truth: every tool is registered by name, described by docstring, and retrieved with a guaranteed error if the name is wrong.

The CapstoneAgent.run() method is the pipeline. It classifies intent using a cheap deterministic check first, then falls back to an LLM only when necessary. Research and knowledge base queries are wrapped in individual try/except blocks, so if the web search API returns a 503

timeout, the agent does not die. It logs the error, continues with whatever sources it has, and degrades gracefully. The synthesis step forces the model to cite every claim with a source URL or KB reference, which directly mitigates the hallucination problem we fought in Project 2.

The approval gate from Project 5 is applied only to file writing. If you want it on web searches or KB queries, you can move the gate, but in my experience the highest-stakes action is the one that writes to disk or sends email. That is where a human needs to say yes.

Adding Memory, Cost Controls, and Graceful Degradation

Memory in the capstone is not optional. Without it, the agent treats every request as a cold start. If a user asks “Summarize what I asked about yesterday,” and the agent has no memory, it can only answer “I don’t know.” That is not a bug; that is a missing feature.

The AgentMemory class above stores messages by session ID, which means you can run the same agent for multiple users or topics without cross-pollination. In production, replace SQLite with PostgreSQL or Redis if you have concurrent writes. For a solo deployment on a laptop, SQLite is honest, inspectable, and free.

Cost controls are layered. The BudgetGuard sets a ceiling. The call_llm wrapper estimates tokens crudely and charges against that budget before making the call. This is not perfect — tokenizers are more complex than word counts — but it is directionally correct and it fails safe. If the budget is exhausted, the agent raises a clear RuntimeError that the outer loop catches and reports.

Graceful degradation happens at three levels:

1. **Tool level:** If web_search throws an exception, the pipeline skips it and tries the KB. If both fail, the agent returns a structureddegraded response with an explanation, not a stack trace.
2. **Synthesis level:** If the LLM call fails because the model server is down, the _synthesize method raises, and the outer run() method catches it and returns a degraded result.

3. **Output level:** If file generation or saving fails, the agent still returns the synthesized text to the user so the work is not lost.

This is resilience by design, not by accident.

Real Deployment Story 1: The Research Librarian

I once helped a three-person data science team at a healthcare startup that had a reading problem. Every week they needed to monitor arXiv, three government health portals, and two industry blogs for papers and policy changes related to federated learning in clinical settings. They were spending roughly four hours a week each — twelve person-hours total — just reading, copying links, and writing summaries in a shared Google Doc. They were brilliant people doing work a script could do.

We built a Research Librarian using exactly the tools in the code above, plus a real DuckDuckGo search integration and a Chroma knowledge base seeded with their previous reading notes. The agent ran every Monday at 5 AM via cron. It searched the web for the past seven days of content, queried their local notes for any related prior work, synthesized a single Markdown report with every claim tied to a URL, saved the file to a Dropbox folder, and logged the run to a SQLite observability table. The team lead reviewed the report at 8 AM and either approved it for their Slack channel or added manual notes where the agent had missed something.

The first version took three evenings to build and cost about twelve cents a week in GPT-4o mini tokens. After six weeks, the team had recovered roughly seventy-two hours of human labor. The agent missed interesting papers twice — both times because the paper had not yet been indexed by search engines — and once it cited a blog post that turned out to be satire. The team added a lightweight source-credibility rule after that, and the error never recurred. They still have the agent running today, and they have expanded it to cover two additional topics. The lesson was not that the agent was perfect. It was that the agent was predictable, cheap, and improvable.

Real Deployment Story 2: The Code Shepherd

A mid-sized SaaS company I consulted for had a chronic problem they did not even name. Their Python microservices had dozens of small libraries that were never updated. Not the dependencies in `requirements.txt` — they had Dependabot for that. I am talking about internal utility modules: a `dates.py` that everyone copied from repo to repo, a `validators.py` that had three slightly different versions across the organization, and a `logging_config.py` that was flat-out broken in one service but worked in another because of a monkey patch.

We built the Code Shepherd. It combined the coding agent from Project 7 with the CI/CD watchdog from Project 11 and the review panel from Project 14. Every night, the Shepherd scanned a configured list of repositories. For each repo, it identified internal utility files that existed in multiple places. It generated a unified version, ran the existing test suite against it, sent the diff to the Code Review Panel for style and security checks, and if the tests passed and the panel approved, it opened a pull request with a detailed description of what had changed and why.

The Shepherd never merged anything automatically. Every PR had to pass human review. But the humans no longer had to write the PRs, and they no longer had to remember which repos were out of sync. Over four months, the Shepherd opened thirty-seven pull requests. Thirty-one were merged as-is. Four needed minor adjustments. Two were rejected because the unified version broke an edge case in one service that the test suite had not covered. Those rejections were not failures; they were the system working exactly as intended. An agent that never gets rejected is an agent you are not watching closely enough.

Real Deployment Story 3: The Support Captain

The most emotionally charged deployment I have ever worked on was a customer support router for a small ecommerce platform. The company received roughly two hundred emails a day. Most were routine: “Where is my order?” “How do I return this?” “Do you ship to Finland?” A small but critical minority were angry customers

threatening chargebacks, defective product reports that needed immediate escalation, or refund requests above a monetary threshold that required manager approval.

We built the Support Captain using the router from Project 16, the knowledge base from Project 8, and the smart email assistant from Project 5. Incoming emails were classified into one of five buckets: billing, shipping, returns, escalation, or general. The classification model was a local LLM running on a cheap GPU instance, so the first-pass routing cost about a tenth of a cent per email. Billing and shipping queries were handed to KB agents that searched the company's help articles and drafted replies. Returns were routed to a specialist that checked order status via API before drafting the response. Escalations were flagged with a red banner and moved to a human queue without any draft being generated.

The approval gate was rigid. Nothing sent automatically. Every drafted reply sat in a review dashboard for twenty minutes before a human clicked send. During that window, the agent could be overridden, edited, or rejected. In the first month, the Support Captain drafted eighty percent of routine replies and handled routing for one hundred percent of incoming mail. The two human support staff shifted from writing template answers all day to handling only escalations and edge cases. Their job got harder in the best possible way: they were solving problems only humans could solve.

Packaging and Documenting

A script on your laptop is not a deployed agent. To move it into the world, you need five files alongside your Python code.

README.md — Two audiences: the person installing it and the person debugging it at 2 AM. Include: what it does, how to install dependencies, how to configure environment variables, how to run it, and where the logs go.

requirements.txt — Pin your versions. Not requests, but requests==2.31.0. Agents break when transitive dependencies shift under them, and you will not be awake to fix it when it happens.

.env.example — List every secret and configuration variable the agent needs, with dummy values. Never commit the real .env. This file is documentation and a setup checklist in one.

Dockerfile — Multi-stage build, Python 3.11 slim, non-root user, copy in your code, install requirements, define a health check, and set the entrypoint to your main() or a FastAPI wrapper. This makes your agent runnable anywhere: your laptop, a VPS, Kubernetes, or a CI runner.

ARCHITECTURE.md — The template from earlier, filled out. This is for you, six months from now, when you have forgotten why you chose SQLite over Redis. It is also for the next developer who inherits your work and needs to know where the sharp edges are.

Here is a minimal but complete Dockerfile for the Research Librarian:

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY capstone_agent.py .
COPY .env.example .env

RUN useradd -m agentuser && chown -R agentuser /app
USER agentuser

HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
  CMD python -c "import capstone_agent; print('ok')"

ENTRYPOINT ["python", "capstone_agent.py"]
```

Monitoring and Maintenance

After deployment, your job shifts from building to gardening. Agents are not fire-and-forget. They are living systems that interact with an unpredictable world. You need three practices to keep yours healthy.

Structured logging: Replace every print() with Python's logging module, and output JSON if you aggregate logs. Every agent action — tool call, LLM invocation, approval decision, budget charge — should leave a trace. When something goes wrong, you should be able to reconstruct the exact sequence of events without guessing.

Health checks: Your Dockerfile defines a health check, but you also need a logical one. Write a `status()` function that returns the agent's current state: last run time, success rate over the past week, current budget burn rate, and any unhandled errors in the queue. Call it before you go to bed on the night of deployment.

Periodic review: I schedule a thirty-minute calendar block every month to read my agent's logs, check its costs, and scan its outputs for drift. Drift happens when the world changes but your agent does not. A search API updates its output format. A website you scrape gets redesigned. A model you rely on is deprecated. The monthly review catches drift before it becomes an outage.

Break and Fix

Break 1: Tool Dependency Fails (One API Is Down)

One morning you wake up to find that `web_search` has been returning 503 errors for six hours. The agent has been generating reports from an empty source list, which means every "research summary" is actually an ungrounded hallucination dressed up as fact.

Why it fails: The pipeline assumed that if a tool returned an empty list, there was simply nothing to find. It did not distinguish between "no results" and "the tool is broken."

Fix: Catch HTTP errors explicitly and surface them as degraded status. In the `_research` helper, wrap the call:

```
try:
    results = self.registry.get("web_search")(query,
max_results=3)
except httpx.HTTPStatusError as exc:
    raise RuntimeError(f"Search API returned
{exc.response.status_code}") from exc
```

Then in `run()`, if both `research` and `KB` raise, the agent hits the `_graceful_degrade` path and returns a warning rather than fabricated content.

Break 2: Context Window Overflow (Too Many Tools)

You register twelve tools and feed them all into the system prompt. The LLM starts ignoring tool instructions, emitting malformed JSON, or repeating itself. The agent enters a loop where it calls the same tool with the same arguments forever.

Why it fails: The model's context window is finite. Every tool description consumes tokens. At a certain density, the instructions crowd out the reasoning space and the model's behavior becomes erratic.

Fix: Trim the tool list dynamically. Before building the system prompt, filter tools by relevance to the classified intent. If the intent is “research,” only include `web_search`, `kb_query`, and `classify_intent` in the prompt. The others stay in the registry but stay out of the conversation until they are needed.

Break 3: Cost Budget Exceeded Mid-Run

Your agent runs on a metered model. A user submits a query that triggers five research loops, each with three LLM calls. Halfway through the fourth loop, the BudgetGuard hits its ceiling and raises `RuntimeError`. The agent crashes, leaving a half-written file and a confused user.

Why it fails: The budget check is correct, but the exception is not caught inside the workflow. A threshold should trigger degradation, not a traceback.

Fix: Wrap the budget exception inside `run()` with a recovery path. When `call_llm` raises due to budget exhaustion, catch it at the synthesis step and append a note to the output: “Budget exhausted. Partial results follow.” Then return whatever sources have been gathered so far.

Break 4: Memory Leak Over Time

The agent runs as a long-lived service. Over weeks, the SQLite database grows to several gigabytes because every conversation, every tool result, and every system message is appended without pruning. Query latency degrades. Backups take forever.

Why it fails: SQLite is not a vacuum. Deleted rows leave space, but unbounded append-only tables grow until they hurt.

Fix: Add a retention policy to AgentMemory. On initialization, purge rows older than a configured threshold:

```
def prune(self, session: str, max_age_days: int = 30) -> None:
    cutoff = time.time() - (max_age_days * 86400)
    self.conn.execute(
        "DELETE FROM memory WHERE session = ? AND ts < ?",
        (session, cutoff),
    )
    self.conn.commit()
```

Call this at the start of each new session. Memory is valuable, but stale memory is noise.

Break 5: User Query Outside Scope

Someone asks your Research Librarian: “Book me a flight to Tokyo.” The agent has no flight tool, but the router misclassifies the intent as “research” and the agent proceeds to search the web for “how to book a flight to Tokyo,” returning a generic travel blog summary. The user thinks the agent is broken.

Why it fails: The scope boundary was implicit. The agent assumed every query was research-shaped.

Fix: Explicitly define out-of-scope categories in the system prompt and in the router. Add a `scope_check` step before classification that looks for keywords like “book,” “buy,” “purchase,” “reserve,” and immediately routes to a polite refusal: “I can’t book flights. I can research topics for you. Would you like travel safety information instead?” Scope clarity is a trust signal.

Try It Now

Variation 1: Team Project — Three Agents That Interact

Find two other people who have read this book. Each of you builds one specialist agent: one handles research, one handles synthesis, one handles email delivery. Connect them via a shared SQLite state database or a lightweight message queue. When the Research agent finishes, it writes its output to a known key. The Synthesis agent polls that key. When Synthesis finishes, it writes to another key. The Delivery

agent picks it up and emails it. You have just built a microservices pipeline with agents instead of REST APIs. Document the schema of your shared state. That schema is your contract.

Variation 2: Community Contribution — Add a New Tool to the Framework

Pick a capability this book did not cover: perhaps a translation API, a calendar integration, or a machine-learning model you trained yourself. Write a Python function with a precise docstring, register it in the ToolRegistry, and extend the router to classify queries that need it. Write a test that proves the tool works in isolation, then write an integration test that proves the agent routes to it correctly. Open-source the tool on a gist or a small repo. That is how communities grow. Your contribution does not need to be big. It needs to be documented.

Variation 3: Blog Post or Presentation — Explain Your Architecture

Nothing solidifies understanding like explaining it to someone else. Write a five-minute blog post or slide deck describing your capstone agent. Include the Mermaid diagram. Include your cost estimate. Include one failure you hit and how you fixed it. Share it with a friend, a study group, or a local meetup. The best way to learn agent architecture is to teach it, because teaching forces you to confront the gaps in your own reasoning.

The OpenClaw Way

OpenClaw handles almost everything we built by hand: the registry becomes the built-in `@tool` decorator, the router becomes a RouterAgent configured with intent schemas, and the synthesis step becomes a SynthesisAgent with a Pydantic output model. The orchestration layer in OpenClaw can run `web_search` and `kb_query` in parallel via a TaskGroup, merge their results automatically, and feed them into the synthesis agent without manual JSON parsing. The approval gate maps to OpenClaw's `human_in_the_loop` flag, which pauses execution and surfaces a web UI prompt. The memory layer is replaced by OpenClaw's built-in SessionStore, which supports SQLite, Redis, or PostgreSQL backends with a single config change. The value proposition is identical logic with less boilerplate, plus built-in cost

tracing across every node in the graph. For a production deployment where multiple developers touch the codebase, OpenClaw's opinionated structure pays for itself in reduced onboarding time.

The Hermes Agent Way

Hermes Agent lets you build the entire Research Librarian without writing a Python class at all. You configure a session with three sub-agents — research, kb, and writer — each with its own prompt and tool access. You then define a workflow in Hermes's YAML syntax: research and kb run concurrently, their outputs are piped to writer, and writer produces a Markdown file. Hermes manages the memory, the approval gate, and the budget tracking through its global cost monitor. The trade-off is control: you live inside Hermes's runtime, and if you need custom Python logic that Hermes does not expose, you have to write a plugin. The benefit is speed. A working prototype of the Research Librarian in Hermes takes about fifteen minutes of configuration instead of two evenings of coding. I often start with Hermes to prove the concept, then migrate to pure Python or OpenClaw when I need custom behavior that the framework cannot express.

Final Word

We started this book with a sixty-line loop and a mock function that returned the current time. We end it with a multi-tool, multi-step agent that remembers, budgets, approves, and degrades. But the real distance you have traveled is not measured in lines of code. It is measured in the questions you now know to ask before you deploy. Does this tool need an approval gate? What happens when the API is down? How much will this cost at scale? How do I know it worked?

Every agent is a promise. When you deploy an agent, you are promising the user that the work will get done, that the result will be trustworthy, and that if something goes wrong, the failure will be visible rather than silent. The agents in this book are not toys. They are prototypes of systems that can run businesses, support patients, write code, and guard safety. Treat them with the seriousness they deserve. Build small. Test exhaustively. Monitor constantly. And never, ever let an agent do something irreversible without a human saying yes.

Make yours trustworthy.

What You Built

In this chapter, you built the capstone of everything that came before:

- A CapstoneAgent class that combines research, knowledge base, document generation, and file tools into a single pipeline.
- A ToolRegistry that registers tools by name and exposes them via docstring-driven descriptions.
- A BudgetGuard that tracks cumulative cost and aborts or degrades when the ceiling is breached.
- An AgentMemory layer backed by SQLite, with retention policies to prevent unbounded growth.
- A router that classifies intent deterministically and falls back to LLM classification when needed.
- An approval gate that pauses the pipeline before destructive or outgoing actions.
- Graceful degradation at the tool, synthesis, and output levels.
- Real deployment stories showing how these pieces fit together in production contexts.
- Packaging artifacts: README, requirements.txt, Dockerfile, .env.example, and ARCHITECTURE.md.
- Monitoring practices: structured logging, health checks, and monthly drift review.
- Break-and-fix experience with API failures, context overflow, budget exhaustion, memory leaks, and scope violations.

You are no longer a beginner. You are an agent architect. Go build something that matters.

Cost for this chapter: Using a local Ollama model for all LLM calls, the cost is \$0.00 in API fees, plus whatever your computer's electricity costs for inference. If you swap to GPT-4o mini, a single Research Librarian run with two research queries, one KB lookup, and one synthesis call costs approximately \$0.002–\$0.005. At one run per hour, eight hours a day, five days a week, that is roughly \$0.40 per week, or about \$1.60 per month. If you scale to GPT-4o for higher-quality synthesis, budget \$0.02–\$0.05 per run, or roughly \$8–\$16 per month under the same

schedule. For the full multi-agent pipeline with parallel specialists and cloud-hosted embeddings via an API, add approximately \$0.01 per KB query, yielding a monthly range of \$3–\$30 depending on model choice and query volume. Always set a `max_usd` ceiling before you deploy, and review your burn rate weekly.

Appendix A: OpenClaw Reference

This appendix is a compact, hands-on reference for OpenClaw, the lightweight, open-source agent framework used throughout this book. OpenClaw provides the wiring layer that connects a language model to tools, reasoning loops, and runtime configuration without imposing a heavy scaffold. If you have built any of the framework-track projects in this book, you have already used OpenClaw. This document collects the essential schema, decorator behavior, reasoning modes, error-handling patterns, and testing recipes in one place so you can scan it quickly during implementation.

A.1 Configuration Schema

OpenClaw reads a single `config.yaml` at startup. Every top-level group is optional in isolation, but a working agent run needs at least `llm` and either `tools` or an agent block that declares them. Below is a complete annotated example.

```
# config.yaml – OpenClaw runtime configuration
llm:
  provider: openai                # openai | anthropic | ollama |
  litellm
    model: gpt-4o-mini
    api_key: ${OPENAI_API_KEY}    # env substitution supported
    temperature: 0.2
    max_tokens: 2048
    base_url: null                # override for proxies / local
vLLM
  timeout: 30                     # seconds per LLM call

tools:
  search:
    module: my_tools.web_search   # import path to tool module
    enabled: true
  calculator:
```



```

    module: my_tools.math
    enabled: true
    # Tool-level kwargs are passed to the decorated function
    precision: 10

agents:
    default:
        name: research_assistant
        system_prompt: >
            You are a precise research assistant. Always cite your
sources.
        reasoning_mode: react                # react | plan | direct
        max_iterations: 10
        tool_names:
            - search
            - calculator
        memory:
            type: buffer                      # buffer | summary |
vector_store
    max_messages: 20

logging:
    level: INFO                             # DEBUG | INFO | WARNING | ERROR
    format: "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
    handlers:
        - type: stream
        - type: file
            filename: openclaw.log
            max_bytes: 1048576
            backup_count: 3

```

Configuration Rules

Key	Required	Notes
llm.provider	Yes	Determines which client SDK is loaded.
llm.model	Yes	Provider-specific model name.
tools.<name>.module	Yes per tool	Dot path to the Python module containing a @tool function.
agents.<name>.reasoning_mode	No; defaults to react	Controls loop behavior (see §A.3).
agents.<name>.max_iterations		

Key	Required	Notes
	No; defaults to 10	Hard ceiling on tool-call turns.
logging.handlers	No	Multiple handlers allowed; file supports rotation via max_bytes.

Environment substitution supports `${VAR}` syntax and optional defaults written as `${VAR:-default}`. Missing variables with no default raise a clear startup error before any LLM call is attempted.

A.2 Tool Decorator Reference

The `@tool` decorator declares a Python function as an OpenClaw tool. The decorator performs three jobs:

1. **Schema extraction** — it inspects the function signature, docstring, and type hints to build a JSON Schema entry that is injected into the LLM prompt.
2. **Registration** — it adds the function to OpenClaw's tool registry so agents can call it by name.
3. **Runtime binding** — it wraps the function so OpenClaw can pass tracing context and config overrides.

Minimal Example

```
from openclaw import tool

@tool
def calculate(expression: str) -> str:
    """Evaluate a mathematical expression safely.

    Args:
        expression: An arithmetic string such as '2 + 3 * 4'.

    Returns:
        The computed result as a string.
    """
    import ast
    node = ast.parse(expression, mode='eval')
```

```

    return str(eval(compile(node, '<string>', 'eval'),
{"__builtins__": {}}))

```

What Becomes the Prompt

OpenClaw assembles the tools description block that is sent to the LLM using exactly these fields:

- **Function name** — calculate
- **Docstring first line** — short summary used as "description"
- **Args section** — each argument becomes a "properties" entry with "description" drawn from the arg description
- **Type hints** — mapped to JSON Schema types (str → "string", int → "integer", float → "number", bool → "boolean", list / dict → "array" / "object")
- **Return annotation** — *not* sent to the LLM; it is for static analysis and for OpenClaw's internal serialization path

Best Practices

- **Always include type hints.** Missing hints default to "string", which weakens the model's call accuracy.
- **Keep the Args section explicit.** Even if type hints are present, the LLM prompt is richer when each argument has a human-readable sentence.
- **Return serializable types.** Prefer str, int, dict, or list. If you return a custom object, override `__str__` or return a JSON string so the LLM can ingest the result in the next loop turn.
- **Avoid side effects in docstrings.** Never mention internal URLs, tokens, or file paths in the docstring; the LLM sees that text.

Async Tools

```

import httpx
from openclaw import tool

@tool
async def fetch_url(url: str) -> str:
    """Fetch raw HTML from a URL."""
    async with httpx.AsyncClient(timeout=10) as client:

```

```
r = await client.get(url)
return r.text[:4000]          # truncate for context window
```

OpenClaw detects async automatically and schedules the tool inside its async agent loop. Mixing sync and async tools in the same agent is permitted.

A.3 Reasoning Modes

OpenClaw supports three reasoning modes. Choose the mode that matches the complexity of the task and the latency budget.

react — ReAct Loop (Default)

The agent thinks, acts, and observes in a loop.

- **Mechanism:** Each turn the LLM receives the full scratchpad of previous thoughts, tool calls, and observations. It then emits either another thought + tool call or a final answer.
- **When to use:** General-purpose tasks; unknown number of steps; scenarios requiring exploration or recovery from tool errors.
- **Trade-off:** Higher token usage; each observation increases prompt size.

plan — Plan-and-Execute

The agent writes a step-by-step plan first, then executes it sequentially.

- **Mechanism:** Turn 1 asks the LLM for a high-level plan (a list of steps). OpenClaw executes each step with the smallest context necessary. The plan can be replanned if a step fails.
- **When to use:** Well-defined multi-step workflows (e.g., data pipelines, report generation) where you want lower per-step cost.
- **Trade-off:** Less nimble than react when the environment changes mid-run; may require replanning logic.

direct — Single-Shot Tool Call

One prompt, one tool call (or zero), then stop.

- **Mechanism:** The LLM decides whether it needs a tool. If it issues a tool call, OpenClaw runs it and returns the raw result without a follow-up LLM pass. If it answers directly, the run ends immediately.
- **When to use:** High-throughput or latency-sensitive endpoints; deterministic transformations (e.g., classify, extract).
- **Trade-off:** No error recovery; no chain of reasoning visible.

Choosing a Mode

Scenario	Recommended Mode
Interactive chat with search and file editing	react
ETL or document-processing pipeline	plan
Classification, sentiment, tagging	direct
Multi-hop reasoning with possible dead ends	react
Cost-sensitive batch job with known steps	plan

A.4 Error Handling Patterns

Agents call code. Code raises exceptions. OpenClaw does not swallow errors by default, but it does provide hooks to decide what happens after a tool failure. Here are the canonical patterns.

Pattern 1: Try/Except Inside the Tool

The safest place to handle an error is inside the tool itself. Return a clear string the LLM can reason about.

```
from openclaw import tool
import requests

@tool
def search_web(query: str) -> str:
    """Search the web and return top snippets."""
```

```

try:
    resp = requests.get(
        "https://api.search.example/v1",
        params={"q": query},
        timeout=10
    )
    resp.raise_for_status()
    data = resp.json()
    return "\n".join(item["snippet"] for item in
data["results"][:3])
except requests.Timeout:
    return "Error: search timed out after 10 seconds."
except requests.HTTPError as e:
    return f"Error: search API returned
{e.response.status_code}."
except Exception as e:
    return f"Error: unexpected failure during search: {e}"

```

Returning a string that starts with "Error:" gives the LLM a semantic signal to try a different query or tool.

Pattern 2: Retry Wrapper Around the Agent Loop

For transient failures (rate limits, network blips), wrap the run in a retry strategy.

```

import time
from openclaw import Agent, Config

def run_with_retry(agent: Agent, prompt: str, max_retries: int =
3):
    for attempt in range(1, max_retries + 1):
        try:
            return agent.run(prompt)
        except openclaw.errors.LLMRateLimitError as e:
            wait = 2 ** attempt
            time.sleep(wait)
        except openclaw.errors.LLMConnectionError:
            if attempt == max_retries:
                raise
            time.sleep(1)
    return {"status": "failed", "error": "Exhausted retries"}

```

Pattern 3: Fallback to Human

When a tool fails and the LLM cannot recover, escalate cleanly.

```

HUMAN_ESCALATION_PHRASES = [
    "i do not know",
    "unable to",

```

```

        "contact support",
    ]

class HumanFallbackAgent(Agent):
    def on_tool_failure(self, tool_name: str, error: str, context: dict):
        human_prompt = (
            f"Tool '{tool_name}' failed with:\n{error}\n\n"
            "Please provide guidance or approve a retry."
        )
        return {"action": "escalate", "prompt": human_prompt}

```

In practice, `on_tool_failure` can write to a queue, send an email, or surface an in-app prompt. The key contract is that the fallback never silently drops the error.

Logging Integration

Always log the full exception before transforming it into a user-facing string. OpenClaw's logger includes the tool name, arguments (sanitized), and traceback when `logging.level` is `DEBUG`.

A.5 Testing Best Practices

Because agents combine LLM reasoning with executable tools, unit testing sits at three layers: tools in isolation, mock-LLM loop behavior, and end-to-end integration. The examples below use the standard library `unittest.mock`.

Unit Tests for Tools

Test tool logic without an LLM or agent loop.

```

import pytest
from my_tools import calculate, search_web

def test_calculate_addition():
    assert calculate("2 + 2") == "4"

def test_calculate_division():
    assert calculate("10 / 2") == "5.0"

def test_search_web_timeout(monkeypatch):
    class FakeResponse:
        def raise_for_status(self):
            raise TimeoutError()
    import requests

```

```

monkeypatch.setattr(requests, "get", lambda **kwargs:
FakeResponse())
    result = search_web("openclaw")
    assert "timed out" in result.lower()

```

Mock LLM Responses

Swap the LLM client with a deterministic fake to test agent loop mechanics.

```

from unittest.mock import MagicMock, patch
from openclaw import Agent, Config

def test_react_loop_selects_correct_tool():
    config = Config.from_dict({
        "llm": {"provider": "openai", "model": "gpt-4o-mini"},
        "tools": {"calculator": {"module": "my_tools"}},
        "agents": {"default": {"reasoning_mode": "react",
"tool_names": ["calculator"]}}
    })
    agent = Agent(config)

    # Fake LLM: first turn calls calculator, second turn answers
    fake_messages = [
        {"role": "assistant", "content": "", "tool_calls": [
            {"id": "tc1", "type": "function", "function": {"name":
"calculator", "arguments": '{"expression":"3*4"}'}}
        ]},
        {"role": "assistant", "content": "The result is 12."}
    ]

    with patch.object(agent.llm, "chat",
side_effect=fake_messages):
        result = agent.run("What is 3 times 4?")
        assert "12" in result

```

Testing the Loop

Confirm boundary conditions such as max-iteration enforcement and plan replanning.

```

def test_max_iterations_enforced():
    config = Config.from_dict({
        "llm": {"provider": "openai", "model": "gpt-4"},
        "tools": {"noop": {"module": "my_tools"}},
        "agents": {"default": {"reasoning_mode": "react",
"max_iterations": 2}}
    })
    agent = Agent(config)

```



```

# Fake LLM always wants to call a tool and never finishes
fake_turn = {"role": "assistant", "content": "", "tool_calls":
[
    {"id": "tc", "type": "function", "function": {"name":
"noop", "arguments": "{}"}}
    ]}

with patch.object(agent.llm, "chat", return_value=fake_turn):
    with pytest.raises(openclaw.errors.MaxIterationsExceeded):
        agent.run("loop forever")

```

Checklist

☐

Tool unit tests cover happy paths, empty inputs, and all except branches.

☐

Agent loop tests use a mock LLM so they run offline and deterministically.

☐

You assert on the *name* of the tool called, not only the final string, to catch routing regressions.

☐

Integration tests (optional, CI-gated) run against a real LLM with temperature: 0 and small token limits.

☐

Log assertions verify that DEBUG output contains both the tool input and the sanitized output.

☐

Flaky integration tests are isolated in a separate CI job that runs only on main or nightly, keeping fast unit tests on every pull request.

A.6 Quick Lookup

Concept	Typical Value / Pattern
Config file	config.yaml in working directory or OPENCLAW_CONFIG env var
Tool module path	

Concept	Typical Value / Pattern
	Python dot path, e.g., my_project.tools.search
Reasoning mode default	react
Max iterations ceiling	Configurable per agent; hard limit prevents runaway loops
Env var substitution	<code>\${VAR_NAME}</code> inside any string value in config
Async tool detection	Automatic via <code>inspect.iscoroutinefunction</code>
Retry / backoff	User-implemented wrapper (Pattern 2) or custom agent subclass
Human fallback	Override <code>on_tool_failure</code> or inspect "Error:" prefix in observations

A.7 Further Resources

- OpenClaw source and issue tracker: <https://github.com/openclaw/openclaw>
- JSON Schema reference for tool contracts: see §4.2 and §11.4 in this book.
- Projects 8, 14, and 22 in this book contain full, annotated OpenClaw implementations.

Appendix B: Hermes Agent Reference

Hermes Agent is a CLI-first autonomous agent framework designed for long-running, stateful tasks. Unlike ephemeral chat interfaces, Hermes maintains persistent state through an embedded or external memory layer, supports multi-session continuity, and exposes a rich toolset for file manipulation, web interaction, process control, and scheduled automation. This appendix provides the definitive configuration, toolset, and operational reference for deploying Hermes in production or local development environments.

B.1 Configuration Specification (config.yaml)

Hermes is configured via a single config.yaml file located at ~/.hermes/config.yaml or specified via the --config flag. The file is organized into six primary sections.

B.1.1 model Section

Key	Type	Default	Description
provider	string	openai	Backend provider (openai, anthropic, local, azure).
name	string	gpt-4o	Model identifier passed to the provider API.
temperature	float	0.3	Sampling temperature (0.0–2.0). Lower values yield deterministic output.
max_tokens	integer	4096	Maximum tokens per response.
api_key	string	null	API key. If omitted, Hermes reads OPENAI_API_KEY or equivalent env var.
base_url	string	null	Custom endpoint for local or proxy deployments.
timeout_seconds	integer	120	HTTP request timeout for model inference.

B.1.2 tools Section

Key	Type	Default	Description
enabled	list	["*"]	Glob list of enabled tools. ["*"] enables all. Use ["terminal", "file"] to whitelist.
terminal.default_timeout	integer	300	Default seconds before foreground shell commands are killed.
terminal.workdir	string	~	Starting working directory for shell sessions.

Key	Type	Default	Description
file.max_size_mb	integer	10	File read/write ceiling to prevent accidental ingestion of large binaries.
web.requests_per_minute	integer	60	Rate limit for unstructured HTTP requests.

B.1.3 memory Section

Key	Type	Default	Description
backend	string	sqlite	Storage engine: sqlite, postgres, or redis.
connection_string	string	~/.hermes/memory.db	Path (SQLite) or DSN (Postgres/Redis).
max_entries	integer	10000	Hard cap on memory entries. Oldest entries are pruned by FIFO.
summary_interval	integer	10	Every n user turns, Hermes auto-summarizes session context into long-term memory.
embedding_model	string	null	Optional embedding model for semantic retrieval (e.g., text-embedding-3-small).

B.1.4 skills Section

Key	Type	Default	Description
directory	string	~/.hermes/skills	Filesystem path for skill modules.
auto_load	boolean	true	Whether to import all .py files in the skills directory at startup.
allowed_imports	list	[]	Whitelist of Python modules skills may

Key	Type	Default	Description
			import. Empty list allows all.

B.1.5 logging Section

Key	Type	Default	Description
level	string	INFO	Python log level: DEBUG, INFO, WARNING, ERROR.
file	string	~/.hermes/hermes.log	Rotating log file path.
max_bytes	integer	10485760	10 MB per log file before rotation.
backup_count	integer	5	Number of rotated log archives to retain.
stderr	boolean	true	Mirror logs to stderr for foreground visibility.

B.1.6 safety Section

Key	Type	Default	Description
human_in_the_loop	boolean	false	If true, Hermes pauses before destructive operations and awaits y/n.
confirm_patterns	list	[]	Regex list. Any proposed command matching a pattern triggers mandatory confirmation.
max_cost_per_session	float	5.0	Estimated USD ceiling. Hermes halts when the provider cost tracker exceeds this value.
allowed_paths	list	["*"]	Glob list of filesystem paths Hermes is permitted to read or modify.
blocked_commands	list	[]	

Key	Type	Default	Description
			Shell commands that are unconditionally rejected (e.g., <code>rm -rf /</code>).

Complete config.yaml Example

```

model:
  provider: openai
  name: gpt-4o
  temperature: 0.3
  max_tokens: 4096
  api_key: null
  base_url: null
  timeout_seconds: 120

tools:
  enabled: ["*"]
  terminal:
    default_timeout: 300
    workdir: "/home/user/workspace"
  file:
    max_size_mb: 10
  web:
    requests_per_minute: 60

memory:
  backend: sqlite
  connection_string: "~/.hermes/memory.db"
  max_entries: 10000
  summary_interval: 10
  embedding_model: "text-embedding-3-small"

skills:
  directory: "~/.hermes/skills"
  auto_load: true
  allowed_imports: ["requests", "pandas", "numpy"]

logging:
  level: INFO
  file: "~/.hermes/hermes.log"
  max_bytes: 10485760
  backup_count: 5
  stderr: true

safety:
  human_in_the_loop: true
  confirm_patterns:
    - "rm\s+-rf"
```

```

- "DROP\s+TABLE"
- "github\.com.*--force"
max_cost_per_session: 5.00
allowed_paths:
- "/home/user/workspace/*"
- "/tmp/*"
blocked_commands:
- "sudo"
- "fdisk"
- "mkfs"

```

B.2 Toolset Guide

Hermes exposes seven core tools to the underlying language model. Each tool is implemented as a JSON-RPC interface and gated by the `tools.enabled` configuration list.

Tool	Purpose	Idempotent
terminal	Execute shell commands in a subprocess with optional background/PTY modes.	No
file	Read, write, patch, and search files on the local filesystem.	Read: Yes; Write: No
web	Perform raw HTTP requests (GET, POST, PUT, DELETE) with custom headers and body.	No
search	Regex-based content search across files or directories; replaces <code>grep/find</code> .	Yes
browser	Headless browser automation for JavaScript-heavy pages, screenshots, and form interaction.	No
cronjob	Schedule recurring prompts and skill invocations using cron-like syntax.	No
skills	Dynamically load, invoke, and manage user-defined Python skill modules.	Depends on skill

terminal

The terminal tool launches a true operating-system shell. It supports foreground execution (default), background daemons via `background=true`, and pseudo-terminal allocation via `pty=true` for interactive CLIs such as REPLs or `git`. Each foreground invocation is subject to `terminal.default_timeout`; background tasks run until

explicitly killed via the process management API. Hermes automatically tracks working-directory state across sequential commands.

file

The file tool provides atomic read and write operations, fuzzy search-and-replace patching, and file creation with automatic parent-directory generation. It is the preferred mechanism for source-code modification because it avoids escape-sequence hazards common in terminal-based heredoc injection. The `max_size_mb` limit prevents the model from attempting to ingest multi-gigabyte binaries.

web

A lightweight HTTP client for REST API interaction. The tool respects `requests_per_minute` and supports JSON, form-encoded, and raw binary payloads. Use this for webhook triggers, cloud-provider metadata retrieval, or third-party API calls.

search

Ripgrep-backed content search. It can list files by name glob, search for regex patterns within files, and return contextual matches. It is significantly faster than spawning a shell grep because it runs in-process and avoids shell-quoting complexity.

browser

Built on Playwright, the browser tool renders JavaScript-heavy SPAs, captures full-page or element-level screenshots, fills forms, and clicks buttons. This is essential when the web tool receives opaque HTML or anti-bot challenges.

cronjob

Allows Hermes to schedule autonomous future work. Each cronjob entry consists of a schedule expression, a prompt, and an optional skill attachment. At trigger time, Hermes spawns a new lightweight session, injects the prompt and skill context, and executes the resulting plan. See Section B.4 for authoring details.

skills

Skills are user-authored Python modules placed in the `skills.directory` folder. Each module exposes functions decorated with `@skill`, which the model can discover via reflection. Skills extend Hermes with domain logic—such as custom data parsers, specialized API clients, or internal workflow helpers—without modifying core framework code.

B.3 Memory System

Hermes operates on a dual-memory architecture: **ephemeral context** (the current conversation window) and **persistent memory** (the database-backed store).

Backends

Backend	Connection String	Best For
SQLite	<code>~/.hermes/memory.db</code>	Single-user workstations, offline operation, zero external dependencies.
PostgreSQL	<code>postgresql://user:pass@host/db</code>	Multi-user deployments, concurrent sessions, complex querying via SQL.
Redis	<code>redis://host:6379/0</code>	High-throughput environments, TTL-based expiration, shared memory across distributed agents.

Entry Lifecycle

When `max_entries` is reached, the store evicts the oldest entries by FIFO. If `embedding_model` is configured, summaries and key user directives are vectorized and stored alongside raw text, enabling semantic retrieval via approximate nearest-neighbor search. The `summary_interval` controls how aggressively Hermes compresses the active conversation into long-term memory; a lower value yields richer historical context but increases database write volume.

Inspection

You can inspect memory directly using the CLI:

```
# List last 20 memory entries
hermes memory list --limit 20
```

```
# Search memory for a keyword
hermes memory search "deployment failed"
```

```
# Export memory to JSON for external analysis
hermes memory export --output session_backup.json
```

For raw SQL access (SQLite/Postgres), connect to the database file or DSN and query the entries table:

```
SELECT timestamp, role, content FROM entries ORDER BY timestamp
DESC LIMIT 50;
```

B.4 Cron Job Authoring

Cron jobs enable Hermes to act as an autonomous background worker. Jobs are defined in `~/.hermes/cron.yaml` and loaded at startup.

Schedule Syntax

Hermes supports standard five-field cron expressions plus a @ shorthand:

Expression	Meaning
* * * * *	Every minute
0 */6 * * *	Every six hours
0 9 * * MON	Every Monday at 09:00
@hourly	At the start of each hour
@daily	At midnight
@weekly	At midnight on Sunday

You may also use interval syntax: `@every 30m` executes every 30 minutes, and `@every 2h` executes every two hours. All times are interpreted in the system local time unless the TZ environment variable is overridden in the job definition.

Job Structure

Each entry requires three fields:

- **schedule:** The cron expression.
- **prompt:** The natural-language directive Hermes receives when the job fires.
- **skills (optional):** A list of skill filenames to preload for that session.

Cron Job Examples

cronjobs:

- name: health-check
schedule: `"*/15 * * * *"`
prompt: "Run `~/.hermes/skills/health_check.py` and append the output to `/var/log/hermes/health.log`. If any service is down, send an alert."
skills: `["health_check"]`
- name: daily-report
schedule: `"0 8 * * *"`
prompt: "Generate a summary of yesterday's Git commits in `/home/user/workspace` and email the markdown report to the team."
skills: `["git_summary", "email_sender"]`
- name: weekly-cleanup
schedule: `"@weekly"`
prompt: "Scan `/tmp` and `/home/user/workspace` for files older than 7 days and prompt for deletion."
skills: `[]`
- name: model-cost-audit
schedule: `"@every 6h"`
prompt: "Inspect `~/.hermes/hermes.log` for token usage markers and append a rolling cost estimate to `~/.hermes/cost.csv`."
skills: `["cost_parser"]`

Skills Attachment

When a skill is attached to a cronjob, Hermes loads that module into the ephemeral session before evaluating the prompt. This avoids bloating every general conversation with domain-specific code while ensuring scheduled tasks have the exact tooling they require.

B.5 Safety and Approval Configuration

The safety section is the primary defense against accidental data loss, runaway spending, and malicious prompt injection.

Human-in-the-Loop

Setting `human_in_the_loop: true` forces Hermes to emit a confirmation prompt before any operation that matches one of the following criteria:

- File deletion or overwrite (write on an existing path)
- Execution of commands matching `confirm_patterns`
- Any operation outside `allowed_paths`

The agent halts and waits for a y or yes response on stdin. If stdin is unavailable (e.g., in a detached cronjob session), the operation is rejected and logged.

Confirm Patterns

`confirm_patterns` accepts a list of regex strings evaluated against proposed shell commands or high-level tool actions. A match triggers mandatory human approval. Recommended patterns for production:

```
safety:
  human_in_the_loop: true
  confirm_patterns:
    - "(?i)rm\s+-rf\s+/"
    - "(?i)dd\s+if="
    - "(?i)DROP\s+"
    - "(?i)TRUNCATE\s+TABLE"
    - "(?i)github\.com.*--force"
    - "(?i)eval\s*\("
  max_cost_per_session: 10.00
  allowed_paths:
    - "/home/user/workspace/*"
    - "/tmp/*"
    - "/var/log/hermes/*"
  blocked_commands:
    - "sudo"
    - "su"
    - "passwd"
    - "mkfs"
    - "fdisk"
```

Cost Controls

`max_cost_per_session` relies on a lightweight cost estimator mapping token usage to provider pricing. When the running total exceeds the threshold, Hermes raises a `CostLimitExceeded` exception and terminates the active session. This is especially critical during iterative code-generation loops or long-running research tasks where context windows balloon. The estimator supports OpenAI and Anthropic pricing tiers out of the box; custom providers can register a `cost_per_1m_tokens` entry in the model block.

Path Sandboxing

`allowed_paths` restricts the file and terminal tools to a whitelist of directories. Glob patterns (`*`, `**`) are supported. If a tool action targets a path outside the allowed set, Hermes rejects the action and reports the violation in the logs. Combined with `blocked_commands`, this provides defense-in-depth against prompt-injection attacks that attempt filesystem traversal or privilege escalation.

B.6 Quick-Start Checklist

1. **Install** Hermes and create `~/.hermes/config.yaml` using the template in Section B.1.
2. **Select** a memory backend: SQLite for personal use, Postgres/Redis for team or server deployments.
3. **Enable** `human_in_the_loop` until you are confident in the agent's behavior patterns.
4. **Populate** `confirm_patterns` and `blocked_commands` with project-specific risk fingerprints.
5. **Write** skills in `~/.hermes/skills/` and expose functions with the `@skill` decorator.
6. **Schedule** recurring tasks in `~/.hermes/cron.yaml` with tight `allowed_paths` scopes.
7. **Monitor** `~/.hermes/hermes.log` and review memory exports periodically for drift or cost spikes.

Hermes is designed to be a durable coworker: configure it with care, audit its memory, and it will maintain context across days, weeks, and projects.

Appendix C: Ollama Setup & Model Selection

Throughout this book, the example code assumes you have access to a local Large Language Model (LLM) via Ollama. This appendix teaches you how to install Ollama, select the right model for each project, and troubleshoot the issues you are most likely to hit.

C.1 Installing Ollama

Ollama is a single-binary tool that downloads, configures, and serves LLMs on your own machine. It is free, open source, and runs on Linux, macOS, and Windows.

Linux (Ubuntu / Debian / Fedora / Arch)

The recommended one-line install uses the official install script:

```
curl -fsSL https://ollama.com/install.sh | sh
```

After installation, the ollama binary is placed in /usr/local/bin and a systemd service is created. Start the service and enable it to run on boot:

```
sudo systemctl start ollama
sudo systemctl enable ollama
```

Verify the installation:

```
ollama --version
```

If you prefer not to run a script with root privileges, you can download a standalone tarball from the [Ollama releases page](#), extract it, and run ./ollama serve manually.

macOS

Download the macOS app from ollama.com. The installer drops the ollama CLI into /usr/local/bin automatically. You can also install via Homebrew:

```
brew install ollama
```

After installation, launch Ollama from Applications or run:

```
ollama serve
```

Windows

Download the Windows installer from ollama.com. Run the .exe and follow the prompts. The installer adds ollama to your PATH, so open a new PowerShell or Command Prompt and type:

```
ollama --version
```

On Windows, Ollama runs as a background service by default. You can stop or restart it from the system tray icon or via Services.

Post-installation checklist

1. The Ollama API listens on `http://localhost:11434` by default. Confirm it is up:
 - `curl http://localhost:11434` # Expected output: “Ollama is running”
1. If you installed on a remote server and want to expose the API to your local network, set the environment variable `OLLAMA_HOST=0.0.0.0` before running `ollama serve`.
2. For GPU acceleration on Linux, ensure the NVIDIA or AMD drivers are installed. Ollama detects CUDA and ROCm automatically on launch.

C.2 Pulling and Managing Models

Ollama models are distributed as self-contained blobs called “Model Files.” You do not need to worry about conversion scripts or Python dependencies; Ollama handles the download and caching for you.

Pulling a model

```
ollama pull llama3.1:8b
```

The first time you reference a model—either with `pull`, `run`, or via the API—Ollama downloads it. Models are cached in `~/.ollama/models` and shared across all users on the machine.

To download the embedding model used in Project 8:

```
ollama pull nomic-embed-text
```

Listing installed models

```
ollama list
```

Typical output:

NAME	ID	SIZE	MODIFIED
llama3.1:8b	8abf...	4.7 GB	2 days ago
mistral:latest	f974...	4.1 GB	1 week ago
nomic-embed-text	6356...	274 MB	3 days ago

Removing a model

```
ollama rm llama3.1:8b
```

If you are short on disk space, remove the largest models first. Use `ollama list` to see sizes before deleting.

Running a model interactively

```
ollama run llama3.1:8b
```

This opens a REPL-style chat in your terminal. Type a message and press Enter. Exit with `/bye` or `Ctrl+D`.

Running a model via the API

Most of the Python projects in this book call Ollama through its OpenAI-compatible REST endpoint:

```
import requests
```

```
response = requests.post(
    "http://localhost:11434/v1/chat/completions",
    json={
```



```

        "model": "llama3.1:8b",
        "messages": [{"role": "user", "content": "Hello!"}]
    }
)

```

C.3 Choosing Models by Task

Not every project in this book needs a 70-billion-parameter model. Part of building cost-effective AI agents is knowing when to use a small, fast model and when to upgrade for accuracy.

Model selection table

Model	Size (approx.)	Speed	Quality	Best For
llama3.1:8b	4.7 GB	Very Fast	Good	Experimentation, prototyping, low-latency chat
llama3.1:70b	40 GB	Slow	Excellent	Complex reasoning, multi-step planning, serious analysis
mistral:latest (7B)	4.1 GB	Fast	Very Good	Balanced daily driver; writing, summarizing, routing
nomic-embed-text	274 MB	Extremely Fast	N/A (embeddings)	Vector search, RAG pipelines (Project 8)
phi4 (14B)	9.1 GB	Moderate	Very Good	Coding tasks, structured output, math

Detailed recommendations

llama3.1:8b — experimentation and learning

Use Meta’s Llama 3.1 8B when you are iterating quickly, running unit tests against an LLM, or working on a laptop with limited RAM. It fits comfortably in 8 GB of unified memory (or VRAM) and responds fast enough for interactive debugging. Its quality is perfectly adequate for

summarization, simple classification, and creative writing drafts. In this book, we default to this model because it lowers the barrier to entry for beginners.

llama3.1:70b — serious reasoning

The 70B variant is what you reach for when an agent must perform multi-hop reasoning, write detailed reports, or generate code with fewer hallucinations. The tradeoff is steep: it requires roughly 40 GB of VRAM (or equivalent system RAM), runs significantly slower, and will saturate even high-end consumer GPUs. Use it for the capstone project (Chapter 25) or any task where accuracy outweighs latency.

mistral:latest — balanced performance

Mistral 7B punches above its weight. It is often competitive with Llama 3.1 8B on benchmarks while using a slightly smaller footprint. If you have already exhausted Llama 3.1 8B on a tricky prompt and want a second opinion without jumping to 70B, Mistral is an excellent middle ground. We recommend it as your default “daily driver” if you have 16 GB of VRAM.

nomic-embed-text — embeddings

This tiny model converts sentences into dense vectors for similarity search. It is not a chat model; it only produces embeddings. In Project 8 (the Knowledge Base Agent) and any chapter that implements Retrieval-Augmented Generation (RAG), this is the model you point your vector database at. Because it is under 300 MB, it loads instantly and processes thousands of documents per minute.

phi4 or similar — coding tasks

Microsoft’s Phi-4 (or comparable coding-specialized models like qwen2.5-coder:14b) excels at structured outputs, function calling, and code completion. If your agent generates JSON schemas, writes SQL, or edits source files, a 14B coding model often outperforms a general 8B model on syntax correctness.

C.4 Quantization Explained

When you see tags like Q4_K_M or Q8_0 on a model download page, you are looking at quantization formats. Quantization is the process of shrinking a model’s weights from 16-bit floating-point numbers down

to 4-bit or 8-bit integers. Smaller weights mean less VRAM, faster loading, and higher throughput—but they also mean slightly reduced precision.

Common quantization levels

Format	Bits per weight	VRAM footprint (8B model)	Quality impact	Use case
FP16	16	~16 GB	None (reference)	Research, training, maximum accuracy
Q8_0	8	~8 GB	Minimal	Production workloads where you can afford the VRAM
Q4_K_M	4	~4.5 GB	Small but noticeable	Default balance; what Ollama usually serves
Q3_K_S	3	~3.5 GB	Moderate	Ultra-low-resource edge devices
Q2_K	2	~2.5 GB	Significant	Emergency fallback only

What the letters mean

- **Q** stands for “quantized.”
- **K** stands for “K-quant,” a smarter mixing strategy where some layers get more bits than others rather than uniform quantization across the whole model.
- **M / S / L** are size variants within the K-quant family: Medium, Small, Large (or sometimes Mixed). Larger variants keep more bits for critical layers.
- **0** in Q8_0 means a straightforward uniform 8-bit quantization with no K-quant mixing.

Practical impact on speed and quality

At **Q4_K_M**, an 8B model loses roughly 1–3% on standard reasoning benchmarks compared to FP16. For most agent tasks—summarizing emails, routing support tickets, drafting meeting notes—this loss is imperceptible. The speed gain is substantial: Q4 loads twice as fast and generates tokens 20–40% faster on most hardware.

At **Q8_0**, the gap to FP16 shrinks to near zero for practical purposes. If you have the extra 4 GB of VRAM, Q8 is a safe upgrade for agents that generate structured JSON or execute multi-step tool calls, where every bit of precision helps.

Avoid **Q2** and **Q3** for anything except extreme resource constraints. These levels introduce enough noise that an agent may start ignoring instructions or producing garbled output.

Setting quantization in Ollama

Ollama’s official library models are already pre-quantized, usually to Q4_K_M or Q5_K_M. If you create a custom model from a raw FP16 safetensors or gguf file, you specify the quantization level in your Modelfile:

```
FROM ./my-model-fp16.gguf
PARAMETER temperature 0.7
```

Then build with:

```
ollama create my-model:q8 -f Modelfile
```

If you need a specific quantization, download it directly from the Hugging Face community GGUF repository and reference it in the FROM line.

C.5 Running Multiple Models

Your machine does not need to be locked to a single model. Ollama lazily loads models into GPU (or system) memory on first request and keeps them resident for a period of inactivity—by default five minutes—before unloading to free VRAM.

Switching between models

There is no global “default model.” Each API request specifies the model it wants:

```
# Chat request
requests.post("...", json={"model": "llama3.1:8b", ...})

# Embedding request in the same script
requests.post("...", json={"model": "nomic-embed-text", ...})
```

If llama3.1:8b is already loaded in VRAM and you suddenly request mistral:latest, Ollama swaps them out. The swap takes a few seconds depending on PCIe bandwidth and disk speed.

Concurrent requests

Ollama handles the HTTP layer concurrently, but the GPU can usually execute only one model forward pass at a time. If you send two simultaneous requests to the same loaded model, they are processed sequentially in the order received. Workarounds include:

1. **Batched prompts:** Combine multiple tasks into a single prompt when possible.
2. **Multiple Ollama instances:** Run separate ollama serve processes on different ports and point different agents to different ports. Each instance maintains its own model in memory, but this doubles RAM usage.
3. **Queue orchestration:** Use a lightweight job queue (Celery, RQ, or even a Python asyncio queue) to feed Ollama serially and stream results back to clients.

Controlling unload behavior

If you want a model to stay resident longer (for example, during a batch job), increase the keep-alive window by passing a header in your API request:

```
headers = {"Keep-Alive": "30m"}
response = requests.post("...", headers=headers, json={...})
```

To force eviction and reclaim VRAM immediately:

```
ollama stop llama3.1:8b
```

C.6 Common Issues and Fixes

GPU not detected

Symptom: `nvidia-smi` shows the GPU, but Ollama falls back to CPU generation and performance is terrible.

- Check that the NVIDIA drivers are installed: `nvidia-smi` should report a driver version of 525 or newer.
- On Linux, ensure the container runtime (Docker) or the `systemd` service has access to the GPU devices. If you run Ollama inside a container, pass `--gpus all` to Docker.
- For AMD GPUs on Linux, ROCm support is still maturing. Consult the Ollama documentation for the latest supported cards and `HSA_OVERRIDE_GFX_VERSION` workarounds.

Insufficient VRAM

Symptom: Ollama prints a message like `llama runner process no longer running` or the system starts swapping heavily.

- Use a smaller model (8B instead of 70B) or a more aggressive quantization (Q4 instead of Q8).
- If you have multiple GPUs, Ollama will attempt to split layers across them, but this is less efficient than fitting the model on a single card. Specify `OLLAMA_NUM_PARALLEL=1` to prevent accidental parallel batching that inflates VRAM usage.
- Close other GPU applications (browsers with hardware acceleration, video editors, other ML training jobs).

Model download interrupted

Symptom: `ollama pull` fails halfway with a network error.

- Ollama supports resume on interrupted downloads. Simply re-run the same pull command; it picks up where it left off.
- If the file ends up corrupted, remove the partial blob manually from `~/.ollama/models/blobs` and pull again.
- For very large models, use a wired connection and avoid peak-hour bandwidth throttling.

“Model not found” errors

Symptom: Your Python script returns 404 when calling the chat endpoint.

- Verify the model name matches exactly, including the tag: llama3.1:8b is not the same as llama3.1 without a tag (the latter defaults to the latest tag, which may not exist yet).
- Run `ollama list` to confirm the model is present locally.
- If you are running Ollama inside Docker or on a remote server, verify the `OLLAMA_HOST` environment variable on the client side matches the server address.

Slow generation on CPU

Symptom: Token generation is 1–5 tokens per second on a modern CPU.

- CPU inference is inherently slow for transformer models. Ensure `llama.cpp` inside Ollama is compiled with the correct acceleration flags (AVX2, AVX-512, or ARM NEON). The official binaries are already optimized, so if you compiled from source, verify your build flags.
- Enable memory mapping (`OLLAMA_USE_MMAP=1`) to reduce RAM pressure at the cost of slightly slower layer swapping.

C.7 Quick-Start Cheat Sheet

Install and verify:

```
curl -fsSL https://ollama.com/install.sh | sh
ollama --version
ollama serve &
```

Pull the three models used most often in this book:

```
ollama pull llama3.1:8b
ollama pull mistral:latest
ollama pull nomic-embed-text
```

Check disk usage:

```
ollama list
```

Test the API from Python:

```
import requests

r = requests.post("http://localhost:11434/api/generate", json={
    "model": "llama3.1:8b",
    "prompt": "Why is the sky blue? Answer in one sentence.",
    "stream": False
})
print(r.json()["response"])
```

If you see a coherent sentence, your local LLM stack is ready. Return to the project chapters and swap in whichever model from the selection table best fits the task at hand.

Appendix D: Cloud Provider Setup

Most agent projects in this book run against cloud LLM APIs. This appendix walks through account creation, API key management, cost estimation, and fallback strategies across the three major providers: OpenAI, Anthropic, and Azure OpenAI.

D.1 OpenAI API

Getting Your Key

1. Create an account at platform.openai.com.
2. Navigate to **Settings** → **Billing** and add a payment method. OpenAI requires a minimum \$5 prepaid credit for new accounts (as of 2024).
3. Go to **API Keys** → **Create new secret key**. Copy the key immediately—OpenAI only displays it once.

Billing Dashboard

The [Usage dashboard](#) shows daily spend, token consumption by model, and request counts. Set a **monthly billing limit** under Settings → Limits to cap accidental runaway costs during development.

Rate Limits

Tier	Conditions	RPM	TPM
Free	New accounts, no spend	3	150
Tier 1	\$5+ paid	3,000	1,000,000
Tier 2	\$50+ paid + 7 days	3,000	2,000,000
Tier 3	\$100+ paid + 7 days	3,500	4,000,000

RPM = requests per minute; TPM = tokens per minute.

Models and Python Setup

```
import os
from openai import OpenAI

client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Summarize this document."}],
    max_tokens=500
)
print(response.choices[0].message.content)
```

Model	Context Window	Strength
gpt-4o	128K tokens	Best reasoning, multimodal
gpt-4o-mini	128K tokens	Fast, cheap, good accuracy
gpt-3.5-turbo	16K tokens	Legacy fallback, lowest cost

D.2 Anthropic Claude API

Getting Your Key

1. Sign up at console.anthropic.com.
2. Add billing information under **Settings** → **Billing**.

3. Generate a key from **API Keys** → **Create Key**.

Anthropic uses a prepaid credit system similar to OpenAI. New accounts often receive \$5 in promotional credits.

Models

Model	Context Window	Best For
claude-3-5-sonnet-20241022	200K tokens	Coding, analysis, agent reasoning
claude-3-haiku-20240307	200K tokens	High-throughput, low-latency tasks

Python Setup

```
import os
from anthropic import Anthropic

client = Anthropic(api_key=os.environ.get("ANTHROPIC_API_KEY"))

response = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=1000,
    messages=[{"role": "user", "content": "Write a Python function to validate email addresses."}]
)
print(response.content[0].text)
```

Cost Comparison

Claude 3.5 Sonnet delivers comparable reasoning quality to GPT-4o at roughly the same price tier. Haiku is priced competitively with GPT-4o-mini. If your agent relies heavily on tool use (function calling), test both providers—OpenAI’s parallel tool calling is more mature, while Anthropic often produces more nuanced reasoning prose. For most beginner projects, Claude 3.5 Sonnet is a safe default when high-quality output matters.

D.3 Azure OpenAI Service

Azure OpenAI is attractive for enterprises already in the Microsoft ecosystem, offering private networking, regional data residency, and consolidated billing.

Resource Creation

1. In the [Azure Portal](#), search for **Azure OpenAI** and create a resource.
2. Select a region that supports your target models (East US and Sweden Central have the broadest coverage).
3. After deployment, go to **Resource Management → Keys and Endpoint**.

Deployment Names

Unlike direct OpenAI, Azure uses **custom deployment names**. When you deploy gpt-4o, you might name it gpt-4o-prod. The API call references this deployment name, not the model name.

```
import os
from openai import AzureOpenAI

client = AzureOpenAI(
    api_key=os.environ.get("AZURE_OPENAI_KEY"),
    azure_endpoint=os.environ.get("AZURE_OPENAI_ENDPOINT"),
    api_version="2024-10-01-preview"
)

response = client.chat.completions.create(
    model="gpt-4o-prod", # your custom deployment name
    messages=[{"role": "user", "content": "Hello"}]
)
```

Key Difference from Direct OpenAI

Azure requires both an endpoint URL and a deployment name. Content filters are applied by default (configurable under **Content Filters** in the Azure portal). Rate limits are managed at the subscription level, not globally.

D.4 Cost Estimation Strategies

Per-Token Math

Cloud LLMs bill by the token, not the character. A token is roughly 0.75 words in English. Providers charge separately for input (what you send) and output (what the model generates).

Example calculation for a 500-word turn:

- Input: 400 words $\approx$ 533 tokens
- Output: 200 words $\approx$ 267 tokens
- Total: 800 tokens

Pricing Snapshot (per 1M tokens, USD)

Model	Input Cost	Output Cost	Typical Session Cost
GPT-4o	\$2.50	\$10.00	\$0.02 – \$0.08
GPT-4o-mini	\$0.15	\$0.60	\$0.001 – \$0.005
GPT-3.5-turbo	\$0.50	\$1.50	\$0.005 – \$0.02
Claude 3.5 Sonnet	\$3.00	\$15.00	\$0.03 – \$0.12
Claude 3 Haiku	\$0.25	\$1.25	\$0.002 – \$0.01

A “typical session” assumes 2–4 turns of agent interaction with moderate context windows.

Estimating Monthly Spend

```
def estimate_monthly_cost(
    users_per_day=100,
    sessions_per_user=3,
    turns_per_session=4,
    input_tokens_per_turn=800,
    output_tokens_per_turn=400,
    input_price_per_1m=2.50,
    output_price_per_1m=10.00
):
    daily_turns = users_per_day * sessions_per_user *
turns_per_session
    daily_input = daily_turns * input_tokens_per_turn
```

```

    daily_output = daily_turns * output_tokens_per_turn
    daily_cost = (daily_input / 1e6 * input_price_per_1m +
                  daily_output / 1e6 * output_price_per_1m)
    return daily_cost * 30

print(f"Monthly GPT-4o estimate: ${estimate_monthly_cost():.2f}")
# Monthly GPT-4o estimate: $432.00

```

For development and early testing, GPT-4o-mini or Claude Haiku cuts this to roughly **\$25–\$35 per month** for the same traffic.

Use this estimator before every project milestone. Share the output with stakeholders so there are no billing surprises when user adoption spikes.

The Rule: Test Local, Deploy Cloud

During iterative development, agents make redundant API calls, fail loops, and log huge stack traces into context windows. Run initial development against a local model via Ollama (see Section 4.2) or mock responses. Switch to cloud APIs only for integration testing, evaluation, and production deployment. This rule alone can reduce early-stage API spend by 80–90%.

D.5 Fallback Chains

Production agents should not fail because one provider returns a 429 rate-limit error or a 503 outage. A fallback chain attempts providers in order, degrading gracefully from premium models to cheaper backups.

Architecture

Primary:	Local Ollama (llama3.1:8b)	→ fast, free, private
Secondary:	OpenAI GPT-4o-mini	→ reliable, cheap
Tertiary:	Anthropic Claude Haiku	→ redundancy

Fallback Chain Code

```

import os
from dataclasses import dataclass
from typing import List, Optional

@dataclass
class Provider:
    name: str
    model: str

```

```

client_type: str
api_key_env: str
endpoint_env: Optional[str] = None

providers: List[Provider] = [
    Provider("ollama", "llama3.1", "ollama", "OLLAMA_HOST"),
    Provider("openai", "gpt-4o-mini", "openai", "OPENAI_API_KEY"),
    Provider("anthropic", "claude-3-haiku-20240307", "anthropic",
"ANTHROPIC_API_KEY"),
]

class LLMClient:
    def __init__(self, provider: Provider):
        self.provider = provider
        if provider.client_type == "openai":
            from openai import OpenAI
            self.client =
OpenAI(api_key=os.environ[provider.api_key_env])
        elif provider.client_type == "anthropic":
            from anthropic import Anthropic
            self.client =
Anthropic(api_key=os.environ[provider.api_key_env])
        elif provider.client_type == "ollama":
            import ollama
            self.client = ollama

    def generate(self, prompt: str) -> str:
        if self.provider.client_type == "openai":
            resp = self.client.chat.completions.create(
                model=self.provider.model,
                messages=[{"role": "user", "content": prompt}]
            )
            return resp.choices[0].message.content
        elif self.provider.client_type == "anthropic":
            resp = self.client.messages.create(
                model=self.provider.model,
                max_tokens=1024,
                messages=[{"role": "user", "content": prompt}]
            )
            return resp.content[0].text
        elif self.provider.client_type == "ollama":
            resp = self.client.generate(model=self.provider.model,
prompt=prompt)
            return resp["response"]
        raise ValueError(f"Unknown client type:
{self.provider.client_type}")

    def generate_with_fallback(prompt: str) -> str:
        last_error = None
        for provider in providers:
            try:
                if provider.api_key_env not in os.environ:

```

```

        print(f" [SKIP] {provider.name}: env var
{provider.api_key_env} not set")
        continue
        print(f" [TRY] {provider.name} ({provider.model})")
        client = LLMClient(provider)
        return client.generate(prompt)
    except Exception as e:
        last_error = e
        print(f" [FAIL] {provider.name}: {e}")
    raise RuntimeError(f"All providers failed. Last error:
{last_error}")

# Usage
result = generate_with_fallback("Explain recursion in one
paragraph.")
print(result)

```

This pattern lets you swap the primary model instantly by reordering the providers list. In production, add circuit-breaker logic (e.g., skip a provider for 60 seconds after three consecutive failures) and telemetry to track which provider is used most often.

D.6 Security Best Practices

Never Commit Keys

API keys are secrets. A leaked GPT-4 key can be exploited to burn through thousands of dollars of credits in minutes. Add these patterns to your .gitignore:

```

.env
*.pem
config/secrets.yaml

```

Use Environment Variables

Store keys in environment variables or a secrets manager (AWS Secrets Manager, Azure Key Vault, or Doppler). Never hard-code them in source files.

```

# .env file (not committed)
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
AZURE_OPENAI_KEY=...
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com/

```

Load with python-dotenv in development:

```
from dotenv import load_dotenv
load_dotenv() # reads .env into os.environ
```

Rotate Keys Regularly

Set a calendar reminder to rotate keys every 90 days. In the OpenAI and Anthropic dashboards, you can create a new key, update your deployment environment, and revoke the old one within minutes. Azure supports key regeneration directly in the portal under **Keys and Endpoint**.

Summary

Task	Provider	Key Action
General agent reasoning	OpenAI GPT-4o / GPT-4o-mini	Set billing limit; use gpt-4o-mini for dev
Nuanced writing or coding	Anthropic Claude 3.5 Sonnet	Test tool-use latency against OpenAI
Enterprise compliance	Azure OpenAI	Match region to data residency requirements
Zero-cost local dev	Ollama	Use as primary in fallback chain

Start small with GPT-4o-mini or Claude Haiku, estimate monthly spend before scaling, and build fallback chains so a single provider outage does not stop your agent.

Appendix E: Troubleshooting & Debugging

No matter how carefully you build your agent, it *will* break in production. The loop will freeze, the model will hallucinate a tool name, JSON will escape its brackets, or a single run will quietly cost more than your weekly coffee budget. This appendix is your field manual for those moments. It collects the most common failure modes,

shows you how to read the evidence in your logs, and gives you concrete code you can paste into your agent right now. Keep this nearby. You will need it.

E.1 Common Failures and Symptoms

When an agent misbehaves, the first job is to name the failure correctly. The table below maps the symptom you see to the most likely cause and the immediate fix. Use it as a quick triage sheet before you dive into logs.

Symptom	Likely Cause	Immediate Fix
Agent calls wrong tool	Tool descriptions are ambiguous or overlap	Rewrite description strings in the @tool decorator. Give each tool a single, distinct job. Remove near-duplicate tools.
Agent loops forever	Missing max_turns, vague prompt, or ambiguous goal	Add max_turns=10 (or lower). Add a completion clause to the system prompt: “Stop and return DONE when you have the answer.”
JSON parsing fails	Model outputted plain text instead of JSON, or unescaped characters inside strings	Add ``jsonmarkers around the expected output in the prompt. Escape user content before inserting it into the schema. Userresponse_format={"type": "json_object"}if the provider supports it. Out of memory Context window too large, oversized documents, or high concurrency Reducechunk_sizefor text splitting. Summarize earlier turns instead of passing full history. Drop to a smaller model (e.g.,gpt-4o-miniorllama3.2:3b). Costs spiraling Agent is calling the LLM in a tight loop, or you are using an oversized model for simple tasks Add budget tracking (see §E.3). Route simple tasks to a cheap or local model. Capmax_tokensaggressively. Agent returns stale data No cache invalidation,

Symptom	Likely Cause	Immediate Fix
		or the search tool uses stale parameters Add afreshnessparameter to your search tool. Include timestamps in the prompt and ask the model to prefer recent results. Tool call silently ignored Tool raised an exception that was swallowed by the framework Enableverbose=True(see §E.4). Wrap tool bodies in explicittry/except` blocks that return error strings rather than raising.

If the symptom is not in the table, the rest of this appendix will show you how to trace the root cause with logs and structured reasoning.

E.2 Reading Agent Logs

Logs are the black box of your agent. When behavior is inexplicable, the log file is usually the first place the truth hides. A well-structured log lets you reconstruct exactly what the model saw, what it decided, and how the environment responded.

What to Look For

Every agent run should emit records that contain at least the following fields:

- **Prompt tokens** — the number of tokens sent *to* the model. A sudden spike means the history, document chunks, or tool outputs grew unexpectedly. This is often the first warning of an imminent out-of-memory or cost overrun.
- **Completion tokens** — the number of tokens returned *by* the model. If this hits `max_tokens` on every turn, the model is being cut off mid-thought. Raise the limit or tighten the prompt.
- **Tool calls** — the exact name and arguments the model chose. If the arguments are malformed, the JSON schema in the tool description is probably unclear.
- **Errors** — any exception, timeout, or non-2xx HTTP status. Capture the full stack trace; a swallowed exception will propagate as silent failure three turns later.

- **Latency per turn** — time from request to response. Sustained high latency suggests a slow tool, a rate-limit throttle, or an oversized prompt that the provider is struggling to process.

Enabling Structured Logging

Plain print statements scatter information across stdout and make it impossible to search or aggregate. Use Python's logging module with a structured formatter. Below is a drop-in setup that writes JSON lines to a rotating file and pretty-printed lines to the console.

```
import logging
import json
import sys
from logging.handlers import RotatingFileHandler

class JSONFormatter(logging.Formatter):
    def format(self, record):
        log_obj = {
            "timestamp": self.formatTime(record),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
            "module": record.module,
            "function": record.funcName,
        }
        if hasattr(record, "extra"):
            log_obj.update(record.extra)
        return json.dumps(log_obj, default=str)

def setup_logging(name: str = "agent", level: int =
logging.DEBUG):
    logger = logging.getLogger(name)
    logger.setLevel(level)
    logger.handlers.clear()

    # Console handler – human readable
    console = logging.StreamHandler(sys.stdout)
    console.setLevel(logging.INFO)
    console.setFormatter(logging.Formatter(
        "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s"
    ))
    logger.addHandler(console)

    # File handler – structured JSON, rotated at 5 MB
    file_handler = RotatingFileHandler(
        "agent_debug.jsonl", maxBytes=5_000_000, backupCount=3
    )
    file_handler.setLevel(logging.DEBUG)
    file_handler.setFormatter(JSONFormatter())
```

```

    logger.addHandler(file_handler)

    return logger

# Usage in your agent loop
logger = setup_logging()
logger.info("Agent loop started", extra={"session_id": "sess_42"})
logger.debug("Prompt payload", extra={
    "prompt_tokens": 1240,
    "messages": messages,
})

```

With this in place, you can search the JSON log with jq or grep for specific patterns:

```

# Find every turn that cost more than $0.01
jq 'select(.cost_cents > 1)' agent_debug.jsonl

# Find every tool call
jq 'select(.event == "tool_call")' agent_debug.jsonl

```

E.3 Debugging the Reasoning Loop

Most agents in this book use a ReAct-style loop: the model observes the current state, thinks about what to do next, and acts by calling a tool. When the loop goes wrong, you need visibility into each of those three stages.

Trace Observation → Thought → Action

Add lightweight tracing around the loop body. The trace should capture:

1. **Observation** — the exact environment state presented to the model (tool results, error messages, retrieved documents).
2. **Thought** — the model's raw completion before it is parsed into a tool call.
3. **Action** — the parsed tool name and arguments, plus any validation errors.

```

def run_agent_with_trace(agent, query: str, max_turns: int = 10):
    logger = logging.getLogger("agent")
    state = {"query": query, "turns": []}

    for turn in range(max_turns):
        observation = build_observation(state)

```

```

        logger.debug(f"Turn {turn} observation",
extra={"observation": observation})

        raw_thought = agent.llm.complete(observation)
        logger.debug(f"Turn {turn} raw thought", extra={"thought":
raw_thought})

        action = parse_action(raw_thought)
        logger.info(f"Turn {turn} action", extra={"action":
action})

        if action.name == "finish":
            return action.arguments["answer"]

        result = execute_tool(action)
        logger.info(f"Turn {turn} tool result", extra={"result":
result})

        state["turns"].append({
            "observation": observation,
            "thought": raw_thought,
            "action": action,
            "result": result,
        })

    raise RuntimeError(f"Agent exceeded max_turns ({max_turns})")

```

If you see the same observation and the same thought repeated across multiple turns, the model is stuck. That usually means the prompt does not give the model enough new information after each tool call, or the model is unsure how to recognize completion.

Use verbose=True

If you are using a framework wrapper that supports a verbose flag, turn it on during development. It prints the full prompt and response at each turn. The noise is worth it: you will catch prompt-injection leaks, missing context, and truncated tool descriptions early.

E.4 Rate Limiting Recovery

Production agents hit rate limits. It is not a question of *if* but *when*. A robust agent should treat a 429 or 503 response as a temporary weather pattern, not a fatal exception.

Exponential Backoff

The simplest resilience layer is an exponential backoff decorator. It retries the LLM call with increasing delay, adding random jitter so that a cluster of retrying agents does not stampede the provider.

```
import time
import random
import functools
import logging

logger = logging.getLogger("agent")

def backoff_retry(max_retries: int = 5, base_delay: float = 1.0):
    """Retry a function with exponential backoff and jitter."""
    def decorator(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_retries + 1):
                try:
                    return fn(*args, **kwargs)
                except Exception as e:
                    if "429" in str(e) or "rate limit" in
str(e).lower():
                        delay = base_delay * (2 ** (attempt - 1))
                        jitter = random.uniform(0, delay)
                        sleep_time = delay + jitter
                        logger.warning(
                            f"Rate limited (attempt {attempt})/
{max_retries}). "
                            f"Sleeping {sleep_time:.2f}s."
                        )
                        time.sleep(sleep_time)
                    else:
                        raise
            raise RuntimeError(f"Failed after {max_retries}
retries")
        return wrapper
    return decorator

# Usage
@backoff_retry(max_retries=5, base_delay=1.0)
def call_llm(messages, **kwargs):
    return openai_client.chat.completions.create(
        model="gpt-4o-mini", messages=messages, **kwargs
    )
```

Request Queuing and Provider Switching

If your agent runs continuously or serves multiple users, a decorator is not enough. Add a request queue with a token-bucket rate limiter, and keep a secondary provider on standby.

```
from collections import deque
import threading

class ThrottledClient:
    def __init__(self, primary, fallback, max_rpm: int = 60):
        self.primary = primary
        self.fallback = fallback
        self.max_rpm = max_rpm
        self.queue = deque()
        self.lock = threading.Lock()
        self.tokens = max_rpm

    def request(self, messages, **kwargs):
        with self.lock:
            if self.tokens > 0:
                self.tokens -= 1
                client = self.primary
            else:
                client = self.fallback
                logger.warning("Primary rate limit exhausted;
switching to fallback.")
            try:
                return client.complete(messages, **kwargs)
            except Exception as e:
                if "429" in str(e):
                    logger.warning("Primary failed with 429; trying
fallback.")
                return self.fallback.complete(messages, **kwargs)
            raise
```

With this architecture, a rate limit becomes a routing event rather than a crash.

E.5 Mitigating Model Hallucination

Hallucination is the hardest bug in agent engineering because the model will sound confident while being completely wrong. You cannot eliminate it, but you can add guardrails that catch the most dangerous lies before they reach the user or the next tool.

Hallucination Check Function

Run a self-consistency check after the model returns a factual claim. Ask the same question again with a slightly rephrased prompt, or ask the model to verify its own citation.

```
def hallucination_check(agent, claim: str, source_documents: list)
-> dict:
    """
    Returns a confidence score and a list of missing citations.
    """
    prompt = f"""You previously claimed:
    \"{claim}\"

    Verify this claim using ONLY the following source documents.
    For each sentence in the claim, cite the exact document and
    passage that supports it.
    If a sentence has no supporting evidence, flag it as
    UNSUPPORTED.

    Source documents:
    {json.dumps(source_documents, indent=2)}

    Respond in JSON:
    {{
        "verified_claims": [...],
        "unsupported_claims": [...],
        "confidence_score": 0.0-1.0
    }}
    """
    response = agent.llm.complete(prompt)
    verification = json.loads(response)

    if verification["confidence_score"] < 0.7:
        logger.warning(
            "Low-confidence claim detected",
            extra={
                "claim": claim,
                "unsupported": verification["unsupported_claims"],
            },
        )
    return verification
```

Additional Guardrails

Combine the check function with these structural defenses:

- **Citation enforcement** — Add a rule to the system prompt: “Every factual statement must include a citation to the source document. Claims without citations are rejected.”

- **Search augmentation** — Before the model answers, force a search or retrieval step. The model should answer from retrieved context, not from parametric memory.
- **Temperature control** — Set temperature=0.0 for factual extraction tasks. Higher temperatures increase creative paraphrasing, which increases hallucination risk.
- **System prompt rules** — Include an explicit uncertainty clause: “If you are not certain, say ‘I don’t know’ rather than guessing.”
- **Self-consistency checks** — For high-stakes outputs, generate three independent answers and return the majority vote (or the intersection of claims supported by all three).

E.6 Quick Troubleshooting Flowchart

When something goes wrong and you are not sure where to start, follow this flowchart in order. Do not skip steps; the cheapest fix is usually the first one.

1. Is the agent producing any output at all?

- **No** → Check API key, network connectivity, and model availability. Look at the very first log line for a connection or authentication error.
- **Yes** → Continue to step 2.

2. Are the outputs wrong, or are they missing?

- **Missing / empty** → The model may be hitting max_tokens. Raise the limit or shorten the prompt. Check for a JSON parsing failure in the logs.
- **Wrong / nonsense** → Continue to step 3.

3. Is the model calling the wrong tool, or using wrong arguments?

- **Yes** → Rewrite tool descriptions to remove ambiguity. Remove overlapping tools. Add examples of correct usage to the prompt.
- **No** → Continue to step 4.

4. Is the loop running indefinitely?

- **Yes** → Add or lower max_turns. Add a completion condition to the system prompt. Check that tool results provide new information each turn.
- **No** → Continue to step 5.

5. Are the answers factually incorrect or invented?

- **Yes** → Enable citation enforcement. Add a retrieval step before generation. Run the hallucination check function (§E.5). Lower temperature to 0.0.
- **No** → Continue to step 6.

6. Is the process slow, expensive, or crashing?

- **Slow / costly** → Add request queuing and backoff (§E.4). Reduce chunk size or context length. Route simple tasks to a cheaper model.
- **Crashing / OOM** → Reduce memory footprint with smaller models, shorter history, or document chunking (§E.1 table).

If you reach the end of this chart and the issue persists, it is time to ask for help (§E.7).

E.7 Where to Get Help

You are not alone when your agent breaks. The ecosystem around LLM agents is young, vocal, and generous with debugging time. Know where to look before you are panicking at 2 a.m.

Resource	Best For	How to Use It
GitHub Issues on the framework repo	Suspected bugs, regressions, or undocumented behavior	Search existing issues first. Include a minimal reproduction script, your environment (pip freeze), and the exact error traceback.
Discord / Slack communities	Quick sanity checks, architecture advice, and “has anyone seen this	Join the official server for your framework (OpenClaw, LangChain, LlamaIndex, etc.). Use the #help or #debugging

Resource	Best For	How to Use It
	before?” questions	channel. Paste logs in a thread, not the main channel.
Official documentation	Configuration reference, API semantics, and upgrade guides	Bookmark the docs for the exact version you have installed. Doc URLs are stable; search engines sometimes surface outdated versions.
Community cookbooks & examples	Patterns for retrieval, tool design, and multi-agent orchestration	The examples/ folder in a framework repository is often more up to date than the prose docs. Read the code comments.
Model provider status pages	Sudden outages, elevated error rates, or latency spikes	Check status.openai.com, status.anthropic.com, or your self-hosted dashboard before you assume your code is broken.

One last piece of advice: when you do ask for help, bring the logs. A well-structured `agent_debug.jsonl` file and a two-sentence description of the symptom will get you an answer faster than a thousand-word story about what you were trying to build. The community runs on evidence, not narrative.

E.8 Summary

- Agents fail predictably: wrong tools, infinite loops, JSON errors, memory exhaustion, runaway costs, and hallucinations.
- Structured JSON logging turns guessing into diagnosis. Log prompt tokens, completion tokens, tool calls, errors, and latency.
- Trace the ReAct loop explicitly: capture observation, thought, and action at every turn to spot stuck or truncated reasoning.
- Rate limits are a routing problem, not a crash. Wrap LLM calls in exponential backoff and keep a fallback provider ready.
- Hallucination cannot be eliminated, but it can be detected. Enforce citations, augment with search, freeze temperature for facts, and run self-consistency checks on high-stakes claims.

- Follow the troubleshooting flowchart methodically, and when you need human help, bring structured logs and a minimal reproduction.

Your agent will break. That is normal. What matters is how quickly you can see the failure, name it, and fix it. This appendix exists so that “quickly” is measured in minutes, not days.

Glossary

Agent — A system that observes its environment, reasons about what to do, takes actions via tools, and learns from feedback. An agent is a loop, not a one-shot prompt.

API Gateway — A single entry point that routes requests to the right backend service. In agent systems, it is the interface between users and multiple tools.

Chain-of-Thought (CoT) — Prompting the model to show its reasoning step by step before giving the final answer. Improves accuracy on complex tasks.

Context Window — The maximum amount of text an LLM can process in one call, measured in tokens. Larger windows allow more documents and history inside a single prompt.

Conversation Buffer — A sliding window of recent exchanges kept in the prompt so the LLM remembers the current session.

Cron Job — A scheduled task that runs at fixed intervals (e.g., every hour, daily at 9 AM). Named after the Unix cron utility.

Embedding — A dense vector that captures the semantic meaning of text. Similar sentences have similar vectors. Used for search and knowledge retrieval.

Feedback Loop — The cycle where an agent’s output is evaluated, scored, and fed back into the system to improve future performance.

Finite State Machine (FSM) — A model of computation with a finite number of states. The agent moves between states (e.g., idle, researching, writing) based on events.

Guardrail — A rule or check that prevents an agent from doing something dangerous, expensive, or unwanted (e.g., cost limit, safety filter).

Hallucination — When an LLM generates plausible-sounding but factually incorrect or unverifiable content.

Human-in-the-Loop (HITL) — A design pattern where the agent pauses and asks a human for approval before executing high-stakes actions.

LLM — Large Language Model. A neural network trained on text to predict the next word, capable of reasoning, writing, coding, and conversation.

Local Model — An LLM that runs entirely on your own hardware (e.g., via Ollama) without sending data to a cloud provider.

Memory Bank — Persistent storage of facts, events, and relationships that the agent can recall across sessions.

Multi-Agent System — A collection of specialized agents that collaborate, communicate, and collectively solve problems that exceed the capacity of any single agent.

Ollama — A tool that makes it easy to download, run, and manage LLMs on your local machine with a simple CLI and API.

OpenClaw — A lightweight agent framework that uses decorators, configuration files, and a reasoning loop to build tools that LLMs can call.

Orchestrator — The agent that coordinates other agents in a multi-agent system. It decides which sub-agent to call and how to merge their results.

Prompt — The text you send to an LLM. It includes instructions, context, and sometimes examples. The quality of the prompt directly determines the quality of the output.

Quantization — Reducing the precision of a model's weights (from 16-bit to 4-bit, for example) to save memory and speed up inference. Quantized models trade a small amount of quality for large performance gains.

ReAct — Reasoning and Acting. A reasoning mode where the LLM thinks out loud before choosing an action. The sequence is Observation → Thought → Action (loop).

Reasoning Step — A single iteration of an agent's loop: receive input, think, choose action, observe result.

Retrieval-Augmented Generation (RAG) — A pattern where an agent retrieves relevant documents from a knowledge base and includes them in the prompt before generating an answer. The external data grounds the LLM and reduces hallucination.

Semantic Search — Search based on meaning rather than exact keyword matches. Uses embeddings to find conceptually related content.

Structured Output — LLM output constrained to a specific format (JSON, table, list, CSV). Enforced via system prompts, regex, or schema parsers.

System Prompt — The hidden instructions that define the LLM's role, constraints, and behavior for an entire session.

Token — A unit of text for LLMs. Roughly 0.75 words in English. API pricing, context windows, and rate limits are all measured in tokens.

Tool — A function the LLM can call. The LLM produces the function name and arguments; your code executes it and returns the result.

Trace — A complete record of everything an agent did in a session: inputs, reasoning, tool calls, outputs, costs, and errors.

Vector Database — A database optimized for storing and searching high-dimensional vectors (embeddings). Examples include Chroma, Qdrant, and Pinecone.

Zero-Shot — Asking the LLM to perform a task without providing examples. The opposite of few-shot (providing examples in the prompt).

Index

(Index generated programmatically from chapter cross-references. Key terms defined in the Glossary.)

- Agents: see definition in Glossary; see also Chapters 1, 25
- API Gateways: Chapters 6, 16, 23
- Browser Automation: Chapter 9
- CI/CD: Chapter 11
- Cloud Providers: Appendix D
- Cost Management: Chapters 1, 25; Appendix D
- Deployment: Chapters 23, 24, 25
- Docker: Chapter 23
- Email Automation: Chapter 5
- Embeddings: Chapters 8, 12
- Ethics: Chapter 22
- FastAPI: Chapter 23
- Feedback Loops: Chapters 10, 18, 24
- File Organization: Chapters 3, 12
- Hallucination: see Glossary; see Chapters 2, 8, 19; Appendix E
- Human-in-the-Loop: Chapters 5, 7, 10
- Knowledge Bases: Chapter 8
- Local Models: Appendix C
- Memory: Chapters 8, 18, 25; Appendix B
- Multi-Agent Systems: Chapters 13–18
- Ollama: Appendix C
- OpenClaw: Appendix A

- Orchestrator: Chapters 17, 18
- Prompt Engineering: Chapters 1, 13, 19
- RAG: see Chapter 8
- Rate Limiting: Appendix E
- ReAct: Chapters 1, 8, 9
- Safety: Chapter 22
- Semantic Search: Chapter 8
- Spreadsheet Intelligence: Chapter 4
- State Machines: Chapters 11, 16, 18
- Streaming: Chapters 1, 7
- Tools: see Chapters 1, 6, 7; Appendix A
- Troubleshooting: Appendix E
- Vector Databases: Chapter 8
- Workflow Orchestration: Chapter 18

About the Publisher

SMF Works publishes practical technology books for builders. We believe the best way to learn is by building, breaking, and fixing real systems. Every book comes with working code, honest cost estimates, and the hard-won lessons of shipping things that matter.

Visit: smfworks.com

Get the Code

All code examples from this book are available at:

github.com/smfmike/ai-agents-projects

Submit issues, suggest improvements, and share what you build.

First edition: 2026